

UNIVERSITÀ DI PISA

Dipartimento di Informatica

Corso di Laurea in Informatica

Programmazione ed Algoritmica

Dispense per il Corso di Laurea in Informatica

Docente: Prof.ssa Anna Bernasconi

Autore: Diego Stefanini

cc: Davide Paolicchi

Anno Accademico 2025/2026

Ultima revisione: 26/04/2026

Prefazione

Queste dispense raccolgono gli argomenti del corso di Programmazione ed Algoritmica. Il testo copre sia la parte algoritmica — complessità, ordinamento, strutture dati — sia la parte di linguaggi di programmazione — grammatiche formali, semantica operativa, sistemi di tipi. L'obiettivo è fornire un riferimento auto-consistente per lo studio autonomo.

Struttura del testo. I primi quattro capitoli trattano i fondamenti dei linguaggi di programmazione: dalla definizione formale della sintassi tramite grammatiche, alla semantica operativa dei programmi, fino ai sistemi di tipi e alle funzioni ricorsive. I capitoli successivi sviluppano la parte algoritmica: complessità computazionale, tecniche di progettazione (divide et impera), algoritmi di ordinamento e strutture dati fondamentali.

Convenzioni tipografiche. I **termini definiti per la prima volta** sono indicati in grassetto all'interno delle definizioni. Gli *enunciati di teoremi e proposizioni* sono in corsivo. Le dimostrazioni si concludono con il simbolo $\square$. Lo pseudocodice è presentato nella sintassi del linguaggio MAO, con `=` per le dichiarazioni e `:=` per gli assegnamenti.

Indice

1. Introduzione	2
1.1. Cos'è l'Informatica	2
1.2. Algoritmo	2
1.3. Programma e programmazione	4
1.4. Computer e problem solving	4
1.5. Problemi computazionali	5
1.6. Correttezza e terminazione	6
1.7. Algoritmi come tecnologia	6
1.8. Linguaggi di Programmazione	7
1.9. Il linguaggio MAO	8
1.10. Sintassi e Semantica	9
1.11. Obiettivi della programmazione	11
2. Linguaggi Formali e Grammatiche	13
2.1. Alfabeti, Stringhe e Linguaggi	13
2.2. Linguaggi formali	15
2.3. Grammatiche formali	16
2.4. Gerarchia di Chomsky	17
2.5. Backus-Naur Form (BNF)	18
2.6. Derivazioni e Linguaggi Generati	19
2.7. Linguaggio generato da un non terminale	20
2.8. Derivazioni canoniche	22
2.9. Alberi di derivazione	23
2.10. Ambiguità	25
2.11. Risoluzione dell'ambiguità	26
3. Semantica Operazionale	28
3.1. Regole di inferenza	28
3.2. Sistemi logici	29
3.3. Induzione	32
3.4. Esercizi: Dimostrazioni Induttive su Grammatiche	35
3.5. MiniMao	41
3.6. Semantica di MiniMao	45
3.7. Regole semantiche per i comandi	48
3.8. Riepilogo ed esempi	55
3.9. Terminazione e divergenza	62
4. Sistemi di Tipi, Funzioni e Ricorsione	64
4.1. Il linguaggio MAO e gli array	64
4.2. Analisi statica e sistema di tipi	69
4.3. Estensioni del linguaggio	78
4.4. Funzioni	82
4.5. Ricorsione	85

5. Complessità Computazionale	91
5.1. Modello di calcolo e costo computazionale	91
5.2. Limiti inferiori alla difficoltà di un problema	101
5.3. Problem solving: ricerca del punto di transizione	104
6. Divide et Impera	108
6.1. Il paradigma Divide et Impera	108
6.2. Ricerca Binaria	109
6.3. Merge Sort	113
6.4. Relazioni di ricorrenza	117
6.5. Master Theorem	123
7. Algoritmi di Ordinamento	127
7.1. Insertion Sort	127
7.2. Selection Sort	129
7.3. Invariante di ciclo: schema generale	132
7.4. QuickSort	133
7.5. Analisi di complessità di QuickSort	135
7.6. QuickSort Randomizzato	137
7.7. Confronto degli algoritmi di ordinamento	139
7.8. Heap e HeapSort	140
7.9. Limite inferiore per l'ordinamento basato su confronti	156
7.10. Ordinamento in tempo lineare	157
8. Strutture Dati	172
8.1. Strutture Dati Lineari	172
8.2. Code con priorità	185
8.3. Riepilogo e confronto	186
8.4. Insiemi dinamici e Dizionari	186
8.5. Alberi Binari	187
8.6. Alberi Binari di Ricerca (BST)	192
8.7. Tabelle Hash	203
9. Programmazione Dinamica	217
9.1. Introduzione alla Programmazione Dinamica	217
9.2. Fibonacci con Programmazione Dinamica	218
9.3. Il problema del Taglio delle Corde	221
9.4. Longest Common Subsequence (LCS)	224
9.5. Partizione di un Insieme di Interi	230
9.6. Riepilogo	234
10. Grafi	235
10.1. Definizione di Grafo	235
10.2. Grafi non orientati	237
10.3. Grafi orientati	239
10.4. Rappresentazione dei grafi in memoria	240
10.5. Visita in Ampiezza (BFS)	242
10.6. Visita in Profondità (DFS)	248
10.7. Cammini minimi da sorgente unica	251
10.8. Algoritmo di Dijkstra	255
11. Calcolabilità e Complessità Computazionale	259
11.1. Problemi computazionali, calcolabilità e complessità	259
11.2. Insiemi numerabili e diagonalizzazione	260
11.3. Esistenza di problemi indecidibili	262
11.4. Il problema dell'arresto	263

11.5. Problemi intrattabili	266
11.6. Le classi di complessità	269
11.7. Verifica e classe NP	271
11.8. Riduzioni polinomiali	272
11.9. NP-hard e NP-completi	273
11.10. La gerarchia completa e la questione di P vs NP	276
11.11. Esercizi	277

CAPITOLO 1

Introduzione

1.1. Cos'è l'Informatica

DEFINIZIONE 1.1.1 (Informatica). Lo studio sistematico degli **algoritmi** che descrivono e trasformano le informazioni: la loro teoria, analisi, progettazione, efficienza, implementazione e applicazione. (ACM – Association for Computing Machinery)

L'informatica non è semplicemente lo studio dei computer, ma lo studio dei *processi algoritmici*: come rappresentare l'informazione, come trasformarla e come farlo in modo efficiente. Il calcolatore è lo strumento che permette di eseguire tali processi in modo rapido e affidabile.

1.2. Algoritmo

DEFINIZIONE 1.2.1 (Algoritmo). Una sequenza **finita** di **passi** (istruzioni) **univocamente determinati** che, se eseguiti da un **esecutore**, portano alla risoluzione di un **problema**.

Le parole chiave di questa definizione meritano un approfondimento:

- **Passi**: operazioni elementari, cioè istruzioni la cui esecuzione è ben definita e non ulteriormente scomponibile dal punto di vista dell'esecutore.
- **Esecutore**: l'entità (umano o macchina) che esegue i passi dell'algoritmo. Nel nostro caso l'esecutore è un calcolatore.
- **Problema**: la domanda a cui l'algoritmo deve rispondere, specificata in modo preciso tramite input e output.

Le tre componenti fondamentali di un algoritmo sono dunque:

1. il **problema** da risolvere,
2. il **procedimento** (la sequenza di passi) da seguire,
3. l'**esecutore** che interpreta ed esegue le istruzioni.

Nota. La descrizione dell'algoritmo deve essere comprensibile dall'esecutore: un algoritmo corretto ma scritto in un linguaggio che l'esecutore non comprende risulta inutile.

1.2.1. Proprietà degli algoritmi

Un algoritmo deve soddisfare le seguenti proprietà:

- **Finitezza:** l'algoritmo è composto da un numero finito di passi e la sua esecuzione deve terminare in tempo finito per ogni input valido.
- **Non ambiguità:** ogni passo è definito in modo preciso, senza possibilità di interpretazioni diverse. L'esecutore non deve mai trovarsi nella condizione di non sapere cosa fare.
- **Determinismo:** dato lo stato corrente dell'esecuzione, il passo successivo è univocamente determinato. A parità di input, l'algoritmo produce sempre lo stesso output.
- **Generalità:** l'algoritmo risolve un'intera *classe* di problemi, non una singola istanza specifica.

ESEMPIO 1.2.2 (Algoritmo per il massimo di un vettore). **Problema:** dato un array $A[1..n]$ di n interi, determinare il valore massimo.

- **Input:** array A di n interi
- **Output:** il valore m tale che $m = \max\{A[i] : 1 \leq i \leq n\}$

Algoritmo (in linguaggio naturale):

1. Assumi il primo elemento come massimo corrente.
2. Per ciascun elemento successivo, confrontalo con il massimo corrente: se è maggiore, aggiorna il massimo corrente.
3. Al termine, restituisci il massimo corrente.

In MAO:

Massimo di un array

```
int max(int[] A, int n){
    int m = A[1];
    int i = 2;
    while(i <= n){
        if(A[i] > m){
            m := A[i];
        }
        i := i + 1;
    }
    return m;
}
```

1.3. Programma e programmazione

DEFINIZIONE 1.3.1 (Programma). La formulazione di un algoritmo in un **linguaggio di programmazione**. Un programma specifica al calcolatore quali operazioni eseguire, in quale ordine, su quali dati e sotto quali condizioni.

DEFINIZIONE 1.3.2 (Programmazione). L'attività di progettare e scrivere programmi, ossia tradurre un algoritmo in una sequenza di istruzioni eseguibili da un calcolatore.

1.3.1. Differenza tra algoritmo e programma

Un algoritmo è un concetto *astratto*, indipendente dal linguaggio in cui viene espresso. Un programma è la sua realizzazione *concreta* in un linguaggio specifico. Lo stesso algoritmo può essere implementato in linguaggi diversi, ottenendo programmi differenti ma semanticamente equivalenti.

Aspetto	Algoritmo	Programma
Livello	Astratto	Concreto
Linguaggio	Naturale / pseudocodice	Linguaggio di programmazione
Esecutore	Umano o macchina	Solo macchina
Dettagli implementativi	Possono essere omessi	Tutti specificati

Tabella 1: Confronto tra algoritmo e programma

1.4. Computer e problem solving

DEFINIZIONE 1.4.1 (Computer). Un **calcolatore** è una macchina in grado di eseguire operazioni elementari con grande **rapidità** e **precisione**, seguendo le istruzioni di un programma.

Un computer riceve in **input** un programma (testo) e un insieme di dati, e produce in **output** il risultato dell'esecuzione del programma sui dati forniti. A differenza di altre macchine automatiche (ad esempio una lavatrice, il cui comportamento è fisso), un computer è **programmabile**: il compito svolto dipende interamente dal programma caricato.

1.4.1. Problem solving

DEFINIZIONE 1.4.2 (Problem solving). Attività sistematica finalizzata all'analisi e alla risoluzione di *problemi computazionali*, ovvero problemi formulabili in termini di input, output e di una relazione tra essi.

Il processo di problem solving si articola nelle seguenti fasi:

1. **Specifica:** definizione precisa del problema, stabilendo chiaramente quali sono gli input ammissibili e qual è l'output atteso.
2. **Progettazione:** ideazione di un algoritmo che risolve il problema, ragionando sulla strategia risolutiva.
3. **Analisi:** studio delle proprietà dell'algoritmo, in particolare la sua *correttezza* (produce l'output corretto per ogni input valido) e la sua *efficienza* (quanto tempo e quanta memoria richiede).
4. **Codifica:** traduzione dell'algoritmo in un linguaggio di programmazione, ottenendo un programma.
5. **Testing e debugging:** verifica sperimentale della correttezza del programma su input di prova e correzione degli eventuali errori.
6. **Esecuzione:** esecuzione del programma sul calcolatore.

Nota (Problemi irrisolvibili). Non tutti i problemi computazionali ammettono una soluzione algoritmica. Esistono problemi per i quali si può dimostrare che *non esiste alcun algoritmo* in grado di risolverli. Un esempio celebre è il **Problema della fermata** (*Halting Problem*): dato un programma P e un input x , stabilire se P termina quando eseguito su x . Alan Turing dimostrò nel 1936 che non esiste alcun algoritmo in grado di rispondere correttamente per ogni coppia (P, x) .

1.5. Problemi computazionali

DEFINIZIONE 1.5.1 (Problema computazionale). Un problema formulato in modo preciso di cui si cerca una soluzione algoritmica. Un problema computazionale è definito da tre elementi:

- **Input:** l'insieme dei dati in ingresso, cioè la descrizione di tutte le possibili *istanze* del problema.
- **Output:** il risultato atteso.
- **Relazione input-output:** il vincolo che lega l'output all'input, specificando quale output è corretto per ciascun input.

DEFINIZIONE 1.5.2 (Istanza di un problema). Un'istanza è un input specifico, cioè un particolare elemento dell'insieme degli input ammissibili. Un algoritmo che risolve un problema deve produrre l'output corretto per *ogni* istanza.

ESEMPIO 1.5.3 (Problema dell'ordinamento).

- **Input:** una sequenza di n numeri $\langle a_1, a_2, \dots, a_n \rangle$
- **Output:** una permutazione $\langle a'_1, a'_2, \dots, a'_n \rangle$ della sequenza di input tale che $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Ad esempio, data l'istanza $\langle 5, 2, 8, 1 \rangle$, l'output corretto è $\langle 1, 2, 5, 8 \rangle$.

ESEMPIO 1.5.4 (Problema della ricerca).

- **Input:** una sequenza di n numeri $\langle a_1, a_2, \dots, a_n \rangle$ e un valore k
- **Output:** un indice i tale che $a_i = k$, oppure -1 se k non compare nella sequenza

Ad esempio, data l'istanza $\langle 3, 7, 1, 9 \rangle$ con $k = 7$, l'output corretto è $i = 2$.

1.6. Correttezza e terminazione

Dato un problema computazionale, un algoritmo che intende risolverlo deve soddisfare due requisiti fondamentali.

DEFINIZIONE 1.6.1 (Correttezza). Un algoritmo si dice **corretto** rispetto a un problema computazionale se, per ogni istanza valida dell'input, produce l'output corretto (cioè l'output che soddisfa la relazione input-output del problema).

DEFINIZIONE 1.6.2 (Terminazione). Un algoritmo si dice **terminante** se, per ogni istanza valida dell'input, la sua esecuzione si conclude in un numero finito di passi.

Nota. Un algoritmo che produce risultati corretti ma non termina su certi input non è considerato un algoritmo valido. Analogamente, un algoritmo che termina sempre ma produce risultati errati non è utile. Sono necessarie *entrambe* le proprietà: correttezza e terminazione.

1.7. Algoritmi come tecnologia

Si potrebbe pensare che, data la velocità dei calcolatori moderni, l'efficienza degli algoritmi sia un aspetto secondario. Non è così: la scelta dell'algoritmo può fare la differenza tra una soluzione praticabile e una inutilizzabile, persino su hardware potente.

ESEMPIO 1.7.1 (Insertion Sort vs Merge Sort). Consideriamo due algoritmi per l'ordinamento: l'Insertion Sort, con complessità $\Theta(n^2)$ nel caso pessimo, e il Merge Sort, con complessità $\Theta(n \log n)$.

Supponiamo di voler ordinare $n = 10^7$ elementi (dieci milioni). Un calcolatore veloce A esegue 10^{10} operazioni al secondo e utilizza l'Insertion Sort; un calcolatore lento B esegue 10^7 operazioni al secondo e utilizza il Merge Sort.

Con costanti ragionevoli ($2n^2$ per l'Insertion Sort, $50n \log n$ per il Merge Sort):

Aspetto	Calcolatore A (Insertion Sort)	Calcolatore B (Merge Sort)
Velocità	10^{10} op/s	10^7 op/s
Costo	$2 \cdot (10^7)^2 = 2 \cdot 10^{14}$ op	$50 \cdot 10^7 \cdot \log_2 10^7 \approx 1.16 \cdot 10^{10}$ op
Tempo	$\frac{2 \cdot 10^{14}}{10^{10}} = 20000$ s ≈ 5.5 ore	$\frac{1.16 \cdot 10^{10}}{10^7} \approx 1163$ s ≈ 19 min

Tabella 2: Confronto tra un calcolatore veloce con un algoritmo lento e un calcolatore lento con un algoritmo efficiente.

Il calcolatore B , pur essendo 1000 volte più lento, termina quasi 20 volte prima del calcolatore A . Per n ancora più grande, il divario aumenta ulteriormente.

Questo esempio illustra un principio fondamentale: gli algoritmi sono una **tecnologia**, al pari dell'hardware. Un buon algoritmo su una macchina modesta può superare un cattivo algoritmo su una macchina potente. Per questa ragione, lo studio dell'efficienza algoritmica è parte integrante dell'informatica.

1.8. Linguaggi di Programmazione

Abbiamo visto che un algoritmo ed un programma sono concetti distinti: il primo è una descrizione astratta di un procedimento risolutivo, il secondo è la sua formulazione concreta in un linguaggio comprensibile da una macchina. Il **linguaggio di programmazione** è lo strumento che consente questa traduzione.

DEFINIZIONE 1.8.1 (Linguaggio di programmazione). Un linguaggio di programmazione è un linguaggio formale dotato di una **sintassi** rigorosa e di una **semantica** precisa, progettato per esprimere algoritmi in una forma eseguibile da un calcolatore.

I linguaggi di programmazione condividono alcune caratteristiche fondamentali:

- sono **formali**: ogni costrutto ha una definizione precisa e non ambigua;
- sono **eseguibili**: un programma scritto correttamente può essere eseguito da una macchina;
- sono **espressivi**: permettono di descrivere algoritmi che risolvono problemi computazionali.

Nota. Un linguaggio di programmazione funge da **ponte** tra il ragionamento umano e l'esecuzione automatica. Esistono centinaia di linguaggi (C, Python, Java, JavaScript, Haskell, ...), ciascuno con caratteristiche proprie, ma tutti fondati sugli stessi concetti di base che studieremo in questo corso.

1.9. Il linguaggio MAO

Per lo studio dei fondamenti della programmazione utilizzeremo un linguaggio didattico appositamente progettato.

DEFINIZIONE 1.9.1 (MAO -- Modello Astratto Operazionale). MAO è un linguaggio di programmazione imperativo semplificato, progettato a fini didattici. Pur essendo sintetico, MAO include tutti i costrutti fondamentali presenti nei linguaggi reali: dichiarazioni, assegnamenti, condizionali, cicli, funzioni e array.

La scelta di un linguaggio didattico permette di concentrarsi sui **concetti** della programmazione senza le complicazioni tecniche dei linguaggi industriali (gestione della memoria, librerie, ambienti di sviluppo, ecc.).

1.9.1. Costrutti principali

I costrutti fondamentali di MAO sono riassunti nella tabella seguente.

Costrutto	Sintassi	Descrizione
Dichiarazione	<code>int x = 5;</code>	Introduce una nuova variabile con tipo e valore iniziale
Assegnamento	<code>x := x + 1;</code>	Modifica il valore di una variabile già dichiarata
Condizionale	<code>if(x > 0){ ... } else { ... }</code>	Esegue un blocco in base a una condizione
Ciclo	<code>while(x > 0){ ... }</code>	Ripete un blocco finché la condizione è vera
Funzione	<code>int f(int a){ return a * 2; }</code>	Definisce un sottoprogramma con parametri e valore di ritorno
Array	<code>int[] A = int[5];</code>	Dichiara una sequenza indicizzata di valori

Tabella 3: Costrutti fondamentali del linguaggio MAO

Nota (Dichiarazione vs. Assegnamento). In MAO la distinzione tra **dichiarazione** e **assegnamento** è resa esplicita dalla sintassi:

- il simbolo `=` si usa esclusivamente nella **dichiarazione**, cioè quando si introduce una nuova variabile (ad esempio `int x = 5;`);
- il simbolo `:=` si usa per l'**assegnamento**, cioè per modificare il valore di una variabile già esistente (ad esempio `x := x + 1;`).

Questa convenzione, diversa da quella adottata dalla maggior parte dei linguaggi reali (che usano `=` per entrambe le operazioni), ha il vantaggio di rendere immediatamente visibile se un'istruzione sta creando una nuova variabile o modificandone una esistente.

ESEMPIO 1.9.2 (Primo programma in MAO).

```
int x = 5;
int y = 3;
int somma = x + y;
```

Il programma dichiara due variabili intere `x` e `y`, poi dichiara una terza variabile `somma` il cui valore iniziale è la somma delle prime due. Al termine dell'esecuzione, `somma` contiene il valore 8.

ESEMPIO 1.9.3 (Uso del ciclo while).

```
int n = 5;
int fatt = 1;
while(n > 1){
    fatt := fatt * n;
    n := n - 1;
}
```

Questo programma calcola il fattoriale di 5. Si noti l'uso di `:=` per gli assegnamenti all'interno del ciclo: le variabili `fatt` e `n` sono già state dichiarate e qui ne modifichiamo il valore.

1.10. Sintassi e Semantica

Lo studio di un linguaggio di programmazione si articola in due aspetti complementari ma distinti: la **sintassi**, che riguarda la forma delle frasi, e la **semantica**, che riguarda il loro significato.

Aspetto	Domanda chiave	Esempio
Sintassi	Come si scrive un programma valido?	<code>if(x > 0){ ... }</code>
Semantica	Cosa significa un programma valido?	Se $x > 0$, esegui il blocco

Tabella 4: I due livelli di analisi di un linguaggio

DEFINIZIONE 1.10.1 (Sintassi). La sintassi di un linguaggio è l'insieme delle regole che stabiliscono quali sequenze di simboli costituiscono frasi (programmi) **ben formate**. In altri termini, la sintassi definisce la struttura grammaticale del linguaggio.

DEFINIZIONE 1.10.2 (Semantica). La semantica di un linguaggio associa un **significato** a ciascuna frase sintatticamente corretta. Nel caso dei linguaggi di programmazione, la semantica specifica quale computazione viene eseguita da un programma: quali operazioni vengono svolte, in quale ordine e con quale effetto sullo stato della macchina.

La distinzione tra sintassi e semantica è cruciale: un programma può essere sintatticamente corretto ma semanticamente errato (ovvero, compila senza errori ma non fa ciò che vogliamo), oppure sintatticamente scorretto e quindi non eseguibile affatto.

ESEMPIO 1.10.3 (Stesso significato, sintassi diversa). L'operazione «incrementa x di 1» si scrive in modo diverso a seconda del linguaggio:

Linguaggio	Sintassi
MAO	<code>x := x + 1;</code>
Python	<code>x = x + 1</code>
C/C++	<code>x++;</code>

Tabella 5: La stessa operazione in linguaggi diversi

Le tre istruzioni hanno la stessa **semantica** (incrementare il valore di x di un'unità) ma **sintassi** differente. Questo dimostra che la sintassi è una proprietà del linguaggio, mentre la semantica è legata al significato dell'operazione.

1.10.1. Perché una sintassi formale?

Nel linguaggio naturale, un interlocutore umano è in grado di interpretare frasi anche in presenza di errori:

ESEMPIO 1.10.4 (Ambiguità del linguaggio naturale). La frase «Dmoani vado al mrae» è comprensibile per un essere umano come «Domani vado al mare», nonostante gli errori di ortografia. Un calcolatore, invece, non possiede questa capacità di interpretazione: richiede istruzioni scritte in modo **esatto** e **non ambiguo**.

Per questo motivo i linguaggi di programmazione hanno una sintassi **formale**, definita in modo rigoroso tramite **grammatiche formali**. Una grammatica formale specifica in modo preciso e completo quali sequenze di simboli appartengono al linguaggio e quali no.

ESEMPIO 1.10.5 (Errori sintattici in MAO).

```
int x = ;           // ERRORE: manca l'espressione dopo =  
if x > 0 { }        // ERRORE: mancano le parentesi tonde  
int 3y = 10;        // ERRORE: identificatore non valido
```

Ciascuna di queste righe viola una regola sintattica di MAO e viene rifiutata prima ancora che il programma possa essere eseguito. Il compilatore (o l'interprete) segnala l'errore indicando quale regola è stata violata.

Nota. Le grammatiche formali non sono una peculiarità dei linguaggi di programmazione: anche le lingue naturali possiedono una grammatica (soggetto + verbo + complemento in italiano). La differenza fondamentale è che le grammatiche dei linguaggi naturali ammettono eccezioni e ambiguità, mentre quelle dei linguaggi formali sono **complete** e **non ambigue** per costruzione.

1.11. Obiettivi della programmazione

Scrivere un programma corretto non è sufficiente: un buon programma deve soddisfare diversi criteri di qualità.

- **Correttezza:** il programma deve risolvere il problema per cui è stato progettato, producendo l'output atteso per ogni input valido. La correttezza può essere verificata tramite **testing** (esecuzione su casi di prova) o dimostrata formalmente tramite **invarianti** e **precondizioni/postcondizioni**.
- **Efficienza:** il programma deve utilizzare le risorse computazionali (tempo di esecuzione e spazio in memoria) in modo ragionevole. Lo studio dell'efficienza è l'oggetto della **complessità computazionale**, trattata nella parte algoritmica del corso.
- **Leggibilità:** il codice deve essere comprensibile da altri programmatori (e dal programmatore stesso a distanza di tempo). Nomi di variabili significativi, commenti appropriati e una struttura logica chiara contribuiscono alla leggibilità.
- **Manutenibilità:** il programma deve essere facilmente modificabile e aggiornabile. Un codice ben strutturato e modulare facilita la correzione di errori e l'aggiunta di nuove funzionalità.

Nota. Questi obiettivi possono talvolta entrare in conflitto: ad esempio, un'ottimizzazione aggressiva per l'efficienza può rendere il codice meno leggibile. La buona pratica di programmazione consiste nel trovare il giusto equilibrio tra questi criteri.

1.11.1. Verso la formalizzazione

Per poter studiare i linguaggi di programmazione con rigore matematico, è necessario formalizzarne sia la sintassi sia la semantica. Nel seguito del corso affronteremo questi temi in modo sistematico:

1. **Linguaggi formali e grammatiche:** definiremo gli strumenti matematici (alfabeti, stringhe, grammatiche) per descrivere la sintassi di un linguaggio.
2. **Semantica operativa:** formalizzeremo il significato dei programmi tramite regole di inferenza che descrivono come un programma modifica lo stato della macchina durante l'esecuzione.
3. **Sistemi di tipi:** studieremo come verificare staticamente (cioè prima dell'esecuzione) che un programma rispetti vincoli di coerenza sui tipi dei dati.

Tutti questi aspetti saranno illustrati sul linguaggio MAO, che verrà introdotto gradualmente: prima nella sua versione base (MiniMao, con sole espressioni e comandi semplici), poi nella versione completa (con array, funzioni e ricorsione).

CAPITOLO 2

Linguaggi Formali e Grammatiche

2.1. Alfabeti, Stringhe e Linguaggi

2.1.1. Alfabeto

DEFINIZIONE 2.1.1 (Alfabeto). Un **alfabeto** A è un insieme finito e non vuoto di simboli. I singoli elementi dell'alfabeto sono chiamati **simboli** o **caratteri**.

ESEMPIO 2.1.2 (Alfabeti comuni).

- Alfabeto latino minuscolo: $A_1 = \{a, b, c, d, \dots, z\}$
- Alfabeto binario: $A_2 = \{0, 1\}$
- Alfabeto di un linguaggio di programmazione: $A_3 = \{a, \dots, z, 0, \dots, 9, +, -, \times, \dots\}$

2.1.2. Stringa

DEFINIZIONE 2.1.3 (Stringa). Una **stringa** (o **parola**) su un alfabeto A è una sequenza finita di simboli appartenenti ad A . Formalmente, dati $a_1, a_2, \dots, a_n \in A$ con $n \geq 0$, la sequenza $a_1 a_2 \dots a_n$ è una stringa su A .

2.1.2.1. Lunghezza di una stringa

DEFINIZIONE 2.1.4 (Lunghezza). La **lunghezza** di una stringa $s = a_1 a_2 \dots a_n$ è il numero naturale n di simboli che la compongono e si denota con $|s|$.

DEFINIZIONE 2.1.5 (Stringa vuota). Se $n = 0$ la stringa è detta **stringa vuota** e si denota con ε . Si ha $|\varepsilon| = 0$.

ESEMPIO 2.1.6 (Lunghezze di stringhe). Sull'alfabeto $A = \{a, b, f, z\}$:

$$|abfbz| = 5$$

$$|aaa| = 3$$

$$|\varepsilon| = 0$$

2.1.3. Operazioni sulle stringhe

2.1.3.1. Concatenazione

DEFINIZIONE 2.1.7 (Concatenazione). Date due stringhe $s = a_1 \cdots a_n$ e $t = b_1 \cdots b_m$ su un alfabeto A , la **concatenazione** di s e t è la stringa $s \cdot t = a_1 \cdots a_n b_1 \cdots b_m$. Si ha $|s \cdot t| = |s| + |t|$.

La stringa vuota ε è l'**elemento neutro** della concatenazione: per ogni stringa s , $s \cdot \varepsilon = \varepsilon \cdot s = s$.

ESEMPIO 2.1.8 (Concatenazione). Sull'alfabeto $A = \{a, b, c\}$:

$$ab \cdot ca = abca$$

$$ab \cdot \varepsilon = ab$$

2.1.4. Insiemi di stringhe di lunghezza fissata

DEFINIZIONE 2.1.9 (Insieme A^n). Dato un alfabeto A e un numero naturale $n \geq 0$, definiamo A^n come l'insieme di tutte e sole le stringhe su A che hanno lunghezza esattamente n .

ESEMPIO 2.1.10 (Stringhe su $A = \{0, 1\}$). $A^0 = \{\varepsilon\}$

$$A^1 = \{0, 1\}$$

$$A^2 = \{00, 01, 10, 11\}$$

$$A^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

In generale, se $|A| = k$, allora $|A^n| = k^n$.

2.1.5. Chiusura di Kleene

DEFINIZIONE 2.1.11 (Chiusura di Kleene A^*). Dato un alfabeto A , la **chiusura di Kleene** (o **chiusura riflessiva e transitiva**) A^* è l'insieme di tutte le stringhe su A , inclusa la stringa vuota:

$$A^* = \bigcup_{n \geq 0} A^n = A^0 \cup A^1 \cup A^2 \cup A^3 \cup \dots = \{\varepsilon\} \cup A \cup A^2 \cup A^3 \cup \dots \quad (1)$$

Nota (Cardinalità di A^*). Se A non è vuoto, A^* è un insieme **infinito numerabile**: contiene infinite stringhe, ma è possibile elencarle tutte (ad esempio ordinandole per lunghezza crescente e, a parità di lunghezza, in ordine lessicografico).

DEFINIZIONE 2.1.12 (Chiusura positiva A^+). La **chiusura positiva** A^+ è l'insieme di tutte le stringhe non vuote su A :

$$A^+ = \bigcup_{n \geq 1} A^n = A^* \setminus \{\varepsilon\} \quad (2)$$

2.2. Linguaggi formali

DEFINIZIONE 2.2.1 (Linguaggio formale). Un **linguaggio formale** L su un alfabeto A è un qualunque sottoinsieme di A^* :

$$L \subseteq A^* \quad (3)$$

Nota (Linguaggi particolari).

- Il **linguaggio vuoto** $\emptyset$ non contiene alcuna stringa (attenzione: $\emptyset \neq \{\varepsilon\}$).
- Il **linguaggio universale** A^* contiene tutte le possibili stringhe su A .
- Il linguaggio $\{\varepsilon\}$ contiene solo la stringa vuota.

ESEMPIO 2.2.2 (Linguaggi sull'alfabeto binario). Sull'alfabeto $A = \{0, 1\}$:

- $L_1 = \{0, 1, 00, 11\}$ è un linguaggio finito.
- $L_2 = \{0^n 1^n \mid n \geq 1\} = \{01, 0011, 000111, \dots\}$ è un linguaggio infinito. Contiene le stringhe formate da n zeri seguiti da n uni, con $n \geq 1$.

2.2.1. Descrizione dei linguaggi

Descrivere un linguaggio di programmazione significa specificare l'insieme delle stringhe ben formate (i **programmi**) del linguaggio. Poiché i programmi ammissibili sono tipicamente infiniti, non è possibile elencarli uno per uno: servono meccanismi finiti per descrivere insiemi infiniti.

Esistono due approcci fondamentali:

- **Metodo generativo:** si definisce una **grammatica** che genera tutte e sole le stringhe del linguaggio, partendo da un simbolo iniziale e applicando regole di riscrittura.
- **Metodo riconoscitivo:** si definisce un **automa** che, data una stringa, decide se essa appartiene o meno al linguaggio.

In questo capitolo ci concentriamo sul metodo generativo.

2.3. Grammatiche formali

DEFINIZIONE 2.3.1 (Grammatica formale). Una **grammatica formale** è una tripla $G = (T, N, P)$ dove:

- T è un insieme finito di simboli **terminali** (l'alfabeto del linguaggio).
- N è un insieme finito di simboli **non terminali** (anche detti **categorie sintattiche** o **variabili**), con $T \cap N = \emptyset$.
- P è un insieme finito di **produzioni** (o **regole di riscrittura**).

Posto $S = T \cup N$ l'insieme di tutti i simboli, ogni produzione ha la forma:

$$\alpha ::= \beta \quad \text{con } \alpha \in S^* N S^* \text{ e } \beta \in S^* \quad (4)$$

dove α è detta **parte sinistra** e β è detta **parte destra**. La condizione $\alpha \in S^* N S^*$ impone che la parte sinistra contenga **almeno un non terminale**.

Nota (Simbolo iniziale). In molte formulazioni la grammatica è definita come una quadrupla $G = (T, N, S_0, P)$, dove $S_0 \in N$ è un **simbolo iniziale** (o **assioma**) distinto. Il linguaggio generato dalla grammatica è il linguaggio del simbolo S_0 . Nella formulazione a tripla, il simbolo iniziale è indicato implicitamente come il primo non terminale delle produzioni.

ESEMPIO 2.3.2 (Grammatica per stringhe di parentesi bilanciate). $T = \{ (,) \}$, $N = \{ \text{Par} \}$, con produzioni:

$\text{Par} ::= \varepsilon$

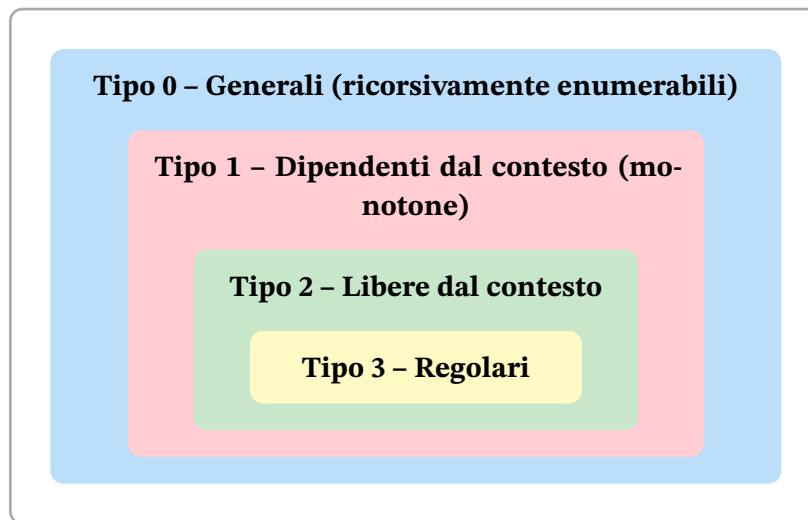
$\text{Par} ::= (\text{Par})$

$\text{Par} ::= \text{Par Par}$

Questa grammatica genera tutte le stringhe di parentesi bilanciate: $\varepsilon, (), ((), () (), ((()), \dots$

2.4. Gerarchia di Chomsky

Le grammatiche formali si classificano in base alla forma delle loro produzioni. La **gerarchia di Chomsky** definisce quattro classi, ognuna contenuta nella precedente.



DEFINIZIONE 2.4.1 (Tipo 0 -- Grammatiche generali). Le produzioni hanno la forma $\alpha ::= \beta$ con $\alpha \in S^*NS^*$ e $\beta \in S^*$, senza alcuna restrizione ulteriore. I linguaggi generati sono detti **ricorsivamente enumerabili** e sono riconosciuti dalle macchine di Turing.

DEFINIZIONE 2.4.2 (Tipo 1 -- Grammatiche dipendenti dal contesto). Le produzioni hanno la forma $\alpha ::= \beta$ con il vincolo $|\alpha| \leq |\beta|$ (le produzioni non possono accorciare la stringa). Si ammette l'eccezione $S_0 ::= \epsilon$ solo se S_0 non compare nella parte destra di alcuna produzione.

I linguaggi generati sono detti **dipendenti dal contesto** e sono riconosciuti dagli automi lineari limitati.

DEFINIZIONE 2.4.3 (Tipo 2 -- Grammatiche libere dal contesto (context-free)). Le produzioni hanno la forma $X ::= \beta$ dove $X \in N$ è un singolo non terminale e $\beta \in S^*$. La riscrittura di X non dipende dal contesto in cui X appare.

I linguaggi generati sono detti **liberi dal contesto** (context-free) e sono riconosciuti dagli automi a pila.

Nota (Importanza delle grammatiche libere dal contesto). La maggior parte dei linguaggi di programmazione ha una **sintassi** descrivibile con grammatiche libere dal contesto (tipo 2). Questo è il tipo di grammatica su cui ci concentreremo nel corso.

DEFINIZIONE 2.4.4 (Tipo 3 -- Grammatiche regolari). Le produzioni hanno una delle seguenti forme:

- $X ::= aY$ oppure $X ::= a$ (**regolari a destra**), oppure
- $X ::= Ya$ oppure $X ::= a$ (**regolari a sinistra**)

dove $X, Y \in N$ e $a \in T$. I linguaggi generati sono detti **regolari** e sono riconosciuti dagli automi a stati finiti. Possono anche essere descritti da **espressioni regolari**.

2.5. Backus-Naur Form (BNF)

La **Backus-Naur Form** (BNF) è una notazione compatta per scrivere grammatiche libere dal contesto. Le convenzioni sono:

- I non terminali sono racchiusi tra parentesi angolari $\langle \dots \rangle$ oppure scritti con iniziale maiuscola.
- Il simbolo $::=$ separa la parte sinistra dalla parte destra.
- Il simbolo $|$ separa le alternative: tutte le produzioni con lo stesso non terminale a sinistra vengono raggruppate in una sola riga.

ESEMPIO 2.5.1 (Grammatica per indicazioni stradali in BNF). $\langle \text{Direzione} \rangle ::= \text{sinistra} \mid \text{destra}$

$\langle \text{Consiglio} \rangle ::= \text{svolta a } \langle \text{Direzione} \rangle \mid \text{prosegui dritto}$

$\langle \text{Percorso} \rangle ::= \langle \text{Consiglio} \rangle \mid \langle \text{Consiglio} \rangle, \text{ poi } \langle \text{Percorso} \rangle$

Questa grammatica genera frasi come: «svolta a sinistra, poi prosegui dritto».

2.5.1. Espressioni aritmetiche in BNF

ESEMPIO 2.5.2 (Grammatica per le espressioni aritmetiche). Dato l'alfabeto dei terminali $T = \{0, 1, 2, \dots, 9, +, \times\}$, definiamo la grammatica $G = (T, N, P)$ con $N = \{\text{Cifra}, \text{Num}, \text{Op}, \text{Esp}\}$ e le produzioni:

$\text{Cifra} ::= 0 \mid 1 \mid 2 \mid \dots \mid 9$

$\text{Num} ::= \text{Cifra} \mid \text{Num Cifra}$

$\text{Op} ::= + \mid \times$

$\text{Esp} ::= \text{Num} \mid \text{Esp Op Esp}$

Ogni produzione definisce come costruire espressioni valide:

- **Cifra** genera una singola cifra decimale.
- **Num** genera un numero naturale come sequenza di cifre (definizione ricorsiva).
- **Op** genera un operatore aritmetico.
- **Esp** genera un'espressione: un numero oppure due espressioni collegate da un operatore.

Nota. Il non terminale Esp è la categoria sintattica principale: il linguaggio delle espressioni aritmetiche è $L(\text{Esp})$. Vedremo nel prossimo paragrafo come formalizzare il concetto di «linguaggio generato da un non terminale».

2.6. Derivazioni e Linguaggi Generati

Il meccanismo fondamentale delle grammatiche formali è la **derivazione**: si parte da un non terminale e, applicando ripetutamente le produzioni, si ottengono stringhe di terminali. In questa sezione formalizziamo questo processo.

2.6.1. Derivazione immediata

DEFINIZIONE 2.6.1 (Derivazione immediata). Data una grammatica $G = (T, N, P)$ con $S = T \cup N$, e date due stringhe $\beta, \beta' \in S^*$, si dice che β' è un **derivato immediato** di β , e si scrive:

$$\beta \rightarrow \beta' \quad (5)$$

se e solo se esistono stringhe $\beta_1, \beta_2 \in S^*$, un non terminale $X \in N$ e una produzione $X ::= \alpha \in P$ tali che:

$$\beta = \beta_1 X \beta_2 \quad \text{e} \quad \beta' = \beta_1 \alpha \beta_2 \quad (6)$$

In altre parole, β' si ottiene da β sostituendo un'occorrenza del non terminale X con la parte destra α di una sua produzione, lasciando invariato il contesto β_1 e β_2 .

ESEMPIO 2.6.2 (Derivazione immediata). Data la grammatica con le produzioni:

$\text{Esp} ::= \text{Num} \mid \text{Esp Op Esp}$

$\text{Op} ::= + \mid \times$

$\text{Num} ::= 0 \mid 1 \mid \dots \mid 9$

Partiamo dalla stringa $\beta = \text{Esp} + \text{Esp Op } 2$. Applicando la produzione $\text{Esp} ::= \text{Num}$ all'occorrenza più a sinistra di Esp , otteniamo:

$$\text{Esp} + \text{Esp Op } 2 \rightarrow \text{Num} + \text{Esp Op } 2 \quad (7)$$

Qui $\beta_1 = \varepsilon$, $X = \text{Esp}$, $\alpha = \text{Num}$, $\beta_2 = + \text{Esp Op } 2$.

Nota. Se una stringa contiene più non terminali, ad ogni passo possiamo scegliere **quale** non terminale espandere e **quale** produzione applicare. Questa libertà di scelta porta, in generale, a derivazioni diverse per la stessa stringa.

2.6.2. Derivazione (in zero o più passi)

DEFINIZIONE 2.6.3 (Derivazione). Data una grammatica $G = (T, N, P)$, e date due stringhe $\beta, \beta' \in S^*$, si dice che β' è un **derivato** di β , e si scrive:

$$\beta \xrightarrow{*} \beta' \quad (8)$$

se e solo se esiste una sequenza finita (eventualmente vuota) di derivazioni immediate:

$$\beta = \beta_0 \rightarrow \beta_1 \rightarrow \beta_2 \rightarrow \dots \rightarrow \beta_n = \beta' \quad (n \geq 0) \quad (9)$$

Il numero n è detto **lunghezza della derivazione**.

- Se $n = 0$, allora $\beta = \beta'$ (ogni stringa deriva da se stessa: **riflessività**).
- Se $\beta \xrightarrow{*} \gamma$ e $\gamma \xrightarrow{*} \beta'$, allora $\beta \xrightarrow{*} \beta'$ (**transitività**).

ESEMPIO 2.6.4 (Derivazione passo per passo). Con la grammatica delle espressioni, deriviamo la stringa $3 + 5$:

$$\begin{aligned} & \text{Esp} \\ \rightarrow & \text{Esp Op Esp} \\ \rightarrow & \text{Num Op Esp} \\ \rightarrow & 3 \text{ Op Esp} \\ \rightarrow & 3 + \text{Esp} \\ \rightarrow & 3 + \text{Num} \\ \rightarrow & 3 + 5 \end{aligned} \quad (10)$$

Poiché $\text{Esp} \xrightarrow{*} 3 + 5$ e $3 + 5 \in T^*$, la stringa $3 + 5$ appartiene al linguaggio generato da Esp.

2.7. Linguaggio generato da un non terminale

DEFINIZIONE 2.7.1 (Linguaggio generato). Data una grammatica $G = (T, N, P)$ e un non terminale $X \in N$, il **linguaggio generato** da X , scritto $L(X)$, è l'insieme di tutte e sole le stringhe di terminali derivabili da X :

$$L(X) = \{w \in T^* \mid X \xrightarrow{*} w\} \quad (11)$$

Quando la grammatica ha un simbolo iniziale S_0 , il **linguaggio generato dalla grammatica** è $L(G) = L(S_0)$.

Nota. Si noti che $L(X) \subseteq T^*$: il linguaggio contiene solo stringhe composte esclusivamente da simboli terminali. Le stringhe intermedie che contengono ancora non terminali (dette **forme sentenziali**) non fanno parte del linguaggio.

2.7.1. Appartenenza al linguaggio

Data una grammatica $G = (T, N, P)$ e una stringa $w \in T^*$, per stabilire se $w \in L(X)$ si ragiona nel modo seguente:

- $w \in L(X)$ se e solo se partendo dal simbolo X è possibile, applicando una o più produzioni, ottenere la stringa w composta da soli terminali.
- $w \notin L(X)$ se e solo se **qualunque** sequenza di applicazioni di produzioni a partire da X non è in grado di generare w .

OSSERVAZIONE 2.7.2. Dimostrare che $w \in L(X)$ è relativamente semplice: basta esibire una derivazione $X \xrightarrow{*} w$. Dimostrare che $w \notin L(X)$ è più difficile, perché richiede di escludere **tutte** le possibili derivazioni.

2.7.2. Esempio completo: numeri naturali senza zeri iniziali

ESEMPIO 2.7.3 (Grammatica per numeri naturali senza zeri iniziali). Consideriamo la grammatica dell'esempio precedente per le espressioni aritmetiche. La produzione $\text{Num} ::= \text{Cifra} \mid \text{Num Cifra}$ permette di generare stringhe come 007, che non è una rappresentazione canonica di un numero naturale.

Per ottenere una grammatica che non generi zeri iniziali, modifichiamo le produzioni:

NonZero ::= 1 | 2 | 3 | ... | 9
 Cifra ::= 0 | NonZero
 Pos ::= NonZero | Pos Cifra
 Num ::= 0 | Pos

In questa grammatica:

- **NonZero** genera una cifra diversa da zero.
- **Cifra** genera una cifra qualsiasi (0–9).
- **Pos** genera un numero positivo: la prima cifra è necessariamente diversa da zero (NonZero), mentre le cifre successive possono essere qualsiasi (Cifra).
- **Num** genera lo zero oppure un numero positivo.

Verifichiamo: $L(\text{Num})$ contiene 0, 5, 42, 100, ma **non** contiene 007 né 00.

2.8. Derivazioni canoniche

Abbiamo visto che, quando una forma sentenziale contiene più non terminali, si può scegliere quale espandere per primo. Le **derivazioni canoniche** eliminano questa libertà di scelta fissando una strategia deterministica.

2.8.1. Derivazione canonica sinistra

DEFINIZIONE 2.8.1 (Derivazione canonica sinistra (leftmost derivation)). Una derivazione $\beta_0 \rightarrow \beta_1 \rightarrow \dots \rightarrow \beta_n$ è **canonica sinistra** se, ad ogni passo $\beta_i \rightarrow \beta_{i+1}$, il non terminale sostituito è sempre quello **più a sinistra** nella stringa β_i .

2.8.2. Derivazione canonica destra

DEFINIZIONE 2.8.2 (Derivazione canonica destra (rightmost derivation)). Una derivazione $\beta_0 \rightarrow \beta_1 \rightarrow \dots \rightarrow \beta_n$ è **canonica destra** se, ad ogni passo $\beta_i \rightarrow \beta_{i+1}$, il non terminale sostituito è sempre quello **più a destra** nella stringa β_i .

ESEMPIO 2.8.3 (Derivazioni canoniche a confronto). Data la grammatica:

$\text{Esp} ::= \text{Num} \mid \text{Esp} + \text{Esp}$

$\text{Num} ::= 0 \mid 1 \mid 2 \mid \dots \mid 9$

Deriviamo la stringa $3 + 5 + 2$.

Derivazione canonica sinistra (ad ogni passo si espande il non terminale più a sinistra):

$$\begin{aligned}
 & \text{Esp} \\
 \rightarrow & \text{Esp} + \text{Esp} \\
 \rightarrow & \text{Esp} + \text{Esp} + \text{Esp} \\
 \rightarrow & \text{Num} + \text{Esp} + \text{Esp} \\
 \rightarrow & 3 + \text{Esp} + \text{Esp} \\
 \rightarrow & 3 + \text{Num} + \text{Esp} \\
 \rightarrow & 3 + 5 + \text{Esp} \\
 \rightarrow & 3 + 5 + \text{Num} \\
 \rightarrow & 3 + 5 + 2
 \end{aligned}
 \tag{12}$$

Derivazione canonica destra (ad ogni passo si espande il non terminale più a destra):

$$\begin{aligned}
 & \text{Esp} \\
 \rightarrow & \text{Esp} + \text{Esp} \\
 \rightarrow & \text{Esp} + \text{Num} \\
 \rightarrow & \text{Esp} + 2 \\
 \rightarrow & \text{Esp} + \text{Esp} + 2 \\
 \rightarrow & \text{Esp} + \text{Num} + 2 \\
 \rightarrow & \text{Esp} + 5 + 2 \\
 \rightarrow & \text{Num} + 5 + 2 \\
 \rightarrow & 3 + 5 + 2
 \end{aligned}
 \tag{13}$$

Nota. Entrambe le derivazioni producono la stessa stringa $3 + 5 + 2$, ma con ordini di sostituzione diversi. La derivazione canonica sinistra e quella destra rappresentano due strategie sistematiche per enumerare le derivazioni.

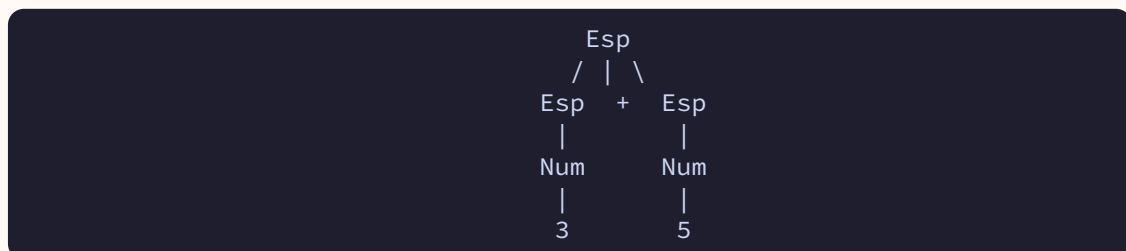
2.9. Alberi di derivazione

Derivazioni diverse (per esempio la canonica sinistra e la canonica destra) possono corrispondere alla stessa «struttura» della derivazione. L'**albero di derivazione** (o **parse tree**) cattura esattamente questa struttura, astruendo dall'ordine in cui le produzioni vengono applicate.

DEFINIZIONE 2.9.1 (Albero di derivazione (parse tree)). Data una grammatica libera dal contesto $G = (T, N, P)$ e un non terminale $X \in N$, un **albero di derivazione** per una stringa $w \in L(X)$ è un albero ordinato con le seguenti proprietà:

1. La **radice** è etichettata con X .
2. Ogni **nodo interno** è etichettato con un non terminale $Y \in N$.
3. Le **foglie** sono etichettate con simboli terminali $a \in T$ oppure con ε .
4. Se un nodo interno è etichettato con Y e i suoi figli (da sinistra a destra) sono etichettati con $X_1, X_2, \dots, X_k$, allora $Y ::= X_1 X_2 \dots X_k$ è una produzione di P .
5. La stringa ottenuta leggendo le foglie da sinistra a destra è w (detta **frontiera** dell'albero).

ESEMPIO 2.9.2 (Albero di derivazione per la stringa 3 + 5). Con la grammatica $\text{Esp} ::= \text{Num} \mid \text{Esp} + \text{Esp}$ e $\text{Num} ::= 0 \mid \dots \mid 9$, l'albero per 3 + 5 è:



Lettura dell'albero:

- La radice Esp si espande con la produzione $\text{Esp} ::= \text{Esp} + \text{Esp}$.
- Il figlio sinistro Esp si espande con $\text{Esp} ::= \text{Num}$, poi $\text{Num} ::= 3$.
- Il figlio destro Esp si espande con $\text{Esp} ::= \text{Num}$, poi $\text{Num} ::= 5$.
- La frontiera (foglie da sinistra a destra) è 3 + 5.

OSSERVAZIONE 2.9.3. Derivazioni canoniche diverse (sinistra e destra) possono produrre lo **stesso albero** di derivazione. L'albero cattura la **struttura** della derivazione, non l'**ordine** delle sostituzioni. In particolare, per una grammatica libera dal contesto, esiste una corrispondenza biunivoca tra alberi di derivazione e derivazioni canoniche sinistre (e analogamente con le derivazioni canoniche destre).

2.9.1. Valutazione basata sull'albero

Quando l'albero di derivazione rappresenta un'espressione, la sua struttura determina l'ordine di valutazione:

- Si valutano prima i **sottoalberi** (ricorsivamente, dalle foglie verso la radice).
- Poi si applica l'operatore del nodo corrente ai risultati dei sottoalberi.

ESEMPIO 2.9.4 (Valutazione dell'espressione $3 + 5$). Dall'albero dell'esempio precedente:

1. Valuta il sottoalbero sinistro: $\text{Num} \rightarrow 3$, valore = 3.
2. Valuta il sottoalbero destro: $\text{Num} \rightarrow 5$, valore = 5.
3. Applica l'operatore $+$: $3 + 5 = 8$.

2.10. Ambiguità

DEFINIZIONE 2.10.1 (Grammatica ambigua). Una grammatica G è **ambigua** se esiste almeno una stringa $w \in L(G)$ che ammette **due o più alberi di derivazione distinti**. Equivalentemente, G è ambigua se esiste una stringa che ammette due derivazioni canoniche sinistre distinte (o due derivazioni canoniche destre distinte).

L'ambiguità è un problema grave perché alberi di derivazione diversi possono assegnare **significati diversi** alla stessa stringa.

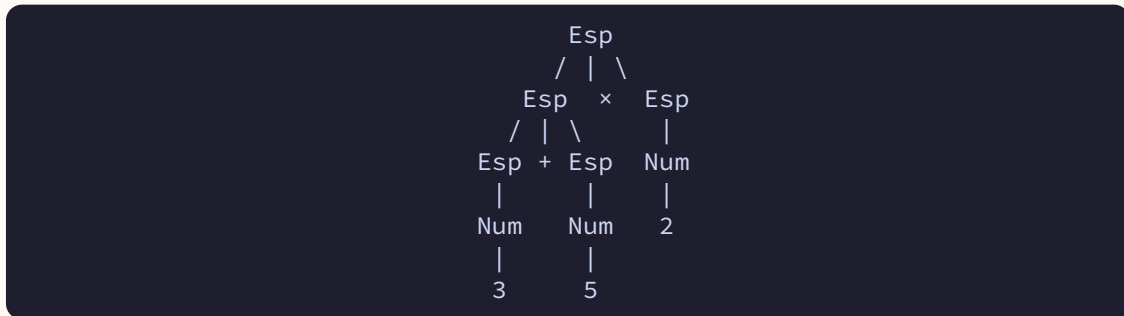
ESEMPIO 2.10.2 (Ambiguità nella stringa $3 + 5 \times 2$). Consideriamo la grammatica:

$\text{Esp} ::= \text{Num} \mid \text{Esp} + \text{Esp} \mid \text{Esp} \times \text{Esp}$

$\text{Num} ::= 0 \mid 1 \mid \dots \mid 9$

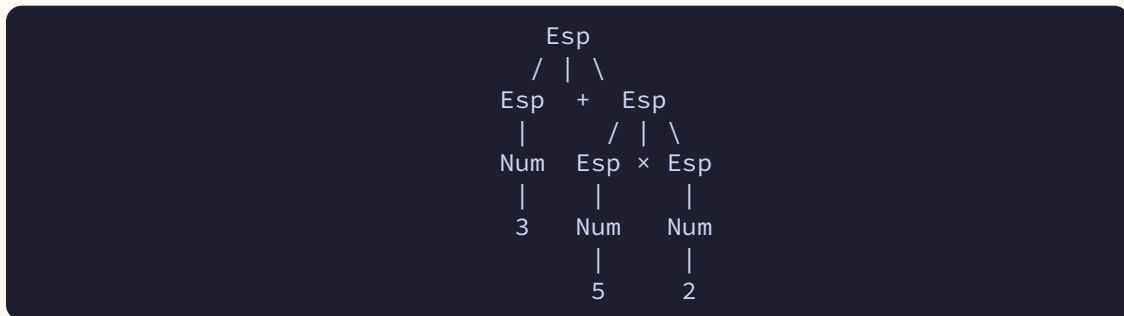
La stringa $3 + 5 \times 2$ ammette due alberi di derivazione distinti.

Albero 1 – interpreta come $(3 + 5) \times 2$:



Valore: $(3 + 5) \times 2 = 16$

Albero 2 – interpreta come $3 + (5 \times 2)$:



Valore: $3 + (5 \times 2) = 13$

La stessa stringa ha due valutazioni diverse: 16 oppure 13, a seconda dell'albero scelto. Per un linguaggio di programmazione, questa situazione è inaccettabile.

2.11. Risoluzione dell'ambiguità

Per eliminare l'ambiguità da una grammatica si possono adottare diverse strategie.

2.11.1. Introduzione di livelli di precedenza

Si ristruttura la grammatica introducendo non terminali aggiuntivi che codificano la **precedenza** e l'**associatività** degli operatori. L'idea è che gli operatori a precedenza più alta vengano «catturati» più in profondità nell'albero.

ESEMPIO 2.11.1 (Grammatica non ambigua con precedenza). $\text{Esp} ::= \text{Term} \mid \text{Esp} + \text{Term}$
 $\text{Term} ::= \text{Factor} \mid \text{Term} \times \text{Factor}$
 $\text{Factor} ::= \text{Num} \mid (\text{Esp})$
 $\text{Num} ::= 0 \mid 1 \mid \dots \mid 9$

In questa grammatica:

- **Esp** gestisce l'addizione: un'espressione è un termine, oppure un'espressione seguita da $+$ e un termine. L'addizione è **associativa a sinistra**.
- **Term** gestisce la moltiplicazione: un termine è un fattore, oppure un termine seguito da $\times$ e un fattore. La moltiplicazione è **associativa a sinistra** e ha **precedenza maggiore** rispetto all'addizione.
- **Factor** gestisce le «unità atomiche»: un numero oppure un'espressione racchiusa tra parentesi.

Con questa grammatica, la stringa $3 + 5 \times 2$ ha un **unico** albero di derivazione, che corrisponde all'interpretazione $3 + (5 \times 2) = 13$, rispettando la precedenza usuale della moltiplicazione sull'addizione.

2.11.2. Uso delle parentesi

Un altro metodo per risolvere l'ambiguità consiste nell'introdurre le **parentesi** nella grammatica per forzare esplicitamente l'ordine di valutazione.

ESEMPIO 2.11.2 (Grammatica con parentesi obbligatorie). $\text{Esp} ::= \text{Num} \mid (\text{Esp Op Esp})$
 $\text{Op} ::= + \mid \times$
 $\text{Num} ::= 0 \mid 1 \mid \dots \mid 9$

Con questa grammatica, ogni operazione binaria deve essere racchiusa tra parentesi, quindi non c'è ambiguità: $((3 + 5) \times 2)$ e $(3 + (5 \times 2))$ sono stringhe diverse.

Nota (Linguaggi inerentemente ambigui). Non sempre è possibile eliminare l'ambiguità modificando la grammatica. Esistono **linguaggi inerentemente ambigui**: linguaggi per i quali **ogni** grammatica che li genera è ambigua. Un esempio classico è il linguaggio $L = \{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$. Tuttavia, per i linguaggi di programmazione questo problema tipicamente non si pone.

CAPITOLO 3

Semantica Operazionale

Le grammatiche formali, introdotte nel capitolo precedente, possono essere lette in due modi complementari. Questa doppia lettura è alla base della semantica formale dei linguaggi di programmazione.

DEFINIZIONE 3.0.1 (Lettura generativa e lettura induttiva). Data una produzione come $S ::= ab \mid aSb$, possiamo interpretarla in due modi:

- **Lettura generativa (produttiva):** la grammatica è vista come un insieme di regole di riscrittura. La produzione si legge da sinistra verso destra: ogni volta che incontro il simbolo S , posso rimpiazzarlo con ab oppure con aSb . Questa lettura descrive come *generare* stringhe.
- **Lettura induttiva (costruttiva):** la grammatica è una definizione induttiva. La produzione si legge da destra verso sinistra: la clausola $S ::= ab$ afferma che $ab \in L(S)$ (caso base), mentre $S ::= aSb$ afferma che se $w \in L(S)$ allora anche $awb \in L(S)$ (passo induttivo). Questa lettura descrive come *costruire* l'insieme delle stringhe valide.

La lettura induttiva è particolarmente importante perché ci permette di utilizzare le **regole di inferenza** per definire la semantica di un linguaggio di programmazione in modo rigoroso.

3.1. Regole di inferenza

Per definire la semantica del linguaggio MAO utilizzeremo le regole di inferenza, uno strumento formale che permette di derivare nuovi giudizi (conclusioni) a partire da giudizi già noti (premesse).

DEFINIZIONE 3.1.1 (Regola di inferenza). Una regola di inferenza ha la seguente forma: date premesse $p_1, \dots, p_n$ e una conclusione q , scriviamo:

$$\frac{p_1 \dots p_n}{q} \quad (\text{Nome-Regola})$$

La regola si legge: «se tutte le premesse $p_1, \dots, p_n$ sono vere, allora si può trarre la conclusione q ».

Quando le premesse sono vuote, la regola si chiama assioma: una verità che vale senza bisogno di giustificazione.

DEFINIZIONE 3.1.2 (Assioma). Un assioma è una regola di inferenza senza premesse:

$$\frac{}{q} \quad (\text{Nome-Assioma})$$

L'assioma afferma che q è vero incondizionatamente.

3.1.1. Produzioni come regole di inferenza

Le produzioni di una grammatica possono essere viste come regole di inferenza. Consideriamo una grammatica per le espressioni aritmetiche con le produzioni:

$$\text{Exp} ::= \text{Exp} + \text{Pro} \mid \text{Pro} \quad \text{Pro} ::= \text{Pro} \times \text{Cifra} \mid \text{Cifra} \quad \text{Cifra} ::= 0 \mid 1 \mid \dots \mid 9$$

Ogni produzione corrisponde a una regola di inferenza letta induttivamente:

$$\begin{array}{ll} \frac{y \in L(\text{Exp}) \quad z \in L(\text{Pro})}{y + z \in L(\text{Exp})} & (\text{Exp-Sum}) \qquad \frac{z \in L(\text{Pro})}{z \in L(\text{Exp})} \quad (\text{Exp-Pro}) \\ \frac{z \in L(\text{Pro}) \quad c \in L(\text{Cifra})}{z \times c \in L(\text{Pro})} & (\text{Pro-Mul}) \qquad \frac{c \in L(\text{Cifra})}{c \in L(\text{Pro})} \quad (\text{Pro-Cifra}) \\ \frac{}{5 \in L(\text{Cifra})} & (\text{Cifra-5}) \end{array}$$

Ad esempio, la regola (Exp-Sum) si legge: «se y è un'espressione e z è un prodotto, allora $y + z$ è un'espressione». La regola (Cifra-5) è un assioma: 5 è una cifra senza bisogno di ulteriori giustificazioni.

3.2. Sistemi logici

DEFINIZIONE 3.2.1 (Sistema logico). Un **sistema logico** è un insieme di regole di inferenza (e assiomi) che possono essere applicate per dimostrare la validità di formule, dette *giudizi*. Fissato un sistema logico, diciamo che un giudizio q è **derivabile** se esiste una sequenza di applicazioni di regole che lo dimostra.

ESEMPIO 3.2.2 (Derivazione in un sistema logico). Consideriamo il sistema logico associato alla grammatica delle espressioni aritmetiche mostrata sopra. Vogliamo mostrare che 2×3 è un'espressione valida, cioè che $2 \times 3 \in L(\text{Exp})$.

Per farlo, costruiamo una derivazione applicando le regole dal basso verso l'alto:

1. $2 \in L(\text{Cifra})$ per l'assioma (Cifra-2), e quindi $2 \in L(\text{Pro})$ per (Pro-Cifra)
2. $3 \in L(\text{Cifra})$ per l'assioma (Cifra-3)
3. Da $2 \in L(\text{Pro})$ e $3 \in L(\text{Cifra})$, otteniamo $2 \times 3 \in L(\text{Pro})$ per la regola (Pro-Mul)
4. Da $2 \times 3 \in L(\text{Pro})$, otteniamo $2 \times 3 \in L(\text{Exp})$ per la regola (Exp-Pro)

3.2.1. Derivazione

DEFINIZIONE 3.2.3 (Derivazione). Una **derivazione** nel sistema logico è una sequenza di passaggi che, partendo dagli assiomi e applicando le regole di inferenza, giustifica una certa conclusione. Si scrive $d \vdash q$ e si legge «la derivazione d dimostra il giudizio q », oppure « d è una derivazione per q ».

Le derivazioni sono definite ricorsivamente:

- **Caso base:** ogni assioma del sistema logico è una derivazione per la sua conclusione q .
- **Passo induttivo:** se esiste una regola

$$\frac{p_1 \dots p_n}{q}$$

del sistema logico, e le premesse sono derivabili ($d_1 \vdash p_1, \dots, d_n \vdash p_n$), allora la composizione delle sotto-derivazioni è una derivazione per q .

ESEMPIO 3.2.4 (Derivazione per 2×3 in $L(\text{"Exp"})$). Rappresentiamo la derivazione come albero di regole applicate (le foglie sono assiomi):

$$\frac{\frac{2 \in L(\text{Cifra})}{2 \in L(\text{Pro})} \quad \frac{3 \in L(\text{Cifra})}{2 \times 3 \in L(\text{Pro})}}{2 \times 3 \in L(\text{Pro})}$$

Da questa derivazione, applicando (Exp-Pro), concludiamo $2 \times 3 \in L(\text{Exp})$.

3.2.2. Alberi di derivazione

Le derivazioni formano naturalmente una **struttura ad albero**, anche se in questo caso la radice (la conclusione finale) è posta verso il basso, e le foglie (gli assiomi) sono in alto. Ogni nodo interno corrisponde all'applicazione di una regola, e i suoi figli sono le sotto-derivazioni delle premesse.

Nota. Non confondere gli alberi di derivazione della semantica (che crescono verso il basso con la conclusione in fondo) con gli alberi di derivazione delle grammatiche (capitolo 2), che hanno la radice in alto.

3.2.3. Grammatiche come sistemi logici

Possiamo formalizzare il legame tra grammatiche e sistemi logici. Invece di usare rappresentazioni grafiche colorate, introduciamo le formule $x \in L(X)$, con il significato che «la stringa x appartiene al linguaggio generato dal non-terminale X ».

Ad ogni produzione della grammatica della forma $X ::= \omega_0 X_1 \omega_1 X_2 \omega_2 \dots X_n \omega_n$ (dove $\omega_i \in T^*$ sono stringhe di terminali e $X_i \in N$ sono non-terminali) associamo una regola di inferenza:

$$\frac{x_1 \in L(X_1) \quad x_2 \in L(X_2) \quad \dots \quad x_n \in L(X_n)}{\omega_0 x_1 \omega_1 x_2 \omega_2 \dots x_n \omega_n \in L(X)}$$

3.2.4. Valutazione di espressioni

I sistemi logici permettono non solo di stabilire se una stringa appartiene a un linguaggio, ma anche di assegnare un **significato** (valore) alle espressioni, seguendone la struttura grammaticale.

ESEMPIO 3.2.5 (Grammatica ambigua delle espressioni). Riprendiamo la grammatica ambigua delle espressioni con sole cifre:

$$E ::= 0 \mid 1 \mid 2 \mid \dots \mid 9 \mid E + E \mid E \times E$$

Per ogni espressione w definiamo un predicato di valutazione $w \Downarrow v$, che si legge «il termine w ha valore v ».

Definiamo le regole di valutazione come segue. Per le cifre abbiamo assiomi (una per ciascuna cifra da 0 a 9):

$$0 \Downarrow 0 \quad \text{---} \quad (\text{Val-0}) \quad 1 \Downarrow 1 \quad \text{---} \quad (\text{Val-1}) \quad \dots \quad 9 \Downarrow 9 \quad \text{---} \quad (\text{Val-9})$$

Per le operazioni abbiamo regole con premesse:

$$\frac{x_1 \Downarrow n_1 \quad x_2 \Downarrow n_2 \quad n = n_1 + n_2}{x_1 + x_2 \Downarrow n} \quad (\text{Val-Sum}) \quad \frac{x_1 \Downarrow n_1 \quad x_2 \Downarrow n_2 \quad n = n_1 \times n_2}{x_1 \times x_2 \Downarrow n} \quad (\text{Val-Mul})$$

ESEMPIO 3.2.6 (Derivazione della valutazione di $1 + (3 \times 2)$). Vogliamo derivare $1 + (3 \times 2) \Downarrow 7$. Costruiamo l'albero di derivazione:

$$\begin{array}{c}
 \begin{array}{ccc}
 & \overline{} & \overline{} \\
 & 3 \Downarrow 3 & 2 \Downarrow 2 \\
 \overline{} & & \overline{} \\
 1 \Downarrow 1 & & 6 = 3 \times 2
 \end{array} \\
 \hline
 1 + (3 \times 2) \Downarrow 7
 \end{array}$$

La derivazione procede come segue:

1. Per l'assioma (Val-1): $1 \Downarrow 1$
2. Per l'assioma (Val-3): $3 \Downarrow 3$ e per (Val-2): $2 \Downarrow 2$
3. Per la regola (Val-Mul): poiché $3 \Downarrow 3$ e $2 \Downarrow 2$ e $6 = 3 \times 2$, concludiamo $3 \times 2 \Downarrow 6$
4. Per la regola (Val-Sum): poiché $1 \Downarrow 1$ e $3 \times 2 \Downarrow 6$ e $7 = 1 + 6$, concludiamo $1 + (3 \times 2) \Downarrow 7$

3.3. Induzione

DEFINIZIONE 3.3.1. L'**induzione** è un principio fondamentale che permette di trattare insiemi infiniti di oggetti attraverso un numero finito di regole o casi.

Il principio di induzione ha tre aspetti fondamentali:

- **Costruzione:** permette di costruire un insieme infinito di oggetti mediante un numero finito di regole (ad esempio, l'insieme dei numeri naturali si definisce con due regole: 0 è un naturale, e se n è un naturale allora $n + 1$ è un naturale).
- **Definizione di funzioni:** permette di definire il comportamento di una funzione su un insieme infinito, descrivendo solo un numero finito di casi.
- **Dimostrazione:** permette di dimostrare che una proprietà vale per tutti gli elementi di un insieme infinito, esaminando un numero finito di casi.

3.3.1. Induzione matematica

L'induzione matematica permette di dimostrare che una proprietà $P(n)$ vale per tutti i numeri naturali $n \geq n_0$.

DEFINIZIONE 3.3.2 (Principio di induzione matematica). Per dimostrare $P(n)$ per ogni $n \geq n_0$:

- **Caso base:** dimostrare che $P(n_0)$ vale.
- **Passo induttivo:** preso un generico $n \geq n_0$, assumendo che $P(n)$ sia vera (**ipotesi induttiva**), dimostrare che $P(n + 1)$ è vera.

ESEMPIO 3.3.3 (Somma dei primi n naturali positivi). Dimostriamo che per ogni naturale positivo n :

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \quad (14)$$

Caso base ($n = 1$): $1 = \frac{1 \cdot 2}{2} = 1$. Verificato.

Passo induttivo: assumiamo che la formula valga per n (ipotesi induttiva):

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \quad (15)$$

Dimostriamo che vale per $n + 1$:

$$1 + 2 + \dots + n + (n + 1) = \frac{n(n+1)}{2} + (n + 1) = \frac{n(n+1) + 2(n+1)}{2} = \frac{(n+1)(n+2)}{2} \quad (16)$$

L'ultimo passaggio si ottiene raccogliendo $(n + 1)$ a fattor comune. La formula è quindi dimostrata per $n + 1$.

3.3.2. Induzione strutturale

L'induzione strutturale estende il principio di induzione matematica alle stringhe di un linguaggio generato da una grammatica. Invece di indurre sui numeri naturali, si induce sulla struttura delle stringhe.

DEFINIZIONE 3.3.4 (Principio di induzione strutturale). Per dimostrare una proprietà $P(w)$ per tutte le stringhe $w \in L(S)$ generate da una grammatica:

- **Casi base:** dimostrare $P(\omega)$ per le stringhe $\omega \in T^*$ che compaiono nella parte destra delle produzioni *atomiche* (cioè produzioni della forma $X ::= \omega$ senza non-terminali a destra).
- **Casi induttivi:** per ogni produzione non atomica della forma $X ::= \omega_0 Y_1 \omega_1 \dots Y_n \omega_n$, assumendo che $P(s_1), \dots, P(s_n)$ valgano per le stringhe s_i generate dai non-terminali Y_i (ipotesi induttiva), dimostrare che $P(\omega_0 s_1 \omega_1 \dots s_n \omega_n)$ vale.

3.3.3. Induzione sulle derivazioni

L'induzione sulle derivazioni permette di dimostrare proprietà valide per tutti i giudizi derivabili in un sistema logico. A differenza dell'induzione strutturale (che lavora sulla struttura delle stringhe), qui si lavora sulla **struttura delle derivazioni** stesse.

DEFINIZIONE 3.3.5 (Principio di induzione sulle derivazioni). Sia $\mathcal{S}$ un sistema logico e P una proprietà sui giudizi. Per dimostrare che $P(J)$ vale per ogni giudizio J derivabile in $\mathcal{S}$:

Casi base (assiomi): Per ogni assioma

$$\frac{}{q}$$

del sistema, dimostrare che $P(q)$ vale.

Casi induttivi (regole): Per ogni regola

$$\frac{p_1 \dots p_n}{q}$$

del sistema, assumendo che $P(p_1), \dots, P(p_n)$ valgano (**ipotesi induttiva**), dimostrare che $P(q)$ vale.

Nota (Intuizione). L'induzione sulle derivazioni riflette il modo in cui i giudizi vengono costruiti: partendo dagli assiomi (verità di base) e applicando regole di inferenza. Se una proprietà vale per le «fondamenta» (assiomi) e si «propaga» attraverso le regole, allora vale per tutto ciò che è derivabile.

ESEMPIO 3.3.6 (Applicazione: correttezza della valutazione). Consideriamo il sistema logico per la valutazione di espressioni aritmetiche con le regole:

$$n \Downarrow n \quad \text{(Val-Num)} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 + e_2 \Downarrow v_1 + v_2} \quad \text{(Val-Sum)}$$

Vogliamo dimostrare: «Se $e \Downarrow v$ è derivabile, allora v è un numero intero».

Caso base (Val-Num): $n \Downarrow n$. Il valore n è un numero intero per definizione.

Caso induttivo (Val-Sum): Assumiamo (ipotesi induttiva) che v_1 e v_2 siano interi. Allora $v_1 + v_2$ è la somma di due interi, quindi è un intero.

ESEMPIO 3.3.7 (Confronto dei tre tipi di induzione).

Induzione matematica	Induzione strutturale	Induzione sulle derivazioni
Sui numeri naturali $\mathbb{N}$	Sulle stringhe di $L(G)$	Sui giudizi derivabili
Caso base: $n = 0$	Caso base: produzioni atomiche	Caso base: assiomi
Passo: $n \rightarrow n + 1$	Passo: produzioni ricorsive	Passo: regole di inferenza
Esempio: somma dei primi n	Esempio: $\#_{a(w)} = \#_{b(w)}$	Esempio: correttezza semantica

Questa tecnica sarà fondamentale per dimostrare proprietà della semantica di MAO, come ad esempio:

- **Determinismo:** se $\langle e, \rho, \sigma \rangle \Downarrow v_1$ e $\langle e, \rho, \sigma \rangle \Downarrow v_2$, allora $v_1 = v_2$
- **Type soundness:** se un'espressione è ben tipata, la sua valutazione non produce errori di tipo

Nota (Anticipazione: regole di tipo per gli operatori di confronto). Il sistema di tipi formale di MAO (presentato nel capitolo dedicato al linguaggio MAO completo) include una regola **T-Cop** per gli operatori di confronto ($<$, $\leq$, $>$, $\geq$, $==$, $\neq$). In tale regola, gli operandi sono vincolati ad avere tipo `int`. Si osservi tuttavia che, a livello semantico, gli operatori di uguaglianza `==` e disuguaglianza `!=` sono definiti anche su operandi booleani (come indicato nella tabella degli operatori di MiniMao). Questa discrepanza è una scelta progettuale comune nei linguaggi didattici: il type checker può essere reso più permissivo estendendo T-Cop con una seconda variante che ammetta operandi di tipo `bool` per `==` e `!=`. Per i dettagli completi si rimanda alla sezione sulle regole di type checking nel capitolo su MAO.

3.4. Esercizi: Dimostrazioni Induttive su Grammatiche

In questa sezione vengono presentati esercizi di dimostrazione per induzione strutturale su grammatiche context-free. Per ogni esercizio si richiede di dimostrare una proprietà valida per tutte le stringhe del linguaggio generato.

3.4.1. Esercizio 1: Bilanciamento di a e b

ESEMPIO 3.4.1 (Grammatica). Data la grammatica:

$$S ::= aSb \mid ab \quad (17)$$

Dimostrare per induzione strutturale che ogni stringa $w \in L(S)$ soddisfa $\#_a(w) = \#_b(w)$, dove $\#_a(w)$ indica il numero di occorrenze del simbolo a nella stringa w .

Dimostrazione. Procediamo per induzione strutturale sulla derivazione di w .

Caso base: $S \rightarrow ab$

La stringa generata è $w = ab$. Contiamo le occorrenze:

- $\#_a(ab) = 1$
- $\#_b(ab) = 1$

Quindi $\#_a(w) = \#_b(w) = 1$. ✓

Caso induttivo: $S \rightarrow aSb$

Assumiamo per ipotesi induttiva che la stringa w' generata da S soddisfi $\#_a(w') = \#_b(w') = k$ per qualche $k \geq 1$.

La nuova stringa è $w = aw'b$. Contiamo le occorrenze:

- $\#_a(aw'b) = 1 + \#_a(w') = 1 + k$
- $\#_b(aw'b) = \#_b(w') + 1 = k + 1$

Quindi $\#_a(w) = \#_b(w) = k + 1$. ✓

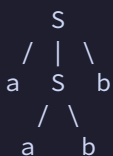
□

ESEMPIO 3.4.2 (Alberi di derivazione). Mostriamo gli alberi di derivazione per alcune stringhe del linguaggio:

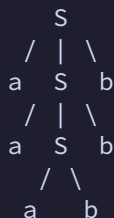
Stringa ab (caso base):



Stringa $aabb$ (una applicazione della regola ricorsiva):



Stringa $aaabbb$ (due applicazioni della regola ricorsiva):



Nota. Il linguaggio $L(S) = \{a^n b^n \mid n \geq 1\}$ è l'esempio classico di linguaggio context-free non regolare. La proprietà dimostrata ($\#_a = \#_b$) è necessaria ma non sufficiente per caratterizzare il linguaggio (ad esempio $abab$ ha lo stesso numero di a e b ma non appartiene a $L(S)$).

3.4.2. Esercizio 2: Stringhe con esattamente una c

ESEMPIO 3.4.3 (Grammatica). Data la grammatica:

$$S ::= bSa \mid aSb \mid c \quad (18)$$

Dimostrare per induzione strutturale che ogni stringa $w \in L(S)$ soddisfa:

1. $\#_c(w) = 1$ (esattamente una occorrenza di c)
2. $\#_a(w) = \#_b(w)$ (stesso numero di a e b)

Dimostrazione. Procediamo per induzione strutturale sulla derivazione di w .

Caso base: $S \rightarrow c$

La stringa generata è $w = c$. Verifichiamo:

- $\#_c(c) = 1$ ✓
- $\#_a(c) = 0 = \#_b(c)$ ✓

Caso induttivo 1: $S \rightarrow bSa$

Assumiamo per ipotesi induttiva che la stringa w' generata da S soddisfi $\#_c(w') = 1$ e $\#_a(w') = \#_b(w') = k$ per qualche $k \geq 0$.

La nuova stringa è $w = bw'a$. Verifichiamo:

- $\#_c(bw'a) = \#_c(w') = 1$ ✓
- $\#_a(bw'a) = \#_a(w') + 1 = k + 1$
- $\#_b(bw'a) = 1 + \#_b(w') = 1 + k = k + 1$

Quindi $\#_a(w) = \#_b(w) = k + 1$. ✓

Caso induttivo 2: $S \rightarrow aSb$

Assumiamo per ipotesi induttiva che la stringa w' generata da S soddisfi $\#_c(w') = 1$ e $\#_a(w') = \#_b(w') = k$ per qualche $k \geq 0$.

La nuova stringa è $w = aw'b$. Verifichiamo:

- $\#_c(aw'b) = \#_c(w') = 1$ ✓
- $\#_a(aw'b) = 1 + \#_a(w') = 1 + k$
- $\#_b(aw'b) = \#_b(w') + 1 = k + 1$

Quindi $\#_a(w) = \#_b(w) = k + 1$. ✓

□

ESEMPIO 3.4.4 (Alberi di derivazione). Mostriamo gli alberi di derivazione per alcune stringhe del linguaggio:

Stringa c (caso base):

```

S
|
c

```

Stringa bca (regola bSa):

```

  S
 / | \
b  S  a
  |
  c

```

Stringa acb (regola aSb):

```

  S
 / | \
a  S  b
  |
  c

```

Stringa $abcab$ (composizione di regole):

```

      S
     / | \
    a  S  b
     / | \
    b  S  a
     |
     c

```

Derivazione: $S \rightarrow aSb \rightarrow abSab \rightarrow abcab$

Nota. Questa grammatica genera il linguaggio delle stringhe palindrome? No! Ad esempio $abcba$ non è generabile. La grammatica genera stringhe dove c è sempre al centro, ma le a e b a sinistra e destra di c non sono necessariamente simmetriche.

3.4.3. Esercizio 3: Almeno una a

ESEMPIO 3.4.5 (Grammatica). Data la grammatica:

$$S ::= Sa \mid Sb \mid a \quad (19)$$

Dimostrare per induzione strutturale che ogni stringa $w \in L(S)$ soddisfa $\#_a(w) \geq 1$.

Dimostrazione. Procediamo per induzione strutturale sulla derivazione di w .

Caso base: $S \rightarrow a$

La stringa generata è $w = a$. Verifichiamo:

- $\#_a(a) = 1 \geq 1$ ✓

Caso induttivo 1: $S \rightarrow Sa$

Assumiamo per ipotesi induttiva che la stringa w' generata da S soddisfi $\#_a(w') \geq 1$.

La nuova stringa è $w = w'a$. Verifichiamo:

- $\#_a(w'a) = \#_a(w') + 1 \geq 1 + 1 = 2 \geq 1$ ✓

Caso induttivo 2: $S \rightarrow Sb$

Assumiamo per ipotesi induttiva che la stringa w' generata da S soddisfi $\#_a(w') \geq 1$.

La nuova stringa è $w = w'b$. Verifichiamo:

- $\#_a(w'b) = \#_a(w') \geq 1$ ✓

(Aggiungere una b non cambia il numero di a , che rimane almeno 1.)

□

ESEMPIO 3.4.6 (Alberi di derivazione). Mostriamo gli alberi di derivazione per alcune stringhe del linguaggio:

Stringa a (caso base):

```

S
|
a

```

Stringa ab :

```

  S
 / \
S   b
|
a

```

Stringa aa :

```

  S
 / \
S   a
|
a

```

Stringa $abab$:

```

      S
     / \
    S   b
   / \
  S   a
 / \
S   b
|
a

```

Derivazione: $S \rightarrow Sb \rightarrow Sab \rightarrow Sbab \rightarrow abab$

Nota. Questa grammatica genera il linguaggio $L(S) = \{a\} \cdot (a \mid b)^*$, cioè tutte le stringhe su $\{a, b\}$ che iniziano con a . La proprietà dimostrata ($\#_a \geq 1$) è una conseguenza immediata di questa caratterizzazione: ogni stringa inizia con almeno una a .

3.4.4. Osservazioni metodologiche

Nota (Schema generale per l'induzione strutturale). Per dimostrare una proprietà $P(w)$ per tutte le stringhe $w \in L(S)$:

1. **Identificare i casi base:** produzioni della forma $X ::= \omega$ dove $\omega \in T^*$ (solo terminali)
2. **Identificare i casi induttivi:** produzioni della forma $X ::= \alpha_1 Y_1 \alpha_2 Y_2 \dots Y_n \alpha_{n+1}$ dove Y_i sono non-terminali
3. **Per ogni caso base:** verificare direttamente che $P(\omega)$ vale
4. **Per ogni caso induttivo:** assumere che $P(w_i)$ valga per le stringhe w_i generate dai non-terminali Y_i (ipotesi induttiva), e dimostrare che $P(\alpha_1 w_1 \alpha_2 w_2 \dots w_n \alpha_{n+1})$ vale

3.5. MiniMao

In questo capitolo introduciamo **MiniMao**, una versione ristretta del linguaggio MAO. L'obiettivo è definire formalmente la *sintassi* (quali programmi sono corretti) e la *semantica operativa* (come i programmi vengono eseguiti) utilizzando i sistemi logici introdotti nella sezione precedente.

3.5.1. Variabili, ambiente e memoria

La programmazione consiste nell'ideare uno o più algoritmi che risolvano un problema, valutarne l'efficacia e codificarli in un linguaggio eseguibile da un calcolatore. Un programma è una sequenza di istruzioni che indicano al calcolatore le operazioni da eseguire. Come istruzione elementare consideriamo la possibilità di memorizzare il risultato di un calcolo. Per riferirsi a questi valori si utilizzano nomi simbolici detti nomi di variabile.

Nota (Equazioni vs assegnamenti). Equazioni matematiche e assegnamenti sono due concetti profondamente diversi. Consideriamo $n = n^2 - 2$:

- Se interpretato come **equazione matematica**, cerchiamo il valore di n che verifica l'uguaglianza: le soluzioni sono $n \in \{2, -1\}$.
- Se interpretato come **assegnamento**, il significato è operativo: se n attualmente vale 5, dopo l'assegnamento n assume il valore $5^2 - 2 = 23$.

Per evitare ambiguità, MAO usa simboli diversi: `=` per la dichiarazione e `:=` per l'assegnamento.

3.5.1.1. Concetto di variabile

Una variabile in MAO è modellata come una «scatola» dotata di un **nome** (l'identificatore), un **tipo** (che determina quali valori può contenere) e un **valore corrente** (il contenuto della scatola). La dichiarazione crea la variabile e le assegna un valore iniziale:

```
int eta = 15;
```

Successivamente, il valore può cambiare tramite assegnamento (si noti l'uso di `:=`):

```
eta := 16;
```

3.5.1.2. Stato del programma

Durante l'esecuzione di un programma possono esistere molte variabili, ciascuna con il proprio valore. Lo stato del programma è l'insieme di tutte le variabili e dei loro valori in un dato istante dell'esecuzione.

ESEMPIO 3.5.1. Uno stato con tre variabili:

```
{eta = 16, studente = true, nome = "Jacob"}
```

3.5.2. Sintassi di MiniMao

Introduciamo ora la sintassi di MiniMao, la versione ristretta del linguaggio MAO. Prima definiamo la sintassi usando grammatiche formali; in un secondo momento definiremo la semantica utilizzando sistemi logici.

Le **categorie sintattiche** di MiniMao sono:

DEFINIZIONE 3.5.2 (Categorie sintattiche).

- **Valori (V):** dati elementari come numeri interi o booleani
- **Identificatori (Id):** simboli per riferirsi a variabili
- **Espressioni (E):** combinano valori, identificatori e operatori per produrre nuovi valori
- **Tipi (T):** indicano l'insieme dei valori ammissibili per una variabile
- **Comandi (C):** descrivono le azioni da eseguire (modificano lo stato)

DEFINIZIONE 3.5.3 (Valori V). Inizialmente consideriamo solo programmi che manipolano valori interi ($\mathbb{Z}$) e booleani ($\mathbb{B}$):

$$V ::= n \mid \text{true} \mid \text{false} \quad (20)$$

dove n indica un qualsiasi numero intero.

DEFINIZIONE 3.5.4 (Identificatori Id). Gli identificatori sono nomi simbolici che permettono al programmatore di riferirsi in modo chiaro e univoco a variabili. In MAO sono stringhe alfanumeriche che possono contenere il carattere underscore (_).

ESEMPIO 3.5.5. `mio_id1`, `x`, `eta`, `contatore`

Nota. Alcune parole chiave del linguaggio (come `if`, `while`, `int`, `bool`, `true`, `false`, `skip`) sono riservate e non possono essere usate come identificatori.

DEFINIZIONE 3.5.6 (Espressioni E). Le espressioni combinano valori, identificatori e operatori per produrre un nuovo valore (intero o booleano):

$E ::= V \mid \text{Id} \mid E \text{ bop } E \mid \text{uop } E \mid (E)$

dove `bop` indica un operatore binario e `uop` un operatore unario.

DEFINIZIONE 3.5.7 (Operatori binari). Gli operatori binari prendono due operandi e producono un risultato:

Simbolo	Nome	Tipo operandi	Tipo risultato
+	addizione	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{Z}$
-	sottrazione	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{Z}$
×	moltiplicazione	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{Z}$
÷	divisione intera	$\mathbb{Z} \times \mathbb{Z}_{\neq 0}$	$\mathbb{Z}$
mod	resto (modulo)	$\mathbb{Z} \times \mathbb{Z}_{\neq 0}$	$\mathbb{Z}$
<	minore	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{B}$
≤	minore o uguale	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{B}$
>	maggiore	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{B}$
≥	maggiore o uguale	$\mathbb{Z} \times \mathbb{Z}$	$\mathbb{B}$
==	uguaglianza	$\mathbb{Z} \times \mathbb{Z} \text{ o } \mathbb{B} \times \mathbb{B}$	$\mathbb{B}$
≠	disuguaglianza	$\mathbb{Z} \times \mathbb{Z} \text{ o } \mathbb{B} \times \mathbb{B}$	$\mathbb{B}$
∧	and logico	$\mathbb{B} \times \mathbb{B}$	$\mathbb{B}$
∨	or logico	$\mathbb{B} \times \mathbb{B}$	$\mathbb{B}$

Tabella 6: Operatori binari in MiniMao

Nota. La divisione intera $\div$ e il modulo mod richiedono che il secondo operando sia diverso da zero; l'applicazione a zero produce un errore.

DEFINIZIONE 3.5.8 (Operatori unari). Gli operatori unari prendono un solo operando:

Simbolo	Nome	Tipo operando	Tipo risultato
$-$	negazione aritmetica	$\mathbb{Z}$	$\mathbb{Z}$
$\neg$	negazione logica	$\mathbb{B}$	$\mathbb{B}$

Tabella 7: Operatori unari in MiniMao

DEFINIZIONE 3.5.9 (Tipi T). Un tipo può essere un tipo base o un tipo composto. Per ogni tipo base assumiamo un valore di default (0 per interi e `false` per booleani):

$$T ::= T_b \mid T_c$$

dove $T_b \in \{\text{int}, \text{bool}\}$ sono tipi base.

Nota: i tipi composti T_c (ad esempio i tipi array $T[]$) verranno introdotti nel capitolo su MAO. In MiniMao si utilizzano esclusivamente i tipi base T_b .

DEFINIZIONE 3.5.10 (Comandi C). I comandi descrivono le azioni che il programma deve eseguire:

$$C ::= \text{skip}; \mid T \text{ Id} = E; \mid \text{Id} := E; \mid CC \mid \{C\} \mid \text{if}(E)\{C\} \text{ else } \{C\} \mid \text{while}(E)\{C\}$$

In dettaglio:

- `skip;` – comando vuoto (non fa nulla)
- `T Id = E;` – dichiarazione di variabile con inizializzazione
- `Id := E;` – assegnamento a variabile esistente
- `C C` – composizione sequenziale di due comandi
- `{C}` – blocco (introduce uno scope)
- `if(E){C}else{C}` – comando condizionale
- `while(E){C}` – ciclo iterativo

ESEMPIO 3.5.11 (Programma in MiniMao).

```

int x = 1;
int n = 0;
while(x <= y) {
    x := x * 2;
    n := n + 1;
}

```

Si noti l'uso di `=` per dichiarare variabili e `:=` per gli assegnamenti.

3.6. Semantica di MiniMao

La sintassi ci dice *quali* programmi sono corretti; la **semantica** ci dice *cosa fanno*. La semantica è lo strumento che ci permette di ragionare formalmente sul comportamento di un programma e quindi di studiare proprietà come correttezza, equivalenza, terminazione e divergenza.

DEFINIZIONE 3.6.1 (Semantica operativa). La **semantica operativa** descrive il comportamento di un programma in termini dei passi che un'opportuna macchina astratta compie per eseguirlo.

DEFINIZIONE 3.6.2 (Macchina astratta). Una **macchina astratta** è una macchina semplificata ideale il cui stato è dato da due componenti: l'**ambiente** ρ e la **memoria** σ .

3.6.1. Ambiente e memoria

L'**ambiente** $\rho : \mathbb{I} \hookrightarrow \mathbb{L}$ è una funzione parziale che associa identificatori a locazioni di memoria. Risponde alla domanda: «dove si trova la variabile x ?»

La **memoria** $\sigma : \mathbb{L} \hookrightarrow \mathbb{V}$ è una funzione parziale che associa locazioni a valori. Risponde alla domanda: «che valore contiene la locazione l ?»

Per leggere il valore di una variabile x servono due passi: prima si cerca la locazione $l = \rho(x)$, poi si legge il valore $v = \sigma(l)$.

ESEMPIO 3.6.3 (Ambiente e memoria). Con due variabili `eta` e `studente`:

$$\rho(\text{eta}) = l_0, \rho(\text{studente}) = l_1 \quad \sigma(l_0) = 15, \sigma(l_1) = \text{true}$$

La variabile `eta` si trova nella locazione l_0 , che contiene il valore 15; la variabile `studente` si trova in l_1 , che contiene `true`.

Nota. Una funzione è *parziale* quando potrebbe non essere definita per tutti gli argomenti del dominio. Ad esempio, $\rho(z)$ non è definita se la variabile z non è stata dichiarata.

3.6.2. Stato del programma

Uno **stato** della macchina astratta è una coppia ambiente-memoria $s = (\rho, \sigma)$.

Quando le funzioni parziali sono definite su un numero finito di argomenti, le rappresentiamo elencando tutte le associazioni argomento-valore:

$$\rho = [x \mapsto l_1, y \mapsto l_2], \quad \sigma = [l_1 \mapsto 15, l_2 \mapsto \text{true}] \quad (21)$$

La notazione $\rho[x \mapsto l]$ indica l'ambiente ottenuto da ρ aggiungendo (o sovrascrivendo) l'associazione $x \mapsto l$. Analogamente $\sigma[l \mapsto v]$ per la memoria.

3.6.3. Predicati semantici

Per definire la semantica dei programmi abbiamo bisogno di specificare formalmente come si valutano le espressioni e come si eseguono i comandi. Introduciamo due famiglie di *giudizi* (predicati semantici):

DEFINIZIONE 3.6.4 (Giudizio di valutazione delle espressioni). $\langle E, \rho, \sigma \rangle \Downarrow v$

Si legge: «l'espressione E , nello stato (ρ, σ) , ha valore v ». L'espressione viene valutata ma lo stato **non cambia** (espressioni pure).

DEFINIZIONE 3.6.5 (Giudizio di esecuzione dei comandi). $\langle C, \rho, \sigma \rangle \rightarrow \langle \rho', \sigma' \rangle$

Si legge: «l'esecuzione del comando C nello stato iniziale (ρ, σ) termina producendo lo stato finale (ρ', σ') ». Il comando può modificare sia l'ambiente (aggiungendo nuove variabili) che la memoria (cambiando valori).

3.6.4. Espressioni pure e con effetti collaterali

Le espressioni con effetti collaterali sono espressioni la cui valutazione può comportare modifiche alla memoria (come allocazione o modifica di celle). In tal caso la valutazione si esprime come $\langle E, \rho, \sigma \rangle \Downarrow \langle v, \sigma' \rangle$.

Le espressioni **pure** non alterano lo stato: la valutazione produce un valore ma lascia ambiente e memoria invariati. MiniMao ammette solo espressioni pure, il che semplifica notevolmente le regole semantiche.

3.6.5. Regole semantiche per le espressioni

Le regole seguenti definiscono formalmente come valutare ciascun tipo di espressione. Per ogni regola, le premesse (sopra la linea) specificano le condizioni necessarie, e la conclusione (sotto la linea) specifica il risultato della valutazione.

3.6.5.1. Valori letterali

I valori letterali (numeri e booleani) si valutano a se stessi. Sono assiomi perché non richiedono premesse:

$$\frac{}{\langle n, \rho, \sigma \rangle \Downarrow n} \quad (\text{Val-Int})$$

$$\frac{}{\langle \text{true}, \rho, \sigma \rangle \Downarrow \text{true}} \quad (\text{Val-True}) \qquad \frac{}{\langle \text{false}, \rho, \sigma \rangle \Downarrow \text{false}} \quad (\text{Val-False})$$

Nota. Nelle regole (Val-Int), (Val-True), (Val-False), l'ambiente ρ e la memoria σ compaiono ma non vengono utilizzati. Il valore di un letterale non dipende dallo stato.

3.6.5.2. Variabili

Per valutare una variabile si effettua una doppia ricerca: prima nell'ambiente (per trovare la locazione), poi nella memoria (per trovare il valore):

$$\frac{\rho(\text{Id}) = l \quad \sigma(l) = v}{\langle \text{Id}, \rho, \sigma \rangle \Downarrow v} \quad (\text{Val-Var})$$

La regola (Val-Var) ha due premesse: $\rho(\text{Id}) = l$ (l'identificatore è associato alla locazione l nell'ambiente) e $\sigma(l) = v$ (la locazione l contiene il valore v nella memoria).

3.6.5.3. Operatori binari e unari

$$\frac{\langle E_1, \rho, \sigma \rangle \Downarrow v_1 \quad \langle E_2, \rho, \sigma \rangle \Downarrow v_2 \quad v = v_1 \text{ bop } v_2}{\langle E_1 \text{ bop } E_2, \rho, \sigma \rangle \Downarrow v} \quad (\text{Val-Bop})$$

La regola (Val-Bop) si legge: «per valutare $E_1 \text{ bop } E_2$, si valutano prima E_1 e E_2 separatamente (ottenendo v_1 e v_2), poi si applica l'operatore `bop` ai risultati».

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow v' \quad v = \text{uop } v'}{\langle \text{uop } E, \rho, \sigma \rangle \Downarrow v} \quad (\text{Val-Uop})$$

La regola (Val-Uop) è analoga ma con un solo operando.

3.6.5.4. Espressioni parentesizzate

Nota (Parentesi nelle espressioni). La grammatica delle espressioni include la produzione (E) , che permette di racchiudere un'espressione tra parentesi tonde. Tuttavia **non è necessaria una regola semantica separata** per le espressioni parentesizzate: le parentesi servono esclusivamente a disambiguare l'ordine di valutazione a livello sintattico (ad esempio, per forzare $(a + b) * c$ invece di $a + (b * c)$). Una volta costruito l'albero sintattico, la struttura delle sotto-espressioni è completamente determinata e le parentesi non influiscono sulla valutazione. In altre parole, valutare (E) equivale esattamente a valutare E : le regole (Val-Bop) e (Val-Uop) si applicano direttamente alla struttura dell'espressione contenuta.

3.7. Regole semantiche per i comandi

I comandi, a differenza delle espressioni, **modificano lo stato**. Le regole seguenti descrivono come ciascun tipo di comando trasforma la coppia (ρ, σ) .

3.7.1. Dichiarazione, assegnamento e sequenza

Per comprendere le regole formali, descriviamo prima informalmente ciascuna operazione.

Dichiarazione ($\text{T Id} = E$;): per eseguire una dichiarazione nello stato (ρ, σ) si deve:

1. Valutare l'espressione E nello stato corrente, ottenendo il valore v .
2. Trovare una locazione di memoria l non ancora usata ($l \notin \text{dom}(\sigma)$).
3. Estendere la memoria σ con l'associazione $l \mapsto v$.
4. Estendere l'ambiente ρ con l'associazione $\text{Id} \mapsto l$.

Assegnamento ($\text{Id} := E$): per eseguire un assegnamento nello stato (ρ, σ) si deve:

1. Valutare l'espressione E nello stato corrente, ottenendo il valore v .
2. Individuare la locazione l che l'ambiente ρ associa all'identificatore Id .
3. Aggiornare la memoria σ con l'associazione $l \mapsto v$ (il vecchio valore viene sovrascritto).
4. L'ambiente rimane invariato (non si creano nuove variabili).

Composizione sequenziale ($C_1 C_2$): per eseguire due comandi in sequenza:

1. Eseguire C_1 nello stato (ρ, σ) , ottenendo lo stato (ρ_1, σ_1) .
2. Eseguire C_2 nello stato (ρ_1, σ_1) , ottenendo lo stato finale (ρ_2, σ_2) .

3.7.2. Skip

Il comando `skip`; non modifica lo stato: è un assioma (nessuna premessa).

$$\langle \text{skip};, \rho, \sigma \rangle \rightarrow \langle \rho, \sigma \rangle \quad (\text{Cmd-Skip})$$

3.7.3. Dichiarazione

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow v \quad l \notin \text{dom}(\sigma)}{\langle \text{T Id} = E; , \rho, \sigma \rangle \rightarrow \langle \rho[\text{Id} \mapsto l], \sigma[l \mapsto v] \rangle} \quad (\text{Cmd-Decl})$$

La regola (Cmd-Decl) modifica **sia** l'ambiente (aggiungendo $\text{Id} \mapsto l$) **sia** la memoria (aggiungendo $l \mapsto v$). La premessa $l \notin \text{dom}(\sigma)$ garantisce che la locazione scelta sia fresca (non già in uso).

3.7.4. Assegnamento

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow v \quad \rho(\text{Id}) = l}{\langle \text{Id} := E; , \rho, \sigma \rangle \rightarrow \langle \rho, \sigma[l \mapsto v] \rangle} \quad (\text{Cmd-Assign})$$

La regola (Cmd-Assign) modifica **solo** la memoria (aggiornando il valore nella locazione l). L'ambiente rimane invariato: ρ compare identico nello stato iniziale e finale.

3.7.5. Sequenza

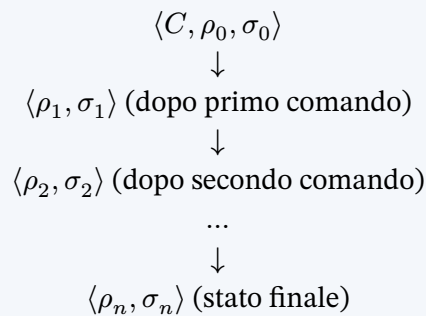
$$\frac{\langle C_1, \rho, \sigma \rangle \rightarrow \langle \rho_1, \sigma_1 \rangle \quad \langle C_2, \rho_1, \sigma_1 \rangle \rightarrow \langle \rho_2, \sigma_2 \rangle}{\langle C_1 C_2, \rho, \sigma \rangle \rightarrow \langle \rho_2, \sigma_2 \rangle} \quad (\text{Cmd-Seq})$$

La regola (Cmd-Seq) mostra chiaramente il «passaggio di stato»: lo stato prodotto da C_1 diventa lo stato iniziale di C_2 .

3.7.6. Sviluppo sequenziale

Le derivazioni ad albero possono diventare complesse e difficili da leggere per programmi con molti comandi. Lo **sviluppo sequenziale** è una notazione alternativa più compatta che descrive l'esecuzione come una sequenza di stati, uno per ogni comando eseguito.

DEFINIZIONE 3.7.1 (Sviluppo sequenziale). Lo sviluppo sequenziale rappresenta l'esecuzione di un programma come una sequenza verticale che alterna stati e comandi:



Notazione semplificata: invece di scrivere la configurazione completa con ambiente e memoria separati, spesso «fondiamo» le due componenti e scriviamo lo stato come un insieme di associazioni variabile $\mapsto$ valore:

$$\{x \mapsto v_1, y \mapsto v_2, \dots\} \quad (22)$$

Questa notazione nasconde le locazioni di memoria e si legge: «la variabile x ha valore v_1 , la variabile y ha valore v_2 , ...».

ESEMPIO 3.7.2 (Derivazione vs sviluppo sequenziale). Consideriamo $C1 = y := x;$ e $C2 = x := y;$ con stato iniziale $\rho = [x \mapsto l_1, y \mapsto l_2]$, $\sigma = [l_1 \mapsto 5, l_2 \mapsto 3]$.

Come derivazione (albero): l'albero di derivazione ha struttura annidata e richiede di leggere dal basso verso l'alto.

$$\frac{\frac{\frac{\langle x, \rho, \sigma \rangle \Downarrow 5}{\langle y := x; \rho, \sigma \rangle \rightarrow \langle \rho, \sigma[l_2 \mapsto 5] \rangle} \quad \frac{\langle y, \rho, \sigma' \rangle \Downarrow 5}{\langle x := y; \rho, \sigma' \rangle \rightarrow \langle \rho, \sigma'[l_1 \mapsto 5] \rangle}}{\langle y := x; x := y; \rho, \sigma \rangle \rightarrow \langle \rho, \sigma'' \rangle}$$

Come sviluppo sequenziale (lineare): molto più leggibile.

$$\begin{aligned} & \{x \mapsto 5, y \mapsto 3\} \\ & \downarrow y := x; \\ & \{x \mapsto 5, y \mapsto 5\} \\ & \downarrow x := y; \\ & \{x \mapsto 5, y \mapsto 5\} \end{aligned}$$

Si noti che il programma **non scambia** i valori di x e y : dopo $y := x;$, entrambe le variabili valgono 5, quindi $x := y;$ assegna 5 a x (che già vale 5).

ESEMPIO 3.7.3 (Sviluppo sequenziale completo). Consideriamo il programma:

```
int x = 3;
int y = 0;
y := x + 1;
x := y * 2;
```

Stato iniziale: $\rho_0 = \emptyset, \sigma_0 = \emptyset$

Passo 1: `int x = 3;` (regola Cmd-Decl)

- Valuto l'espressione: $\langle 3, \rho_0, \sigma_0 \rangle \Downarrow 3$
- Alloco una nuova locazione: $l_1 \notin \text{dom}(\sigma_0)$
- Estendo ambiente: $\rho_1 = \rho_0[x \mapsto l_1] = [x \mapsto l_1]$
- Estendo memoria: $\sigma_1 = \sigma_0[l_1 \mapsto 3] = [l_1 \mapsto 3]$

Passo 2: `int y = 0;` (regola Cmd-Decl)

- Valuto: $\langle 0, \rho_1, \sigma_1 \rangle \Downarrow 0$
- Alloco: $l_2 \notin \text{dom}(\sigma_1)$
- $\rho_2 = \rho_1[y \mapsto l_2] = [x \mapsto l_1, y \mapsto l_2]$
- $\sigma_2 = \sigma_1[l_2 \mapsto 0] = [l_1 \mapsto 3, l_2 \mapsto 0]$

Passo 3: `y := x + 1;` (regola Cmd-Assign)

- Valuto $x + 1$: $\langle x + 1, \rho_2, \sigma_2 \rangle \Downarrow 3 + 1 = 4$
- Trovo la locazione di y : $\rho_2(y) = l_2$
- Aggiorno memoria: $\sigma_3 = \sigma_2[l_2 \mapsto 4] = [l_1 \mapsto 3, l_2 \mapsto 4]$
- L'ambiente resta: $\rho_3 = \rho_2$

Passo 4: `x := y * 2;` (regola Cmd-Assign)

- Valuto $y * 2$: $\langle y * 2, \rho_3, \sigma_3 \rangle \Downarrow 4 * 2 = 8$
- Trovo la locazione di x : $\rho_3(x) = l_1$
- Aggiorno memoria: $\sigma_4 = \sigma_3[l_1 \mapsto 8] = [l_1 \mapsto 8, l_2 \mapsto 4]$

Stato finale: $\rho_4 = [x \mapsto l_1, y \mapsto l_2], \sigma_4 = [l_1 \mapsto 8, l_2 \mapsto 4]$

Quindi $x = 8$ e $y = 4$.

3.7.7. Blocchi e scoping

3.7.7.1. Il blocco $\{C\}$

Un **blocco** è un comando racchiuso tra parentesi graffe $\{C\}$. Tutte le variabili dichiarate all'interno del blocco sono visibili solo al suo interno. Lo scope (ambito di visibilità) di una variabile definisce la porzione di codice nella quale la variabile può essere dichiarata, letta o modificata.

In MAO una variabile dichiarata all'interno di un blocco è visibile:

- In tutti i comandi successivi alla dichiarazione, nello stesso blocco.
- All'interno dei blocchi annidati.

All'uscita dal blocco, le variabili dichiarate al suo interno cessano di essere visibili.

3.7.7.2. Shadowing

È possibile dichiarare una variabile all'interno di un blocco con lo stesso nome di una variabile già dichiarata in un blocco esterno. In questo caso la variabile interna **nasconde** (shadow) quella esterna: all'interno del blocco, il nome si riferisce alla nuova variabile; all'uscita dal blocco, il nome torna a riferirsi alla variabile originale.

3.7.8. Blocco

$$\frac{\langle C, \rho, \sigma \rangle \rightarrow \langle \rho', \sigma' \rangle}{\langle \{C\}, \rho, \sigma \rangle \rightarrow \langle \rho, \sigma' \rangle} \quad (\text{Cmd-Block})$$

Nota (Semantica del blocco). La regola (Cmd-Block) è cruciale: lo stato finale del blocco ha l'**ambiente originale** ρ (non ρ'), ma la **memoria modificata** σ' . Questo significa che:

- Le variabili dichiarate nel blocco vengono «dimenticate» (l'ambiente torna quello di prima).
- Le modifiche alla memoria persistono (se il blocco ha modificato il valore di una variabile esterna, la modifica rimane).
- Le locazioni allocate nel blocco restano in memoria ma diventano inaccessibili (nessun nome punta più ad esse).

3.7.9. Comando condizionale (if-then-else)

I comandi condizionali servono per prendere decisioni durante l'esecuzione. L'espressione E (detta **guardia**) viene valutata: se produce `true`, si esegue il ramo «then» (C_1); se produce `false`, si esegue il ramo «else» (C_2).

```
if (E) {C1} else {C2}
```

ESEMPIO 3.7.4 (Comando condizionale in MAO).

```
if (piove) {
    ombrello := true;
} else {
    ombrello := false;
}
```

Le regole semantiche distinguono i due casi:

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow \text{true} \quad \langle C_1, \rho, \sigma \rangle \rightarrow \langle \rho', \sigma' \rangle}{\langle \text{if}(E)\{C_1\}\text{else}\{C_2\}, \rho, \sigma \rangle \rightarrow \langle \rho, \sigma' \rangle} \quad (\text{Cmd-IfTrue})$$

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow \text{false} \quad \langle C_2, \rho, \sigma \rangle \rightarrow \langle \rho', \sigma' \rangle}{\langle \text{if}(E)\{C_1\}\text{else}\{C_2\}, \rho, \sigma \rangle \rightarrow \langle \rho, \sigma' \rangle} \quad (\text{Cmd-IfFalse})$$

3.7.10. Comando if-then (senza else)

Il comando `if(E){C}` senza ramo `else` si esprime in MAO come un `if-else` con `skip;` nel ramo falso:

```
if (E) {C1} else {skip;}
```

ESEMPIO 3.7.5 (Comando condizionale senza else).

```
if (piove) {
    ombrello := true;
}
```

Equivale a: `if (piove) { ombrello := true; } else { skip; }`

Nota. Le parentesi tonde e graffe svolgono due compiti distinti: le parentesi tonde `()` delimitano la guardia e fissano l'ordine di valutazione nelle espressioni; le parentesi graffe `{ }` delimitano un blocco di comandi e introducono uno scope.

ESEMPIO 3.7.6 (Sviluppo sequenziale con condizionale). Consideriamo:

```
int a = 5;
int b = 3;
int max = 0;
if(a > b){
    max := a;
} else {
    max := b;
}
```

Dopo le dichiarazioni: $\rho = [a \mapsto l_1, b \mapsto l_2, \max \mapsto l_3]$, $\sigma = [l_1 \mapsto 5, l_2 \mapsto 3, l_3 \mapsto 0]$

Valutazione della guardia: $\langle a > b, \rho, \sigma \rangle \Downarrow \langle 5 > 3 \rangle \Downarrow \text{true}$

Poiché la guardia è vera, si applica (Cmd-IfTrue) e si esegue il ramo `then` :

Esecuzione `max := a;` :

- Valuto a : $\sigma(\rho(a)) = \sigma(l_1) = 5$
- Aggiorno: $\sigma' = \sigma[l_3 \mapsto 5] = [l_1 \mapsto 5, l_2 \mapsto 3, l_3 \mapsto 5]$

Stato finale: $\max = 5$, che è corretto poiché $\max(5, 3) = 5$.

3.7.11. Cicli

3.7.11.1. Comando iterativo (*while*)

Il ciclo `while` ripete l'esecuzione di un corpo di comandi finché la guardia è vera. È l'unico costrutto iterativo presente in MAO.

```
while (E) {C}
```

L'espressione E è una guardia booleana: se vale `true`, viene eseguito il corpo C e poi si ripete il controllo della guardia; se vale `false`, il ciclo termina e l'esecuzione prosegue con il comando successivo.

ESEMPIO 3.7.7 (Comando iterativo in MAO).

```
while(cioccolatini > 0) {
  cioccolatini := cioccolatini - 1;
}
```

3.7.11.2. Semantica del *while*

Il `while` richiede due regole: una per il caso in cui la guardia è falsa (il ciclo termina) e una per il caso in cui è vera (il ciclo prosegue).

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow \text{false}}{\langle \text{while}(E)\{C\}, \rho, \sigma \rangle \rightarrow \langle \rho, \sigma \rangle} \quad (\text{Cmd-WhileFalse})$$

Se la guardia è falsa, il ciclo termina immediatamente: lo stato non cambia.

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow \text{true} \quad \langle C, \rho, \sigma \rangle \rightarrow \langle \rho', \sigma' \rangle \quad \langle \text{while}(E)\{C\}, \rho, \sigma' \rangle \rightarrow \langle \rho, \sigma'' \rangle}{\langle \text{while}(E)\{C\}, \rho, \sigma \rangle \rightarrow \langle \rho, \sigma'' \rangle} \quad (\text{Cmd-WhileTrue})$$

Nota (Ricorsività della regola *WhileTrue*). La regola (Cmd-WhileTrue) è **ricorsiva**: tra le premesse compare di nuovo il giudizio `while(E){C}`, ma con la memoria aggiornata σ' e l'ambiente originale ρ (le parentesi graffe ripristinano lo scope, come in Cmd-Block). Il ciclo viene quindi «srotolato» un passo alla volta: si esegue il corpo una volta, e poi si riesegue l'intero `while` sullo stato risultante.

3.8. Riepilogo ed esempi

Comando	Regole
<code>skip;</code>	(Cmd-Skip)
<code>T Id = E;</code>	(Cmd-Decl)
<code>Id := E;</code>	(Cmd-Assign)
<code>C1 C2</code>	(Cmd-Seq)
<code>{C}</code>	(Cmd-Block)
<code>if(E){C1}else{C2}</code>	(Cmd-IfTrue), (Cmd-IfFalse)
<code>while(E){C}</code>	(Cmd-WhileFalse), (Cmd-WhileTrue)

Tabella 8: Riepilogo delle regole semantiche di MiniMao

3.8.1. Esempi di sviluppo sequenziale

3.8.1.1. Ciclo while: somma dei primi n

ESEMPIO 3.8.1 (Sviluppo sequenziale con ciclo while). Consideriamo:

```
int n = 3;
int s = 0;
while(n > 0){
    s := s + n;
    n := n - 1;
}
```

Stato iniziale dopo le dichiarazioni: $\rho = [n \mapsto l_1, s \mapsto l_2]$, $\sigma_0 = [l_1 \mapsto 3, l_2 \mapsto 0]$

Iterazione 1 (regola Cmd-WhileTrue):

- Guardia: $\langle n > 0, \rho, \sigma_0 \rangle \Downarrow 3 > 0 = \text{true}$
- `s := s + n;` : $s = 0 + 3 = 3$ — $\sigma_1 = [l_1 \mapsto 3, l_2 \mapsto 3]$
- `n := n - 1;` : $n = 3 - 1 = 2$ — $\sigma_{1'} = [l_1 \mapsto 2, l_2 \mapsto 3]$

Iterazione 2 (regola Cmd-WhileTrue):

- Guardia: $2 > 0 = \text{true}$
- `s := s + n;` : $s = 3 + 2 = 5$ — $\sigma_2 = [l_1 \mapsto 2, l_2 \mapsto 5]$
- `n := n - 1;` : $n = 2 - 1 = 1$ — $\sigma_{2'} = [l_1 \mapsto 1, l_2 \mapsto 5]$

Iterazione 3 (regola Cmd-WhileTrue):

- Guardia: $1 > 0 = \text{true}$
- `s := s + n;` : $s = 5 + 1 = 6$ — $\sigma_3 = [l_1 \mapsto 1, l_2 \mapsto 6]$
- `n := n - 1;` : $n = 1 - 1 = 0$ — $\sigma_{3'} = [l_1 \mapsto 0, l_2 \mapsto 6]$

Uscita dal ciclo (regola Cmd-WhileFalse):

- Guardia: $0 > 0 = \text{false}$ — il ciclo termina

Stato finale: $n = 0, s = 6$ (somma dei primi 3 naturali: $3 + 2 + 1 = 6$).

3.8.1.2. Ciclo `while`: calcolo del fattoriale

ESEMPIO 3.8.2 (Sviluppo sequenziale: calcolo del fattoriale). Consideriamo il programma che calcola il fattoriale di 4:

```
int n = 4;
int f = 1;
while (n > 0) {
  f := f * n;
  n := n - 1;
}
```

Stato iniziale dopo le dichiarazioni:

- $\rho = [n \mapsto l_n, f \mapsto l_f]$
- $\sigma_0 = [l_n \mapsto 4, l_f \mapsto 1]$

Iterazione 1:

- Guardia: $\langle n > 0, \rho, \sigma_0 \rangle \Downarrow 4 > 0 = \text{true}$
- $f := f * n; : \langle f * n, \rho, \sigma_0 \rangle \Downarrow 1 \times 4 = 4$
 - $\sigma_{0'} = \sigma_0[l_f \mapsto 4] = [l_n \mapsto 4, l_f \mapsto 4]$
- $n := n - 1; : \langle n - 1, \rho, \sigma_{0'} \rangle \Downarrow 4 - 1 = 3$
 - $\sigma_1 = \sigma_{0'}[l_n \mapsto 3] = [l_n \mapsto 3, l_f \mapsto 4]$

Iterazione 2:

- Guardia: $\langle n > 0, \rho, \sigma_1 \rangle \Downarrow 3 > 0 = \text{true}$
- $f := f * n; : \langle f * n, \rho, \sigma_1 \rangle \Downarrow 4 \times 3 = 12$
 - $\sigma_{1'} = \sigma_1[l_f \mapsto 12] = [l_n \mapsto 3, l_f \mapsto 12]$
- $n := n - 1; : \langle n - 1, \rho, \sigma_{1'} \rangle \Downarrow 3 - 1 = 2$
 - $\sigma_2 = \sigma_{1'}[l_n \mapsto 2] = [l_n \mapsto 2, l_f \mapsto 12]$

Iterazione 3:

- Guardia: $\langle n > 0, \rho, \sigma_2 \rangle \Downarrow 2 > 0 = \text{true}$
- $f := f * n; : \langle f * n, \rho, \sigma_2 \rangle \Downarrow 12 \times 2 = 24$
 - $\sigma_{2'} = \sigma_2[l_f \mapsto 24] = [l_n \mapsto 2, l_f \mapsto 24]$
- $n := n - 1; : \langle n - 1, \rho, \sigma_{2'} \rangle \Downarrow 2 - 1 = 1$
 - $\sigma_3 = \sigma_{2'}[l_n \mapsto 1] = [l_n \mapsto 1, l_f \mapsto 24]$

Iterazione 4:

- Guardia: $\langle n > 0, \rho, \sigma_3 \rangle \Downarrow 1 > 0 = \text{true}$
- $f := f * n; : \langle f * n, \rho, \sigma_3 \rangle \Downarrow 24 \times 1 = 24$
 - $\sigma_{3'} = \sigma_3[l_f \mapsto 24] = [l_n \mapsto 1, l_f \mapsto 24]$
- $n := n - 1; : \langle n - 1, \rho, \sigma_{3'} \rangle \Downarrow 1 - 1 = 0$
 - $\sigma_4 = \sigma_{3'}[l_n \mapsto 0] = [l_n \mapsto 0, l_f \mapsto 24]$

Uscita dal ciclo:

- Guardia: $\langle n > 0, \rho, \sigma_4 \rangle \Downarrow 0 > 0 = \text{false}$ — il ciclo termina

Riepilogo delle iterazioni:

Iterazione	n	f	Guardia
Inizio	4	1	-
1	3	4	true
2	2	12	true
3	1	24	true
4	0	24	true
Fine	0	24	false

Tabella 2. Evoluzione dello stato nel calcolo di 4!

3.8.1.3. Blocchi e shadowing

ESEMPIO 3.8.3 (Sviluppo sequenziale: blocchi e shadowing). Consideriamo il programma che illustra lo shadowing delle variabili:

```
int x = 10;
{
  int x = 5;
  x := x + 1;
}
x := x * 2;
```

Stato iniziale: $\rho_0 = \emptyset, \sigma_0 = \emptyset$

Passo 1: `int x = 10;` (dichiarazione esterna)

- Valuto: $\langle 10, \rho_0, \sigma_0 \rangle \Downarrow 10$
- Alloco nuova locazione: $l_1 \notin \text{dom}(\sigma_0)$
- $\rho_1 = [x \mapsto l_1]$
- $\sigma_1 = [l_1 \mapsto 10]$

Passo 2: Ingresso nel blocco `{ ... }`

- L'ambiente corrente è: $\rho_1 = [x \mapsto l_1]$
- La memoria corrente è: $\sigma_1 = [l_1 \mapsto 10]$

Passo 2a: `int x = 5;` (dichiarazione interna — shadowing)

- Valuto: $\langle 5, \rho_1, \sigma_1 \rangle \Downarrow 5$
- Alloco **nuova** locazione: $l_2 \notin \text{dom}(\sigma_1)$
- $\rho_2 = \rho_1[x \mapsto l_2] = [x \mapsto l_2]$ (la nuova x **nasconde** quella esterna)
- $\sigma_2 = \sigma_1[l_2 \mapsto 5] = [l_1 \mapsto 10, l_2 \mapsto 5]$

Nota. Ora esistono **due** locazioni: l_1 (la x esterna, valore 10) e l_2 (la x interna, valore 5). L'ambiente ρ_2 «vede» solo l_2 perché il binding $x \mapsto l_2$ ha sostituito $x \mapsto l_1$.

Passo 2b: `x := x + 1;` (assegnamento alla x interna)

- Valuto $x + 1$: $\langle x + 1, \rho_2, \sigma_2 \rangle$
 - $\rho_2(x) = l_2, \sigma_2(l_2) = 5$
 - $5 + 1 = 6$
- Aggiorno: $\sigma_3 = \sigma_2[l_2 \mapsto 6] = [l_1 \mapsto 10, l_2 \mapsto 6]$

Passo 3: Uscita dal blocco (regola Cmd-Block)

- L'ambiente torna a $\rho_1 = [x \mapsto l_1]$
- La memoria **rimane** $\sigma_3 = [l_1 \mapsto 10, l_2 \mapsto 6]$

Nota. Dopo l'uscita dal blocco, la variabile x si riferisce di nuovo a l_1 (che contiene ancora 10). La locazione l_2 esiste ancora in memoria ma non è più accessibile tramite nessun nome.

Passo 4: `x := x * 2;` (assegnamento alla x esterna)

- Valuto $x * 2$: $\langle x * 2, \rho_1, \sigma_3 \rangle$
 - $\rho_1(x) = l_1, \sigma_3(l_1) = 10$
 - $10 * 2 = 20$
- Aggiorno: $\sigma_4 = \sigma_3[l_1 \mapsto 20] = [l_1 \mapsto 20, l_2 \mapsto 6]$

Riepilogo dell'evoluzione degli ambienti:

- $\rho_{\text{fin}} = [x \mapsto l_1]$

Punto	Ambiente ρ	x punta a
Dopo <code>int x = 10;</code>	$[x \mapsto l_1]$	l_1 (valore 10)
Dentro blocco, dopo <code>int x = 5;</code>	$[x \mapsto l_2]$	l_2 (valore 5)

3.8.1.4. Condizionali: calcolo del massimo

ESEMPIO 3.8.4 (Sviluppo sequenziale: condizionali annidati (calcolo del massimo)).
Consideriamo il programma che trova il massimo tra tre numeri:

```
int a = 7;
int b = 3;
int c = 5;
int max = a;
if (b > max) { max := b; }
if (c > max) { max := c; }
```

Stato iniziale: $\rho_0 = \emptyset, \sigma_0 = \emptyset$

Passo 1–4: Dichiarazioni

- `int a = 7;` : alloco l_a , $\rho_1 = [a \mapsto l_a]$, $\sigma_1 = [l_a \mapsto 7]$
- `int b = 3;` : alloco l_b , $\rho_2 = [a \mapsto l_a, b \mapsto l_b]$, $\sigma_2 = [l_a \mapsto 7, l_b \mapsto 3]$
- `int c = 5;` : alloco l_c , $\rho_3 = [a \mapsto l_a, b \mapsto l_b, c \mapsto l_c]$, $\sigma_3 = [l_a \mapsto 7, l_b \mapsto 3, l_c \mapsto 5]$
- `int max = a;` : valuto $a (= 7)$, alloco l_m
 - $\rho_4 = [a \mapsto l_a, b \mapsto l_b, c \mapsto l_c, \text{max} \mapsto l_m]$
 - $\sigma_4 = [l_a \mapsto 7, l_b \mapsto 3, l_c \mapsto 5, l_m \mapsto 7]$

Stato dopo le dichiarazioni:

$$\{a \mapsto 7, b \mapsto 3, c \mapsto 5, \text{max} \mapsto 7\}$$

Passo 5: Primo condizionale `if (b > max) { max := b; }`

- Valuto la guardia: $\langle b > \text{max}, \rho_4, \sigma_4 \rangle$
 - $\sigma_4(\rho_4(b)) = \sigma_4(l_b) = 3$
 - $\sigma_4(\rho_4(\text{max})) = \sigma_4(l_m) = 7$
 - $3 > 7 = \text{false}$
- Poiché la guardia è falsa, applichiamo (Cmd-IfFalse) con il ramo else implicito `skip`;
- Lo stato **non cambia**: $\sigma_5 = \sigma_4$

$$\{a \mapsto 7, b \mapsto 3, c \mapsto 5, \text{max} \mapsto 7\} \text{ (invariato)}$$

Passo 6: Secondo condizionale `if (c > max) { max := c; }`

- Valuto la guardia: $\langle c > \text{max}, \rho_4, \sigma_5 \rangle$
 - $\sigma_5(\rho_4(c)) = \sigma_5(l_c) = 5$
 - $\sigma_5(\rho_4(\text{max})) = \sigma_5(l_m) = 7$
 - $5 > 7 = \text{false}$
- Poiché la guardia è falsa, applichiamo (Cmd-IfFalse)
- Lo stato **non cambia**: $\sigma_6 = \sigma_5$

Stato finale:

$$\{a \mapsto 7, b \mapsto 3, c \mapsto 5, \text{max} \mapsto 7\}$$

Il risultato è corretto: $\text{max} = 7 = \max(7, 3, 5)$.

—

Variante: Consideriamo lo stesso programma con valori diversi: $a = 2, b = 8, c = 5$

Dopo le dichiarazioni: $\{a \mapsto 2, b \mapsto 8, c \mapsto 5, \text{max} \mapsto 2\}$

Primo co: `if (b > max) { max := b; }`
Secondo co: `if (c > max) { max := c; }`
 • Guardia: $8 > 2 = \text{true}$
 • Guardia: $5 > 8 = \text{false}$

Altra variante: $a = 2, b = 3, c = 9$

Dopo le dichiarazioni: $\{a \mapsto 2, b \mapsto 3, c \mapsto 9, \text{max} \mapsto 2\}$

3.9. Terminazione e divergenza

Con l'introduzione dei cicli, i programmi possono non terminare mai. Questa distinzione tra terminazione e divergenza è fondamentale nello studio dei linguaggi di programmazione.

DEFINIZIONE 3.9.1 (Terminazione). La **terminazione** è la proprietà di un programma che garantisce il completamento della sua esecuzione in tempo finito, producendo uno stato finale.

Condizioni **sufficienti** (ma non necessarie) per la terminazione di un ciclo `while(E){C}` sono:

- La guardia E contiene almeno una variabile.
- Il corpo C contiene almeno un assegnamento che modifica quella variabile.
- Le modifiche successive portano la guardia a diventare falsa in un numero finito di passi.

Queste condizioni sono euristiche utili per verificare la terminazione, ma non sono necessarie: ad esempio, `while(false){C}` termina immediatamente indipendentemente dal corpo, anche se la guardia non contiene variabili.

DEFINIZIONE 3.9.2 (Divergenza). La **divergenza** si verifica quando un programma (o un ciclo) non termina mai, continuando a eseguire istruzioni indefinitamente. In questo caso non si produce nessuno stato finale.

ESEMPIO 3.9.3 (Programma divergente).

```
int x = 1;
while(x > 0){
    x := x + 1;
}
```

La guardia $x > 0$ è sempre vera (poiché x cresce ad ogni iterazione), quindi il ciclo non termina mai.

Nota. Non esiste un algoritmo che, dato un programma arbitrario, sia in grado di decidere se esso termina o meno. Questo risultato fondamentale è noto come **indecidibilità del problema della terminazione** (Halting Problem, Turing 1936).

3.9.1. Costrutti iterativi alternativi

In MAO è presente solamente il costrutto iterativo `while`, ma i linguaggi di programmazione comuni offrono diversi costrutti iterativi. Tutti questi costrutti possono essere espressi usando un ciclo `while`.

3.9.1.1. Ciclo for

Il ciclo `for` è un costrutto particolarmente compatto: in una sola riga è possibile leggere l'inizializzazione, la condizione di terminazione e l'aggiornamento.

ESEMPIO 3.9.4 (for in JavaScript).

```
let somma = 0;
for (let i = 1; i < n; i = i + 1) {
    somma = somma + i;
}
```

Questo ciclo è equivalente al seguente codice MAO:

```
int somma = 0;
int i = 1;
while(i < n) {
    somma := somma + i;
    i := i + 1;
}
```

3.9.1.2. Ciclo do-while

Il ciclo `do-while` si utilizza quando il corpo deve essere eseguito almeno una volta, senza controllare preventivamente la condizione.

ESEMPIO 3.9.5 (do-while in JavaScript).

```
let eta;
do {
    eta = parseInt(prompt("Anni?"));
} while(eta < 0);
```

In MAO, il `do-while` può essere simulato eseguendo il corpo una volta prima del `while`:

```
int eta = -1;
eta := /* leggi input */;
while(eta < 0) {
    eta := /* leggi input */;
}
```

CAPITOLO 4

Sistemi di Tipi, Funzioni e Ricorsione

4.1. Il linguaggio MAO e gli array

Il linguaggio MAO estende MiniMao con costrutti fondamentali per la programmazione reale: gli **array**, le **funzioni**, la **ricorsione** e un **sistema di tipi** formale. In questo capitolo si presenta ciascuno di questi aspetti, a partire dalla loro sintassi formale fino alla semantica operativa e al type checking.

4.1.1. Array

DEFINIZIONE 4.1.1 (Array -- definizione informale). Un **array** è una struttura dati che permette di trattare in modo efficiente un insieme finito di dati omogenei, cioè tutti dello stesso tipo. Gli elementi sono memorizzati in celle contigue di memoria e sono accessibili tramite un indice posizionale intero.

In MAO, gli array si dichiarano specificando il tipo degli elementi seguito da `[]`. Ad esempio, la dichiarazione

```
int[] voti = [18, 30, 23];
```

alloca in memoria un blocco contiguo di celle. La prima cella, situata all'indirizzo base l_b , contiene la lunghezza dell'array. Le celle successive $l_b + 1, l_b + 2, \dots$ contengono i singoli elementi. Per accedere all'elemento in posizione i si usa la notazione $l_b[i]$, che corrisponde all'indirizzo $l_b + 1 + i$.

4.1.1.1. Rappresentazione in memoria

Nella struttura memoria-ambiente di MAO, un array viene rappresentato tramite una catena di indirizzazione. L'ambiente ρ associa il nome della variabile a una locazione l_x , e la memoria σ associa l_x all'indirizzo base l_b dell'array:

$$\begin{aligned} \rho(\text{voti}) &= l_x, & \sigma(l_x) &= l_b \\ \sigma(l_b) &= 3 & (\text{lunghezza}) \\ \sigma(l_b + 1) &= 18, & \sigma(l_b + 2) &= 30, & \sigma(l_b + 3) &= 23 \end{aligned}$$

Si osservi che la variabile `voti` non contiene direttamente i dati dell'array, ma un *riferimento* (locazione) all'indirizzo base. Questa scelta progettuale ha conseguenze importanti, come vedremo nella sezione sull'aliasing.

DEFINIZIONE 4.1.2 (Array -- definizione formale). Un array è una collezione finita di elementi dello stesso tipo, memorizzati in celle contigue di memoria. Il numero degli elementi è detto lunghezza dell'array. Tipo e lunghezza sono fissati al momento della dichiarazione e sono **statici**: non possono essere modificati durante l'esecuzione del programma.

Un array permette di trattare come entità atomiche intere collezioni di dati e, al contempo, consente l'accesso ai singoli elementi tramite indici posizionali. Gli indici ammissibili appartengono all'intervallo

$$[0, \text{lunghezza})$$

cioè da 0 (incluso) a «lunghezza» - 1 (incluso). Un accesso con indice fuori da questo intervallo costituisce un errore a tempo di esecuzione.

4.1.2. Sintassi degli array

Per ottenere la lunghezza di un array si utilizza il costrutto `.length`, che restituisce un valore intero.

ESEMPIO 4.1.3 (Lunghezza di un array).

```
int[] voti = [18, 30, 23];
voti.length = 3
```

La grammatica di MAO viene estesa con le seguenti produzioni per supportare gli array:

$$\begin{aligned} T &::= \dots \mid T[] \\ E &::= \dots \mid [S] \mid \text{new } T [E] \mid E.\text{length} \mid E [E] \\ S &::= E \mid E, S \\ C &::= \dots \mid E [E] := E \end{aligned}$$

Dove ciascuna produzione ha il seguente significato:

- $T[]$: tipo array di elementi di tipo T (ad esempio `int[]`, `bool[]`)
- $[S]$: **letterale array**, cioè una lista esplicita di espressioni che definisce i valori iniziali
- $\text{new } T [E]$: **allocazione** di un nuovo array di tipo T con dimensione data dalla valutazione di E , con elementi inizializzati ai valori di default
- $E.\text{length}$: espressione che restituisce la **lunghezza** dell'array
- $E [E]$: **accesso** a un singolo elemento dell'array, dove la prima espressione denota l'array e la seconda l'indice
- $E [E] := E$: **assegnamento** a un elemento dell'array

4.1.3. Valori e riferimenti

In MiniMao le celle di memoria contenevano solamente valori interi e booleani. L'ambiente e la memoria erano definiti come:

$$\rho : \mathbb{I} \hookrightarrow \mathbb{L} \quad \sigma : \mathbb{L} \hookrightarrow \mathbb{V} \text{ dove } \mathbb{V} = \mathbb{Z} \cup \mathbb{B}$$

Con l'introduzione degli array in MAO, il dominio dei valori viene esteso per includere anche le *locazioni* (indirizzi di memoria), poiché una variabile di tipo array contiene un riferimento all'indirizzo base dell'array:

$$\rho : \mathbb{I} \hookrightarrow \mathbb{L} \quad \sigma : \mathbb{L} \hookrightarrow \mathbb{V} \text{ dove } \mathbb{V} = \mathbb{Z} \cup \mathbb{B} \cup \mathbb{L}$$

Le locazioni che compaiono come valori prendono il nome di **riferimenti**. Un riferimento non è un dato del programma, ma un indirizzo di memoria che permette di accedere indirettamente a una struttura dati.

4.1.4. Espressioni pure e con effetti collaterali

DEFINIZIONE 4.1.4 (Espressione pura). Un'espressione si dice **pura** se la sua valutazione non modifica lo stato della memoria. Il risultato dipende solo dai valori correnti delle variabili, senza effetti collaterali.

In **MiniMao** tutte le espressioni erano pure: la valutazione di un'espressione E produceva un valore v lasciando la memoria σ invariata. Ciò permetteva di usare la notazione semplificata:

$$\langle E, \rho, \sigma \rangle \Downarrow v$$

ESEMPIO 4.1.5 (Espressioni pure in MiniMao). Le seguenti espressioni non modificano la memoria:

- $x + 3 \rightarrow$ legge il valore di x , calcola la somma, restituisce un intero
- $a > b \text{ and } c \rightarrow$ legge a, b, c , restituisce un booleano
- $(x + 1) * (y - 2) \rightarrow$ solo letture e calcoli aritmetici

In **MAO**, con l'introduzione degli array, alcune espressioni possono **allocare nuova memoria**. Queste espressioni producono **effetti collaterali** e restituiscono, oltre al valore, una memoria modificata σ' :

$$\langle E, \rho, \sigma \rangle \Downarrow (v, \sigma')$$

ESEMPIO 4.1.6 (Espressioni con effetti collaterali). Le seguenti espressioni modificano la memoria allocando nuove celle:

- `new int[5]` $\rightarrow$ alloca 6 celle consecutive (1 per la lunghezza + 5 per gli elementi)
- `[1, 2, 3]` $\rightarrow$ alloca 4 celle consecutive (1 per la lunghezza + 3 per i valori)
- `new bool[n+1]` $\rightarrow$ prima valuta l'espressione `n+1`, poi alloca il numero risultante di celle

Nota (Conseguenza sulla semantica). In MAO la valutazione di *qualsiasi* espressione restituisce sempre una coppia (v, σ') , anche quando $\sigma' = \sigma$ (cioè quando l'espressione è pura). Questo garantisce uniformità nella semantica: ogni regola di valutazione segue lo stesso schema, indipendentemente dalla presenza o meno di effetti collaterali.

La distinzione tra espressioni pure e non pure ha importanti implicazioni pratiche:

- **Ordine di valutazione:** se due sottoespressioni hanno effetti collaterali, l'ordine in cui vengono valutate può influenzare il risultato finale
- **Ottimizzazioni del compilatore:** il compilatore può riordinare o eliminare espressioni pure senza alterare la semantica del programma, ma non può fare altrettanto con espressioni che hanno effetti collaterali
- **Ragionamento formale:** le espressioni pure sono più semplici da analizzare perché il loro comportamento è completamente determinato dai valori correnti delle variabili

4.1.5. Semantica operativa degli array

Le regole seguenti definiscono la semantica operativa degli array in stile *big-step*. In ciascuna regola, le premesse (sopra la linea) stabiliscono le condizioni necessarie, e la conclusione (sotto la linea) descrive il risultato della valutazione.

4.1.5.1. Allocazione di un array letterale

Quando si valuta un letterale array $[E_1, \dots, E_n]$, si valutano in ordine le n espressioni, ottenendo i valori $v_1, \dots, v_n$. Si allocano poi $n + 1$ celle consecutive a partire dall'indirizzo base l_b : la prima cella memorizza la lunghezza n , le successive i valori degli elementi.

$$\frac{\langle E_1, \rho, \sigma \rangle \Downarrow (v_1, \sigma_1) \quad \dots \quad \langle E_n, \rho, \sigma_{n-1} \rangle \Downarrow (v_n, \sigma_n) \quad l_b, l_b + 1, \dots, l_b + n \notin \text{dom}(\sigma_n)}{\langle [E_1, \dots, E_n], \rho, \sigma \rangle \Downarrow (l_b, \sigma_{n[l_b \mapsto n]}[l_b + 1 \mapsto v_1] \dots [l_b + n \mapsto v_n])} \quad (\text{Array-Lit})$$

Nota. Si osservi che il valore restituito è l'indirizzo base l_b , non l'array stesso. Ciò riflette il fatto che in MAO gli array sono gestiti per riferimento. Inoltre, ogni espressione E_i viene valutata nella memoria σ_{i-1} risultante dalla valutazione precedente, garantendo la corretta propagazione degli effetti collaterali.

4.1.5.2. Allocazione con new

L'espressione `new T[E]` valuta prima E per ottenere la dimensione n , poi alloca $n + 1$ celle consecutive: la prima per la lunghezza, le restanti inizializzate al valore di default del tipo T (tipicamente 0 per `int`, `false` per `bool`).

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow (n, \sigma') \quad l_b, \dots, l_b + n \notin \text{dom}(\sigma')}{\langle \text{new } T[E], \rho, \sigma \rangle \Downarrow (l_b, \sigma'[l_b \mapsto n][l_b + 1 \mapsto \text{default}] \dots [l_b + n \mapsto \text{default}])} \quad (\text{Array-New})$$

4.1.5.3. Lunghezza

L'espressione `E.length` valuta E ottenendo l'indirizzo base l_b , quindi legge il valore memorizzato in l_b , che per costruzione è la lunghezza dell'array.

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow (l_b, \sigma') \quad \sigma'(l_b) = n}{\langle E.\text{length}, \rho, \sigma \rangle \Downarrow (n, \sigma')} \quad (\text{Array-Length})$$

4.1.5.4. Accesso a un elemento

L'espressione $E_1[E_2]$ valuta prima E_1 per ottenere l'indirizzo base l_b , poi E_2 per ottenere l'indice i . La premessa $0 \leq i < \sigma_2(l_b)$ garantisce che l'indice sia nell'intervallo valido. L'elemento cercato si trova all'indirizzo $l_b + 1 + i$ (si aggiunge 1 perché la prima cella contiene la lunghezza).

$$\frac{\langle E_1, \rho, \sigma \rangle \Downarrow (l_b, \sigma_1) \quad \langle E_2, \rho, \sigma_1 \rangle \Downarrow (i, \sigma_2) \quad 0 \leq i < \sigma_2(l_b)}{\langle E_1[E_2], \rho, \sigma \rangle \Downarrow (\sigma_2(l_b + 1 + i), \sigma_2)} \quad (\text{Array-Access})$$

4.1.5.5. Assegnamento in array

Il comando $E_1[E_2] := E_3$ valuta le tre espressioni in ordine: E_1 per l'indirizzo base, E_2 per l'indice, E_3 per il nuovo valore. La premessa $0 \leq i < \sigma_2(l_b)$ garantisce che l'indice sia nell'intervallo valido, analogamente alla regola (Array-Access). La cella all'indirizzo $l_b + 1 + i$ viene aggiornata con il valore v .

$$\frac{\langle E_1, \rho, \sigma \rangle \Downarrow (l_b, \sigma_1) \quad \langle E_2, \rho, \sigma_1 \rangle \Downarrow (i, \sigma_2) \quad \langle E_3, \rho, \sigma_2 \rangle \Downarrow (v, \sigma_3) \quad 0 \leq i < \sigma_2(l_b)}{\langle E_1[E_2] := E_3; \rho, \sigma \rangle \rightarrow \langle \rho, \sigma_3[l_b + 1 + i \mapsto v] \rangle} \quad (\text{Array-Assign})$$

4.1.6. Aliasing

DEFINIZIONE 4.1.7 (Aliasing). Si ha **aliasing** quando due o più variabili distinte fanno riferimento alla stessa zona di memoria. Poiché in MAO gli array sono gestiti tramite riferimenti (indirizzi base), l'assegnamento di un array a un'altra variabile copia il *riferimento*, non i dati. Di conseguenza, entrambe le variabili puntano allo stesso blocco di memoria.

ESEMPIO 4.1.8 (Problema dell'aliasing). Consideriamo il seguente frammento di codice:

```
int[] a = [1, 2, 3];
int[] b = a;           // b punta allo stesso array di a!
b[0] := 99;           // modifica anche a[0]!
```

Dopo l'esecuzione, lo stato è il seguente:

- $\rho = [a \mapsto l_a, b \mapsto l_b]$
- $\sigma(l_a) = \sigma(l_b) = l_{\text{base}}$ (stesso indirizzo base!)

Quindi `a[0]` e `b[0]` accedono alla stessa cella di memoria $l_{\text{base}} + 1$. La modifica effettuata tramite `b` è visibile anche tramite `a`, perché non è stata creata una copia indipendente dell'array.

Nota (Conseguenza sul passaggio di parametri). L'aliasing ha implicazioni dirette sul passaggio di parametri alle funzioni. Quando si passa un array come argomento a una funzione, si passa il suo **riferimento** (indirizzo base). Di conseguenza, eventuali modifiche agli elementi dell'array effettuate nel corpo della funzione si riflettono sull'array originale del chiamante. Questo comportamento è equivalente a un passaggio per *riferimento implicito*.

4.2. Analisi statica e sistema di tipi

In un linguaggio di programmazione, le grammatiche definiscono in modo rigoroso le categorie sintattiche delle espressioni e dei comandi. Tuttavia, molti programmi sintatticamente validi possono contenere errori che si manifestano solo a tempo di esecuzione: ad esempio, sommare un intero con un booleano, oppure accedere a un array con un indice di tipo booleano. Per prevenire questa classe di errori si ricorre all'**analisi statica**.

DEFINIZIONE 4.2.1 (Analisi statica). L'**analisi statica** consiste nei controlli effettuati sul codice sorgente *senza eseguire il programma*. Il suo scopo è individuare errori e anomalie prima dell'esecuzione, basandosi esclusivamente sulla struttura sintattica e sulle informazioni di tipo.

La maggior parte dei linguaggi di programmazione moderni prevede diversi controlli di analisi statica (ad esempio, il controllo di raggiungibilità del codice, l'analisi di variabili non inizializzate, e altri). In MAO ci si limita al controllo dei tipi (*type checking*), che verifica la coerenza tra i tipi dichiarati per le variabili e l'uso che ne viene fatto nelle espressioni e nei comandi.

4.2.1. Sistemi di tipi

In MAO il controllo dei tipi avviene in modo formale attraverso regole di tipo che stabiliscono le condizioni necessarie affinché un'espressione o un comando vengano considerati ben tipati. Il controllo è **composizionale**: le regole di tipo per un costrutto composito sono definite in funzione dei giudizi di tipo sulle sue sottocomponenti. Per poter effettuare questo controllo è necessario conoscere i tipi associati alle variabili che occorrono nelle espressioni e nei comandi; a tal fine si introduce l'*ambiente di tipo*.

4.2.2. Ambiente di tipo

DEFINIZIONE 4.2.2 (Ambiente di tipo). L'**ambiente di tipo** $\Gamma : \mathbb{I} \hookrightarrow \mathbb{T}$ è una funzione parziale che associa a ciascun identificatore nel suo dominio un tipo. La scrittura $\Gamma(x) = \text{int}$ si legge: «nell'ambiente Γ si assume che la variabile x abbia tipo `int`».

Per comodità si scrive $\text{Id} : T$ in luogo di $\Gamma(\text{Id}) = T$. Un ambiente di tipo si rappresenta come un insieme di associazioni:

$$\Gamma = \{\text{Id}_1 : T_1, \text{Id}_2 : T_2, \dots, \text{Id}_n : T_n\} \quad (23)$$

L'ambiente di tipo è un concetto esclusivamente statico: esiste solo a tempo di compilazione e serve al type checker per verificare la correttezza del programma. Non va confuso con l'ambiente ρ (che mappa identificatori a locazioni a tempo di esecuzione).

ESEMPIO 4.2.3 (Costruzione di un ambiente di tipo). Dopo le seguenti dichiarazioni:

```
int temp = 2;
bool y = true;
```

L'ambiente di tipo risultante è $\Gamma = \{\text{temp} : \text{int}, y : \text{bool}\}$.

4.2.3. Variabili libere

Le variabili libere di un'espressione o di un comando sono quelle variabili che vi occorrono senza essere state dichiarate localmente. La loro definizione formale è necessaria per garantire che il type checking possa procedere: ogni variabile libera deve essere presente nell'ambiente di tipo Γ .

4.2.3.1. Variabili libere in un'espressione

Data un'espressione $E \in \mathbb{E}$, la funzione $\text{fv}(\cdot) : \mathbb{E} \rightarrow \mathcal{P}_{\text{fin}}(\mathbb{I})$ restituisce l'insieme finito di tutte le variabili che occorrono in E , dette variabili libere. La funzione è definita per induzione sulla struttura dell'espressione:

$$\begin{aligned} \text{fv}(n) &= \emptyset \\ \text{fv}(\text{true}) &= \emptyset \\ \text{fv}(\text{false}) &= \emptyset \\ \text{fv}(\text{Id}) &= \{\text{Id}\} \\ \text{fv}(E_1 \text{ bop } E_2) &= \text{fv}(E_1) \cup \text{fv}(E_2) \\ \text{fv}(\text{uop } E) &= \text{fv}(E) \\ \text{fv}((E)) &= \text{fv}(E) \\ \text{fv}(E_1[E_2]) &= \text{fv}(E_1) \cup \text{fv}(E_2) \\ \text{fv}(E.\text{length}) &= \text{fv}(E) \\ \text{fv}(\text{new } T[E]) &= \text{fv}(E) \end{aligned}$$

Le costanti (numeriche e booleane) non contengono variabili libere. Un identificatore ha se stesso come unica variabile libera. Per le espressioni composte, le variabili libere sono l'unione delle variabili libere delle sottoespressioni.

ESEMPIO 4.2.4. $\text{fv}(\text{x}+3) = \text{fv}(\text{x}) \cup \text{fv}(3) = \{\text{x}\} \cup \emptyset = \{\text{x}\}$

ESEMPIO 4.2.5. $\text{fv}(\text{new int}[\text{a.length}]) = \text{fv}(\text{a.length}) = \text{fv}(\text{a}) = \{\text{a}\}$

ESEMPIO 4.2.6. $\text{fv}(x\%7 == z*x) = \text{fv}(x\%7) \cup \text{fv}(z*x) = \{x\} \cup \{z, x\} = \{x, z\}$

4.2.3.2. Variabili libere in un comando

Dato un comando $C \in \mathbb{C}$, la funzione $\text{fv}(\cdot) : \mathbb{C} \rightarrow \mathcal{P}_{\text{fin}}(\mathbb{I})$ restituisce l'insieme di tutte le variabili che occorrono in C senza essere state dichiarate all'interno di C stesso. La definizione per induzione richiede una funzione ausiliaria $\text{dv}(\cdot) : \mathbb{C} \rightarrow \mathcal{P}_{\text{fin}}(\mathbb{I})$, che restituisce l'insieme delle variabili *introdotte* da dichiarazioni.

$$\begin{aligned} \text{fv}(\text{skip};) &= \emptyset \\ \text{fv}(T \text{ Id} = E;) &= \text{fv}(E) \\ \text{fv}(\text{Id} := E;) &= \{\text{Id}\} \cup \text{fv}(E) \\ \text{fv}(E_1[E_2] := E_3;) &= \text{fv}(E_1) \cup \text{fv}(E_2) \cup \text{fv}(E_3) \\ \text{fv}(C_1 C_2) &= \text{fv}(C_1) \cup (\text{fv}(C_2) \setminus \text{dv}(C_1)) \\ \text{fv}(\{C\}) &= \text{fv}(C) \\ \text{fv}(\text{if}(E)\{C_1\}\text{else}\{C_2\}) &= \text{fv}(E) \cup \text{fv}(C_1) \cup \text{fv}(C_2) \\ \text{fv}(\text{while}(E)\{C\}) &= \text{fv}(E) \cup \text{fv}(C) \end{aligned}$$

Nota. Nella regola per la sequenza $C_1 C_2$, le variabili dichiarate in C_1 (cioè $\text{dv}(C_1)$) vengono sottratte dalle variabili libere di C_2 , perché le dichiarazioni di C_1 rendono disponibili quei nomi nel contesto di C_2 .

La funzione $\text{dv}(C)$ è definita come segue:

$$\begin{aligned} \text{dv}(T \text{ Id} = E;) &= \{\text{Id}\} \\ \text{dv}(C_1 C_2) &= \text{dv}(C_1) \cup \text{dv}(C_2) \\ \text{dv}(\text{altri comandi}) &= \emptyset \end{aligned}$$

4.2.4. Giudizi di tipo

I giudizi di tipo sono le asserzioni fondamentali del sistema di tipi. Ve ne sono di due forme, una per le espressioni e una per i comandi.

4.2.4.1. Giudizi di tipo per le espressioni

Dato un ambiente di tipo Γ e un'espressione E tale che $\text{fv}(E) \subseteq \text{dom}(\Gamma)$, possiamo derivare un giudizio di tipo della forma

$$\Gamma \vdash E : T$$

che si legge: «nell'ambiente Γ , l'espressione E è ben tipata e ha tipo T ». La condizione $\text{fv}(E) \subseteq \text{dom}(\Gamma)$ garantisce che tutte le variabili che compaiono in E abbiano un tipo noto.

4.2.4.2. Giudizi di tipo per i comandi

Dato un ambiente di tipo Γ e un comando C tale che $\text{fv}(C) \subseteq \text{dom}(\Gamma)$, possiamo derivare un giudizio della forma

$$\Gamma \vdash C : \Gamma'$$

che si legge: «nell'ambiente Γ , il comando C è ben tipato e produce l'ambiente locale Γ' ». L'ambiente Γ' contiene le associazioni introdotte dalle eventuali dichiarazioni presenti in C ; per i comandi che non dichiarano variabili (come l'assegnamento, il condizionale o il ciclo), si ha $\Gamma' = \emptyset$.

4.2.5. Regole di type checking per le espressioni

Le regole seguenti definiscono come derivare i giudizi di tipo per le espressioni. Ogni regola è presentata in stile *regola di inferenza*: le premesse si trovano sopra la linea orizzontale, la conclusione sotto.

4.2.5.1. Costanti

Una costante intera n ha sempre tipo `int`, indipendentemente dall'ambiente. Le costanti booleane `true` e `false` hanno tipo `bool`. Queste sono regole *assiomatiche* (senza premesse sostanziali).

$$\frac{n \in \mathbb{Z}}{\Gamma \vdash n : \text{int}} \quad (\text{T-Int})$$

$$\frac{}{\Gamma \vdash \text{true} : \text{bool}} \quad (\text{T-True}) \qquad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \quad (\text{T-False})$$

4.2.5.2. Variabili

Il tipo di un identificatore è quello assegnato dall'ambiente Γ . La premessa richiede che l'identificatore sia presente nel dominio di Γ .

$$\frac{\Gamma(\text{Id}) = T}{\Gamma \vdash \text{Id} : T} \quad (\text{T-Var})$$

4.2.5.3. Operatori aritmetici

Gli operatori aritmetici ($+$, $-$, $\times$, $\div$, mod) richiedono che entrambi gli operandi abbiano tipo `int` e producono un risultato di tipo `int`.

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 \text{ aop } E_2 : \text{int}} \quad (\text{T-Aop}) \text{ dove } \text{aop} \in \{+, -, \times, \div, \text{mod}\}$$

4.2.5.4. Operatori di confronto

Gli operatori di confronto di ordinamento ($<$, $\leq$, $>$, $\geq$) confrontano due espressioni intere e producono un risultato booleano:

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 \text{ cop } E_2 : \text{bool}} \quad (\text{T-Cop}) \text{ dove } \text{cop} \in \{<, \leq, >, \geq\}$$

Gli operatori di uguaglianza e disuguaglianza ($==$, $\neq$) accettano operandi dello stesso tipo, sia `int` che `bool`, e producono un risultato booleano. Si richiede che entrambi gli operandi abbiano lo stesso tipo $T \in \{\text{int}, \text{bool}\}$:

$$\frac{\Gamma \vdash E_1 : T \quad \Gamma \vdash E_2 : T \quad T \in \{\text{int}, \text{bool}\}}{\Gamma \vdash E_1 \text{ eop } E_2 : \text{bool}} \quad (\text{T-Eop}) \text{ dove } \text{eop} \in \{==, \neq\}$$

Nota. La distinzione tra (T-Cop) e (T-Eop) riflette il fatto che gli operatori di ordinamento hanno senso solo su valori interi (non ha significato confrontare `true < false`), mentre l'uguaglianza e la disuguaglianza sono definite anche per i booleani. Ad esempio, l'espressione `(x > 0) == (y > 0)` è ben tipata: entrambi i lati hanno tipo `bool` e l'operatore `==` accetta operandi booleani.

4.2.5.5. Operatori logici

Gli operatori logici binari ($\wedge$, $\vee$) richiedono operandi booleani e producono un booleano. L'operatore unario $\neg$ nega un'espressione booleana.

$$\frac{\Gamma \vdash E_1 : \text{bool} \quad \Gamma \vdash E_2 : \text{bool}}{\Gamma \vdash E_1 \text{ lop } E_2 : \text{bool}} \quad (\text{T-Lop}) \text{ dove } \text{lop} \in \{\wedge, \vee\}$$

$$\frac{\Gamma \vdash E : \text{bool}}{\Gamma \vdash \neg E : \text{bool}} \quad (\text{T-Not})$$

$$\frac{\Gamma \vdash E : \text{int}}{\Gamma \vdash -E : \text{int}} \quad (\text{T-Neg})$$

4.2.5.6. Regole di tipo per gli array

Le seguenti regole gestiscono il type checking dei costrutti relativi agli array.

$$\frac{\Gamma \vdash E : \text{int}}{\Gamma \vdash \text{new } T[E] : T[]} \quad (\text{T-NewArray})$$

Un letterale array $[E_1, \dots, E_n]$ è ben tipato se tutte le espressioni hanno lo stesso tipo T . Il risultato ha tipo $T[]$:

$$\frac{\Gamma \vdash E_1 : T \quad \dots \quad \Gamma \vdash E_n : T}{\Gamma \vdash [E_1, \dots, E_n] : T[]} \quad (\text{T-ArrayLit})$$

Nota. La regola (T-ArrayLit) garantisce l'omogeneità del tipo degli elementi: un letterale come `[1, 2, 3]` ha tipo `int[]`, mentre `[true, false]` ha tipo `bool[]`. Un letterale misto come `[1, true, 3]` è *mal tipato* perché non esiste un tipo T tale che tutte le espressioni abbiano tipo T .

$$\frac{\Gamma \vdash E : T[]}{\Gamma \vdash E.\text{length} : \text{int}} \quad (\text{T-Length})$$

$$\frac{\Gamma \vdash E_1 : T[] \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1[E_2] : T} \quad (\text{T-ArrayAccess})$$

4.2.6. Regole di type checking per i comandi

Le regole per i comandi stabiliscono quando un comando è ben tipato e quale ambiente locale Γ' esso produce.

4.2.6.1. Skip

Il comando `skip` è sempre ben tipato e non introduce nuove variabili.

$$\Gamma \vdash \text{skip}; : \emptyset \quad \text{(T-Skip)}$$

4.2.6.2. Dichiarazione

Una dichiarazione `T Id = E;` è ben tipata se l'espressione E ha tipo T . La dichiarazione introduce l'associazione $\text{Id} : T$ nell'ambiente locale.

$$\Gamma \vdash T \quad \text{Id} = E; : \{\text{Id} : T\} \quad \text{(T-Decl)}$$

4.2.6.3. Assegnamento

Un assegnamento `Id := E;` è ben tipato se la variabile `Id` è già presente nell'ambiente con tipo T e l'espressione E ha lo stesso tipo T . L'assegnamento non introduce nuove variabili.

$$\Gamma(\text{Id}) = T \quad \Gamma \vdash E : T \quad \Gamma \vdash \text{Id} := E; : \emptyset \quad \text{(T-Assign)}$$

4.2.6.4. Assegnamento in array

L'assegnamento a un elemento di array richiede che E_1 abbia tipo $T[]$, che E_2 abbia tipo `int` (l'indice) e che E_3 abbia tipo T (coerente con il tipo degli elementi).

$$\Gamma \vdash E_1 : T[] \quad \Gamma \vdash E_2 : \text{int} \quad \Gamma \vdash E_3 : T \quad \Gamma \vdash E_1[E_2] := E_3; : \emptyset \quad \text{(T-ArrayAssign)}$$

4.2.6.5. Sequenza

La composizione sequenziale $C_1 C_2$ verifica prima C_1 nell'ambiente Γ , ottenendo l'ambiente locale Γ_1 . Poi verifica C_2 nell'ambiente esteso $\Gamma \cup \Gamma_1$, ottenendo Γ_2 . L'ambiente locale complessivo è l'unione $\Gamma_1 \cup \Gamma_2$.

$$\Gamma \vdash C_1 : \Gamma_1 \quad \Gamma \cup \Gamma_1 \vdash C_2 : \Gamma_2 \quad \Gamma \vdash C_1 C_2 : \Gamma_1 \cup \Gamma_2 \quad \text{(T-Seq)}$$

4.2.6.6. Blocco

Un blocco `{C}` incapsula un comando: le dichiarazioni all'interno del blocco sono *locali* e non visibili all'esterno. Per questo motivo la regola restituisce l'ambiente vuoto $\emptyset$.

$$\Gamma \vdash C : \Gamma' \quad \Gamma \vdash \{C\} : \emptyset \quad \text{(T-Block)}$$

4.2.6.7. Condizionale

Il condizionale richiede che la guardia E sia di tipo `bool` e che entrambi i rami C_1 e C_2 siano ben tipati. Non introduce nuove variabili nell'ambiente esterno, perché i rami sono racchiusi in blocchi.

$$\frac{\Gamma \vdash E : \text{bool} \quad \Gamma \vdash C_1 : \Gamma_1 \quad \Gamma \vdash C_2 : \Gamma_2}{\Gamma \vdash \text{if}(E)\{C_1\}\text{else}\{C_2\} : \emptyset} \quad (\text{T-If})$$

4.2.6.8. Ciclo while

Il ciclo `while` richiede che la guardia E abbia tipo `bool` e che il corpo C sia ben tipato. Come il condizionale, non introduce nuove variabili nell'ambiente esterno.

$$\frac{\Gamma \vdash E : \text{bool} \quad \Gamma \vdash C : \Gamma'}{\Gamma \vdash \text{while}(E)\{C\} : \emptyset} \quad (\text{T-While})$$

4.2.7. Esempi di derivazioni di tipo

I seguenti esempi illustrano come le regole di type checking vengano applicate per costruire *alberi di derivazione* che dimostrano la correttezza dei tipi in un programma.

ESEMPIO 4.2.7 (Derivazione di tipo per un'espressione aritmetica). Verifichiamo che l'espressione `x + y * 2` sia ben tipata nell'ambiente $\Gamma = \{x : \text{int}, y : \text{int}\}$.

L'albero di derivazione si costruisce dal basso verso l'alto, partendo dalle foglie (assiomi) e applicando le regole fino a raggiungere il giudizio desiderato.

Prima si verifica la sottoespressione `y * 2`:

$$\frac{\frac{\Gamma(y) = \text{int} \quad \Gamma \vdash y : \text{int}}{\Gamma \vdash y : \text{int}} \quad (\text{T-Var}) \quad \frac{2 \in \mathbb{Z} \quad \Gamma \vdash 2 : \text{int}}{\Gamma \vdash 2 : \text{int}} \quad (\text{T-Int})}{\Gamma \vdash y * 2 : \text{int}} \quad (\text{T-Aop})$$

Poi si combina con `x`:

$$\frac{\Gamma(x) = \text{int} \quad \Gamma \vdash y * 2 : \text{int}}{\Gamma \vdash x + y * 2 : \text{int}} \quad (\text{T-Aop})$$

L'espressione è **ben tipata** con tipo `int`.

ESEMPIO 4.2.8 (Derivazione di tipo per una sequenza di comandi). Verifichiamo il seguente frammento:

```
int z = 0;
z := x + 1;
```

nell'ambiente iniziale $\Gamma_0 = \{x : \text{int}\}$.

Passo 1: Type checking della dichiarazione `int z = 0;`

$$\frac{\Gamma_0 \vdash 0 : \text{int}}{\Gamma_0 \vdash \text{int } z = 0; : \{z : \text{int}\}} \quad (\text{T-Decl})$$

Dopo la dichiarazione, l'ambiente viene esteso: $\Gamma_1 = \Gamma_0 \cup \{z : \text{int}\} = \{x : \text{int}, z : \text{int}\}$

Passo 2: Type checking dell'assegnamento `z := x + 1;` nell'ambiente Γ_1

Prima verifichiamo l'espressione `x + 1`:

$$\frac{\Gamma_1 \vdash x : \text{int} \quad \Gamma_1 \vdash 1 : \text{int}}{\Gamma_1 \vdash x + 1 : \text{int}} \quad (\text{T-Aop})$$

Poi l'assegnamento, verificando la coerenza dei tipi:

$$\frac{\Gamma_1(z) = \text{int} \quad \Gamma_1 \vdash x + 1 : \text{int}}{\Gamma_1 \vdash z := x + 1; : \emptyset} \quad (\text{T-Assign})$$

Il comando è **ben tipato**.

ESEMPIO 4.2.9 (Errore di tipo). Consideriamo il seguente programma:

```
int x = 5;
bool y = true;
x := x + y; // ERRORE!
```

Con $\Gamma = \{x : \text{int}, y : \text{bool}\}$, tentiamo di derivare il tipo di `x + y`.

- $\Gamma \vdash x : \text{int}$ (corretto per T-Var)
- $\Gamma \vdash y : \text{bool}$ (corretto per T-Var)
- La regola (T-Aop) richiede che **entrambi** gli operandi siano di tipo `int`:

$$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 + E_2 : \text{int}}$$

Poiché y ha tipo `bool` e non `int`, la premessa $\Gamma \vdash y : \text{int}$ non è derivabile. La derivazione **fallisce** e il compilatore segnala un errore di tipo. Il programma è **mal tipato**.

ESEMPIO 4.2.10 (Type checking di un programma con while). Verifichiamo il type checking del seguente programma completo:

```
int x = 5;
int y = 6;
while (x != y) {
  if (x < y) { x := x + 2; }
  else { y := y + 1; }
}
```

Passo 1: Partiamo dall'ambiente vuoto $\Gamma_0 = \emptyset$.

Passo 2: Type checking di `int x = 5;`

$$\frac{5 \in \mathbb{Z}}{\Gamma_0 \vdash 5 : \text{int}} \quad \Gamma_0 \vdash \text{int } x = 5; : \{x : \text{int}\} \quad (\text{T-Decl})$$

Dopo la dichiarazione: $\Gamma_1 = \Gamma_0 \cup \{x : \text{int}\} = \{x : \text{int}\}$

Passo 3: Type checking di `int y = 6;` nell'ambiente Γ_1

$$\frac{6 \in \mathbb{Z}}{\Gamma_1 \vdash 6 : \text{int}} \quad \Gamma_1 \vdash \text{int } y = 6; : \{y : \text{int}\} \quad (\text{T-Decl})$$

Dopo la dichiarazione: $\Gamma_2 = \Gamma_1 \cup \{y : \text{int}\} = \{x : \text{int}, y : \text{int}\}$

Passo 4: Verifica della guardia `x != y` nell'ambiente Γ_2

$$\frac{\Gamma_2(x) = \text{int} \quad \Gamma_2(y) = \text{int}}{\Gamma_2 \vdash x : \text{int} \quad \Gamma_2 \vdash y : \text{int}} \quad \Gamma_2 \vdash x \neq y : \text{bool} \quad (\text{T-Eop})$$

Passo 5: Verifica del ramo then `{ x := x + 2; }`

Prima l'espressione `x + 2`:

$$\frac{\Gamma_2(x) = \text{int} \quad 2 \in \mathbb{Z}}{\Gamma_2 \vdash x : \text{int} \quad \Gamma_2 \vdash 2 : \text{int}} \quad \Gamma_2 \vdash x + 2 : \text{int} \quad (\text{T-Aop})$$

Poi l'assegnamento e il blocco:

$$\frac{\Gamma_2(x) = \text{int} \quad \Gamma_2 \vdash x + 2 : \text{int}}{\Gamma_2 \vdash x := x + 2; : \emptyset} \quad (\text{T-Assign})$$

$$\frac{\Gamma_2 \vdash x := x + 2; : \emptyset}{\Gamma_2 \vdash \{x := x + 2;\} : \emptyset} \quad (\text{T-Block})$$

Passo 6: Verifica del ramo else `{ y := y + 1; }` (procedimento analogo)

$$\frac{\Gamma_2(y) = \text{int} \quad \Gamma_2 \vdash y + 1 : \text{int}}{\Gamma_2 \vdash y := y + 1; : \emptyset} \quad (\text{T-Assign})$$

$$\frac{\Gamma_2 \vdash y := y + 1; : \emptyset}{\Gamma_2 \vdash \{y := y + 1;\} : \emptyset} \quad (\text{T-Block})$$

4.2.8. Controllo di tipi e inferenza di tipo

Il sistema di tipi di MAO è un sistema a **controllo di tipi** (type checking): il programmatore dichiara esplicitamente il tipo di ogni variabile e il type checker verifica che l'uso sia coerente con le dichiarazioni. Questa non è l'unica strategia possibile.

Molti linguaggi moderni adottano approcci diversi:

- **Linguaggi senza controllo di tipi** (ad esempio JavaScript): i tipi delle variabili non vengono verificati staticamente; eventuali errori di tipo emergono solo a tempo di esecuzione
- **Linguaggi con inferenza di tipo** (ad esempio Go, Haskell, OCaml): il compilatore *deduce* automaticamente il tipo di una variabile dal contesto in cui viene usata, senza che il programmatore debba dichiararlo esplicitamente

ESEMPIO 4.2.11 (Inferenza di tipo). In un linguaggio con inferenza di tipo, la dichiarazione

```
x := 5 + 3;
```

permette al compilatore di dedurre che `x` ha tipo `int` dal fatto che `5 + 3` è un'espressione aritmetica il cui risultato è un intero.

4.3. Estensioni del linguaggio

4.3.1. Tipi base aggiuntivi: char e stringhe

4.3.1.1. Caratteri

Il tipo `char` rappresenta singoli simboli, lettere e altri caratteri alfanumerici. In MAO i caratteri possono essere codificati secondo lo standard `ASCII` o `Unicode`. I caratteri *speciali* (come il ritorno a capo o la tabulazione) vengono rappresentati tramite *sequenze di escape*, cioè combinazioni di caratteri che iniziano con il backslash.

ESEMPIO 4.3.1 (Dichiarazione di caratteri).

```
char lettera = 'R';  
char a_capo = '\n';
```

Il valore `'\n'` è la sequenza di escape che rappresenta il carattere di ritorno a capo (*newline*).

4.3.1.2. Stringhe

Le stringhe in MAO sono trattate come array di caratteri, ovvero hanno tipo `char[]`. Questa scelta semplifica la semantica: tutte le operazioni sugli array (accesso per indice, `.length`, assegnamento) si applicano direttamente alle stringhe.

ESEMPIO 4.3.2 (Stringhe come array di caratteri). La stringa "Ciao" è equivalente all'array:

```
char[] saluto = ['C', 'i', 'a', 'o'];
```

Pertanto `saluto.length` restituisce 4 e `saluto[0]` restituisce 'C'.

4.3.2. Assegnamento multiplo

Molti linguaggi di programmazione permettono di dichiarare o assegnare più variabili contemporaneamente in un unico comando. Questa funzionalità, detta **assegnamento multiplo**, consente di scrivere codice più compatto e, in alcuni casi, di realizzare operazioni altrimenti impossibili con assegnamenti singoli.

ESEMPIO 4.3.3 (Dichiarazione multipla).

```
let x, y, z = 6, 7, 42;
```

Questa singola istruzione dichiara tre variabili e assegna a ciascuna il valore corrispondente nella lista di destra.

L'assegnamento multiplo è particolarmente utile per lo **scambio di variabili** (swap), che con assegnamenti singoli richiederebbe una variabile temporanea:

Swap tramite assegnamento multiplo

```
x, y := y, x;
```

Tutte le espressioni a destra vengono valutate *prima* che qualsiasi assegnamento abbia luogo. Ciò significa che i valori originali di `x` e `y` vengono letti, e solo successivamente scritti nelle posizioni scambiate.

4.3.2.1. Sintassi dell'assegnamento multiplo

Si introducono due nuove categorie sintattiche, LHS (*Left-Hand Side*) e RHS (*Right-Hand Side*):

$$\text{LHS} ::= \text{Id} \mid \text{Id}, \text{LHS}$$

$$\text{RHS} ::= E \mid E, \text{RHS}$$

I comandi atomici vengono generalizzati per includere la forma multipla:

$$C ::= \dots \mid T \quad \text{LHS} = \text{RHS}; \mid \text{LHS} := \text{RHS};$$

4.3.2.2. Type checking per l'assegnamento multiplo

Le variabili libere di LHS e RHS sono definite come:

$$\begin{aligned} \text{fv}(\text{Id}) &= \{\text{Id}\} & \text{fv}(\text{Id}, \text{LHS}) &= \{\text{Id}\} \cup \text{fv}(\text{LHS}) \\ \text{fv}(E) &= \text{fv}(E) & \text{fv}(E, \text{RHS}) &= \text{fv}(E) \cup \text{fv}(\text{RHS}) \end{aligned}$$

La regola di tipo per l'assegnamento multiplo richiede che ogni espressione E_i abbia un tipo coerente con il tipo della variabile corrispondente Id_i . Si noti che ciascuna coppia variabile-espressione viene verificata in modo indipendente: variabili diverse possono avere tipi diversi.

$$\frac{\Gamma(\text{Id}_i) = T_i \quad \Gamma \vdash E_i : T_i \quad \text{per } i = 1..n}{\Gamma \vdash \text{Id}_1, \dots, \text{Id}_n := E_1, \dots, E_n; : \emptyset} \quad (\text{T-MultiAssign})$$

Nota. La regola (T-MultiAssign) si applica all'assegnamento di variabili semplici. Per l'assegnamento a elementi di array all'interno di un assegnamento multiplo (ad esempio `a[0], b[1] := 5, 10;`) si applicano le stesse verifiche di tipo della regola (T-ArrayAssign) per ciascuna posizione del lato sinistro che sia un accesso ad array.

4.3.2.3. Semantica operativa dell'assegnamento multiplo

La semantica della dichiarazione multipla prevede la valutazione sequenziale di tutte le espressioni, seguita dall'allocazione simultanea delle variabili:

Dichiarazione multipla:

$$\frac{\langle E_i, \rho, \sigma_{i-1} \rangle \Downarrow (v_i, \sigma_i) \quad \text{per } i = 1..n \quad l_1, \dots, l_n \notin \text{dom}(\sigma_n)}{\langle T \text{ Id}_1, \dots, \text{Id}_n = E_1, \dots, E_n; \rho, \sigma \rangle \rightarrow \langle \rho[\text{Id}_1 \mapsto l_1] \dots [\text{Id}_n \mapsto l_n], \sigma_n[l_1 \mapsto v_1] \dots [l_n \mapsto v_n] \rangle}$$

Assegnamento multiplo:

$$\frac{\langle E_i, \rho, \sigma_{i-1} \rangle \Downarrow (v_i, \sigma_i) \quad \text{per } i = 1..n \quad \rho(\text{Id}_i) = l_i \quad \text{per } i = 1..n}{\langle \text{Id}_1, \dots, \text{Id}_n := E_1, \dots, E_n; \rho, \sigma \rangle \rightarrow \langle \rho, \sigma_n[l_1 \mapsto v_1] \dots [l_n \mapsto v_n] \rangle}$$

Nota (Semantica dello swap). Nell'assegnamento multiplo `x, y := y, x`, la semantica garantisce che tutte le espressioni del lato destro vengano valutate *prima* di effettuare gli assegnamenti. Questo significa che `y` e `x` vengono letti con i loro valori originali, e solo dopo i risultati vengono scritti nelle locazioni di `x` e `y` rispettivamente. Lo swap avviene correttamente senza bisogno di variabili temporanee.

4.3.3. Direttiva return

La direttiva `return` permette a un blocco di codice (tipicamente il corpo di una funzione) di restituire un valore al chiamante. Il valore restituito è il risultato della valutazione dell'espressione che segue la parola chiave `return`.

ESEMPIO 4.3.4 (Uso della direttiva return).

```
{ count := count + 1; return count; }
```

Questo blocco incrementa la variabile `count` e restituisce il suo nuovo valore.

4.3.3.1. Sintassi

La grammatica dei comandi viene estesa con la produzione:

$$C ::= \dots \mid \text{return } E;$$

4.3.3.2. Variabili libere

Le variabili libere di un comando `return` sono quelle dell'espressione restituita:

$$\text{fv}(\text{return } E;) = \text{fv}(E)$$

4.3.3.3. Type checking

La regola di tipo verifica che l'espressione restituita sia ben tipata. Il comando `return` non introduce nuove variabili.

$$\frac{\Gamma \vdash E : T}{\Gamma \vdash \text{return } E; : \emptyset} \quad (\text{T-Return})$$

Nota. In un sistema di tipi completo, il tipo T dell'espressione restituita dovrebbe essere confrontato con il tipo di ritorno dichiarato nella firma della funzione. In MAO questo controllo viene effettuato dalla regola (T-Fun) o (T-RecFun).

4.3.3.4. Semantica operativa

L'esecuzione di `return E;` valuta l'espressione E nello stato corrente e produce una terna (v, ρ, σ') che segnala la terminazione con il valore v :

$$\frac{\langle E, \rho, \sigma \rangle \Downarrow (v, \sigma')}{\langle \text{return } E; , \rho, \sigma \rangle \rightarrow (v, \rho, \sigma')} \quad (\text{Return})$$

4.3.3.5. Propagazione del return nei comandi composti

Quando un comando `return` viene eseguito all'interno di un comando composto (sequenza, condizionale o ciclo), il valore restituito deve **propagarsi verso l'alto** fino al contesto della funzione chiamante. In pratica, l'esecuzione di un `return` interrompe immediatamente l'esecuzione del corpo della funzione e restituisce il valore al chiamante.

Nella semantica big-step di MAO, questa propagazione si realizza nel modo seguente: quando l'esecuzione di un sotto-comando produce una terna (v, ρ, σ') anziché una coppia (ρ', σ') , ciò segnala che un `return` è stato eseguito. I comandi composti che lo contengono devono propagare questa terna senza eseguire ulteriori istruzioni.

- **Sequenza:** se nella sequenza $C_1 C_2$ l'esecuzione di C_1 produce (v, ρ', σ') , allora C_2 non viene eseguito e il risultato complessivo è (v, ρ', σ') . Se C_1 termina normalmente e C_2 produce un return, il risultato è quello di C_2 .
- **Condizionale:** se il ramo selezionato (then o else) produce (v, ρ', σ') , questo risultato viene propagato.
- **Ciclo while:** se il corpo del while produce (v, ρ', σ') , il ciclo si interrompe e il valore viene propagato.

Nota. La distinzione tra terminazione normale (coppia (ρ', σ')) e terminazione con return (terna (v, ρ', σ')) è il meccanismo chiave che permette al `return` di interrompere il flusso di esecuzione e propagare il valore restituito attraverso qualsiasi livello di annidamento fino alla regola (Call).

4.4. Funzioni

Le funzioni sono un meccanismo di astrazione fondamentale nella programmazione. Permettono di associare un nome a un frammento di codice parametrico, che calcola un valore a partire da uno o più argomenti. Una volta definita, una funzione può essere *invocata* (chiamata) più volte con argomenti diversi, favorendo il riuso del codice e la modularità del programma.

Si distinguono due momenti:

- **Definizione della funzione:** si scrive il codice del corpo della funzione, specificando i *parametri formali* (nomi simbolici che rappresentano gli input)
- **Invocazione (chiamata) della funzione:** si esegue il corpo della funzione con *parametri attuali* (espressioni concrete i cui valori vengono passati ai parametri formali)

ESEMPIO 4.4.1 (Definizione e invocazione di una funzione).

```
int max(int a, int b){
    int m = a;
    if (b > m) {
        m := b;
    }
    return m;
}
...
if (max(x, y) < 10) {
    z := max(x + 2, y * 3);
} else {
    z := max(x / 10, y - 10);
}
```

La funzione `max` è definita con due parametri formali `a` e `b`. Viene poi invocata tre volte con parametri attuali diversi.

4.4.1. Corrispondenza tra parametri formali e attuali

Quando si invoca una funzione, i parametri attuali devono corrispondere ai parametri formali **in numero e in tipo**. La corrispondenza è *posizionale*: il primo parametro attuale viene associato al primo parametro formale, il secondo al secondo, e così via.

4.4.2. Passaggio di parametri per valore

Il **passaggio per valore** è la modalità di default in MAO per i tipi scalari (`int`, `bool`, `char`). Ogni parametro formale viene inizializzato con una *copia* del valore del corrispondente parametro attuale. Di conseguenza, eventuali modifiche ai parametri formali all'interno del corpo della funzione *non influenzano* i parametri attuali nel contesto del chiamante.

4.4.3. Passaggio di parametri per riferimento (array)

Quando un array viene passato come parametro attuale, il valore copiato nel parametro formale è l'*indirizzo base* l_b dell'array. Si tratta tecnicamente ancora di un passaggio per valore (si copia il riferimento), ma il risultato pratico è un **passaggio per riferimento implicito**: la funzione e il chiamante condividono lo stesso array in memoria, quindi le modifiche agli elementi dell'array effettuate nel corpo della funzione sono visibili anche nel contesto del chiamante.

Nota. Questa è una conseguenza diretta del meccanismo di aliasing descritto in precedenza. Poiché la variabile di tipo array contiene un riferimento e non i dati stessi, copiare la variabile equivale a creare un alias.

4.4.4. Sintassi delle funzioni

4.4.4.1. Dichiarazione

La sintassi della dichiarazione di funzione è la seguente:

$$C ::= \dots \mid T_R \text{ Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n) \{C\}$$

dove T_R è il **tipo di ritorno** (può essere `void` per funzioni che non restituiscono un valore), Id è il nome della funzione, $T_i \text{ Id}_i$ sono le coppie tipo-nome dei parametri formali, e C è il corpo della funzione.

4.4.4.2. Invocazione

La sintassi della chiamata di funzione è:

$$E ::= \dots \mid \text{Id}(E_1, \dots, E_n)$$

dove le espressioni $E_1, \dots, E_n$ sono i parametri attuali.

4.4.4.3. Tipo di una funzione

Il tipo di una funzione viene rappresentato come un tipo freccia:

$$(T_1, \dots, T_n) \rightarrow T_R$$

dove $(T_1, \dots, T_n)$ sono i tipi dei parametri formali e T_R è il tipo di ritorno.

4.4.4.4. Variabili libere di una funzione

Le variabili libere nel corpo di una funzione escludono i parametri formali, che sono dichiarati nell'intestazione:

$$\text{fv}(T_R \text{ Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n)\{C\}) = \text{fv}(C) \setminus \{\text{Id}_1, \dots, \text{Id}_n\}$$

4.4.5. Type checking delle funzioni

4.4.5.1. Dichiarazione di funzione

La regola di tipo per la dichiarazione di una funzione verifica che il corpo C sia ben tipato in un ambiente esteso con i parametri formali. La dichiarazione introduce nell'ambiente l'associazione tra il nome della funzione e il suo tipo freccia.

$$\frac{\Gamma \cup \{\text{Id}_1 : T_1, \dots, \text{Id}_n : T_n\} \vdash C : \Gamma'}{\Gamma \vdash T_R \text{ Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n)\{C\} : \{\text{Id} : (T_1, \dots, T_n) \rightarrow T_R\}} \quad (\text{T-Fun})$$

Nota. Si osservi che il corpo della funzione viene verificato nell'ambiente $\Gamma \cup \{\text{Id}_1 : T_1, \dots, \text{Id}_n : T_n\}$, in cui i parametri formali sono disponibili con i loro tipi dichiarati. L'ambiente Γ del chiamante resta invariato tranne per l'aggiunta del tipo della funzione.

4.4.5.2. Invocazione di funzione

La regola per l'invocazione verifica che la funzione sia presente nell'ambiente con un tipo freccia compatibile e che i tipi dei parametri attuali corrispondano (posizionalmente) ai tipi dei parametri formali.

$$\frac{\Gamma(\text{Id}) = (T_1, \dots, T_n) \rightarrow T_R \quad \Gamma \vdash E_1 : T_1 \quad \dots \quad \Gamma \vdash E_n : T_n}{\Gamma \vdash \text{Id}(E_1, \dots, E_n) : T_R} \quad (\text{T-Call})$$

4.4.6. Semantica operativa delle funzioni

4.4.6.1. Dichiarazione di funzione

La dichiarazione di una funzione non esegue il corpo, ma memorizza nell'ambiente una **chiusura** (*closure*), ovvero una terna composta dai nomi dei parametri formali, dal corpo della funzione e dall'ambiente al momento della dichiarazione:

$$\langle T_R \text{ Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n)\{C\}, \rho, \sigma \rangle \rightarrow \langle \rho[\text{Id} \mapsto (\text{Id}_1, \dots, \text{Id}_n, C, \rho)], \sigma \rangle$$

Nota (Chiusura (closure)). La chiusura cattura l'ambiente ρ al momento della dichiarazione. Questo è essenziale per lo **scoping statico** (o *lessicale*): quando la funzione verrà invocata, le variabili libere nel corpo saranno risolte nell'ambiente della dichiarazione, non in quello della chiamata. Senza la chiusura, una funzione definita in un certo contesto e chiamata in un altro potrebbe accedere a variabili diverse da quelle previste dal programmatore.

4.4.6.2. Chiamata di funzione

La chiamata di funzione si articola nei seguenti passi:

1. Si recupera la chiusura della funzione dall'ambiente del chiamante
2. Si valutano i parametri attuali $E_1, \dots, E_n$ da sinistra a destra
3. Si allocano nuove locazioni $l_1, \dots, l_n$ per i parametri formali
4. Si costruisce un nuovo ambiente ρ' estendendo l'ambiente della chiusura ρ_f con le associazioni tra parametri formali e locazioni
5. Si esegue il corpo C nel nuovo ambiente ρ'
6. Il valore restituito dal `return` diventa il risultato della chiamata

$$\begin{array}{c}
 \rho(\text{Id}) = (\text{Id}_1, \dots, \text{Id}_n, C, \rho_f) \\
 \langle E_i, \rho, \sigma_{i-1} \rangle \Downarrow (v_i, \sigma_i) \text{ per } i = 1..n \\
 l_1, \dots, l_n \notin \text{dom}(\sigma_n) \\
 \rho' = \rho_f[\text{Id}_1 \mapsto l_1] \dots [\text{Id}_n \mapsto l_n] \\
 \sigma' = \sigma_n[l_1 \mapsto v_1] \dots [l_n \mapsto v_n] \\
 \hline
 \langle C, \rho', \sigma' \rangle \rightarrow (v_r, \rho'', \sigma'') \\
 \hline
 \langle \text{Id}(E_1, \dots, E_n), \rho, \sigma \rangle \Downarrow (v_r, \sigma'') \quad (\text{Call})
 \end{array}$$

Nota (Scoping statico). Si osservi che il nuovo ambiente ρ' è costruito a partire da ρ_f (l'ambiente catturato nella chiusura), **non** da ρ (l'ambiente del chiamante). Questo implementa lo scoping statico: le variabili libere nel corpo della funzione vengono risolte nel contesto in cui la funzione è stata *definita*, non in quello in cui viene *chiamata*.

4.5. Ricorsione

In molti linguaggi di programmazione è ammessa la possibilità di definire **funzioni ricorsive**, cioè funzioni che invocano se stesse (direttamente o indirettamente) nel proprio corpo. La ricorsione è un meccanismo potente che permette di risolvere problemi definiti in modo induttivo, decomponendoli in sottoproblemi della stessa natura ma di dimensione ridotta.

La chiamata ricorsiva può essere:

- **Diretta:** la funzione f contiene nel proprio corpo una chiamata a f stessa
- **Indiretta:** la funzione f chiama una funzione g , che a sua volta chiama f (oppure tramite una catena più lunga di chiamate intermedie)

4.5.1. Variabili libere nelle funzioni ricorsive

Per supportare la ricorsione, la definizione di variabili libere di una funzione viene modificata includendo anche il nome della funzione stessa tra le variabili escluse:

$$\text{fv}(T_R \text{ Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n)\{C\}) = \text{fv}(C) \setminus \{\text{Id}, \text{Id}_1, \dots, \text{Id}_n\}$$

In questo modo il nome della funzione non è considerato una variabile libera nel corpo, anche se vi compare come chiamata ricorsiva. Senza questa modifica, il type checker richiederebbe che `Id` fosse già presente nell'ambiente Γ prima della dichiarazione, rendendo impossibile la ricorsione.

4.5.2. Type checking delle funzioni ricorsive

La regola di tipo per le funzioni ricorsive differisce da (T-Fun) per il fatto che l'ambiente in cui viene verificato il corpo include anche l'associazione tra il nome della funzione e il suo tipo freccia. Questo permette al type checker di verificare le chiamate ricorsive:

$$\frac{\Gamma \cup \{\text{Id} : (T_1, \dots, T_n) \rightarrow T_R\} \cup \{\text{Id}_1 : T_1, \dots, \text{Id}_n : T_n\} \vdash C : \Gamma'}{\Gamma \vdash T_R \quad \text{Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n)\{C\} : \{\text{Id} : (T_1, \dots, T_n) \rightarrow T_R\} \quad (\text{T-RecFun})}$$

Nota (Differenza tra T-Fun e T-RecFun). La differenza chiave è nell'ambiente usato per verificare il corpo C . In (T-Fun), l'ambiente è $\Gamma \cup \{\text{Id}_1 : T_1, \dots, \text{Id}_n : T_n\}$; in (T-RecFun), è $\Gamma \cup \{\text{Id} : (T_1, \dots, T_n) \rightarrow T_R\} \cup \{\text{Id}_1 : T_1, \dots, \text{Id}_n : T_n\}$. La presenza del binding $\text{Id} : (T_1, \dots, T_n) \rightarrow T_R$ permette di verificare le chiamate ricorsive nel corpo della funzione come normali invocazioni tramite la regola (T-Call).

4.5.3. Semantica operativa delle funzioni ricorsive

Nella semantica operativa, la ricorsione funziona perché la chiusura di una funzione ricorsiva include il nome della funzione stessa nell'ambiente catturato. Al momento della dichiarazione, l'ambiente ρ viene esteso con il binding della funzione *prima* di costruire la chiusura:

$$\langle T_R \quad \text{Id}(T_1 \text{ Id}_1, \dots, T_n \text{ Id}_n)\{C\}, \rho, \sigma \rangle \rightarrow \langle \rho', \sigma \rangle$$

dove $\rho' = \rho[\text{Id} \mapsto (\text{Id}_1, \dots, \text{Id}_n, C, \rho')]$

Nota (Autoreferenza nella chiusura). Si osservi la natura circolare della definizione: l'ambiente ρ' contenuto nella chiusura include il binding di Id alla chiusura stessa. Questa autoreferenza è ciò che rende possibile la ricorsione a livello semantico. Quando il corpo C viene eseguito durante una chiamata, il nome della funzione è presente nell'ambiente ρ' e punta alla stessa chiusura, permettendo di effettuare chiamate ricorsive. Senza questo meccanismo, il corpo della funzione non troverebbe il binding per Id nell'ambiente e la chiamata ricorsiva fallirebbe. La regola (Call) non necessita di modifiche: funziona identicamente per funzioni ricorsive e non ricorsive, perché la ricorsione è interamente gestita dalla struttura della chiusura.

4.5.4. Esempi completi

ESEMPIO 4.5.1 (Type checking della funzione abs). Verifichiamo che la funzione `abs` (valore assoluto) sia ben tipata:

```
int abs(int n) {
  int m = n;
  if (m < 0) { m := -m; }
  return m;
}
```

Vogliamo dimostrare: $\Gamma \vdash \text{abs} : (\text{int}) \rightarrow \text{int}$

Passo 1: Costruzione dell'ambiente per il corpo.

Secondo la regola (T-Fun), il corpo deve essere verificato nell'ambiente:

$$\Gamma' = \Gamma \cup \{n : \text{int}\} \quad (24)$$

Passo 2: Type checking di `int m = n;` in Γ'

$$\frac{\frac{\Gamma'(n) = \text{int}}{\Gamma' \vdash n : \text{int}}}{\Gamma' \vdash \text{int } m = n; : \{m : \text{int}\}} \quad (\text{T-Decl})$$

Ambiente aggiornato: $\Gamma'' = \Gamma' \cup \{m : \text{int}\} = \Gamma \cup \{n : \text{int}, m : \text{int}\}$

Passo 3: Type checking della guardia `m < 0` in Γ''

$$\frac{\frac{\frac{\Gamma''(m) = \text{int} \quad 0 \in \mathbb{Z}}{\Gamma'' \vdash m : \text{int}} \quad \Gamma'' \vdash 0 : \text{int}}{\Gamma'' \vdash m < 0 : \text{bool}} \quad (\text{T-Cop})$$

Passo 4: Type checking della negazione `-m` in Γ''

L'operatore unario meno richiede un operando intero e produce un intero:

$$\frac{\Gamma'' \vdash m : \text{int}}{\Gamma'' \vdash -m : \text{int}} \quad (\text{T-Neg})$$

Passo 5: Type checking dell'assegnamento `m := -m;` in Γ''

$$\frac{\Gamma''(m) = \text{int} \quad \Gamma'' \vdash -m : \text{int}}{\Gamma'' \vdash m := -m; : \emptyset} \quad (\text{T-Assign})$$

Passo 6: Type checking del blocco `{ m := -m; }` in Γ''

$$\frac{\Gamma'' \vdash m := -m; : \emptyset}{\Gamma'' \vdash \{m := -m;\} : \emptyset} \quad (\text{T-Block})$$

Passo 7: Type checking del condizionale (con else implicito uguale a skip)

$$\frac{\Gamma'' \vdash m < 0 : \text{bool} \quad \Gamma'' \vdash \{m := -m;\} : \emptyset \quad \Gamma'' \vdash \text{skip}; : \emptyset}{\Gamma'' \vdash \text{if}(m < 0)\{m := -m;\} : \emptyset} \quad (\text{T-If})$$

Passo 8: Type checking di `return m;` in Γ''

$$\frac{\frac{\Gamma''(m) = \text{int}}{\Gamma'' \vdash m : \text{int}}}{\Gamma'' \vdash \text{return } m; : \emptyset} \quad (\text{T-Return})$$

ESEMPIO 4.5.2 (Type checking di una funzione ricorsiva su array). Verifichiamo il type checking della funzione ricorsiva `azzera`, che cerca l'elemento `k` nell'array `a` a partire dalla posizione `p` e, se lo trova, lo sostituisce con 0:

```
bool azzera(int[] a, int k, int p) {
  bool res = false;
  if ((p >= 0) && (p < a.length)) {
    if (a[p] == k) { a[p] := 0; res := true; }
    res := azzera(a, k, p+1);
  }
  return res;
}
```

Passo 1: Poiché la funzione è ricorsiva, usiamo la regola (T-RecFun).

L'ambiente per il corpo deve includere il tipo della funzione stessa (per permettere la chiamata ricorsiva) e i parametri formali:

$$\Gamma' = \Gamma \cup \{azzera : (\text{int}[], \text{int}, \text{int}) \rightarrow \text{bool}\} \cup \{a : \text{int}[], k : \text{int}, p : \text{int}\} \quad (25)$$

Passo 2: Type checking di `bool res = false;` in Γ'

$$\frac{}{\Gamma' \vdash \text{false} : \text{bool}} \quad \text{(T-Decl)}$$

Ambiente aggiornato: $\Gamma'' = \Gamma' \cup \{\text{res} : \text{bool}\}$

Passo 3: Type checking della guardia `(p >= 0) && (p < a.length)` in Γ''

Verifica di `p >= 0`:

$$\frac{\frac{\Gamma''(p) = \text{int} \quad 0 \in \mathbb{Z}}{\Gamma'' \vdash p : \text{int}} \quad \Gamma'' \vdash 0 : \text{int}}{\Gamma'' \vdash p \geq 0 : \text{bool}} \quad \text{(T-Cop)}$$

Verifica di `a.length`:

$$\frac{\frac{\Gamma''(a) = \text{int}[]}{\Gamma'' \vdash a : \text{int}[]}}{\Gamma'' \vdash a.\text{length} : \text{int}} \quad \text{(T-Length)}$$

Verifica di `p < a.length`:

$$\frac{\Gamma'' \vdash p : \text{int} \quad \Gamma'' \vdash a.\text{length} : \text{int}}{\Gamma'' \vdash p < a.\text{length} : \text{bool}} \quad \text{(T-Cop)}$$

Congiunzione logica:

$$\frac{\Gamma'' \vdash p \geq 0 : \text{bool} \quad \Gamma'' \vdash p < a.\text{length} : \text{bool}}{\Gamma'' \vdash (p \geq 0) \wedge (p < a.\text{length}) : \text{bool}} \quad \text{(T-Lop)}$$

Passo 4: Type checking dell'accesso `a[p]` in Γ''

$$\frac{\Gamma'' \vdash a : \text{int}[] \quad \Gamma'' \vdash p : \text{int}}{\Gamma'' \vdash a[p] : \text{int}} \quad \text{(T-ArrayAccess)}$$

Passo 5: Type checking della condizione `a[p] == k`

4.5.5. Valori iniziali ammissibili per i parametri

Quando una funzione ricorsiva viene definita per risolvere un problema specifico (ad esempio «controllare se l'array contiene almeno k occorrenze di 'z'»), il type checking garantisce che la funzione sia **ben tipata**, ma non dice con quali valori i parametri debbano essere inizializzati per ottenere il comportamento desiderato. Questa è una domanda **semantica**, non sintattica, e richiede di ragionare sul significato della funzione.

Nota (Determinare i valori iniziali). Per determinare i valori iniziali ammissibili dei parametri di una funzione ricorsiva, si procede così:

1. **Leggere la specifica:** cosa deve fare la funzione? (es. «controllare se gli ultimi k caratteri sono tutti 'z' »)
2. **Identificare il caso base:** quale condizione ferma la ricorsione? (es. un flag booleano b , un indice fuori dai limiti)
3. **Ragionare all'indietro:** con quali valori iniziali il caso base viene raggiunto correttamente e il risultato è quello voluto?

In particolare, se la funzione usa:

- un **parametro indice** p che avanza ad ogni chiamata ricorsiva, il valore iniziale deve corrispondere alla posizione di partenza della scansione (ad esempio $p = 0$ per scandire dal primo elemento, oppure $p = a.length - 1$ per scandire dall'ultimo);
- un **parametro flag booleano** b che controlla il caso base, il valore iniziale deve essere quello che permette alla ricorsione di procedere (tipicamente $b = false$ se il caso base è `if (b) { return b; }`).

ESEMPIO 4.5.3 (Valori iniziali per il controllo di caratteri). Consideriamo una funzione ricorsiva:

```
bool f1(char[] a, int k, bool b, int p) {
    bool r = false;
    if (b || p >= a.length || p < 0) {
        r := b;
    } else {
        if (a[p] == 'z') { k := k-1; }
        r := f1(a, k, k<0, p+1);
    }
    return r;
}
```

La funzione controlla se la stringa a contiene strettamente più di k caratteri 'z'. Ad ogni passo incrementa p e decrementa k quando trova 'z'. La ricorsione si ferma quando b diventa vero (cioè $k < 0$, trovati abbastanza 'z') oppure quando p esce dai limiti.

I valori iniziali ammissibili sono: $b = false$ (perché con $b = true$ la funzione ritornerebbe immediatamente senza scandire) e $p = 0$ (per partire dall'inizio dell'array).

CAPITOLO 5

Complessità Computazionale

5.1. Modello di calcolo e costo computazionale

In questo capitolo introduciamo gli strumenti fondamentali per analizzare l'efficienza degli algoritmi. Dato un problema computazionale, esistono in generale molteplici algoritmi che lo risolvono: il nostro obiettivo è confrontarli in modo rigoroso, determinando quale sia il più efficiente in termini di risorse utilizzate. Per fare ciò, abbiamo bisogno di un modello di calcolo di riferimento e di un linguaggio matematico per esprimere il costo degli algoritmi.

5.1.1. Modello di calcolo RAM

Per poter contare le operazioni eseguite da un algoritmo, occorre fissare un modello di calcolo. Il modello adottato nel corso è il **modello RAM** (Random Access Machine), che formalizza un calcolatore idealizzato.

DEFINIZIONE 5.1.1 (Modello RAM a costo unitario). Nel modello RAM a costo unitario, le seguenti operazioni hanno ciascuna costo costante (pari a 1):

- **Operazioni aritmetiche:** $+$, $-$, $\times$, $\div$, $\%$
- **Operazioni di confronto:** $<$, $>$, $\leq$, $\geq$, $==$, $\neq$
- **Operazioni logiche:** $\wedge$, $\vee$, $\neg$
- **Operazioni di trasferimento:** lettura e scrittura in memoria, assegnamento
- **Operazioni di controllo:** `return`, chiamata a funzione, salto condizionale

La memoria è ad accesso casuale: l'accesso a qualsiasi cella ha costo costante, indipendentemente dal suo indirizzo.

Nota. Il modello RAM a costo unitario è una semplificazione: in un calcolatore reale, il costo di un'operazione può dipendere dalla dimensione degli operandi (ad esempio, moltiplicare due numeri di 1000 cifre è più costoso che moltiplicare due numeri di una cifra). Tuttavia, per gli scopi del corso, questa approssimazione è adeguata e consente di concentrarsi sugli aspetti strutturali degli algoritmi.

5.1.2. Costo computazionale di un algoritmo

Dato un algoritmo A e un input di dimensione n , definiamo il **costo computazionale** come la quantità di risorse richieste dalla sua esecuzione.

DEFINIZIONE 5.1.2 (Costo in tempo). Il **costo in tempo** $T_A(n)$ di un algoritmo A è il numero di operazioni elementari (passi) che A esegue su un input di dimensione n nel modello RAM.

DEFINIZIONE 5.1.3 (Costo in spazio). Il **costo in spazio** $S_A(n)$ di un algoritmo A è il numero di celle di memoria utilizzate durante l'esecuzione di A su un input di dimensione n , incluse quelle occupate dall'input stesso.

In generale, il costo in tempo di un algoritmo non dipende solo dalla dimensione dell'input, ma anche dalla specifica istanza. Per questo si distinguono tre casi.

DEFINIZIONE 5.1.4 (Caso ottimo, pessimo e medio). Sia I_n l'insieme di tutte le istanze di dimensione n per un dato problema. Il costo in tempo di un algoritmo A si descrive come:

- **Caso ottimo:** $T_A^{\text{best}}(n) = \min_{I \in I_n} T_A(I)$ – il minimo numero di operazioni su tutte le istanze di dimensione n .
- **Caso pessimo:** $T_A^{\text{worst}}(n) = \max_{I \in I_n} T_A(I)$ – il massimo numero di operazioni su tutte le istanze di dimensione n .
- **Caso medio:** $T_A^{\text{avg}}(n) = \sum_{I \in I_n} P(I) \cdot T_A(I)$ – il costo atteso, dove $P(I)$ è la probabilità dell'istanza I .

Nell'analisi degli algoritmi ci concentreremo principalmente sul **caso pessimo**, poiché fornisce una garanzia sul costo massimo. Il caso medio è spesso più informativo nella pratica, ma richiede ipotesi sulla distribuzione degli input.

A parità di complessità in tempo, si cerca di minimizzare anche la complessità in spazio.

5.1.3. Esempio: Minimo in un vettore

Consideriamo il problema di trovare il valore minimo in un array.

Input: array $A[0..n-1]$ di interi.

Output: il valore minimo m tale che $m = A[i]$ per qualche $i \in \{0, \dots, n-1\}$ e $m \leq A[j]$ per ogni $j \in \{0, \dots, n-1\}$.

Minimo di un array

```
int min(int[] A, int n){
    int min = A[0];
    int i = 1;
    while(i <= n-1){
        if(A[i] < min){
            min := A[i];
        }
        i := i + 1;
    }
    return min;
}
```

Analisi del costo. Tutte le operazioni nel corpo del ciclo (`if`, confronto, eventuale assegnamento, incremento) hanno costo costante nel modello RAM. Il ciclo `while` viene eseguito esattamente $n-1$ volte, indipendentemente dai valori contenuti nell'array. Il costo totale è quindi:

$$T(n) = c_1 + (n-1) \cdot c_2 + c_3 \quad (26)$$

dove c_1, c_3 sono costanti per le operazioni fuori dal ciclo e c_2 è il costo costante di ciascuna iterazione. La complessità in tempo è **lineare** nella dimensione n dell'input. Si noti che in questo caso il costo non dipende dalla specifica istanza: caso ottimo, pessimo e medio coincidono.

5.1.4. Esempio: Ricerca di un elemento

Consideriamo il problema della ricerca lineare (o sequenziale).

Input: array $A[0..n-1]$ di interi, intero k .

Output: il minimo indice i tale che $A[i] = k$, oppure -1 se $k \notin A$.

Ricerca lineare (CercaK)

```

int cercaK(int[] A, int n, int k){
    int i = 0;
    bool trovato = false;
    while((!trovato) && (i <= n-1)){
        if(A[i] == k){
            trovato := true;
        } else {
            i := i + 1;
        }
    }
    if(trovato){
        return i;
    } else {
        return -1;
    }
}

```

Analisi del costo. A differenza dell'esempio precedente, il numero di iterazioni dipende dalla posizione di k nell'array:

- **Caso ottimo:** $A[0] = k$, il ciclo esegue una sola iterazione. Il costo è $T^{\text{best}(n)} = \Theta(1)$ (costante).
- **Caso pessimo:** $k \notin A$, il ciclo scorre l'intero array senza trovare k . Il costo è $T^{\text{worst}(n)} = \Theta(n)$ (lineare).
- **Caso medio:** assumendo che k sia presente e che ciascuna posizione sia equiprobabile, il numero medio di confronti è $\frac{n+1}{2}$, dunque $T^{\text{avg}(n)} = \Theta(n)$.

Questo esempio mostra come lo stesso algoritmo possa avere costi significativamente diversi a seconda dell'istanza, motivando la distinzione tra caso ottimo e pessimo.

5.1.5. Complessità asintotica

L'obiettivo dell'analisi di complessità non è calcolare il numero esatto di operazioni, ma determinare l'ordine di grandezza della funzione di costo $T(n)$ al crescere di n . Si trascurano:

- le **costanti moltiplicative**, che dipendono dal modello di calcolo e dall'implementazione;
- i **termini di ordine inferiore**, che diventano trascurabili per n grande.

ESEMPIO 5.1.5 (Complessità lineare).

$$T(n) = 3n + 2 \quad (27)$$

Il termine dominante è $3n$. Trascurando la costante moltiplicativa 3 e il termine di ordine inferiore 2, si conclude che $T(n)$ ha ordine di grandezza **lineare**.

ESEMPIO 5.1.6 (Complessità quadratica).

$$T(n) = 8n^2 + \log n + 4 \quad (28)$$

Il termine dominante è $8n^2$. I termini $\log n$ e 4 sono di ordine inferiore e vengono trascurati. La complessità è **quadratica**.

Per formalizzare queste nozioni, introduciamo la **notazione asintotica**.

5.1.6. Notazione Θ – Limite asintotico stretto

DEFINIZIONE 5.1.7 (Θ (Theta)). Sia $g(n)$ una funzione. L'insieme $\Theta(g(n))$ è definito come:

$$\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0 > 0 : \forall n \geq n_0, \quad 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\} \quad (29)$$

Se $f(n) \in \Theta(g(n))$, si dice che $g(n)$ è un **limite asintotico stretto** per $f(n)$.

Si scrive $f(n) = \Theta(g(n))$ (con abuso di notazione, poiché $\Theta(g(n))$ è un insieme).

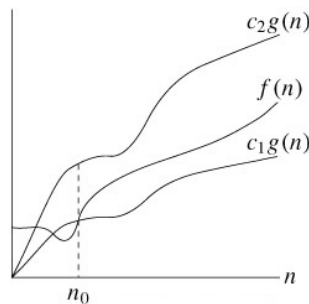


Figura 1: Rappresentazione grafica di $\Theta(g(n))$: per $n \geq n_0$, la funzione $f(n)$ è compresa tra $c_1 \cdot g(n)$ e $c_2 \cdot g(n)$.

Intuitivamente, $f(n) = \Theta(g(n))$ significa che, per n sufficientemente grande, $f(n)$ cresce **allo stesso ritmo** di $g(n)$, a meno di costanti moltiplicative. Esempio: $\Theta(n)$ vuol dire «cresce proporzionalmente come n »

5.1.7. Notazione O – Limite asintotico superiore

DEFINIZIONE 5.1.8 (O (O grande)). Sia $g(n)$ una funzione. L'insieme $O(g(n))$ è definito come:

$$O(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : \forall n \geq n_0, \quad 0 \leq f(n) \leq c \cdot g(n)\} \quad (30)$$

Se $f(n) \in O(g(n))$, si dice che $g(n)$ è un **limite asintotico superiore** per $f(n)$.

Si scrive $f(n) = O(g(n))$.

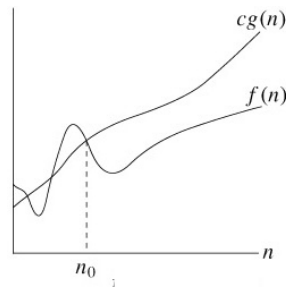


Figura 2: Rappresentazione grafica di $O(g(n))$: per $n \geq n_0$, la funzione $f(n)$ non supera $c \cdot g(n)$.

La notazione O fornisce un **maggiorante** alla crescita di $f(n)$. Si utilizza tipicamente per esprimere il costo nel caso pessimo. Esempio: $O(n)$ vuol dire che «al massimo cresce come n », non puoi fare peggio di n

5.1.8. Notazione Ω – Limite asintotico inferiore

DEFINIZIONE 5.1.9 (Ω (Omega grande)). Sia $g(n)$ una funzione. L'insieme $\Omega(g(n))$ è definito come:

$$\Omega(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : \forall n \geq n_0, \quad 0 \leq c \cdot g(n) \leq f(n)\} \quad (31)$$

Se $f(n) \in \Omega(g(n))$, si dice che $g(n)$ è un **limite asintotico inferiore** per $f(n)$.

Si scrive $f(n) = \Omega(g(n))$.

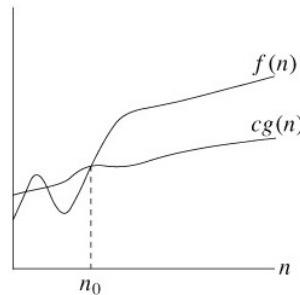


Figura 3: Rappresentazione grafica di $\Omega(g(n))$: per $n \geq n_0$, la funzione $f(n)$ è sempre almeno $c \cdot g(n)$.

La notazione Ω fornisce un **minorante** alla crescita di $f(n)$. Si utilizza tipicamente per esprimere il costo nel caso ottimo o per dimostrare limiti inferiori. Esempio: $\Omega(n)$ dice che «cresce almeno come n », non puoi fare meglio di n

5.1.9. Relazione tra le notazioni

TEOREMA 5.1.10 (Relazione tra Θ , O e Ω). Per ogni coppia di funzioni $f(n)$ e $g(n)$:

$$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n)) \quad (32)$$

Dimostrazione. ($\Rightarrow$) Se $f(n) = \Theta(g(n))$, allora esistono $c_1, c_2, n_0 > 0$ tali che per ogni $n \geq n_0$:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad (33)$$

La disuguaglianza $f(n) \leq c_2 \cdot g(n)$ implica $f(n) = O(g(n))$ con $c = c_2$. La disuguaglianza $c_1 \cdot g(n) \leq f(n)$ implica $f(n) = \Omega(g(n))$ con $c = c_1$.

($\Leftarrow$) Se $f(n) = O(g(n))$, esiste $c_2, n_1 > 0$ con $f(n) \leq c_2 \cdot g(n)$ per $n \geq n_1$. Se $f(n) = \Omega(g(n))$, esiste $c_1, n_2 > 0$ con $c_1 \cdot g(n) \leq f(n)$ per $n \geq n_2$. Prendendo $n_0 = \max(n_1, n_2)$, entrambe le disuguaglianze valgono simultaneamente per $n \geq n_0$, ovvero $f(n) = \Theta(g(n))$. $\square$

5.1.10. Proprietà della notazione asintotica

Le notazioni Θ , O e Ω godono di proprietà algebriche che ne facilitano l'uso. Per ciascuna proprietà riportiamo la formulazione e, ove opportuno, una giustificazione intuitiva.

TEOREMA 5.1.11 (Transitività). Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$, allora $f(n) = \Theta(h(n))$.

Analogamente per O e Ω :

- $f(n) = O(g(n)) \wedge g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$
- $f(n) = \Omega(g(n)) \wedge g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$

Dimostrazione. Dimostriamo il caso O . Per ipotesi, esistono c_1, n_1 con $f(n) \leq c_1 \cdot g(n)$ per $n \geq n_1$, e c_2, n_2 con $g(n) \leq c_2 \cdot h(n)$ per $n \geq n_2$. Per $n \geq \max(n_1, n_2)$:

$$f(n) \leq c_1 \cdot g(n) \leq c_1 \cdot c_2 \cdot h(n) \quad (34)$$

Ponendo $c = c_1 \cdot c_2$ e $n_0 = \max(n_1, n_2)$, si ha $f(n) = O(h(n))$. Le dimostrazioni per Ω e Θ sono analoghe. $\square$

TEOREMA 5.1.12 (Riflessività). Per ogni funzione $f(n)$:

$$f(n) = \Theta(f(n)), \quad f(n) = O(f(n)), \quad f(n) = \Omega(f(n)) \quad (35)$$

Basta prendere $c_1 = c_2 = c = 1$ e $n_0 = 1$ nelle rispettive definizioni.

TEOREMA 5.1.13 (Simmetria).

$$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n)) \quad (36)$$

La simmetria vale **solo** per Θ . Non vale per O e Ω .

TEOREMA 5.1.14 (Simmetria trasposta).

$$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n)) \quad (37)$$

Intuitivamente: dire che f cresce al più come g equivale a dire che g cresce almeno come f .

Nota (Analogia con le relazioni d'ordine). Le notazioni asintotiche si possono interpretare come relazioni d'ordine tra funzioni:

- $f(n) = O(g(n))$ corrisponde a $f \leq g$ (informalmente)
- $f(n) = \Omega(g(n))$ corrisponde a $f \geq g$
- $f(n) = \Theta(g(n))$ corrisponde a $f \approx g$

Le proprietà di transitività, riflessività e simmetria ricalcano quelle delle relazioni $\leq$, $\geq$ e $=$.

5.1.11. Notazione o – Limite superiore non stretto

Le notazioni O e Ω forniscono limiti che possono essere stretti o meno. Per esprimere un limite superiore che **non** è asintoticamente stretto, si utilizza la notazione o (o piccolo).

DEFINIZIONE 5.1.15 (o (o piccolo)). Sia $g(n)$ una funzione. L'insieme $o(g(n))$ è definito come:

$$o(g(n)) = \{f(n) \mid \forall c > 0, \exists n_0 > 0 : \forall n \geq n_0, \quad 0 \leq f(n) < c \cdot g(n)\} \quad (38)$$

Se $f(n) \in o(g(n))$, si dice che $f(n)$ è **asintoticamente trascurabile** rispetto a $g(n)$.

La differenza cruciale con O è nel quantificatore: nella definizione di O la costante $c > 0$ è **esistenziale** (basta trovarne una), mentre nella definizione di o è **universale** (la disuguaglianza deve valere per **ogni** $c > 0$). Intuitivamente, $f(n) = o(g(n))$ significa che $f(n)$ diventa insignificante rispetto a $g(n)$ al crescere di n :

$$f(n) = o(g(n)) \Leftrightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \quad (39)$$

ESEMPIO 5.1.16 ($2n = o(n^2)$ ma $2n^2 \neq o(n^2)$). Per $2n = o(n^2)$: si ha $\lim_{n \rightarrow \infty} \frac{2n}{n^2} = \lim_{n \rightarrow \infty} \frac{2}{n} = 0$, dunque $2n \in o(n^2)$.

Per $2n^2 \neq o(n^2)$: si ha $\lim_{n \rightarrow \infty} \frac{2n^2}{n^2} = 2 \neq 0$, dunque $2n^2 \notin o(n^2)$. Si noti che $2n^2 \in O(n^2)$, poiché basta scegliere $c = 2$: la notazione O ammette limiti stretti, la notazione o no.

5.1.12. Notazione ω – Limite inferiore non stretto

Analogamente, per esprimere un limite inferiore **non** asintoticamente stretto, si usa la notazione ω (omega piccolo).

DEFINIZIONE 5.1.17 (ω (omega piccolo)). Sia $g(n)$ una funzione. L'insieme $\omega(g(n))$ è definito come:

$$\omega(g(n)) = \{f(n) \mid \forall c > 0, \exists n_0 > 0 : \forall n \geq n_0, \quad 0 \leq c \cdot g(n) < f(n)\} \quad (40)$$

Se $f(n) \in \omega(g(n))$, si dice che $f(n)$ **domina asintoticamente** $g(n)$.

La relazione tra o e ω è analoga a quella tra O e Ω :

$$f(n) = \omega(g(n)) \Leftrightarrow g(n) = o(f(n)) \quad (41)$$

Equivalentemente:

$$f(n) = \omega(g(n)) \Leftrightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \quad (42)$$

ESEMPIO 5.1.18 ($\frac{n^2}{2} = \omega(n)$ **ma** $\frac{n^2}{2} \neq \omega(n^2)$). Per $\frac{n^2}{2} = \omega(n)$: si ha $\lim_{n \rightarrow \infty} \frac{n^2/2}{n} = \lim_{n \rightarrow \infty} \frac{n}{2} = \infty$, dunque $\frac{n^2}{2} \in \omega(n)$.

Per $\frac{n^2}{2} \neq \omega(n^2)$: si ha $\lim_{n \rightarrow \infty} \frac{n^2/2}{n^2} = \frac{1}{2} \neq \infty$, dunque $\frac{n^2}{2} \notin \omega(n^2)$.

Nota (Analogia completa con le relazioni d'ordine). Estendendo l'analogia introdotta in precedenza:

- $f(n) = o(g(n))$ corrisponde a $f < g$ (strettamente minore)
- $f(n) = \omega(g(n))$ corrisponde a $f > g$ (strettamente maggiore)
- $f(n) = O(g(n))$ corrisponde a $f \leq g$
- $f(n) = \Omega(g(n))$ corrisponde a $f \geq g$
- $f(n) = \Theta(g(n))$ corrisponde a $f \approx g$

Si noti che $f(n) = \Theta(g(n))$ implica $f(n) \notin o(g(n))$ e $f(n) \notin \omega(g(n))$, proprio come $a = b$ implica $a \not< b$ e $a \not> b$.

«claude» inserire tabella con complessità più comuni e come si calcola in pratica.

5.1.13. Esercizi sulla notazione asintotica

ESEMPIO 5.1.19 (Dimostrare che $3n^2 - 2n - 1 \in \Theta(n^2)$). Dobbiamo trovare costanti $c_1, c_2 > 0$ e $n_0 \geq 1$ tali che:

$$\forall n \geq n_0 : c_1 \cdot n^2 \leq 3n^2 - 2n - 1 \leq c_2 \cdot n^2 \quad (43)$$

Limite superiore (O): Dimostriamo che $3n^2 - 2n - 1 \leq c_2 \cdot n^2$.

$$\text{Per } n \geq 1: 3n^2 - 2n - 1 \leq 3n^2$$

Quindi con $c_2 = 3$ il limite superiore è verificato per ogni $n \geq 1$.

Limite inferiore (Ω): Dimostriamo che $c_1 \cdot n^2 \leq 3n^2 - 2n - 1$.

$$\text{Riscriviamo: } 3n^2 - 2n - 1 \geq c_1 \cdot n^2$$

$$\text{Dividendo per } n^2 \text{ (per } n \geq 1\text{): } 3 - \frac{2}{n} - \frac{1}{n^2} \geq c_1$$

Per $n \geq 2$:

- $\frac{2}{n} \leq 1$
- $\frac{1}{n^2} \leq \frac{1}{4}$

$$\text{Quindi: } 3 - 1 - \frac{1}{4} = \frac{7}{4} \geq c_1$$

Scegliendo $c_1 = 1$, verifichiamo per $n = 2$:

$$3(4) - 2(2) - 1 = 12 - 4 - 1 = 7 \geq 1 \cdot 4 = 4 \quad \checkmark \quad (44)$$

Conclusione: Con $c_1 = 1$, $c_2 = 3$, $n_0 = 2$ abbiamo dimostrato che:

$$3n^2 - 2n - 1 \in \Theta(n^2) \quad (45)$$

ESEMPIO 5.1.20 (Dimostrare che $5n + 3 \in O(n)$). Dobbiamo trovare $c > 0$ e $n_0 \geq 1$ tali che $5n + 3 \leq c \cdot n$ per ogni $n \geq n_0$.

$$\text{Per } n \geq 1: 5n + 3 \leq 5n + 3n = 8n \text{ (poiché } 3 \leq 3n \text{ per } n \geq 1\text{)}.$$

Dunque con $c = 8$ e $n_0 = 1$ la disuguaglianza è verificata. Si ha $5n + 3 \in O(n)$.

ESEMPIO 5.1.21 (Dimostrare che $n^2 \notin O(n)$). Per assurdo, supponiamo $n^2 \in O(n)$. Allora esistono $c, n_0 > 0$ tali che $n^2 \leq c \cdot n$ per ogni $n \geq n_0$, cioè $n \leq c$ per ogni $n \geq n_0$. Ma questo è impossibile, perché n cresce senza limite. Contraddizione.

ESEMPIO 5.1.22 (Ordinamento per crescita asintotica). Ordinare le seguenti funzioni in ordine crescente di crescita asintotica:

$$2, \log n, (\log n)^2, \sqrt{n}, n, n \log n, n(\log n)^2, n^2, 2^n, 3^n, n! \quad (46)$$

Soluzione:

$$2 \prec \log n \prec (\log n)^2 \prec \sqrt{n} \prec n \prec n \log n \prec n(\log n)^2 \prec n^2 \prec 2^n \prec 3^n \prec n! \quad (47)$$

Dove $f \prec g$ significa $f(n) = o(g(n))$, ovvero $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

Classificazione per famiglie di crescita:

- **Costanti:** $2 = \Theta(1)$
- **Logaritmiche:** $\log n$
- **Polilogaritmiche:** $(\log n)^2$
- **Sublineari:** $\sqrt{n} = n^{1/2}$
- **Lineari:** n
- **Linearitriche:** $n \log n$
- **Superlinearitriche:** $n(\log n)^2$
- **Polinomiali:** n^2
- **Esponenziali:** $2^n \prec 3^n$ (base maggiore $\Rightarrow$ crescita più rapida)
- **Fattoriali:** $n!$ (cresce più rapidamente di qualsiasi c^n con c costante)

Nota. La notazione $\log^2 n$ si intende come $(\log n)^2$, non come $\log(\log n)$. Per evitare ambiguità, in queste dispense si usa sempre la scrittura estesa $(\log n)^2$.

5.2. Limiti inferiori alla difficoltà di un problema

La notazione asintotica ci consente di classificare la complessità dei singoli algoritmi. Un'ulteriore domanda fondamentale è: dato un problema, qual è il minimo costo necessario per risolverlo? Per rispondere, occorre stabilire dei **limiti inferiori**, cioè delle soglie al di sotto delle quali nessun algoritmo può scendere.

DEFINIZIONE 5.2.1 (Difficoltà di un problema). Dato un problema π , la **difficoltà** di π è la complessità al caso peggior del miglior algoritmo che risolve π , espressa in funzione della dimensione dell'input e in termini asintotici. Un algoritmo che risolve π con complessità $T(n)$ fornisce un **limite superiore** alla difficoltà di π .

DEFINIZIONE 5.2.2 (Limite inferiore). Una funzione $L(n)$ è un **limite inferiore** per il problema π se, per ogni algoritmo A che risolve π , la complessità al caso peggior di A soddisfa $T_{A(n)} \in \Omega(L(n))$.

In altre parole, $L(n)$ è il numero minimo di operazioni necessarie per risolvere π al caso peggiore.

Nota (Algoritmo ottimo). Un algoritmo è **ottimo** per il problema π se la sua complessità al caso peggior coincide (asintoticamente) con il limite inferiore. In tal caso il limite inferiore è detto **stretto**.

Esistono tre criteri principali per stabilire limiti inferiori.

5.2.1. Criterio della dimensione dell'input

Se la soluzione di un problema richiede necessariamente l'esame di tutti i dati in input, allora la dimensione dell'input n è un limite inferiore:

$$L(n) = \Omega(n) \quad (48)$$

ESEMPIO 5.2.3 (Ricerca del massimo). La ricerca del massimo in un vettore non ordinato deve necessariamente esaminare tutti gli n elementi: un elemento non esaminato potrebbe essere il massimo. Dunque $L(n) = \Omega(n)$.

Poiché l'algoritmo di scansione lineare ha complessità $\Theta(n)$, il limite inferiore è stretto e l'algoritmo è **ottimo**.

5.2.2. Criterio dell'albero di decisione

Questo criterio si applica a problemi risolvibili attraverso una sequenza di **decisioni binarie** (tipicamente confronti tra valori) che via via riducono lo spazio delle soluzioni possibili.

DEFINIZIONE 5.2.4 (Albero di decisione). Un **albero di decisione** per un problema π è un albero binario in cui:

- ogni **nodo interno** rappresenta un confronto (decisione);
- ogni **foglia** rappresenta una possibile soluzione;
- ogni **percorso radice-foglia** corrisponde a una possibile esecuzione dell'algoritmo.

Il **caso peggior** di un algoritmo basato su confronti corrisponde alla lunghezza del percorso più lungo dalla radice a una foglia, ossia all'**altezza** dell'albero di decisione.

DEFINIZIONE 5.2.5 (Altezza di un albero). L'altezza di un albero è la lunghezza (in numero di archi) del più lungo percorso dalla radice ad una foglia.

TEOREMA 5.2.6 (Limite inferiore dall'albero di decisione). Se $\text{SOL}(n)$ è il numero di soluzioni possibili per un'istanza di dimensione n del problema π , allora ogni albero di decisione per π ha almeno $\text{SOL}(n)$ foglie. Poiché un albero binario con F foglie ha altezza almeno $\log_2 F$, si ha:

$$L(n) = \Omega(\log_2(\text{SOL}(n))) \quad (49)$$

Dimostrazione. Un albero binario di altezza h ha al massimo 2^h foglie. L'albero di decisione deve avere almeno $\text{SOL}(n)$ foglie (una per ogni soluzione possibile). Dunque $2^h \geq \text{SOL}(n)$, da cui $h \geq \log_2(\text{SOL}(n))$. $\square$

Nota (Percorsi nell'albero di decisione). In un albero di decisione, il percorso più breve dalla radice a una foglia corrisponde al **caso ottimo**, mentre il percorso più lungo corrisponde al **caso pessimo**. Un **albero bilanciato** è un albero in cui il caso ottimo e il caso pessimo differiscono al più di una costante.

DEFINIZIONE 5.2.7 (Albero bilanciato). Un albero si dice **bilanciato** se, per ogni nodo, le altezze dei suoi sottoalberi differiscono al più di una costante. Come conseguenza, un albero bilanciato con n nodi ha altezza $\Theta(\log n)$.

Nota (Algoritmo ottimo per problemi basati su confronti). L'algoritmo ottimo al caso pessimo è quello che minimizza l'altezza dell'albero di decisione. Questo corrisponde a un albero bilanciato, con altezza $\Theta(\log(\text{SOL}(n)))$.

5.2.3. Criterio degli eventi contabili

Se la soluzione di un problema richiede che un certo **evento** si ripeta un numero minimo di volte, e ogni occorrenza ha un certo costo, allora si ottiene un limite inferiore:

$$L(n) = (\text{numero minimo di ripetizioni dell'evento}) \times (\text{costo per evento}) \quad (50)$$

ESEMPIO 5.2.8 (Ordinamento per confronti). Nell'ordinamento per confronti, l'evento elementare è il **confronto** tra due elementi (costo $\Theta(1)$). Le soluzioni possibili sono le $n!$ permutazioni dell'array. Dall'albero di decisione:

$$L(n) = \Omega(\log_2(n!)) = \Omega(n \log n) \quad (51)$$

dove l'ultima uguaglianza segue dall'approssimazione di Stirling: $\log(n!) = \Theta(n \log n)$.

Poiché il Merge Sort ha complessità $\Theta(n \log n)$, esso è un algoritmo **ottimo** per l'ordinamento basato su confronti.

5.3. Problem solving: ricerca del punto di transizione

In molti problemi su array, la struttura dell'input presenta una **proprietà** che vale fino a un certo indice i' e poi cambia. Questo schema ricorrente richiede due tipi di algoritmi:

1. **Verifica:** controllare che l'array soddisfi effettivamente la proprietà. Questo richiede tipicamente una scansione lineare $O(n)$.
2. **Ricerca del punto di transizione i' :** individuare l'indice in cui la proprietà cambia. Poiché la proprietà è **monotona** (vale per gli indici $< i'$ e non vale per gli indici $\geq i'$), si può usare una ricerca binaria adattata $O(\log n)$.

Entrambi gli algoritmi ammettono prove di **ottimalità** che sfruttano i criteri di limite inferiore.

5.3.1. Pattern generale

Sia $A[0..n-1]$ un array e sia P una proprietà strutturale che vale per le prime posizioni e poi cambia. Formalmente, esiste un indice i' ($0 \leq i' \leq n-1$) tale che:

- per $0 \leq j < i'$, la coppia $(A[j], A[j+1])$ soddisfa un certo predicato φ ;
- da $A[i']$ in poi, vale una condizione diversa (ad esempio, tutti gli elementi hanno lo stesso segno, la stessa parità, ecc.).

Verifica. Per verificare che un array soddisfi questa proprietà, occorre:

1. trovare il punto i' in cui il predicato φ smette di valere;
2. controllare che da i' in poi valga la seconda condizione.

Ricerca. Per trovare i' (assumendo che l'array soddisfi la proprietà), si osserva che il predicato definisce una partizione dell'array in due regioni contigue: la regione in cui φ vale e quella in cui non vale. Si tratta di trovare il **punto di transizione** di un predicato monotono — lo stesso schema della ricerca binaria.

5.3.2. Verifica in tempo lineare

Verifica proprietà di alternanza

```
// Verifica che A alterni una proprietà P tra coppie consecutive
// fino a un indice i', poi mantenga P costante.
// prop(x) restituisce la proprietà dell'elemento (es. segno, parità)
Verifica(A[0..n-1]):
    n = length(A)
    if n == 1:
        return true
    j = 0
    // Fase 1: trova dove smette di alternare
    while j <= n-2 and prop(A[j]) != prop(A[j+1]):
        j := j + 1
    if j >= n-1:
        return true // alterna fino alla fine, i' = n-1
    // Fase 2: verifica che da j+1 in poi sia costante
    s = prop(A[j+1])
    t = j + 1
    while t <= n-1:
        if prop(A[t]) != s:
            return false
        t := t + 1
    return true
```

La complessità è $\Theta(n)$ nel caso pessimo: l'algoritmo visita ogni elemento al più una volta.

TEOREMA 5.3.1 (Ottimalità della verifica). *La verifica di una proprietà strutturale su un array richiede $\Omega(n)$ nel caso pessimo.*

Dimostrazione. Si usa il **criterio della dimensione dell'input** (o *fooling argument*): un avversario può costruire due istanze che differiscono solo nell'ultimo elemento — una che soddisfa la proprietà e una che non la soddisfa. Qualsiasi algoritmo che non esamina l'ultimo elemento non può distinguerle. Dunque ogni algoritmo corretto deve ispezionare $\Omega(n)$ elementi nel caso pessimo. $\square$

5.3.3. Ricerca del punto di transizione in tempo logaritmico

Se **assumiamo** che l'array soddisfi la proprietà, la ricerca di i' diventa una ricerca binaria sul predicato monotono: «la coppia $(A[q], A[q + 1])$ ha la stessa proprietà?»

Ricerca del punto di transizione

```
// Trova i' assumendo che A soddisfi la proprietà.
// prop(x) restituisce la proprietà dell'elemento
Transizione(A, p, r):
    if p == r:
        return p
    q = (p + r) / 2
    if prop(A[q]) == prop(A[q+1]):
        // q e q+1 nella zona costante => i' è a sinistra
        return Transizione(A, p, q)
    else:
        // q e q+1 ancora alternano => i' è a destra
        return Transizione(A, q+1, r)
```

La chiamata iniziale è `Transizione(A, 0, n-1)`.

La ricorrenza associata è $T(n) = T(n/2) + \Theta(1)$, da cui $T(n) = \Theta(\log n)$, come nella ricerca binaria classica.

TEOREMA 5.3.2 (Ottimalità della ricerca del punto di transizione). *La ricerca del punto di transizione di un predicato monotono su n posizioni richiede $\Omega(\log n)$ confronti nel caso pessimo.*

Dimostrazione. Si applica il **criterio dell'albero di decisione**. Il punto di transizione i' può trovarsi in una qualsiasi delle n posizioni, quindi $\text{SOL}(n) = n$. L'albero di decisione ha almeno n foglie, dunque ha altezza almeno $\log_2 n$:

$$L(n) = \Omega(\log_2 n) \quad (52)$$

Poiché l'algoritmo proposto raggiunge questo limite, esso è **ottimo**. □

ESEMPIO 5.3.3 (Alternanza di segno). Sia A un array di interi non nulli che alterna segno positivo e negativo fino a un indice i' , da cui tutti gli elementi mantengono il segno di $A[i']$.

Con $A = [-2, 1, -1, 5, 7, 3]$, si ha $i' = 3$ poiché $(-2, 1, -1)$ alternano segno e da $A[3] = 5$ in poi tutti sono positivi.

Verifica ($\Theta(n)$): si usa `prop(x) = sign(x)` con `sign(x) = "+" "se" x > 0, "-" "altrimenti"`. L'algoritmo scorre l'array verificando che i segni alternino fino a un punto, e siano costanti dopo.

Ricerca di i' ($\Theta(\log n)$): si usa la ricerca binaria. Al passo corrente con indici $[p, r]$:

- si calcola $q = (p + r) / 2$;
- se `sign(A[q]) == sign(A[q+1])`, siamo nella zona costante: i' è in $[p, q]$;
- altrimenti, siamo ancora nella zona alternante: i' è in $[q + 1, r]$.

ESEMPIO 5.3.4 (Alternanza di parità). Sia A un array di interi che alterna elementi pari e dispari fino a un indice i' , da cui tutti gli elementi mantengono la parità di $A[i']$.

Con $A = [0, 3, 2, 8, 4, 10]$, si ha $i' = 2$ poiché $(0, 3, 2)$ alternano parità e da $A[2] = 2$ in poi tutti sono pari.

Si usa `prop(x) = x mod 2`. La struttura degli algoritmi è identica al caso precedente: cambia solo la funzione `prop`.

Nota (Lo schema generale degli esercizi di problem solving). In un esercizio di problem solving su array, la traccia tipica chiede:

1. un **esempio** concreto di array con la proprietà;
2. lo **pseudocodice** di un algoritmo di verifica, la sua **complessità** e la sua **ottimalità**;
3. lo **pseudocodice** di un algoritmo di ricerca (assumendo la proprietà), la sua **complessità** e la sua **ottimalità**.

Per la verifica, l'ottimalità si dimostra con il **fooling argument** (criterio della dimensione dell'input): un avversario che modifica l'ultimo elemento forza $\Omega(n)$.

Per la ricerca, l'ottimalità si dimostra con l'**albero di decisione**: n soluzioni possibili implicano $\Omega(\log n)$ confronti.

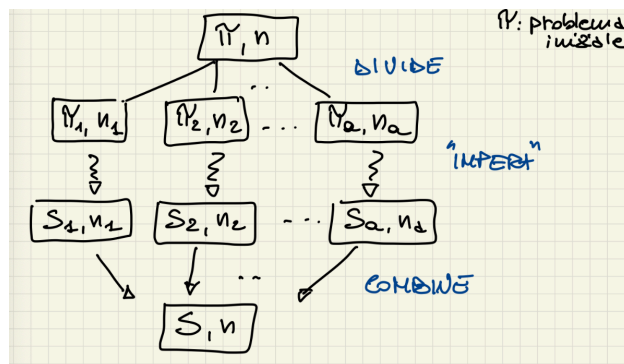
CAPITOLO 6

Divide et Impera

6.1. Il paradigma Divide et Impera

Il **Divide et Impera** (dal latino *divide et impera*) è uno dei paradigmi algoritmici fondamentali. Alla base della sua struttura vi sono tre fasi:

1. **Divide**: si suddivide il problema originale in due o più sotto-problemi *dello stesso tipo*, ciascuno operante su un sottoinsieme dei dati originali. La dimensione di ciascun sotto-problema è strettamente minore di quella del problema di partenza.
2. **Impera** (conquista): si risolvono i sotto-problemi in modo ricorsivo, applicando la stessa tecnica. Quando la dimensione del sotto-problema raggiunge un caso base (dimensione sufficientemente piccola), lo si risolve direttamente.
3. **Combina**: si fondono le soluzioni dei sotto-problemi per costruire la soluzione del problema originale.



La struttura generale di un algoritmo Divide et Impera è la seguente:

Schema generale Divide et Impera

```

soluzione DivideEtImpera(problema P) {
  if (P è un caso base) {
    return risolviDirettamente(P);
  }
  dividi P in sotto-problemi P1, P2, ..., Pa;
  soluzione s1 = DivideEtImpera(P1);
  soluzione s2 = DivideEtImpera(P2);
  ...
  soluzione sa = DivideEtImpera(Pa);
  return combina(s1, s2, ..., sa);
}

```

La complessità di un algoritmo Divide et Impera è descritta da una **relazione di ricorrenza** della forma:

$$T(n) = \underbrace{a}_{\text{num. sotto-problemi}} \cdot T(n/b) + \underbrace{f(n)}_{\text{costo divide + combina}} \quad (53)$$

dove $a \geq 1$ è il numero di sotto-problemi, $b > 1$ è il fattore di riduzione, e $f(n)$ rappresenta il costo delle fasi di divisione e combinazione.

6.2. Ricerca Binaria

La ricerca binaria è un algoritmo classico di tipo Divide et Impera per cercare un elemento k in un array $A[p..r]$ **ordinato**. L'idea è la seguente: si confronta k con l'elemento centrale dell'array; se k è uguale all'elemento centrale, la ricerca termina; altrimenti si prosegue ricorsivamente nella metà sinistra (se k è minore) o nella metà destra (se k è maggiore).

6.2.1. Implementazione e complessità

Input: array ordinato $A[p..r]$ di interi, elemento k da cercare.

Output: indice i tale che $A[i] = k$, oppure -1 se k non è presente.

Ricerca Binaria

```

int binarySearch(int[] A, int p, int r, int k){
    if(p > r){
        return -1;
    }
    if(p == r){
        if(A[p] == k){
            return p;
        } else {
            return -1;
        }
    }
    int q = (p + r) / 2;
    if(A[q] == k){
        return q;
    }
    if(A[q] > k){
        return binarySearch(A, p, q - 1, k);
    } else {
        return binarySearch(A, q + 1, r, k);
    }
}

```

Analizziamo la complessità. Sia $n = r - p + 1$ la dimensione del sotto-array corrente:

- **Divide:** il calcolo del punto medio q costa $\Theta(1)$.
- **Impera:** si effettua **una sola** chiamata ricorsiva su un sotto-array di dimensione al più $n/2$.
- **Combina:** il risultato della chiamata ricorsiva è già la risposta, quindi il costo di combinazione è $\Theta(1)$.

La relazione di ricorrenza è:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1 \\ T(n/2) + \Theta(1) & \text{se } n \geq 2 \end{cases} \quad (54)$$

dove $f(n) = D(n) + C(n) = \Theta(1)$ (costo di divide + combina).

TEOREMA 6.2.1 (Complessità della Ricerca Binaria). *La ricerca binaria su un array ordinato di n elementi ha complessità nel caso pessimo $\Theta(\log n)$ e nel caso ottimo $\Theta(1)$ (quando l'elemento cercato si trova esattamente nella posizione centrale al primo confronto).*

6.2.2. Correttezza

La correttezza di un algoritmo Divide et Impera si dimostra tipicamente per **induzione forte** sulla dimensione n del problema.

- **Caso base** ($n \leq 1$): se $p > r$ l'array è vuoto e si restituisce -1 ; se $p = r$ si confronta direttamente $A[p]$ con k . In entrambi i casi l'algoritmo è corretto.

- **Passo induttivo:** si assume, per **ipotesi induttiva**, che l'algoritmo sia corretto per ogni input di dimensione $n' < n$ (con $n' \geq 0$). Si deve dimostrare che è corretto per input di dimensione n . Poiché l'array è ordinato, dopo il confronto con $A[q]$:
 - se $A[q] = k$, l'indice q è restituito correttamente;
 - se $A[q] > k$, allora k può trovarsi solo in $A[p..q - 1]$ (dimensione $< n$), e per ipotesi induttiva la chiamata ricorsiva restituisce il risultato corretto;
 - se $A[q] < k$, il ragionamento è simmetrico su $A[q + 1..r]$.

6.2.3. Varianti della Ricerca Binaria

La ricerca binaria standard trova *una* occorrenza di un elemento, ma non garantisce quale (prima, ultima, o una qualsiasi). Le seguenti varianti permettono di trovare specificamente la prima o l'ultima occorrenza.

6.2.3.1. Ricerca Binaria Sinistra

Trova la **prima occorrenza** (più a sinistra) di un elemento k in un array ordinato.

Ricerca Binaria Sinistra

```
int ricercaBinariaSx(int[] a, int sx, int dx, int k) {
    if (sx > dx) {
        return -1;
    }
    int cx = (sx + dx) / 2;
    if (a[cx] == k and (cx == sx or a[cx - 1] != k)) {
        return cx;
    }
    if (a[cx] >= k) {
        return ricercaBinariaSx(a, sx, cx - 1, k);
    } else {
        return ricercaBinariaSx(a, cx + 1, dx, k);
    }
}
```

Idea: quando troviamo k in posizione cx , verifichiamo se è la prima occorrenza controllando che:

- $cx = sx$ (siamo al bordo sinistro del sotto-array), oppure
- $a[cx - 1] \neq k$ (l'elemento precedente è diverso da k).

Se la condizione non è soddisfatta, k compare anche a sinistra di cx , quindi si prosegue la ricerca nella metà sinistra. La complessità resta $O(\log n)$.

6.2.3.2. Ricerca Binaria Destra

Trova l'**ultima occorrenza** (più a destra) di un elemento k in un array ordinato.

Ricerca Binaria Destra

```
int ricercaBinariaDx(int[] a, int sx, int dx, int k) {
    if (sx > dx) {
        return -1;
    }
    int cx = (sx + dx) / 2;
    if (a[cx] == k and (cx == dx or a[cx + 1] != k)) {
        return cx;
    }
    if (a[cx] <= k) {
        return ricercaBinariaDx(a, cx + 1, dx, k);
    } else {
        return ricercaBinariaDx(a, sx, cx - 1, k);
    }
}
```

Idea: simmetricamente alla variante sinistra, quando troviamo k in posizione cx , verifichiamo se è l'ultima occorrenza controllando che:

- $cx = dx$ (siamo al bordo destro del sotto-array), oppure
- $a[cx + 1] \neq k$ (l'elemento successivo è diverso da k).

Se la condizione non è soddisfatta, si prosegue nella metà destra. La complessità resta $O(\log n)$.

6.2.3.3. Conta Occorrenze

Combinando le due varianti si può contare il numero di occorrenze di k in un array ordinato in tempo $\Theta(\log n)$.

Conta Occorrenze

```
int contaOccorrenze(int[] a, int n, int k) {
    int prima = ricercaBinariaSx(a, 0, n - 1, k);
    if (prima == -1) {
        return 0;
    }
    int ultima = ricercaBinariaDx(a, 0, n - 1, k);
    return ultima - prima + 1;
}
```

Nota (Complessità di contaOccorrenze). Si eseguono al massimo due ricerche binarie, ciascuna con complessità $O(\log n)$: la complessità totale è quindi $\Theta(\log n)$. Un approccio lineare che scorre l'intero array richiederebbe $\Theta(n)$, dunque le varianti della ricerca binaria offrono un miglioramento significativo per array di grandi dimensioni.

6.3. Merge Sort

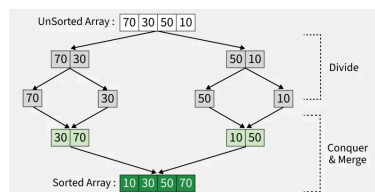
Il Merge Sort è un algoritmo di ordinamento basato sul paradigma Divide et Impera. L'idea è la seguente:

1. **Divide**: si divide l'array $A[p..r]$ in due metà $A[p..q]$ e $A[q + 1..r]$, dove $q = \lfloor (p + r)/2 \rfloor$.
2. **Impera**: si ordinano ricorsivamente le due metà.
3. **Combina**: si fondono (*merge*) le due metà ordinate in un unico array ordinato.

6.3.1. Implementazione

Merge Sort

```
mergeSort(int[] A, int p, int r){           // -- T(n)
    if(p < r){                             // -- Theta(1)
        int q = (p + r) / 2;               // divide -- Theta(1)
        mergeSort(A, p, q);                // impera -- T(n/2)
        mergeSort(A, q + 1, r);            // impera -- T(n/2)
        merge(A, p, q, r);                 // combina -- Theta(n)
    }
}
```



La procedura `merge` fonde due sotto-array ordinati $A[p..q]$ e $A[q + 1..r]$ in un unico sotto-array ordinato $A[p..r]$, utilizzando due array ausiliari L e R .

Procedura Merge

```

merge(int[] A, int p, int q, int r){
    int n1 = q - p + 1;
    int n2 = r - q;
    int[] L = new int[n1 + 1];
    int[] R = new int[n2 + 1];

    int i = 1;
    while(i <= n1){
        L[i] := A[p + i - 1];
        i := i + 1;
    }
    int j = 1;
    while(j <= n2){
        R[j] := A[q + j];
        j := j + 1;
    }
    L[n1 + 1] := +∞;
    R[n2 + 1] := +∞;

    i := 1;
    j := 1;
    int k = p;
    while(k <= r){
        if(L[i] <= R[j]){
            A[k] := L[i];
            i := i + 1;
        } else {
            A[k] := R[j];
            j := j + 1;
        }
        k := k + 1;
    }
}

```

Nota (Funzionamento di Merge). La procedura `merge` utilizza due **sentinelle** $+\infty$ alla fine degli array ausiliari L e R : quando un array ausiliario è stato completamente percorso, la sentinella garantisce che il confronto selezioni sempre l'elemento dall'altro array, senza necessità di controlli aggiuntivi sugli indici. Ogni iterazione del ciclo `while` copia esattamente un elemento in A , per un totale di $n_1 + n_2 = r - p + 1$ iterazioni. La complessità di `merge` è dunque $\Theta(n)$.

6.3.2. Correttezza di Merge

La correttezza della procedura `merge` si dimostra tramite la seguente **invariante di ciclo** sul ciclo `while(k <= r)`:

Invariante: all'inizio di ogni iterazione del ciclo, il sotto-array $A[p..k-1]$ contiene i $k-p$ elementi più piccoli di $L[1..n_1+1]$ e $R[1..n_2+1]$, ordinati. Inoltre, $L[i]$ e $R[j]$ sono i più piccoli elementi dei rispettivi array che non sono ancora stati copiati in A .

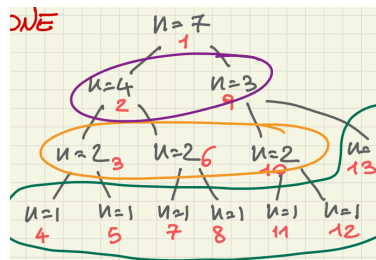
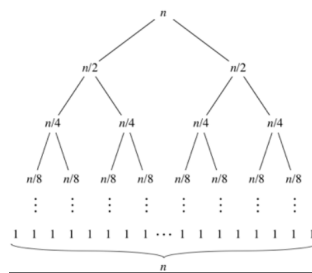
- **Inizializzazione:** prima della prima iterazione, $k = p$ e il sotto-array $A[p..k-1]$ è vuoto. Si ha $i = 1$ e $j = 1$, dunque $L[1]$ e $R[1]$ sono i più piccoli elementi non ancora copiati.
- **Conservazione:** supponiamo che l'invariante sia vera all'inizio di un'iterazione. Se $L[i] \leq R[j]$, allora $L[i]$ è il più piccolo elemento non ancora copiato in A ; viene posto in $A[k]$ e i viene incrementato. Il sotto-array $A[p..k]$ contiene ora $i - p + 1$ elementi più piccoli, ordinati. Il caso $L[i] > R[j]$ è simmetrico. In entrambi i casi, incrementando k si ripristina l'invariante.
- **Conclusione:** alla terminazione, $k = r + 1$. Per l'invariante, il sotto-array $A[p..r]$ contiene gli $r - p + 1$ elementi di L e R in ordine, escluse le sentinelle. Dunque la procedura `merge` è corretta.

6.3.3. Relazione di ricorrenza e complessità

La relazione di ricorrenza del Merge Sort è:

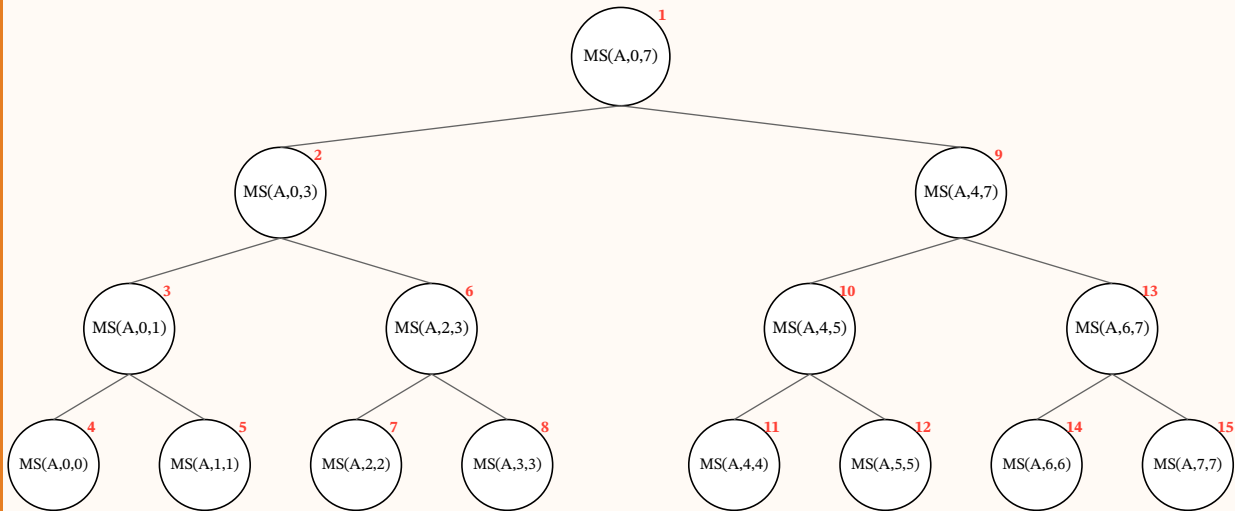
$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{se } n > 1 \end{cases} \quad (55)$$

Per comprendere intuitivamente la complessità, si può analizzare l'**albero di ricorsione**: a ciascun livello i dell'albero vi sono 2^i sotto-problemi, ciascuno di dimensione $n/2^i$. Il costo al livello i è $2^i \cdot c \cdot (n/2^i) = c \cdot n = \Theta(n)$. L'albero ha $\log_2 n$ livelli, dunque il costo totale è $\Theta(n \log n)$.



ESEMPIO 6.3.1 (Simulazione del Merge Sort). Simuliamo l'esecuzione di `mergeSort(A, 0, 7)` sull'array $A = [11, 7, 15, 19, 26, 9, 13, 6]$ con indici da 0 a $n - 1 = 7$.

1. Albero di ricorsione delle chiamate. Ogni nodo rappresenta una chiamata `mergeSort(A, p, r)`. I numeri in rosso indicano l'ordine in cui avvengono le chiamate ricorsive (visita in profondità, figlio sinistro prima):



L'albero ha $\log_2 8 = 3$ livelli (più il livello delle foglie). Le chiamate con $p = r$ (indici uguali) sono i **casi base**: sotto-array di un solo elemento, già ordinati.

2. Ordine delle chiamate. L'esecuzione segue una visita in **profondità** (*depth-first*), esplorando prima il sotto-albero sinistro:

- Si scende completamente a sinistra: **#1** $\rightarrow$ **#2** $\rightarrow$ **#3** $\rightarrow$ **#4** (caso base [11]), poi **#5** (caso base [7]). Dopo le chiamate **#4** e **#5**, si esegue `merge(A, 0, 0, 1)` che fonde [11] e [7] ottenendo [7, 11].
- Si risale ed esplora il fratello destro: **#6** $\rightarrow$ **#7** (caso base [15]), **#8** (caso base [19]). Poi `merge(A, 2, 2, 3)` produce [15, 19].
- Ritornati alla chiamata **#2**, si esegue `merge(A, 0, 1, 3)` che fonde [7, 11] e [15, 19] ottenendo [7, 11, 15, 19].
- Analogamente per il sotto-albero destro (chiamate **#9–#15**), producendo [6, 9, 13, 26].
- Infine, `merge(A, 0, 3, 7)` fonde le due metà ordinate: [6, 7, 9, 11, 13, 15, 19, 26].

3. Quando avvengono le operazioni di merge. Le chiamate a `merge` avvengono al **ritorno** dalla ricorsione, cioè dopo che entrambi i sotto-array sono stati ordinati:

Operazione	Fonde	Risultato
<code>merge(A, 0, 0, 1)</code>	[11] + [7]	[7, 11]
<code>merge(A, 2, 2, 3)</code>	[15] + [19]	[15, 19]
<code>merge(A, 0, 1, 3)</code>	[7, 11] + [15, 19]	[7, 11, 15, 19]
<code>merge(A, 4, 4, 5)</code>	[26] + [9]	[9, 26]
<code>merge(A, 6, 6, 7)</code>	[13] + [6]	[6, 13]
<code>merge(A, 4, 5, 7)</code>	[9, 26] + [6, 13]	[6, 9, 13, 26]
<code>merge(A, 0, 3, 7)</code>	[7, 11, 15, 19] + [6, 9, 13, 26]	[6, 7, 9, 11, 13, 15, 19, 26]

TEOREMA 6.3.2 (Complessità del Merge Sort). La complessità in tempo del Merge Sort è $\Theta(n \log n)$ in **tutti i casi** (ottimo, medio, pessimo). La complessità in spazio è $O(n)$, dovuta agli array ausiliari utilizzati dalla procedura `merge`.

Nota (Relazione di ricorrenza vs. complessità). La **relazione di ricorrenza** è la definizione matematica del costo di un algoritmo ricorsivo in funzione dell'input: descrive $T(n)$ in termini di T applicata a sotto-problemi più piccoli. La **complessità** è il risultato della risoluzione di tale relazione, espressa in notazione asintotica.

6.4. Relazioni di ricorrenza

Le relazioni di ricorrenza sono lo strumento matematico per descrivere la complessità $T(n)$ di algoritmi ricorsivi. Distinguiamo due forme principali.

6.4.1. Relazioni bilanciate

Una relazione di ricorrenza si dice **bilanciata** quando i sotto-problemi hanno tutti la stessa dimensione:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq n_0 \\ \underbrace{a}_{\text{sotto-problemi}} \cdot T(n/b) + \underbrace{f(n)}_{\text{forzante}} & \text{se } n > n_0 \end{cases} \quad (56)$$

dove $a \in \mathbb{N}^+$ (numero di sotto-problemi), $b > 1, b \in \mathbb{Q}$ (fattore di riduzione), e $f(n)$ è il termine **forzante** che rappresenta il costo delle fasi di divisione e combinazione.

La ricerca binaria ($a = 1, b = 2, f(n) = \Theta(1)$) e il Merge Sort ($a = 2, b = 2, f(n) = \Theta(n)$) sono esempi di relazioni bilanciate.

6.4.2. Relazioni di ordine k

Una relazione è di **ordine k** quando $T(n)$ dipende dai k valori precedenti:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq n_0 \\ \alpha_1 T(n-1) + \alpha_2 T(n-2) + \dots + \alpha_k T(n-k) + f(n) & \text{se } n > n_0 \end{cases} \quad (57)$$

Queste relazioni si risolvono con il **metodo dell'equazione caratteristica**. L'idea è la seguente: per la parte omogenea (cioè con $f(n) = 0$), si cerca una soluzione della forma $T(n) = x^n$. Sostituendo nella ricorrenza si ottiene:

$$x^n = \alpha_1 x^{n-1} + \alpha_2 x^{n-2} + \dots + \alpha_k x^{n-k} \quad (58)$$

Dividendo per x^{n-k} si ricava l'**equazione caratteristica**:

$$x^k - \alpha_1 x^{k-1} - \alpha_2 x^{k-2} - \dots - \alpha_k = 0 \quad (59)$$

Se le k radici $x_1, x_2, \dots, x_k$ sono distinte, la soluzione generale è:

$$T(n) = c_1 x_1^n + c_2 x_2^n + \dots + c_k x_k^n \quad (60)$$

dove le costanti $c_1, \dots, c_k$ si determinano dalle condizioni iniziali. Dal punto di vista asintotico, il termine dominante è quello con la radice di modulo massimo.

ESEMPIO 6.4.1 (Fibonacci come relazione di ordine 2). La successione di Fibonacci soddisfa $T(n) = T(n-1) + T(n-2)$, con $\alpha_1 = 1$ e $\alpha_2 = 1$.

L'equazione caratteristica è:

$$x^2 - x - 1 = 0 \quad (61)$$

Le radici sono:

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \quad \hat{\varphi} = \frac{1 - \sqrt{5}}{2} \approx -0.618 \quad (62)$$

La soluzione generale è $T(n) = c_1 \varphi^n + c_2 \hat{\varphi}^n$. Poiché $|\hat{\varphi}| < 1$, il termine $\hat{\varphi}^n \rightarrow 0$ e il termine dominante è φ^n , da cui:

$$T(n) = \Theta(\varphi^n) \approx \Theta(1.618^n) \quad (63)$$

La complessità della successione di Fibonacci è dunque **esponenziale**.

6.4.3. Metodi di risoluzione

Esistono quattro metodi principali per risolvere le relazioni di ricorrenza:

1. **Metodo iterativo:** si espande (srotola) la ricorrenza fino a raggiungere i casi base, poi si somma il lavoro a tutti i livelli.
2. **Metodo di sostituzione:** si ipotizza una soluzione e la si dimostra per induzione.
3. **Albero di ricorsione:** ausilio grafico che rappresenta i costi ai vari livelli della ricorsione; spesso usato per formulare un'ipotesi da verificare col metodo di sostituzione.
4. **Master Theorem:** formula chiusa applicabile alle relazioni bilanciate (vedi sezione successiva).

ESEMPIO 6.4.2 (Metodo di sostituzione: MergeSort). Il metodo di sostituzione consiste in due passi: (1) si **ipotizza** la forma della soluzione, (2) si usa l'**induzione matematica** per dimostrare che l'ipotesi è corretta e per trovare le costanti.

Consideriamo la ricorrenza $T(n) = 2T(n/2) + n$. Ipotizziamo che $T(n) = O(n \log n)$, cioè che esista una costante $c > 0$ tale che $T(n) \leq c \cdot n \cdot \log n$ per ogni $n \geq n_0$.

Passo induttivo: supponiamo $T(n/2) \leq c \cdot (n/2) \cdot \log(n/2)$ (ipotesi induttiva). Sostituendo nella ricorrenza:

$$\begin{aligned} T(n) &\leq 2 \cdot c \cdot (n/2) \cdot \log(n/2) + n \\ &= c \cdot n \cdot \log(n/2) + n \\ &= c \cdot n \cdot \log n - c \cdot n \cdot \log 2 + n \\ &= c \cdot n \cdot \log n - c \cdot n + n \\ &\leq c \cdot n \cdot \log n \end{aligned} \tag{64}$$

dove l'ultima disuguaglianza vale per $c \geq 1$.

Caso base: per $n = 1$ si ha $T(1) = \Theta(1)$, e dobbiamo verificare $T(1) \leq c \cdot 1 \cdot \log 1 = 0$, che non è soddisfatta. Tuttavia, possiamo scegliere $n_0 = 2$ come caso base: la condizione al contorno non influisce sul comportamento asintotico, purché la costante c sia sufficientemente grande. In particolare, basta scegliere $c \geq T(2)/(2 \cdot \log 2)$.

Si conclude che $T(n) = O(n \log n)$.

Nota (Insidie del metodo di sostituzione). Il metodo di sostituzione richiede di ipotizzare la forma esatta della soluzione, e un'ipotesi troppo debole (per esempio $T(n) \leq c \cdot n$) o un errore nel trattamento dei termini di ordine inferiore possono far fallire la dimostrazione. È fondamentale che l'ipotesi induttiva sia applicata *esattamente* nella stessa forma in cui si vuole dimostrare il risultato.

ESEMPIO 6.4.3 (Metodo iterativo: MergeSort). Consideriamo la ricorrenza del MergeSort: $T(n) = 2T(n/2) + n$ (con $T(1) = \Theta(1)$).

Si **srotola** (unrolling) la ricorrenza sostituendo ripetutamente la definizione di T :

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n \\ &= 4[2T(n/8) + n/4] + 2n = 8T(n/8) + 3n \\ &= \dots \\ &= 2^k T(n/2^k) + kn \end{aligned} \tag{65}$$

Il processo termina quando il sotto-problema raggiunge il caso base, cioè quando $n/2^k = 1$, ovvero $k = \log_2 n$. Sostituendo:

$$T(n) = 2^{\log_2 n} \cdot T(1) + n \log_2 n = n \cdot \Theta(1) + n \log n = \Theta(n \log n) \tag{66}$$

Il risultato coincide con quello ottenuto tramite il Master Theorem.

6.4.4. Metodo dell'albero di ricorsione

In un **albero di ricorsione** ogni nodo rappresenta il costo di un singolo sotto-problema nell'insieme delle chiamate ricorsive di funzione. Sommiamo i costi all'interno di ogni livello dell'albero per ottenere un insieme di costi per livello; poi sommiamo tutti i costi per livello per determinare il costo totale di tutti i livelli della ricorsione.

L'albero di ricorsione è particolarmente efficace in due situazioni:

1. quando si desidera una comprensione **visiva** della distribuzione del costo tra i livelli della ricorsione;
2. quando la ricorrenza non è nella forma standard $T(n) = aT(n/b) + f(n)$ e il Master Theorem non è applicabile (ad esempio ricorrenze con sotto-problemi di dimensioni diverse).

Il metodo si articola in tre passi:

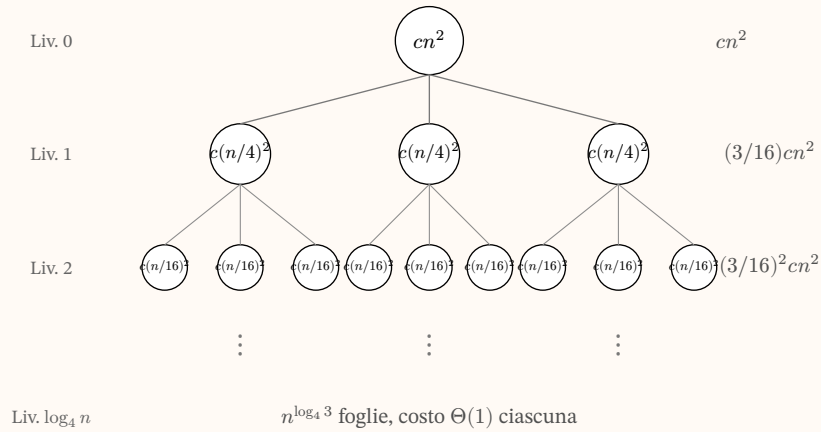
1. **Costruire l'albero:** la radice contiene il costo della forzante $f(n)$; ogni nodo genera tanti figli quante sono le chiamate ricorsive, ciascuno col costo del sotto-problema corrispondente.
2. **Sommare per livello:** si calcola il costo totale di ogni livello dell'albero.
3. **Sommare i livelli:** si sommano i costi di tutti i livelli, ottenendo una formula (spesso una serie geometrica) che fornisce un'ipotesi per $T(n)$.

L'ipotesi ottenuta dall'albero di ricorsione può essere usata direttamente come soluzione (quando i calcoli sono esatti), oppure può essere verificata rigorosamente col metodo di sostituzione.

ESEMPIO 6.4.4 (Albero di ricorsione: ricorrenza bilanciata). L'albero di ricorsione è particolarmente utile per formulare un'ipotesi sulla soluzione di ricorrenze per le quali il Master Theorem non è immediatamente applicabile, o per le quali si desidera una comprensione visiva del costo.

Consideriamo la ricorrenza $T(n) = 3T(n/4) + cn^2$.

Si costruisce l'albero ponendo alla radice il costo cn^2 . Ogni nodo genera 3 figli, ciascuno con un sotto-problema di dimensione $n/4$:



- **Livello 0** (radice): un nodo con costo cn^2 . Costo totale: cn^2 .
- **Livello 1:** 3 nodi, ciascuno con costo $c(n/4)^2 = cn^2/16$. Costo totale: $3 \cdot cn^2/16 = (3/16) \cdot cn^2$.
- **Livello 2:** $3^2 = 9$ nodi, ciascuno con costo $c(n/16)^2 = cn^2/256$. Costo totale: $9 \cdot cn^2/256 = (3/16)^2 \cdot cn^2$.
- **Livello i :** 3^i nodi, ciascuno con costo $c(n/4^i)^2$. Costo totale: $(3/16)^i \cdot cn^2$.

L'albero ha $\log_4 n$ livelli (i sotto-problemi raggiungono dimensione 1 quando $n/4^i = 1$). Le foglie al livello $\log_4 n$ sono $3^{\log_4 n} = n^{\log_4 3}$, ciascuna con costo $\Theta(1)$.

Il costo totale è la somma su tutti i livelli:

$$T(n) = \sum_{i=0}^{\log_4 n - 1} (3/16)^i \cdot cn^2 + \Theta(n^{\log_4 3}) \quad (67)$$

Poiché $3/16 < 1$, la serie geometrica converge:

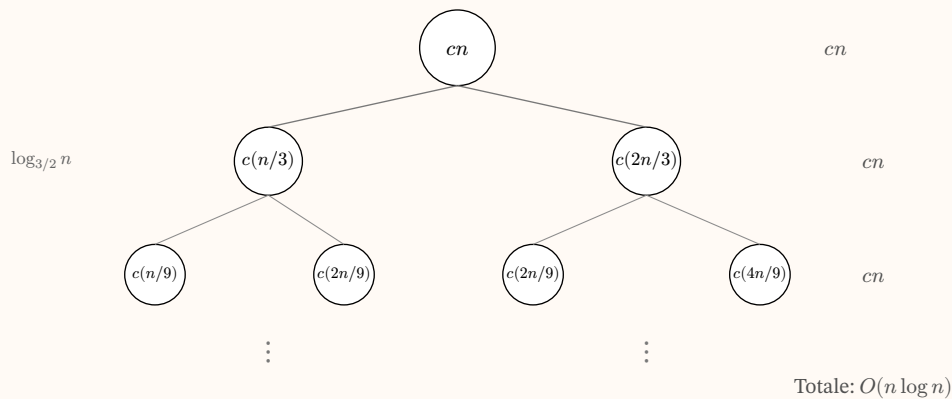
$$\sum_{i=0}^{\infty} (3/16)^i = \frac{1}{1 - 3/16} = 16/13 \quad (68)$$

Dunque il costo dei nodi interni è limitato superiormente da $(16/13) \cdot cn^2 = O(n^2)$. Poiché $\log_4 3 \approx 0.793 < 2$, il costo delle foglie $\Theta(n^{\log_4 3})$ è di ordine inferiore. Si ottiene $T(n) = \Theta(n^2)$.

Questa ipotesi può essere verificata rigorosamente con il metodo di sostituzione, oppure confermata direttamente applicando il Master Theorem (Caso 3, con $a = 3$, $b = 4$, $f(n) = cn^2$, $c_{\text{crit}} = \log_4 3 \approx 0.793$).

ESEMPIO 6.4.5 (Albero di ricorsione: ricorrenza con sotto-problemi di dimensioni diverse). Consideriamo la ricorrenza $T(n) = T(n/3) + T(2n/3) + cn$, dove $c > 0$ è una costante. Questa ricorrenza **non** ha la forma standard $aT(n/b) + f(n)$ perché i due sotto-problemi hanno dimensioni diverse ($n/3$ e $2n/3$), quindi il Master Theorem non è direttamente applicabile. L'albero di ricorsione è lo strumento ideale.

Costruiamo l'albero. La radice ha costo cn . Il figlio sinistro ha costo $c(n/3)$ e il figlio destro ha costo $c(2n/3)$:



Analisi per livello. Al livello 0 il costo è cn . Al livello 1 il costo è $c(n/3) + c(2n/3) = cn$. Al livello 2, sommando i quattro nodi: $c(n/9) + c(2n/9) + c(2n/9) + c(4n/9) = cn$. Ogni livello contribuisce esattamente cn al costo totale.

Altezza dell'albero. Il cammino più lungo dalla radice a una foglia è quello che segue sempre il figlio destro (sotto-problema di dimensione $2n/3$): $n \rightarrow (2/3)n \rightarrow (2/3)^2 n \rightarrow \dots \rightarrow 1$. Poiché $(2/3)^k n = 1$ quando $k = \log_{3/2} n$, l'altezza dell'albero è $\log_{3/2} n$.

Non tutti i livelli dell'albero contribuiscono con costo esattamente cn : i livelli più bassi hanno meno nodi, poiché il ramo sinistro ($n/3$) raggiunge le foglie prima del ramo destro ($2n/3$). Tuttavia, poiché il costo per livello è **al più** cn e ci sono $\log_{3/2} n$ livelli, possiamo concludere:

$$T(n) = O(n \log_{3/2} n) = O(n \log n) \quad (69)$$

dove l'ultimo passaggio segue dal fatto che $\log_{3/2} n = (\log n)/(\log 3/2) = \Theta(\log n)$.

Verifica con il metodo di sostituzione. Dimostriamo che $T(n) \leq dn \log n$ per un'opportuna costante $d > 0$. Per $d \geq c/(\log 3 - 2/3)$, si ha:

$$\begin{aligned} T(n) &\leq T(n/3) + T(2n/3) + cn \\ &\leq d(n/3) \log(n/3) + d(2n/3) \log(2n/3) + cn \\ &= d(n/3)(\log n - \log 3) + d(2n/3)(\log n - \log(3/2)) + cn \\ &= dn \log n - dn(1/3 \cdot \log 3 + 2/3 \cdot \log(3/2)) + cn \\ &\leq dn \log n \end{aligned} \quad (70)$$

dove l'ultima disuguaglianza vale scegliendo d sufficientemente grande affinché $d(1/3 \cdot \log 3 + 2/3 \cdot \log(3/2)) \geq c$, che è sempre possibile poiché la quantità tra parentesi è una costante positiva.

Nota (Quando usare l'albero di ricorsione). L'albero di ricorsione è il metodo di elezione quando:

- i sotto-problemi hanno **dimensioni diverse** (es. $T(n) = T(n/3) + T(2n/3) + cn$), rendendo inapplicabile il Master Theorem;
- si desidera **intuire** la soluzione prima di dimostrarla formalmente;
- si vuole capire **quale parte domina** il costo: le foglie (ricorsione pesante) o la radice (forzante pesante).

Per ricorrenze bilanciate nella forma $T(n) = aT(n/b) + f(n)$, il Master Theorem è più rapido e diretto.

6.5. Master Theorem

Il Master Theorem fornisce una soluzione in forma chiusa per le relazioni di ricorrenza bilanciate.

TEOREMA 6.5.1 (Master Theorem). Sia data la relazione di ricorrenza:

$$T(n) = aT(n/b) + f(n) \quad (71)$$

con $a \geq 1$, $b > 1$ e $f(n) > 0$ definitivamente. Sia $c_{\text{crit}} = \log_b a$. Allora:

1. **Caso 1** – $f(n)$ cresce più lentamente di $n^{c_{\text{crit}}}$:
Se $\exists \varepsilon > 0 : f(n) = O(n^{\log_b a - \varepsilon})$, allora $T(n) = \Theta(n^{\log_b a})$.
2. **Caso 2** – $f(n)$ cresce come $n^{c_{\text{crit}}}$ (a meno di fattori logaritmici):
Se $f(n) = \Theta(n^{\log_b a})$, allora $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.
3. **Caso 3** – $f(n)$ cresce più velocemente di $n^{c_{\text{crit}}}$:
Se $\exists \varepsilon > 0 : f(n) = \Omega(n^{\log_b a + \varepsilon})$ e inoltre $\exists c < 1 : a \cdot f(n/b) \leq c \cdot f(n)$ (condizione di regolarità), allora $T(n) = \Theta(f(n))$.

Dimostrazione. La dimostrazione si basa sull'analisi dell'**albero di ricorsione**. Consideriamo la ricorrenza $T(n) = aT(n/b) + f(n)$.

Struttura dell'albero. L'albero di ricorsione ha:

- **Radice** con costo $f(n)$.
- Ogni nodo interno ha esattamente a figli.
- Al livello i , ci sono a^i nodi, ciascuno con un sotto-problema di dimensione n/b^i .
- Il costo al livello i è $a^i \cdot f(n/b^i)$.
- L'albero ha $\log_b n$ livelli (il sotto-problema raggiunge dimensione 1 quando $n/b^i = 1$, cioè $i = \log_b n$).
- Le foglie sono $a^{\log_b n} = n^{\log_b a}$, ciascuna con costo $\Theta(1)$.

Costo totale. Il costo complessivo è la somma su tutti i livelli:

$$T(n) = \underbrace{n^{\log_b a} \cdot \Theta(1)}_{\text{foglie}} + \underbrace{\sum_{i=0}^{\log_b n - 1} a^i \cdot f(n/b^i)}_{\text{nodi interni}} \quad (72)$$

Poniamo $g(n) = \sum_{i=0}^{\log_b n - 1} a^i \cdot f(n/b^i)$. Il comportamento di $g(n)$ dipende dal confronto tra $f(n)$ e $n^{\log_b a}$:

Caso 1 ($f(n) = O(n^{\log_b a - \epsilon})$): la somma $g(n)$ è dominata dalle foglie. Intuitivamente, scendendo nell'albero il costo per livello **cresce** (il fattore a^i cresce più velocemente di quanto $f(n/b^i)$ diminuisca). Il costo delle foglie $n^{\log_b a}$ domina la somma, e si ottiene $T(n) = \Theta(n^{\log_b a})$.

Caso 2 ($f(n) = \Theta(n^{\log_b a})$): ogni livello contribuisce approssimativamente lo stesso costo. Infatti:

$$a^i \cdot f(n/b^i) = a^i \cdot \Theta\left((n/b^i)^{\log_b a}\right) = a^i \cdot \Theta(n^{\log_b a} / a^i) = \Theta(n^{\log_b a}) \quad (73)$$

Ci sono $\log_b n$ livelli, ciascuno di costo $\Theta(n^{\log_b a})$, dunque $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.

Caso 3 ($f(n) = \Omega(n^{\log_b a + \epsilon})$ con condizione di regolarità): la somma $g(n)$ è dominata dalla radice. Scendendo nell'albero il costo per livello **decresce** geometricamente (la condizione di regolarità $a \cdot f(n/b) \leq c \cdot f(n)$ garantisce che $a^i \cdot f(n/b^i) \leq c^i \cdot f(n)$). La serie geometrica converge e si ottiene $T(n) = \Theta(f(n))$. $\square$

Nota (Caso 2 generalizzato). Il secondo caso ammette una formulazione più generale:

$$\exists k \geq 0 : f(n) = \Theta(n^{\log_b a} \cdot \log^k n) \Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) \quad (74)$$

Il caso standard corrisponde a $k = 0$.

Nota (Interpretazione intuitiva). Il Master Theorem confronta il lavoro svolto dalla forzante $f(n)$ con il costo intrinseco della ricorsione, misurato da $n^{\log_b a}$:

- **Caso 1:** la ricorsione domina: il costo totale è determinato dal numero di foglie dell'albero di ricorsione.
- **Caso 2:** forzante e ricorsione contribuiscono in modo bilanciato: il costo è amplificato da un fattore $\log n$ (uno per livello dell'albero).
- **Caso 3:** la forzante domina: il costo totale è determinato dal lavoro alla radice dell'albero.

6.5.1. Come applicare il Master Theorem

Nota (Procedimento).

1. Identificare i parametri a , b e $f(n)$.
2. Calcolare l'**esponente critico** $c_{\text{crit}} = \log_b a$.
3. Confrontare la crescita di $f(n)$ con $n^{c_{\text{crit}}}$:
 - se $f(n)$ cresce **polinomialmente più lentamente** di $n^{c_{\text{crit}}} \Rightarrow$ Caso 1;
 - se $f(n)$ cresce **allo stesso modo** (a meno di fattori logaritmici) $\Rightarrow$ Caso 2;
 - se $f(n)$ cresce **polinomialmente più velocemente** di $n^{c_{\text{crit}}} \Rightarrow$ Caso 3 (verificare la condizione di regolarità).

6.5.2. Esempi di applicazione

ESEMPIO 6.5.2 (Caso 2: Ricerca Binaria). $T(n) = T(n/2) + \Theta(1)$

Parametri: $a = 1, b = 2, f(n) = \Theta(1)$.

Esponente critico: $c_{\text{crit}} = \log_2 1 = 0$, dunque $n^{c_{\text{crit}}} = n^0 = 1$.

Confronto: $f(n) = \Theta(1) = \Theta(n^0) = \Theta(n^{\log_b a})$.

$\Rightarrow$ **Caso 2:** $T(n) = \Theta(n^0 \cdot \log n) = \Theta(\log n)$.

ESEMPIO 6.5.3 (Caso 2: Merge Sort). $T(n) = 2T(n/2) + \Theta(n)$

Parametri: $a = 2, b = 2, f(n) = \Theta(n)$.

Esponente critico: $c_{\text{crit}} = \log_2 2 = 1$, dunque $n^{c_{\text{crit}}} = n$.

Confronto: $f(n) = \Theta(n) = \Theta(n^{\log_b a})$.

$\Rightarrow$ **Caso 2:** $T(n) = \Theta(n \cdot \log n)$.

ESEMPIO 6.5.4 (Caso 1: Algoritmo ipotetico). $T(n) = 4T(n/2) + n$

Parametri: $a = 4, b = 2, f(n) = n$.

Esponente critico: $c_{\text{crit}} = \log_2 4 = 2$, dunque $n^{c_{\text{crit}}} = n^2$.

Confronto: $f(n) = n = O(n^{2-\varepsilon})$ con $\varepsilon = 1$. La forzante cresce più lentamente di n^2 .

$\Rightarrow$ **Caso 1:** $T(n) = \Theta(n^2)$.

Interpretazione: il costo è dominato dalle $4^{\log_2 n} = n^2$ foglie dell'albero di ricorsione.

ESEMPIO 6.5.5 (Caso 3: Algoritmo ipotetico). $T(n) = T(n/2) + n$

Parametri: $a = 1, b = 2, f(n) = n$.

Esponente critico: $c_{\text{crit}} = \log_2 1 = 0$, dunque $n^{c_{\text{crit}}} = 1$.

Confronto: $f(n) = n = \Omega(n^{0+\varepsilon})$ con $\varepsilon = 1$. La forzante cresce più velocemente di $n^0 = 1$.

Verifica condizione di regolarità: $a \cdot f(n/b) = 1 \cdot n/2 = n/2 \leq c \cdot n$ con $c = 1/2 < 1$. ✓

⇒ **Caso 3:** $T(n) = \Theta(n)$.

Interpretazione: il costo è dominato dal lavoro alla radice.

ESEMPIO 6.5.6 (Caso non coperto dal Master Theorem). $T(n) = 2T(n/2) + n \log n$

Parametri: $a = 2, b = 2, f(n) = n \log n$.

Esponente critico: $c_{\text{crit}} = \log_2 2 = 1$, dunque $n^{c_{\text{crit}}} = n$.

Si ha $f(n) = n \log n$, che cresce più di n ma **non polinomialmente** più di n (non esiste $\varepsilon > 0$ tale che $n \log n = \Omega(n^{1+\varepsilon})$). Non si applica il Caso 3.

Tuttavia, applicando il **Caso 2 generalizzato** con $k = 1$: $f(n) = \Theta(n \cdot \log^1 n)$, si ottiene $T(n) = \Theta(n \cdot \log^2 n)$.

CAPITOLO 7

Algoritmi di Ordinamento

7.1. Insertion Sort

L'Insertion Sort è un algoritmo di ordinamento iterativo. L'idea è quella di mantenere una porzione iniziale dell'array già ordinata e, ad ogni passo, inserire il prossimo elemento nella posizione corretta all'interno di tale porzione.

Input: array $A[1..n]$ di interi.

Output: A ordinato in modo non decrescente: $A[1] \leq A[2] \leq \dots \leq A[n]$.

InsertionSort

```
insertionSort(int[] A, int n){
    int j = 2;
    while(j <= n){
        int k = A[j];
        int i = j - 1;
        while((i > 0) && (A[i] > k)){
            A[i + 1] := A[i];
            i := i - 1;
        }
        A[i + 1] := k;
        j := j + 1;
    }
}
```

Funzionamento. All'iterazione j -esima del ciclo esterno, l'elemento $A[j]$ viene salvato in una variabile k (la *chiave*). Il ciclo interno scorre il sottoarray $A[1..j-1]$ da destra verso sinistra, spostando a destra tutti gli elementi maggiori di k . Quando si trova la posizione corretta (un elemento minore o uguale a k , oppure l'inizio dell'array), la chiave viene inserita.

ESEMPIO 7.1.1 (Esecuzione di Insertion Sort). Sia $A = [5, 2, 4, 6, 1, 3]$ con $n = 6$.

- $j = 2$: chiave $k = 2$. Confronto con $A[1] = 5 > 2$: spostato 5 a destra. Raggiunto l'inizio dell'array: inserisco k .
 $A = [2, 5, 4, 6, 1, 3]$
- $j = 3$: chiave $k = 4$. Confronto con $A[2] = 5 > 4$: spostato 5. Confronto con $A[1] = 2 \leq 4$: stop. Inserisco k .
 $A = [2, 4, 5, 6, 1, 3]$
- $j = 4$: chiave $k = 6$. Confronto con $A[3] = 5 \leq 6$: stop. Nessuno spostamento.
 $A = [2, 4, 5, 6, 1, 3]$
- $j = 5$: chiave $k = 1$. Spostato 6, 5, 4, 2 (tutti maggiori di 1). Raggiunto l'inizio: inserisco k .
 $A = [1, 2, 4, 5, 6, 3]$
- $j = 6$: chiave $k = 3$. Spostato 6, 5, 4. Confronto con $A[2] = 2 \leq 3$: stop. Inserisco k .
 $A = [1, 2, 3, 4, 5, 6]$

7.1.1. Dimostrazione di correttezza con invariante di ciclo

Per dimostrare che l'Insertion Sort produce effettivamente un array ordinato, utilizziamo la tecnica dell'**invariante di ciclo**.

Invariante: All'inizio dell'iterazione j -esima del ciclo esterno, il sottoarray $A[1..j-1]$ contiene gli stessi elementi che erano in $A[1..j-1]$ prima dell'esecuzione dell'algoritmo, disposti in ordine non decrescente.

Dimostrazione. Inizializzazione ($j = 2$):

Prima della prima iterazione, il sottoarray $A[1..1]$ contiene un solo elemento. Un singolo elemento è banalmente ordinato, e coincide con l'elemento originale. L'invariante è verificato.

Mantenimento: Supponiamo che l'invariante sia vero all'inizio dell'iterazione j -esima, ovvero $A[1..j-1]$ è ordinato e contiene gli elementi originali. Il corpo del ciclo:

1. Salva $A[j]$ nella variabile k .
2. Il ciclo interno sposta a destra gli elementi di $A[1..j-1]$ che sono maggiori di k , preservando il loro ordine relativo.
3. Inserisce k nella posizione corretta, cioè immediatamente dopo l'ultimo elemento $\leq k$.

Al termine, $A[1..j]$ contiene tutti gli elementi originali di $A[1..j]$ ed è ordinato. L'invariante vale quindi per $j+1$.

Terminazione ($j = n+1$):

Il ciclo termina quando $j = n+1$. Per l'invariante, $A[1..n]$ contiene gli elementi originali in ordine non decrescente. Questo è esattamente la specifica dell'output desiderato. $\square$

7.1.2. Analisi della complessità

Sia d_j il numero di volte in cui il ciclo interno viene eseguito (con guardia vera) per un dato valore di j . Al variare di j da 2 a n , il numero totale di confronti è:

$$C(n) = \sum_{j=2}^n d_j \quad (75)$$

- **Caso ottimo:** l'array è già ordinato. Per ogni j , la condizione $A[i] > k$ è subito falsa, dunque $d_j = 0$ spostamenti vengono effettuati. Tuttavia, anche quando $d_j = 0$, si esegue comunque un confronto: il test di guardia del ciclo interno ($A[i] > k$) che risulta immediatamente falso. Per questo il costo per ogni j è di 1 confronto. Il numero totale di confronti è:

$$C(n) = \sum_{j=2}^n 1 = n - 1 = \Theta(n) \quad (76)$$

La complessità è **lineare**.

- **Caso pessimo:** l'array è ordinato in ordine decrescente. Per ogni j , tutti gli elementi di $A[1..j-1]$ sono maggiori della chiave, e il ciclo interno esegue $j-1$ spostamenti. Il numero totale di confronti è:

$$C(n) = \sum_{j=2}^n (j-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2) \quad (77)$$

La complessità è **quadratica**.

In sintesi:

- **Caso ottimo:** $T(n) = \Theta(n)$ – lineare
- **Caso pessimo:** $T(n) = \Theta(n^2)$ – quadratico

L'Insertion Sort è dunque un algoritmo efficiente su array quasi ordinati, ma inadeguato per array in ordine inverso o casuale di grandi dimensioni.

7.2. Selection Sort

Il Selection Sort è un algoritmo di ordinamento iterativo basato su un'idea diversa dall'Insertion Sort: ad ogni passo, si seleziona il minimo tra gli elementi non ancora ordinati e lo si colloca nella posizione corretta tramite uno scambio.

Input: array $A[1..n]$ di interi.

Output: A ordinato in modo non decrescente: $A[1] \leq A[2] \leq \dots \leq A[n]$.

SelectionSort

```

selectionSort(int[] A, int n){
    int i = 1;
    while(i <= n - 1){
        int min = i;
        int j = i + 1;
        while(j <= n){
            if(A[j] < A[min]){
                min := j;
            }
            j := j + 1;
        }
        // swap(A[i], A[min])
        int temp = A[i];
        A[i] := A[min];
        A[min] := temp;
        i := i + 1;
    }
}

```

Funzionamento. All'iterazione i -esima del ciclo esterno, il ciclo interno cerca l'indice dell'elemento minimo nel sottoarray $A[i..n]$. Una volta trovato, l'elemento minimo viene scambiato con $A[i]$. Dopo questa operazione, $A[1..i]$ contiene gli i elementi più piccoli dell'array, in ordine non decrescente.

ESEMPIO 7.2.1 (Esecuzione di Selection Sort). Sia $A = [5, 2, 4, 6, 1, 3]$ con $n = 6$.

- $i = 1$: cerco il minimo in $A[1..6]$. Minimo: $A[5] = 1$. Scambio $A[1]$ con $A[5]$.
 $A = [1, 2, 4, 6, 5, 3]$
- $i = 2$: cerco il minimo in $A[2..6]$. Minimo: $A[2] = 2$. Scambio $A[2]$ con se stesso.
 $A = [1, 2, 4, 6, 5, 3]$
- $i = 3$: cerco il minimo in $A[3..6]$. Minimo: $A[6] = 3$. Scambio $A[3]$ con $A[6]$.
 $A = [1, 2, 3, 6, 5, 4]$
- $i = 4$: cerco il minimo in $A[4..6]$. Minimo: $A[6] = 4$. Scambio $A[4]$ con $A[6]$.
 $A = [1, 2, 3, 4, 5, 6]$
- $i = 5$: cerco il minimo in $A[5..6]$. Minimo: $A[5] = 5$. Scambio $A[5]$ con se stesso.
 $A = [1, 2, 3, 4, 5, 6]$

Nota (Invariante del ciclo interno del Selection Sort). Il ciclo interno del Selection Sort mantiene la seguente invariante: all'inizio dell'iterazione j -esima del ciclo interno, $A[\text{min}]$ è il minimo di $A[i..j-1]$. Questo garantisce che, al termine del ciclo interno (quando $j = n+1$), $A[\text{min}]$ sia effettivamente il minimo dell'intero sottoarray $A[i..n]$.

7.2.1. Dimostrazione di correttezza con invariante di ciclo

Invariante: All'inizio dell'iterazione i -esima del ciclo esterno:

1. Il sottoarray $A[1..i - 1]$ è ordinato in modo non decrescente.
2. Ogni elemento di $A[1..i - 1]$ è minore o uguale ad ogni elemento di $A[i..n]$.

Dimostrazione. Inizializzazione ($i = 1$):

Il sottoarray $A[1..0]$ è vuoto. Entrambe le condizioni dell'invariante sono banalmente verificate (proprietà dell'insieme vuoto).

Mantenimento: Supponiamo l'invariante vero per i . Dimostriamo che vale per $i + 1$.

- Il ciclo interno esamina $A[i]$, $A[i + 1]$, ..., $A[n]$ e individua l'indice $\min$ dell'elemento minimo in questo sottoarray.
- Lo scambio tra $A[i]$ e $A[\min]$ colloca in posizione i il più piccolo elemento di $A[i..n]$.
- Per l'ipotesi induttiva (condizione 2), tutti gli elementi di $A[1..i - 1]$ sono $\leq$ ad ogni elemento di $A[i..n]$, e in particolare $\leq A[i]$ dopo lo scambio.
- Dunque $A[1..i]$ è ordinato e contiene gli i elementi più piccoli dell'array.
- Ogni elemento di $A[i + 1..n]$ è $\geq A[i]$ (perché $A[i]$ era il minimo di $A[i..n]$).

L'invariante vale per $i + 1$.

Terminazione ($i = n$):

Per l'invariante, $A[1..n - 1]$ è ordinato e contiene gli $n - 1$ elementi più piccoli. L'unico elemento rimasto, $A[n]$, è necessariamente il più grande. L'intero array è ordinato. $\square$

7.2.2. Analisi della complessità

Il ciclo interno, per un dato valore di i , esegue esattamente $n - i$ confronti. Il numero totale di confronti è:

$$C(n) = \sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \sum_{k=1}^{n-1} k = \frac{n(n - 1)}{2} = \Theta(n^2) \quad (78)$$

A differenza dell'Insertion Sort, il Selection Sort esegue sempre $\Theta(n^2)$ confronti, **indipendentemente dall'input**. Il caso ottimo, pessimo e medio coincidono:

$$T(n) = \Theta(n^2) \quad (79)$$

Nota (Confronto tra Insertion Sort e Selection Sort).

- L'Insertion Sort ha complessità $\Theta(n)$ nel caso ottimo (array ordinato) e $\Theta(n^2)$ nel caso pessimo.
- Il Selection Sort ha complessità $\Theta(n^2)$ in tutti i casi.
- L'Insertion Sort è quindi preferibile quando l'input può essere parzialmente ordinato.
- Il Selection Sort ha il vantaggio di eseguire al più $n - 1$ scambi (utile quando le operazioni di scrittura sono costose).

Nota (Stabilità degli algoritmi di ordinamento). Un algoritmo di ordinamento si dice **stabile** se preserva l'ordine relativo degli elementi con la stessa chiave.

- **Insertion Sort è stabile:** il ciclo interno si arresta quando incontra un elemento $\leq k$ (la condizione è $A[i] > k$, strettamente maggiore). Pertanto, elementi uguali alla chiave non vengono spostati e mantengono la loro posizione relativa originale.
- **Selection Sort non è stabile:** lo scambio tra $A[i]$ e $A[\min]$ può alterare l'ordine relativo di elementi uguali. Ad esempio, con $A = [2_a, 2_b, 1]$, alla prima iterazione si scambia 2_a con 1, ottenendo $[1, 2_b, 2_a]$: l'ordine relativo di 2_a e 2_b è invertito.

7.3. Invariante di ciclo: schema generale

La tecnica dell'invariante di ciclo è uno strumento fondamentale per dimostrare la **correttezza** degli algoritmi iterativi. Formalizziamo la sua struttura.

DEFINIZIONE 7.3.1 (Invariante di ciclo). Un **invariante di ciclo** è una proprietà logica P tale che:

1. **Inizializzazione:** P è vera prima della prima iterazione del ciclo.
2. **Mantenimento:** se P è vera all'inizio di un'iterazione, allora è vera anche all'inizio dell'iterazione successiva.
3. **Terminazione:** quando il ciclo termina, l'invariante P , combinata con la condizione di uscita dal ciclo, implica la correttezza dell'algoritmo.

Nota (Analogia con l'induzione matematica). La struttura della dimostrazione con invariante di ciclo ricalca il **principio di induzione**:

- L'inizializzazione corrisponde al **caso base**.
- Il mantenimento corrisponde al **passo induttivo**.
- La terminazione sfrutta il fatto che il ciclo ha un numero finito di iterazioni (analogamente alla buona fondatezza dell'induzione sui naturali).

Nota (Schema pratico per le dimostrazioni). Per dimostrare la correttezza di un algoritmo iterativo con invariante di ciclo:

1. **Identificare l'invariante:** determinare quale proprietà rimane vera ad ogni iterazione.
2. **Inizializzazione:** verificare che l'invariante valga prima della prima iterazione.
3. **Mantenimento:** assumere che l'invariante valga all'iterazione k e dimostrare che vale per $k + 1$.
4. **Terminazione:** combinare l'invariante con la condizione di uscita dal ciclo per dedurre la correttezza del risultato.

7.4. QuickSort

Il **QuickSort** è un algoritmo di ordinamento basato sul paradigma Divide et Impera, inventato da Tony Hoare nel 1960. È uno degli algoritmi di ordinamento più utilizzati nella pratica grazie alla sua eccellente efficienza nel caso medio e al basso overhead di memoria, poiché opera **in loco** (in-place) senza richiedere array ausiliari.

L'idea chiave consiste nello scegliere un elemento detto **pivot** e nel partizionare l'array in due sottoinsiemi:

- tutti gli elementi $\leq$ pivot finiscono nella parte sinistra
- tutti gli elementi $>$ pivot finiscono nella parte destra

Successivamente si ordina ricorsivamente ciascuna delle due parti. A differenza del MergeSort, il lavoro principale avviene nella fase di **divide** (la procedura Partition), mentre la fase di **combine** è banale: una volta che le due parti sono ordinate, l'intero array è automaticamente ordinato.

7.4.1. Algoritmo

QuickSort

```
quickSort(int[] A, int p, int r){  
    if(p < r){  
        int q = partition(A, p, r);    // divide  
        quickSort(A, p, q - 1);        // impera (parte sinistra)  
        quickSort(A, q + 1, r);        // impera (parte destra)  
    }  
}
```

La chiamata iniziale è `quickSort(A, 1, n)` dove n è la lunghezza dell'array. Il caso base ($p \geq r$) corrisponde ad un sotto-array di zero o un elemento, che è già ordinato per definizione.

7.4.2. Procedura Partition

La procedura Partition riorganizza il sotto-array $A[p..r]$ **in loco** e restituisce un indice q tale che:

- $A[q]$ contiene il pivot nella sua posizione finale
- ogni elemento in $A[p..q - 1]$ è $\leq A[q]$
- ogni elemento in $A[q + 1..r]$ è $> A[q]$

Il pivot viene scelto come l'ultimo elemento $A[r]$. Si mantiene un indice i che delimita il confine della regione degli elementi $\leq x$: tutti gli elementi in $A[p..i]$ sono $\leq x$ e tutti quelli in $A[i + 1..j - 1]$ sono $> x$.

Partition

```

int partition(int[] A, int p, int r){
    int x = A[r];           // pivot (ultimo elemento)
    int i = p - 1;          // bordo della partizione sinistra
    int j = p;
    while(j <= r - 1){
        if(A[j] <= x){
            i := i + 1;
            // swap A[i] e A[j]
            int temp = A[i];
            A[i] := A[j];
            A[j] := temp;
        }
        j := j + 1;
    }
    // swap A[i+1] e A[r]: colloca il pivot nella posizione finale
    int temp = A[i + 1];
    A[i + 1] := A[r];
    A[r] := temp;
    return i + 1;
}

```

ESEMPIO 7.4.1 (Esecuzione di Partition). Consideriamo $A = \langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle$ con $p = 1$ e $r = 8$. Il pivot è $x = A[8] = 4$.

j	i	Stato di A
1	0	$\langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle - A[1] = 2 \leq 4$: $i := 1$, swap
2	1	$\langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle - A[2] = 8 > 4$: nessuno swap
3	1	$\langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle - A[3] = 7 > 4$: nessuno swap
4	1	$\langle 2, 1, 7, 8, 3, 5, 6, 4 \rangle - A[4] = 1 \leq 4$: $i := 2$, swap
5	2	$\langle 2, 1, 3, 8, 7, 5, 6, 4 \rangle - A[5] = 3 \leq 4$: $i := 3$, swap
6	3	$\langle 2, 1, 3, 8, 7, 5, 6, 4 \rangle - A[6] = 5 > 4$: nessuno swap
7	3	$\langle 2, 1, 3, 8, 7, 5, 6, 4 \rangle - A[7] = 6 > 4$: nessuno swap

Al termine del ciclo, $i = 3$. Si esegue lo swap di $A[4]$ con $A[8]$ (il pivot):

$$A = \langle 2, 1, 3, 4, 7, 5, 6, 8 \rangle \quad (80)$$

Partition restituisce $q = 4$. Il pivot 4 è nella sua posizione finale; tutti gli elementi a sinistra sono ≤ 4 e tutti quelli a destra sono > 4 .

7.4.3. Invariante di Partition

DEFINIZIONE 7.4.2 (Invariante del ciclo while in Partition). All'inizio di ogni iterazione del ciclo `while`, per l'indice corrente j :

1. Per ogni $k \in [p, i]$: $A[k] \leq x$ (regione degli elementi piccoli)
2. Per ogni $k \in [i + 1, j - 1]$: $A[k] > x$ (regione degli elementi grandi)
3. $A[r] = x$ (il pivot rimane in posizione r)

7.4.4. Correttezza di Partition

Dimostrazione. La dimostrazione procede verificando le tre proprietà dell'invariante (inizializzazione, conservazione, terminazione).

Inizializzazione ($j = p$): le regioni $A[p..i]$ e $A[i + 1..j - 1]$ sono vuote (poiché $i = p - 1$ e $j - 1 = p - 1$), quindi l'invariante è banalmente soddisfatto. Il pivot $A[r] = x$ è al suo posto.

Conservazione: supponiamo che l'invariante valga all'inizio dell'iterazione j -esima. Due casi:

- Se $A[j] > x$: si incrementa solo j . L'elemento $A[j]$ entra nella regione $A[i + 1..j - 1]$ degli elementi $> x$, preservando l'invariante.
- Se $A[j] \leq x$: si incrementa i e si scambia $A[i]$ con $A[j]$. L'elemento che era in $A[i]$ (che era $> x$) va in posizione j , estendendo la regione $> x$; l'elemento $A[j] \leq x$ va in posizione i , estendendo la regione $\leq x$.

Terminazione ($j = r$): l'invariante garantisce che $A[p..i]$ contiene elementi $\leq x$ e $A[i + 1..r - 1]$ contiene elementi $> x$. Lo swap finale di $A[i + 1]$ con $A[r]$ posiziona il pivot in $A[i + 1]$, ottenendo la partizione corretta. $\square$

7.4.5. Complessità di Partition

La procedura Partition esegue un singolo scorrimento dell'array da p a $r - 1$, effettuando un confronto per ogni elemento. Pertanto la sua complessità è:

$$T_{\text{partition}}(n) = \Theta(n) \quad (81)$$

dove $n = r - p + 1$ è il numero di elementi nel sotto-array.

7.5. Analisi di complessità di QuickSort

La complessità di QuickSort dipende interamente dal bilanciamento delle partizioni prodotte dalla scelta del pivot. La relazione di ricorrenza generale è:

$$T(n) = T(q - 1) + T(n - q) + \Theta(n) \quad (82)$$

dove q è l'indice restituito da Partition: la partizione sinistra $A[p..q - 1]$ contiene $q - 1$ elementi e quella destra $A[q + 1..r]$ ne contiene $n - q$.

7.5.1. Caso pessimo – $\Theta(n^2)$

Il caso pessimo si verifica quando le partizioni sono **massimamente sbilanciate** ad ogni chiamata ricorsiva: una partizione contiene $n - 1$ elementi e l'altra 0. Questo accade, ad esempio, quando l'array è già ordinato (o ordinato in modo decrescente) e il pivot è sempre l'ultimo (o il primo) elemento.

La ricorrenza diventa:

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n) \quad (83)$$

Svolgendo per sostituzione:

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta(n^2) \quad (84)$$

7.5.1.1. Dimostrazione formale: $T(n) = O(n^2)$ per sostituzione

Per dimostrare rigorosamente il limite superiore, si osserva che nel caso pessimo la ricorrenza è:

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \Theta(n) \quad (85)$$

dove q varia da 0 a $n - 1$ perché Partition genera due sottoproblemi con dimensione totale $n - 1$.

Supponiamo $T(n) \leq cn^2$ per qualche costante $c > 0$. Sostituendo nella ricorrenza:

$$T(n) \leq \max_{0 \leq q \leq n-1} (cq^2 + c(n - q - 1)^2) + \Theta(n) \quad (86)$$

L'espressione $q^2 + (n - q - 1)^2$ raggiunge il massimo agli estremi dell'intervallo $[0, n - 1]$ (la derivata seconda rispetto a q è positiva, quindi il punto critico è un minimo). Il massimo è $(n - 1)^2 \leq n^2 - 2n + 1$. Riprendendo:

$$T(n) \leq cn^2 - c(2n - 1) + \Theta(n) \leq cn^2 \quad (87)$$

purché c sia sufficientemente grande da far prevalere $c(2n - 1)$ sul termine $\Theta(n)$.

Analogamente si dimostra $T(n) = \Omega(n^2)$ nel caso sbilanciato, quindi $T(n) = \Theta(n^2)$ nel caso pessimo.

Nota (Worst case uguale a Insertion Sort). Nel caso pessimo, QuickSort ha la stessa complessità $\Theta(n^2)$ di Insertion Sort e Selection Sort. Tuttavia, questo caso è raro in pratica e può essere evitato con la randomizzazione del pivot.

7.5.2. Caso ottimo – $\Theta(n \log n)$

Il caso ottimo si verifica quando il pivot divide sempre l'array in due parti di **uguale dimensione** (o con al più un elemento di differenza):

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \quad (88)$$

Per il Master Theorem (caso 2 con $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$):

$$T(n) = \Theta(n \log n) \quad (89)$$

7.5.3. Caso medio – $\Theta(n \log n)$

TEOREMA 7.5.1 (Complessità nel caso medio di QuickSort). *Se tutte le permutazioni dell'input sono equiprobabili, la complessità attesa di QuickSort è $\Theta(n \log n)$.*

L'intuizione fondamentale è che anche partizioni **moderatamente sbilanciate** producono un albero di ricorsione di altezza $O(\log n)$. Ad esempio, anche con uno split costante 9:1 (ogni partizione divide gli elementi in proporzione 90%-10%), la ricorrenza:

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) \quad (90)$$

si risolve comunque in $\Theta(n \log n)$, poiché l'altezza dell'albero di ricorsione è $\log_{10/9} n = O(\log n)$.

Nella pratica, è estremamente improbabile che il pivot produca sempre partizioni degeneri. Il mix di «buone» e «cattive» partizioni che si presenta nel caso medio produce comunque un comportamento $\Theta(n \log n)$.

OSSERVAZIONE 7.5.2. Partizioni buone e cattive alternate. Supponiamo che le partizioni buone e cattive si alternino nei livelli dell'albero di ricorsione. Una partizione «cattiva» nella radice produce sottoproblemi di dimensione 0 e $n - 1$ con costo $\Theta(n)$; al livello successivo, una partizione «buona» divide $n - 1$ in $\frac{n-1}{2} - 1$ e $\frac{n-1}{2}$ con costo $\Theta(n - 1)$. Il costo combinato dei due livelli è $\Theta(n) + \Theta(n - 1) = \Theta(n)$, lo stesso costo di un singolo livello con partizione bilanciata che produce due sottoproblemi di dimensione $\frac{n-1}{2}$. Quindi il costo della partizione cattiva viene **assorbito** da quello della partizione buona, e il tempo complessivo resta $O(n \log n)$.

7.6. QuickSort Randomizzato

Per eliminare la dipendenza dal caso peggiore su input particolari (ad esempio, array già ordinati), si può **randomizzare la scelta del pivot**. Invece di scegliere sempre l'ultimo elemento, si sceglie un elemento casuale uniforme nell'intervallo $[p, r]$ e lo si scambia con $A[r]$ prima di chiamare Partition.

Randomized Partition e Randomized QuickSort

```

int randomizedPartition(int[] A, int p, int r){
    int i = random(p, r);      // indice casuale uniforme in [p, r]
    // swap A[i] e A[r]
    int temp = A[i];
    A[i] := A[r];
    A[r] := temp;
    return partition(A, p, r);
}

randomizedQuickSort(int[] A, int p, int r){
    if(p < r){
        int q = randomizedPartition(A, p, r);
        randomizedQuickSort(A, p, q - 1);
        randomizedQuickSort(A, q + 1, r);
    }
}

```

Nota (Vantaggi della randomizzazione). Con la randomizzazione, nessun input specifico può causare sistematicamente il caso pessimo. La complessità $\Theta(n^2)$ è ancora teoricamente possibile, ma la probabilità che si verifichi è trascurabile. Il tempo di esecuzione **atteso** è $O(n \log n)$ **per qualunque input**.

7.6.1. Analisi del numero atteso di confronti

Il tempo di esecuzione di QuickSort è dominato dal tempo impiegato nella procedura Partition. Ogni volta che viene chiamata Partition, viene selezionato un pivot che non sarà più incluso nelle successive chiamate ricorsive. Quindi ci possono essere al massimo n chiamate di Partition durante l'intera esecuzione dell'algoritmo. Ogni chiamata di Partition svolge una quantità costante di lavoro più un tempo proporzionale al numero di confronti effettuati nel ciclo.

LEMMA 7.6.1 (Tempo di esecuzione e confronti). Sia X il numero totale di confronti svolti in tutte le chiamate di Partition durante l'esecuzione di QuickSort su un array di n elementi. Allora il tempo di esecuzione di QuickSort è $O(n + X)$.

Dimostrazione. L'algoritmo effettua al massimo n chiamate di Partition, ciascuna delle quali svolge una quantità costante di lavoro e poi esegue il ciclo un certo numero di volte. Ogni iterazione del ciclo effettua esattamente un confronto. Sommando il lavoro costante ($O(n)$ in totale) e i confronti (X in totale), il tempo complessivo è $O(n + X)$. $\square$

Per analizzare formalmente il caso medio, si conta dunque il **numero atteso di confronti** eseguiti dall'algoritmo.

Siano $z_1, z_2, \dots, z_n$ gli elementi di A in ordine crescente di valore. Definiamo la variabile aleatoria indicatrice:

$$X_{ij} = \begin{cases} 1 & \text{se } z_i \text{ e } z_j \text{ vengono confrontati durante l'esecuzione} \\ 0 & \text{altrimenti} \end{cases} \quad (91)$$

Il numero totale di confronti è:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \quad (92)$$

TEOREMA 7.6.2 (Probabilità di confronto tra due elementi).

$$P(X_{ij} = 1) = \frac{2}{j - i + 1} \quad (93)$$

Giustificazione. Consideriamo l'insieme $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ di cardinalità $j - i + 1$. Gli elementi z_i e z_j vengono confrontati se e solo se uno dei due è il **primo** elemento di Z_{ij} ad essere scelto come pivot (in una qualche chiamata ricorsiva). Se venisse scelto come pivot un qualunque elemento z_k con $i < k < j$, allora z_i e z_j finirebbero in partizioni diverse e non sarebbero mai confrontati tra loro. Poiché ogni elemento di Z_{ij} ha la stessa probabilità di essere scelto per primo come pivot, la probabilità cercata è $\frac{2}{j-i+1}$.

Applicando la linearità del valore atteso:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \quad (94)$$

Con la sostituzione $k = j - i$:

$$E[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = 2(n-1)H_n = O(n \log n) \quad (95)$$

dove $H_n = \sum_{k=1}^n \frac{1}{k} = \Theta(\log n)$ è il numero armonico n -esimo.

7.7. Confronto degli algoritmi di ordinamento

Algoritmo	Caso ottimo	Caso medio	Caso pessimo	Atteso	In-place
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	–	Si
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	–	Si
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	–	No
QuickSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	–	Si
QuickSort Rand.	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n \log n)$	Si

Tabella 11: Confronto delle complessità in tempo degli algoritmi di ordinamento basati su confronti. La colonna «Atteso» indica la complessità attesa del QuickSort Randomizzato: il caso pessimo deterministico resta $\Theta(n^2)$ ma si verifica con probabilità trascurabile.

7.8. Heap e HeapSort

7.8.1. Definizione di Heap

DEFINIZIONE 7.8.1 (Albero binario quasi completo). Un albero binario è **quasi completo** (o *nearly complete*) se tutti i livelli sono completamente riempiti, eccetto eventualmente l'ultimo, che deve essere riempito da sinistra verso destra senza interruzioni.

DEFINIZIONE 7.8.2 (Heap binario). Un **heap binario** è un albero binario quasi completo che soddisfa la **proprietà di heap**:

- **Max-Heap:** per ogni nodo i diverso dalla radice vale $A[\text{Parent}(i)] \geq A[i]$, ossia ogni nodo ha chiave non superiore a quella del padre.
- **Min-Heap:** per ogni nodo i diverso dalla radice vale $A[\text{Parent}(i)] \leq A[i]$, ossia ogni nodo ha chiave non inferiore a quella del padre.

Nel seguito tratteremo esclusivamente il caso del max-heap; le considerazioni per il min-heap sono simmetriche.

Da queste definizioni seguono immediatamente alcune proprietà fondamentali.

TEOREMA 7.8.3 (Proprietà strutturali di un max-heap). Sia $A[1..n]$ un max-heap con n elementi. Allora:

1. L'elemento massimo si trova nella radice: $A[1] \geq A[i]$ per ogni $i \in \{1, \dots, n\}$.
2. L'altezza dell'heap è $h = \lfloor \log_2 n \rfloor$, dunque $h = \Theta(\log n)$.
3. Il numero di nodi a un'altezza h' (contata dalle foglie) è al massimo $\lceil n/2^{h'+1} \rceil$.
4. Le foglie occupano le posizioni $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$.

Dimostrazione. (1) Segue direttamente dalla proprietà di max-heap applicata ripetutamente lungo il cammino dalla radice a qualsiasi nodo.

(2) In un albero binario quasi completo con n nodi, il livello ℓ (con la radice al livello 0) contiene al massimo 2^ℓ nodi. Poiché tutti i livelli fino a $h - 1$ sono pieni, si ha $n \geq 2^h$, da cui $h \leq \log_2 n$. Inoltre $n < 2^{h+1}$, da cui $h > \log_2 n - 1$. Pertanto $h = \lfloor \log_2 n \rfloor$.

(3) Sia $h' \in \{0, 1, \dots, h\}$ un'altezza misurata dalle foglie. I nodi a altezza h' si trovano al livello $h - h'$. Poiché l'heap ha al massimo n nodi e il numero di nodi fino al livello $h - h' - 1$ vale $2^{h-h'} - 1$, i nodi al livello $h - h'$ sono al massimo $\lceil n/2^{h'+1} \rceil$.

(4) Un nodo in posizione i ha figlio sinistro in posizione $2i$. Se $2i > n$, allora il nodo non ha figli ed è dunque una foglia. Questo accade per $i > \lfloor n/2 \rfloor$. $\square$

7.8.2. Rappresentazione implicita come array

Un heap può essere memorizzato in modo compatto in un array $A[1..n]$, senza bisogno di puntatori espliciti. La radice si trova in $A[1]$ e, per un nodo in posizione i :

$$\text{Parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor, \quad \text{Left}(i) = 2i, \quad \text{Right}(i) = 2i + 1 \quad (96)$$

Navigazione nell'heap

```
int parent(int i){ return i / 2; }  
int left(int i)  { return 2 * i; }  
int right(int i) { return 2 * i + 1; }
```

Nota (Attributi dell'array). Un array A che rappresenta un heap possiede due attributi distinti:

- $A.length$: il numero totale di elementi allocati nell'array.
- $A.heap\text{-}size$: il numero di elementi dell'heap effettivamente memorizzati in A , con $0 \leq A.heap\text{-}size \leq A.length$.

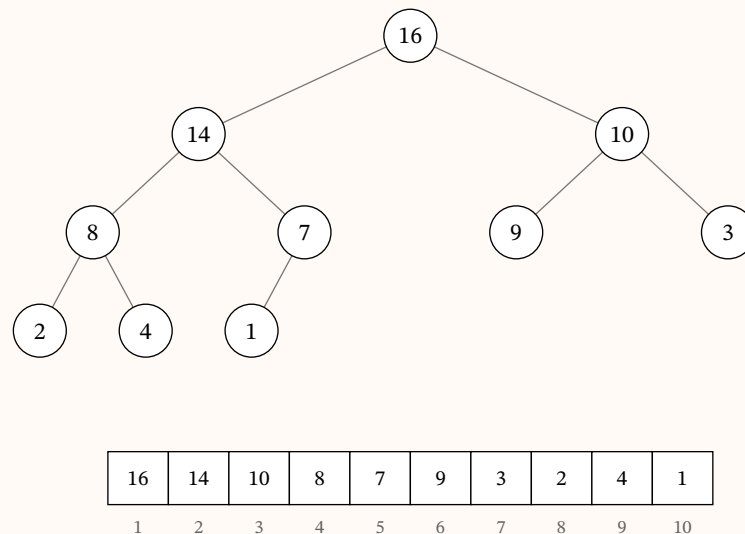
Solo gli elementi in $A[1..A.heap\text{-}size]$ sono elementi validi dell'heap. Le posizioni $A[A.heap\text{-}size + 1..A.length]$ possono contenere valori che non appartengono all'heap (ad esempio, gli elementi già estratti durante HeapSort).

Nota (Efficienza della rappresentazione). Questa rappresentazione implicita presenta diversi vantaggi:

- **Nessun puntatore:** la relazione padre-figlio è codificata dall'aritmetica sugli indici, con notevole risparmio di spazio.
- **Navigazione in $O(1)$:** le operazioni `parent`, `left` e `right` sono semplici divisioni e moltiplicazioni intere.
- **Località in memoria:** gli elementi sono memorizzati in posizioni contigue, il che favorisce le prestazioni della cache.

ESEMPIO 7.8.4 (Rappresentazione array di un max-heap). Consideriamo l'array $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ con $n = 10$.

Esso corrisponde al seguente max-heap (l'albero sopra e la rappresentazione array sotto):



Si può verificare che la proprietà di max-heap vale per ogni nodo: $A[\text{Parent}(i)] \geq A[i]$ per $i = 2, \dots, 10$.

7.8.3. Max-Heapify

La procedura `Max-Heapify` è il mattone fondamentale su cui si costruiscono tutte le operazioni sugli heap. Essa ripristina la proprietà di max-heap nel sottoalbero radicato in posizione i , sotto l'ipotesi che i sottoalberi sinistro e destro di i siano già max-heap validi. L'unica possibile violazione si trova dunque nel nodo i stesso.

L'idea è semplice: si confronta $A[i]$ con i suoi due figli; si scambia $A[i]$ con il **maggiore** dei due figli (se almeno uno è maggiore) e si prosegue ricorsivamente verso il basso (*sift-down*).

Max-Heapify

```

maxHeapify(int[] A, int i, int heapsize){
    int l = left(i);
    int r = right(i);
    int largest = i;

    // Trova il massimo tra A[i], A[l], A[r]
    if((l <= heapsize) && (A[l] > A[i])){
        largest := l;
    }

    if((r <= heapsize) && (A[r] > A[largest])){
        largest := r;
    }

    // Se il massimo non è il nodo corrente, scambia e ricorri
    if(largest != i){
        int temp = A[i];
        A[i] := A[largest];
        A[largest] := temp;
        maxHeapify(A, largest, heapsize);
    }
}

```

7.8.3.1. Correttezza di Max-Heapify

TEOREMA 7.8.5 (Correttezza di Max-Heapify). *Se i sottoalberi radicati in $\text{Left}(i)$ e $\text{Right}(i)$ sono max-heap, allora dopo la chiamata $\text{Max-Heapify}(A, i, \text{heapsize})$ il sottoalbero radicato in i è un max-heap.*

Dimostrazione. Procediamo per induzione sull'altezza h del nodo i nell'heap.

Caso base ($h = 0$): il nodo i è una foglia, non ha figli, e dunque è banalmente radice di un max-heap. La procedura non esegue alcuno scambio.

Passo induttivo ($h > 0$): supponiamo che Max-Heapify funzioni correttamente per ogni nodo di altezza minore di h . La procedura calcola l'indice `largest` del massimo tra $A[i]$, $A[l]$ e $A[r]$.

- Se $\text{largest} = i$, allora $A[i] \geq A[l]$ e $A[i] \geq A[r]$. Poiché i sottoalberi di l e r sono max-heap per ipotesi, il sottoalbero radicato in i è già un max-heap.
- Se $\text{largest} \neq i$, si scambia $A[i]$ con $A[\text{largest}]$. Dopo lo scambio, il nodo i soddisfa la proprietà di max-heap rispetto ai suoi figli. Il sottoalbero radicato in `largest` potrebbe però violare la proprietà, ma ha altezza al più $h - 1$. Per ipotesi induttiva, la chiamata ricorsiva $\text{Max-Heapify}(A, \text{largest}, \text{heapsize})$ ripristina la proprietà.

□

7.8.3.2. Complessità di Max-Heapify

TEOREMA 7.8.6 (Complessità di Max-Heapify). La procedura `Max-Heapify` su un nodo di altezza h ha complessità $O(h)$. Per un heap di n elementi, nel caso pessimo si ha:

$$T(n) = O(\log n) \quad (97)$$

Dimostrazione. A ogni chiamata ricorsiva, la procedura scende di un livello nell'heap. Il numero massimo di livelli che può attraversare è l'altezza h del nodo di partenza. Poiché ogni livello richiede lavoro $O(1)$ (un confronto e un eventuale scambio), il costo totale è $O(h)$.

L'altezza massima di un nodo nell'heap è $\lfloor \log_2 n \rfloor$, da cui $T(n) = O(\log n)$.

In alternativa, il risultato si ottiene mediante una ricorrenza sulla dimensione del sottoalbero. I sottoalberi dei figli di un nodo hanno ciascuno una dimensione che non supera $2n/3$ — il caso peggiore si verifica quando l'ultimo livello dell'albero è riempito esattamente a metà, cosicché il sottoalbero sinistro è il più grande possibile. Il tempo di esecuzione di `Max-Heapify` soddisfa dunque la ricorrenza:

$$T(n) \leq T(2n/3) + \Theta(1) \quad (98)$$

La soluzione, per il caso 2 del Teorema dell'esperto (con $a = 1$, $b = 3/2$, $f(n) = \Theta(1)$), è $T(n) = O(\log n)$. $\square$

ESEMPIO 7.8.7 (Max-Heapify passo-passo). Consideriamo l'array $A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ con $\text{heapsize} = 10$ e invochiamo `Max-Heapify(A, 2, 10)`.

Il nodo in posizione 2 ha valore 4. I sottoalberi sinistro (radice $A[4] = 14$) e destro (radice $A[5] = 7$) sono già max-heap, ma $A[2] = 4 < A[4] = 14$, quindi la proprietà di max-heap è violata in posizione 2.

Passo 1: confronto in posizione 2

- Nodo corrente: $A[2] = 4$
- Figlio sinistro: $A[4] = 14$
- Figlio destro: $A[5] = 7$
- $\text{largest} = 4$ (poiché $14 > 4$ e $14 > 7$)
- Scambio $A[2] \leftrightarrow A[4]$

Array dopo il passo 1: $[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$

Passo 2: chiamata ricorsiva in posizione 4

- Nodo corrente: $A[4] = 4$
- Figlio sinistro: $A[8] = 2$
- Figlio destro: $A[9] = 8$
- $\text{largest} = 9$ (poiché $8 > 4$ e $8 > 2$)
- Scambio $A[4] \leftrightarrow A[9]$

Array dopo il passo 2: $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

Passo 3: chiamata ricorsiva in posizione 9

- Nodo corrente: $A[9] = 4$
- $\text{Left}(9) = 18 > \text{heapsize}$: nessun figlio
- Il nodo è una foglia: la procedura termina

Risultato finale: $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

Il valore 4 è «sceso» dalla posizione 2 alla posizione 9, attraversando 2 scambi (altezza percorsa = 2).

7.8.4. Build-Max-Heap

Data un array arbitrario $A[1..n]$, la procedura `Build-Max-Heap` lo trasforma in un max-heap. L'idea è applicare `Max-Heapify` a tutti i nodi interni, procedendo dal basso verso l'alto (dalle foglie verso la radice). Poiché le foglie sono già heap banali (sottoalberi di un solo nodo), si parte dalla posizione $\lfloor n/2 \rfloor$ e si scende fino a 1.

Build-Max-Heap

```

buildMaxHeap(int[] A, int n){
    int heapsize = n;
    int i = n / 2;
    while(i >= 1){
        maxHeapify(A, i, heapsize);
        i := i - 1;
    }
}

```

Nota. Si parte da $i = \lfloor n/2 \rfloor$ e non da $i = n$ perché i nodi nelle posizioni $\lfloor n/2 \rfloor + 1, \dots, n$ sono foglie (non hanno figli) e sono pertanto max-heap banali di un singolo elemento. Iniziare da essi sarebbe corretto ma inutile.

7.8.4.1. Correttezza di Build-Max-Heap

DEFINIZIONE 7.8.8 (Invariante di ciclo per Build-Max-Heap). All'inizio di ogni iterazione del ciclo `while`, ogni nodo $j \in \{i + 1, i + 2, \dots, n\}$ è radice di un max-heap.

Dimostrazione. Inizializzazione: prima della prima iterazione si ha $i = \lfloor n/2 \rfloor$. I nodi $\lfloor n/2 \rfloor + 1, \dots, n$ sono foglie, dunque radici di max-heap banali. L'invariante vale.

Mantenimento: all'iterazione corrente, l'invariante garantisce che i figli del nodo i (che si trovano nelle posizioni $2i$ e $2i + 1$, entrambe $> i$) sono radici di max-heap. Questo è esattamente il prerequisito di `Max-Heapify(A, i, heapsize)`, che rende il sottoalbero radicato in i un max-heap. Dopo il decremento $i := i - 1$, l'invariante si estende al nodo $i + 1$ appena trattato.

Terminazione: il ciclo termina quando $i = 0$. A quel punto l'invariante afferma che ogni nodo $j \in \{1, 2, \dots, n\}$ è radice di un max-heap. In particolare, il nodo 1 (la radice dell'intero albero) è radice di un max-heap, il che significa che l'intero array è un max-heap. $\square$

7.8.4.2. Complessità di Build-Max-Heap**7.8.4.2.1. Analisi ingenua**

Si effettuano $O(n)$ chiamate a `Max-Heapify`, ciascuna di costo $O(\log n)$. Questo porta a un limite superiore di $O(n \log n)$, che è tuttavia non stretto.

7.8.4.2.2. Analisi stretta

TEOREMA 7.8.9 (Complessità di Build-Max-Heap). La procedura `Build-Max-Heap` ha complessità $\Theta(n)$.

Dimostrazione. L'altezza dell'heap è $h = \lfloor \log_2 n \rfloor$. Il costo di `Max-Heapify` su un nodo a altezza h' (misurata dalle foglie) è $O(h')$.

Per la proprietà (3) degli heap, a altezza h' vi sono al massimo $\lceil n/2^{h'+1} \rceil$ nodi. Sommando su tutte le altezze:

$$T(n) = \sum_{h'=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h'+1}} \right\rceil \cdot O(h') = O\left(n \sum_{h'=0}^{\lfloor \log_2 n \rfloor} \frac{h'}{2^{h'}}\right) \quad (99)$$

La serie $\sum_{h'=0}^{\infty} \frac{h'}{2^{h'}}$ converge e vale esattamente 2, essendo derivata della serie geometrica. Pertanto:

$$T(n) = O(n \cdot 2) = O(n) \quad (100)$$

Il limite inferiore $\Omega(n)$ segue banalmente dal fatto che ogni elemento dell'array deve essere esaminato almeno una volta. Dunque $T(n) = \Theta(n)$. $\square$

Nota (Intuizione della complessità lineare). Il risultato $\Theta(n)$ può sembrare sorprendente, poiché si eseguono $O(n)$ chiamate a `Max-Heapify`, ciascuna potenzialmente $O(\log n)$. Il punto chiave è che la *maggior parte* dei nodi ha altezza piccola: circa $n/2$ nodi sono foglie (altezza 0), circa $n/4$ hanno altezza 1, e così via. Solo un nodo (la radice) ha altezza $\lfloor \log n \rfloor$. Il lavoro effettivo è quindi dominato dai nodi bassi.

ESEMPIO 7.8.10 (Build-Max-Heap passo-passo). Costruiamo un max-heap dall'array $A = [7, 4, 3, 8, 9, 6]$ con $n = 6$.

Calcoliamo $\lfloor n/2 \rfloor = 3$: partiamo da $i = 3$ e procediamo fino a $i = 1$.

Stato iniziale:

Array: $[7, 4, 3, 8, 9, 6]$ (indici 1, ..., 6)

Iterazione $i = 3$: Max-Heapify(A, 3, 6)

- Nodo: $A[3] = 3$
- Figlio sinistro: $A[6] = 6$, figlio destro: non esiste
- $\text{largest} = 6$ (poiché $6 > 3$), scambio $A[3] \leftrightarrow A[6]$

Array: $[7, 4, 6, 8, 9, 3]$

Iterazione $i = 2$: Max-Heapify(A, 2, 6)

- Nodo: $A[2] = 4$
- Figlio sinistro: $A[4] = 8$, figlio destro: $A[5] = 9$
- $\text{largest} = 5$ (poiché $9 > 4$ e $9 > 8$), scambio $A[2] \leftrightarrow A[5]$
- Chiamata ricorsiva su posizione 5: è una foglia, termina

Array: $[7, 9, 6, 8, 4, 3]$

Iterazione $i = 1$: Max-Heapify(A, 1, 6)

- Nodo: $A[1] = 7$
- Figlio sinistro: $A[2] = 9$, figlio destro: $A[3] = 6$
- $\text{largest} = 2$ (poiché $9 > 7$ e $9 > 6$), scambio $A[1] \leftrightarrow A[2]$

Array intermedio: $[9, 7, 6, 8, 4, 3]$

- Chiamata ricorsiva su posizione 2:
 - Nodo: $A[2] = 7$
 - Figlio sinistro: $A[4] = 8$, figlio destro: $A[5] = 4$
 - $\text{largest} = 4$ (poiché $8 > 7$), scambio $A[2] \leftrightarrow A[4]$

Array: $[9, 8, 6, 7, 4, 3]$

- Chiamata ricorsiva su posizione 4: figli $A[8]$ e $A[9]$ non esistono, termina

Risultato finale: $A = [9, 8, 6, 7, 4, 3]$ – un max-heap valido.

Iterazione	i	Operazione	Array risultante
Iniziale	-	-	$[7, 4, 3, 8, 9, 6]$
1	3	scambio $3 \leftrightarrow 6$	$[7, 4, 6, 8, 9, 3]$
2	2	scambio $4 \leftrightarrow 9$	$[7, 9, 6, 8, 4, 3]$
3	1	scambio $7 \leftrightarrow 9$, poi $7 \leftrightarrow 8$	$[9, 8, 6, 7, 4, 3]$

7.8.5. HeapSort

L'algoritmo HeapSort sfrutta la struttura del max-heap per ordinare un array in loco.

7.8.5.1. Idea dell'algoritmo

1. Si costruisce un max-heap dall'array con `Build-Max-Heap`. A questo punto il massimo è in $A[1]$.
2. Si scambia $A[1]$ (il massimo) con l'ultimo elemento dell'heap $A[\text{heapsize}]$.
3. Si riduce `heapsize` di 1: l'elemento appena spostato in fondo si trova nella sua posizione finale.
4. Si invoca `Max-Heapify(A, 1, heapsize)` per ripristinare la proprietà di max-heap sulla parte rimanente.
5. Si ripetono i passi 2–4 finché `heapsize` vale 1.

HeapSort

```

heapsort(int[] A, int n){
    buildMaxHeap(A, n);
    int heapsize = n;
    int i = n;
    while(i >= 2){
        // swap(A[1], A[i]) - metti il massimo in fondo
        int temp = A[1];
        A[1] := A[i];
        A[i] := temp;

        heapsize := heapsize - 1;
        maxHeapify(A, 1, heapsize);
        i := i - 1;
    }
}

```

7.8.5.2. Correttezza di HeapSort

DEFINIZIONE 7.8.11 (Invariante di ciclo per HeapSort). All'inizio di ogni iterazione del ciclo `while` con indice i :

1. $A[1..i]$ è un max-heap contenente gli i elementi più piccoli di $A[1..n]$.
2. $A[i+1..n]$ contiene gli $n - i$ elementi più grandi di $A[1..n]$, in ordine crescente.

Dimostrazione. Inizializzazione: prima della prima iterazione, $i = n$. L'array $A[1..n]$ è un max-heap (appena costruito da `Build-Max-Heap`) e $A[n+1..n]$ è vuoto. L'invariante vale banalmente.

Mantenimento: all'inizio dell'iterazione con indice i , $A[1]$ contiene il massimo di $A[1..i]$ (proprietà del max-heap). Scambiandolo con $A[i]$, l'elemento più grande finisce in posizione i . Ora $A[i..n]$ contiene gli $n - i + 1$ elementi più grandi in ordine crescente. Dopo il decremento di `heapsize` e la chiamata a `Max-Heapify(A, 1, heapsize)`, il sotto-array $A[1..i-1]$ è nuovamente un max-heap. L'invariante vale con $i - 1$.

Terminazione: quando $i = 1$, l'invariante afferma che $A[2..n]$ contiene gli $n - 1$ elementi più grandi in ordine crescente, e $A[1]$ è il minimo. L'array è ordinato. $\square$

7.8.5.3. Complessità di HeapSort

TEOREMA 7.8.12 (Complessità di HeapSort). *HeapSort ha complessità $\Theta(n \log n)$ nel caso pessimo, ottimo e medio.*

Dimostrazione.

$$T(n) = \underbrace{\Theta(n)}_{\text{Build-Max-Heap}} + \underbrace{(n-1) \cdot O(\log n)}_{\text{ciclo principale}} = O(n \log n) \quad (101)$$

La chiamata a `Build-Max-Heap` costa $\Theta(n)$. Il ciclo esegue $n - 1$ iterazioni, ciascuna con una chiamata a `Max-Heapify` sulla radice di un heap di dimensione decrescente, per un costo di $O(\log n)$ per iterazione. Questo stabilisce il limite superiore $O(n \log n)$.

Per il limite inferiore, si osserva che qualsiasi algoritmo di ordinamento basato su confronti richiede $\Omega(n \log n)$ confronti nel caso pessimo (limite inferiore informazionale). Pertanto $T(n) = \Theta(n \log n)$. $\square$

Nota (Caratteristiche di HeapSort).

- **Complessità:** $\Theta(n \log n)$ nel caso pessimo, ottimo e medio. A differenza di QuickSort, non ha un caso pessimo $O(n^2)$.
- **In-place:** utilizza solo una quantità costante di memoria aggiuntiva (nessun array ausiliario).
- **Non stabile:** lo scambio tra radice e ultimo elemento può alterare l'ordine relativo di elementi con chiave uguale.
- Nella pratica, QuickSort è spesso più veloce di HeapSort a causa di una migliore località di cache, nonostante il caso pessimo peggiore.

ESEMPIO 7.8.13 (HeapSort passo-passo). Ordiniamo l'array $A = [7, 4, 3, 8, 9, 6]$ usando HeapSort.

Fase 1: Build-Max-Heap

Come mostrato nell'esempio precedente, dopo **Build-Max-Heap** si ottiene:

$$A = [9, 8, 6, 7, 4, 3] \text{ con } \text{heapsize} = 6$$

Fase 2: estrazione iterativa del massimo

Iter.	heapsize	Scambio	Dopo scambio	Dopo Max-Heapify
1	6	$A[1] \leftrightarrow A[6]$	$[3, 8, 6, 7, 4 \mid 9]$	$[8, 7, 6, 3, 4 \mid 9]$
2	5	$A[1] \leftrightarrow A[5]$	$[4, 7, 6, 3 \mid 8, 9]$	$[7, 4, 6, 3 \mid 8, 9]$
3	4	$A[1] \leftrightarrow A[4]$	$[3, 4, 6 \mid 7, 8, 9]$	$[6, 4, 3 \mid 7, 8, 9]$
4	3	$A[1] \leftrightarrow A[3]$	$[3, 4 \mid 6, 7, 8, 9]$	$[4, 3 \mid 6, 7, 8, 9]$
5	2	$A[1] \leftrightarrow A[2]$	$[3 \mid 4, 6, 7, 8, 9]$	$[3 \mid 4, 6, 7, 8, 9]$

La barra verticale «|» separa la parte heap (sinistra) dalla parte già ordinata (destra).

Dettaglio iterazione 1 ($i = 6$):

- Scambio $A[1] = 9$ con $A[6] = 3$: array = $[3, 8, 6, 7, 4, 9]$
- Riduco **heapsize** a 5 (il 9 è nella sua posizione finale)
- **Max-Heapify**($A, 1, 5$):
 - $A[1] = 3$, figli: $A[2] = 8, A[3] = 6$. Massimo: 8, scambio $A[1] \leftrightarrow A[2]$
 - Risultato intermedio: $[8, 3, 6, 7, 4, 9]$
 - Ricorsione su posizione 2: $A[2] = 3$, figli: $A[4] = 7, A[5] = 4$. Massimo: 7, scambio $A[2] \leftrightarrow A[4]$
 - Risultato: $[8, 7, 6, 3, 4, 9]$

Dettaglio iterazione 2 ($i = 5$):

- Scambio $A[1] = 8$ con $A[5] = 4$: array = $[4, 7, 6, 3, 8, 9]$
- Riduco **heapsize** a 4
- **Max-Heapify**($A, 1, 4$):
 - $A[1] = 4$, figli: $A[2] = 7, A[3] = 6$. Massimo: 7, scambio $A[1] \leftrightarrow A[2]$
 - Risultato: $[7, 4, 6, 3, 8, 9]$
 - Ricorsione su posizione 2: $A[2] = 4$, figli: $A[4] = 3$, nessun figlio destro. $4 > 3$: nessuno scambio

Dettaglio iterazione 3 ($i = 4$):

- Scambio $A[1] = 7$ con $A[4] = 3$: array = $[3, 4, 6, 7, 8, 9]$
- Riduco **heapsize** a 3
- **Max-Heapify**($A, 1, 3$):
 - $A[1] = 3$, figli: $A[2] = 4, A[3] = 6$. Massimo: 6, scambio $A[1] \leftrightarrow A[3]$
 - Risultato: $[6, 4, 3, 7, 8, 9]$

Dettaglio iterazione 4 ($i = 3$):

- Scambio $A[1] = 6$ con $A[3] = 3$: array = $[3, 4, 6, 7, 8, 9]$
- Riduco **heapsize** a 2
- **Max-Heapify**($A, 1, 2$):
 - $A[1] = 3$, figli: $A[2] = 4$. Massimo: 4, scambio $A[1] \leftrightarrow A[2]$

Risultato finale: $A = [3, 4, 6, 7, 8, 9]$ – l'array è ordinato in ordine crescente.

- Scambio $A[1] = 4$ con $A[2] = 3$: array = $[3, 4, 6, 7, 8, 9]$

7.8.6. Code di priorità

Una coda di priorità è una struttura dati astratta che mantiene una collezione dinamica di elementi, ciascuno dotato di una **chiave** (o *priorità*), e supporta operazioni efficienti di inserimento ed estrazione dell'elemento con chiave massima (o minima). Le code di priorità sono alla base di numerosi algoritmi fondamentali.

DEFINIZIONE 7.8.14 (Coda di priorità (max)). Una **coda di priorità max** è una struttura dati che mantiene un insieme dinamico S di elementi, ciascuno con una chiave associata, e supporta le seguenti operazioni:

- `Maximum( $S$ )` : restituisce l'elemento di S con chiave massima, senza modificare S .
- `Extract-Max( $S$ )` : rimuove e restituisce l'elemento di S con chiave massima.
- `Increase-Key( $S, x, k$ )` : aumenta la chiave dell'elemento x al nuovo valore $k \geq A[x]$.
- `Insert( $S, x$ )` : inserisce un nuovo elemento x in S .

7.8.6.1. Applicazioni

Le code di priorità trovano impiego in numerosi contesti:

- **Scheduling di processi:** il sistema operativo seleziona il processo con priorità più alta da eseguire.
- **Simulazione di eventi discreti:** gli eventi vengono estratti in ordine cronologico (priorità = tempo dell'evento).
- **Algoritmo di Dijkstra:** coda di priorità min per estrarre il vertice con distanza stimata minima.
- **Algoritmo di Prim:** per la costruzione dell'albero di copertura minimo (MST).

7.8.6.2. Implementazione con Max-Heap

Un max-heap fornisce un'implementazione naturale ed efficiente di una coda di priorità max.

7.8.6.2.1. Maximum

L'operazione più semplice: il massimo è sempre la radice dell'heap.

Heap-Maximum

```
int heapMaximum(int[] A){
    return A[1];
}
```

Complessità: $O(1)$.

7.8.6.2.2. Extract-Max

Si salva il valore della radice, si sostituisce la radice con l'ultimo elemento dell'heap, si riduce la dimensione e si invoca `Max-Heapify` per ripristinare la proprietà.

Heap-Extract-Max

```
int heapExtractMax(int[] A, int heapsize){
    if(heapsize < 1){
        // errore: heap vuoto (underflow)
        return -1;
    }
    int max = A[1];
    A[1] := A[heapsize];
    heapsize := heapsize - 1;
    maxHeapify(A, 1, heapsize);
    return max;
}
```

Complessità: $O(\log n)$, dominata dalla chiamata a `Max-Heapify`.

7.8.6.2.3. Increase-Key

Per aumentare la chiave di un elemento, si aggiorna il valore e si fa «risalire» l'elemento verso la radice (*sift-up*), scambiandolo con il padre finché la proprietà di max-heap non è ripristinata.

Heap-Increase-Key

```
heapIncreaseKey(int[] A, int i, int key){
    if(key < A[i]){
        // errore: la nuova chiave è minore della precedente
        return;
    }
    A[i] := key;
    // Risali verso la radice finché necessario
    while((i > 1) && (A[parent(i)] < A[i])){
        int temp = A[i];
        A[i] := A[parent(i)];
        A[parent(i)] := temp;
        i := parent(i);
    }
}
```

Complessità: $O(\log n)$. Nel caso pessimo l'elemento risale dalla foglia fino alla radice, percorrendo $O(\log n)$ livelli.

La correttezza della procedura si dimostra mediante il seguente invariante di ciclo.

DEFINIZIONE 7.8.15 (Invariante di ciclo per Heap-Increase-Key). All'inizio di ogni iterazione del ciclo `while`, l'array $A[1.. \text{heapsize}]$ soddisfa la proprietà del max-heap, con al più una possibile violazione: $A[i]$ potrebbe essere maggiore di $A[\text{Parent}(i)]$.

Dimostrazione. Inizializzazione: dopo l'assegnamento $A[i] := \text{key}$ alla riga 3 dello pseudocodice, l'unica possibile violazione della proprietà di max-heap si trova nella coppia $(A[i], A[\text{Parent}(i)])$, poiché il valore di $A[i]$ è stato aumentato. L'invariante vale.

Mantenimento: se $A[\text{Parent}(i)] < A[i]$, lo scambio fa sì che il nodo i soddisfi la proprietà rispetto ai propri figli (il padre, ora in posizione i , è almeno grande quanto il vecchio valore di $A[i]$, che a sua volta era $\geq$ dei figli di i). Dopo lo scambio, i viene aggiornato a $\text{Parent}(i)$: l'unica possibile violazione si è spostata un livello più in alto.

Terminazione: il ciclo termina quando $i = 1$ (radice, nessun padre) oppure $A[\text{Parent}(i)] \geq A[i]$ (nessuna violazione). In entrambi i casi l'intero array soddisfa la proprietà del max-heap. $\square$

7.8.6.2.4. Insert

Per inserire un nuovo elemento, si estende l'heap di una posizione in fondo, si inserisce un valore «sentinella» $-\infty$ e si invoca `Increase-Key` per portare la chiave al valore desiderato.

Heap-Insert

```
heapInsert(int[] A, int key, int heapsize){
    heapsize := heapsize + 1;
    A[heapsize] :=  $-\infty$ ;
    heapIncreaseKey(A, heapsize, key);
}
```

Complessità: $O(\log n)$, dominata dalla chiamata a `Increase-Key`.

ESEMPIO 7.8.16 (Operazioni su una coda di priorità). Partiamo dal max-heap $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ con $\text{heapsize} = 10$.

Operazione 1: Maximum

- Restituisce $A[1] = 16$. L'heap non cambia.

Operazione 2: Extract-Max

- Si salva $\text{max} = A[1] = 16$
- Si pone $A[1] := A[10] = 1$, si riduce heapsize a 9
- Array: $[1, 14, 10, 8, 7, 9, 3, 2, 4]$
- $\text{Max-Heapify}(A, 1, 9)$:
 - $A[1] = 1$, figli: 14, 10. Scambio con 14 (posizione 2)
 - $A[2] = 1$, figli: 8, 7. Scambio con 8 (posizione 4)
 - $A[4] = 1$, figli: 2, 4. Scambio con 4 (posizione 9)
- Risultato: $A = [14, 8, 10, 4, 7, 9, 3, 2, 1]$, restituisce 16

Operazione 3: Increase-Key(A, 9, 15) (aumento la chiave in posizione 9 a 15)

- Si pone $A[9] := 15$
- Array: $[14, 8, 10, 4, 7, 9, 3, 2, 15]$
- Risalita: $A[9] = 15 > A[4] = 4$, scambio. Array: $[14, 8, 10, 15, 7, 9, 3, 2, 4]$
- Risalita: $A[4] = 15 > A[2] = 8$, scambio. Array: $[14, 15, 10, 8, 7, 9, 3, 2, 4]$
- Risalita: $A[2] = 15 > A[1] = 14$, scambio. Array: $[15, 14, 10, 8, 7, 9, 3, 2, 4]$
- $i = 1$: radice raggiunta, terminazione

Operazione 4: Insert(A, 12) con $\text{heapsize} = 9$

- Si pone $\text{heapsize} := 10$, $A[10] := -\infty$
- $\text{Increase-Key}(A, 10, 12) : A[10] := 12$
- Risalita: $A[10] = 12 > A[5] = 7$, scambio. Array con $A[5] = 12$, $A[10] = 7$
- Risalita: $A[5] = 12 < A[2] = 14$: nessuno scambio, terminazione
- Risultato: $A = [15, 14, 10, 8, 12, 9, 3, 2, 4, 7]$

7.8.6.3. Riepilogo complessità

Operazione	Max-Heap	Array non ordinato
Maximum	$O(1)$	$O(n)$
Extract-Max	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(1)$
Increase-Key	$O(\log n)$	$O(1)$

Tabella 12: Confronto delle complessità per le operazioni sulle code di priorità

Nota (Compromesso dell'implementazione con heap). L'implementazione con max-heap offre un buon compromesso: tutte e quattro le operazioni hanno costo al più logaritmico. Con un array non ordinato, **Insert** e **Increase-Key** sono $O(1)$, ma **Maximum** ed **Extract-Max** richiedono una scansione lineare $O(n)$. Simmetricamente, con un array ordinato **Maximum** ed **Extract-Max** sono $O(1)$, ma l'inserimento richiede $O(n)$ per mantenere l'ordinamento.

7.9. Limite inferiore per l'ordinamento basato su confronti

Nota. La teoria generale dei limiti inferiori alla difficoltà dei problemi (criterio della dimensione dell'input, criterio dell'albero di decisione, criterio degli eventi contabili) è trattata nel capitolo sulla complessità computazionale. In questa sezione applichiamo quei risultati al caso specifico dell'ordinamento.

Tutti gli algoritmi di ordinamento studiati finora (Insertion Sort, Selection Sort, MergeSort, QuickSort, HeapSort) si basano esclusivamente su **confronti** tra coppie di elementi per determinare l'ordinamento. Una domanda naturale è: esiste un limite inferiore alla complessità di qualsiasi algoritmo di questo tipo?

TEOREMA 7.9.1 (Limite inferiore per l'ordinamento basato su confronti). *Qualsiasi algoritmo di ordinamento basato su confronti richiede $\Omega(n \log n)$ confronti nel caso pessimo per ordinare n elementi.*

Per dimostrare questo risultato, si introduce il modello dell'**albero di decisione**.

7.9.1. Albero di decisione

DEFINIZIONE 7.9.2 (Albero di decisione). Un **albero di decisione** è un albero binario che modella tutti i possibili confronti effettuati da un algoritmo di ordinamento su un input di dimensione n . Diversi algoritmi producono alberi di forma diversa, ma la proprietà fondamentale è che l'albero deve avere almeno $n!$ foglie (una per ogni possibile permutazione), e un albero binario di altezza h ha al più 2^h foglie:

- Ogni **nodo interno** rappresenta un confronto del tipo $a_i \leq a_j$, dove i, j sono indici degli elementi.
- Il **sottoalbero sinistro** di un nodo corrisponde all'esecuzione nel caso in cui il confronto dia esito positivo ($a_i \leq a_j$).
- Il **sottoalbero destro** corrisponde al caso in cui il confronto dia esito negativo ($a_i > a_j$).
- Ogni **foglia** rappresenta una permutazione dell'output, cioè un possibile ordinamento finale degli elementi. Una foglia si dice **raggiungibile** se esiste un input per il quale l'algoritmo segue il cammino dalla radice a quella foglia.

Affinché un algoritmo di ordinamento per confronti sia corretto, ciascuna delle $n!$ permutazioni deve apparire come foglia raggiungibile dell'albero di decisione. Consideriamo soltanto alberi di decisione in cui ogni permutazione compare in una foglia raggiungibile.

L'altezza dell'albero corrisponde al numero massimo di confronti effettuati dall'algoritmo nel caso pessimo.

7.9.2. Dimostrazione del limite inferiore

Dimostrazione. Sia h l'altezza dell'albero di decisione associato a un algoritmo che ordina n elementi.

Osservazione 1. L'algoritmo deve essere in grado di produrre ogni possibile permutazione come output. Poiché un array di n elementi distinti ammette $n!$ permutazioni, l'albero deve avere almeno $n!$ foglie.

Osservazione 2. Un albero binario di altezza h ha al più 2^h foglie.

Combinando le due osservazioni:

$$2^h \geq n! \quad (102)$$

$$h \geq \log_2(n!) \quad (103)$$

Applicando l'**approssimazione di Stirling** ($n! \approx \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n$), si ottiene:

$$\log_2(n!) = \sum_{i=1}^n \log_2(i) \geq \sum_{i=\lceil n/2 \rceil}^n \log_2(i) \geq \frac{n}{2} \cdot \log_2\left(\frac{n}{2}\right) = \Omega(n \log n) \quad (104)$$

Pertanto:

$$h = \Omega(n \log n) \quad (105)$$

Qualsiasi algoritmo basato su confronti deve eseguire almeno $\Omega(n \log n)$ confronti nel caso pessimo. $\square$

COROLLARIO 7.9.3 (Ottimalità asintotica di HeapSort e MergeSort). *HeapSort e MergeSort sono algoritmi di ordinamento per confronti asintoticamente ottimali.*

Dimostrazione. I limiti superiori $O(n \log n)$ sui tempi di esecuzione di HeapSort e MergeSort corrispondono al limite inferiore $\Omega(n \log n)$ nel caso pessimo stabilito dal teorema precedente. Pertanto, entrambi gli algoritmi raggiungono il limite inferiore a meno di un fattore costante. $\square$

Nota (Conseguenze del limite inferiore).

- MergeSort e HeapSort, avendo complessità $\Theta(n \log n)$ nel caso pessimo, sono **asintoticamente ottimi** tra gli algoritmi basati su confronti.
- QuickSort ha caso pessimo $O(n^2)$, ma caso medio $O(n \log n)$.
- Per superare la barriera $\Omega(n \log n)$ è necessario **non basarsi esclusivamente su confronti**, sfruttando informazioni aggiuntive sulla struttura dei dati (ad esempio, il fatto che siano interi in un intervallo limitato).

7.10. Ordinamento in tempo lineare

Gli algoritmi presentati di seguito raggiungono complessità lineare o quasi-lineare sfruttando ipotesi aggiuntive sull'input. Non si basano esclusivamente su confronti tra coppie di elementi e quindi non sono soggetti al limite inferiore $\Omega(n \log n)$.

7.10.1. Counting Sort

Il **Counting Sort** è un algoritmo di ordinamento che opera su interi nell'intervallo $[0, k]$. L'idea fondamentale è **contare** il numero di occorrenze di ciascun valore e utilizzare queste informazioni per determinare direttamente la posizione finale di ogni elemento nell'array ordinato.

7.10.1.1. Idea

L'algoritmo si articola in quattro fasi:

1. **Inizializzazione:** si crea un array ausiliario $C[0...k]$ inizializzato a zero.
2. **Conteggio:** si scorre l'array di input A e per ogni elemento $A[j]$ si incrementa $C[A[j]]$. Al termine, $C[i]$ contiene il numero di elementi uguali a i .
3. **Somme prefisse:** si calcolano le somme prefisse su C , cioè $C[i] := C[i] + C[i - 1]$. Dopo questa fase, $C[i]$ contiene il numero di elementi $\leq i$, e quindi indica la posizione finale dell'ultimo elemento con valore i nell'array ordinato.
4. **Costruzione dell'output:** si scorre A da destra a sinistra. Per ogni elemento $A[j]$, si piazza $A[j]$ nella posizione $C[A[j]]$ dell'array di output B , e si decrementa $C[A[j]]$.

7.10.1.2. Algoritmo

Counting Sort

```
countingSort(int[] A, int[] B, int n, int k){
    // Fase 1: Inizializzazione di C
    int[] C = new int[k + 1];
    int i = 0;
    while(i <= k){
        C[i] := 0;
        i := i + 1;
    }

    // Fase 2: Conteggio delle occorrenze
    int j = 1;
    while(j <= n){
        C[A[j]] := C[A[j]] + 1;
        j := j + 1;
    }

    // Fase 3: Somme prefisse
    i := 1;
    while(i <= k){
        C[i] := C[i] + C[i - 1];
        i := i + 1;
    }

    // Fase 4: Costruzione dell'output (da destra a sinistra)
    j := n;
    while(j >= 1){
        B[C[A[j]]] := A[j];
        C[A[j]] := C[A[j]] - 1;
        j := j - 1;
    }
}
```

7.10.1.3. Esempio passo-passo

ESEMPIO 7.10.1 (Esecuzione di Counting Sort). Input: $A = [2, 5, 3, 0, 2, 3, 0, 3]$, con $n = 8$ e $k = 5$.

Fase 1 – Inizializzazione. Si crea l'array C di dimensione $k + 1 = 6$:

$$C = [0, 0, 0, 0, 0, 0] \quad (\text{indici } 0, 1, 2, 3, 4, 5) \quad (106)$$

Fase 2 – Conteggio. Si scorre A da sinistra a destra, incrementando $C[A[j]]$ ad ogni passo:

j	A[j]	C dopo l'aggiornamento
1	2	[0, 0, 1, 0, 0, 0]
2	5	[0, 0, 1, 0, 0, 1]
3	3	[0, 0, 1, 1, 0, 1]
4	0	[1, 0, 1, 1, 0, 1]
5	2	[1, 0, 2, 1, 0, 1]
6	3	[1, 0, 2, 2, 0, 1]
7	0	[2, 0, 2, 2, 0, 1]
8	3	[2, 0, 2, 3, 0, 1]

Lettura di C : ci sono 2 zeri, 0 uni, 2 due, 3 tre, 0 quattro, 1 cinque.

Fase 3 – Somme prefisse. Si trasforma C in modo che $C[i]$ contenga il numero di elementi $\leq i$:

i	Calcolo	C
0	invariato	[2, 0, 2, 3, 0, 1]
1	$C[1] := 0 + 2 = 2$	[2, 2, 2, 3, 0, 1]
2	$C[2] := 2 + 2 = 4$	[2, 2, 4, 3, 0, 1]
3	$C[3] := 3 + 4 = 7$	[2, 2, 4, 7, 0, 1]
4	$C[4] := 0 + 7 = 7$	[2, 2, 4, 7, 7, 1]
5	$C[5] := 1 + 7 = 8$	[2, 2, 4, 7, 7, 8]

Interpretazione: ci sono 2 elementi ≤ 0 , 2 elementi ≤ 1 , 4 elementi ≤ 2 , 7 elementi ≤ 3 , 7 elementi ≤ 4 , 8 elementi ≤ 5 .

Fase 4 – Costruzione dell'output. Si scorre A da destra a sinistra. Per ogni $A[j]$, si inserisce $A[j]$ in posizione $C[A[j]]$ di B , poi si decrementa $C[A[j]]$:

j	A[j]	C[A[j]]	Posizionamento	B
8	3	7	$B[7] := 3$	[−, −, −, −, −, −, 3, −]
7	0	2	$B[2] := 0$	[−, 0, −, −, −, −, 3, −]
6	3	6	$B[6] := 3$	[−, 0, −, −, −, 3, 3, −]
5	2	4	$B[4] := 2$	[−, 0, −, 2, −, 3, 3, −]
4	0	1	$B[1] := 0$	[0, 0, −, 2, −, 3, 3, −]
3	3	5	$B[5] := 3$	[0, 0, −, 2, 3, 3, 3, −]
2	5	8	$B[8] := 5$	[0, 0, −, 2, 3, 3, 3, 5]
1	2	3	$B[3] := 2$	[0, 0, 2, 2, 3, 3, 3, 5]

Output finale: $B = [0, 0, 2, 2, 3, 3, 3, 5]$.

7.10.1.4. Stabilità

DEFINIZIONE 7.10.2 (Algoritmo di ordinamento stabile). Un algoritmo di ordinamento è **stabile** se elementi con la stessa chiave di ordinamento mantengono l'ordine relativo che avevano nell'input originale.

Nota (Stabilità del Counting Sort). Il Counting Sort è **stabile** grazie al fatto che la Fase 4 scorre l'array A **da destra a sinistra**. Consideriamo due elementi $A[i]$ e $A[j]$ con $i < j$ e $A[i] = A[j]$. Poiché j viene elaborato prima di i (si parte da n e si decrementa), $A[j]$ viene posizionato in una posizione più alta (a destra) rispetto ad $A[i]$, preservando così l'ordine relativo originale.

Questa proprietà è fondamentale per il corretto funzionamento del Radix Sort, che richiede un sotto-algoritmo stabile per l'ordinamento su ogni cifra.

7.10.1.5. Complessità

Analizziamo separatamente ciascuna fase:

- **Fase 1** – Inizializzazione di C : $\Theta(k)$
- **Fase 2** – Conteggio degli elementi: $\Theta(n)$
- **Fase 3** – Calcolo delle somme prefisse: $\Theta(k)$
- **Fase 4** – Costruzione dell'output: $\Theta(n)$

La complessità totale è quindi:

$$T(n, k) = \Theta(n + k) \quad (107)$$

Nota (Quando il Counting Sort è lineare). Se $k = O(n)$, cioè il range dei valori è proporzionale al numero di elementi, allora la complessità diventa $\Theta(n)$: ordinamento in tempo lineare. Tuttavia, se $k \gg n$ (ad esempio $k = n^2$), il Counting Sort diventa meno efficiente degli algoritmi basati su confronti.

7.10.1.6. Limitazioni

- Gli elementi devono essere **interi non negativi** (o mappabili a tali).
- Il range $[0, k]$ deve essere **noto a priori**.
- Lo **spazio ausiliario** richiesto è $\Theta(n + k)$ (per gli array B e C), quindi l'algoritmo **non è in-place**.
- Se k è molto grande rispetto a n , sia il tempo che lo spazio diventano proibitivi.

OSSERVAZIONE 7.10.3. Il Counting Sort supera il limite inferiore $\Omega(n \log n)$ perché **non è un algoritmo di ordinamento per confronti**: nel suo codice non compare alcun confronto tra elementi dell'input. Il Counting Sort utilizza i valori effettivi degli elementi come indici di un array, operando in modo completamente diverso dal modello dell'albero di decisione. Il limite inferiore $\Omega(n \log n)$ non si applica quando ci si allontana dal modello di ordinamento basato esclusivamente su confronti.

7.10.2. Radix Sort

Il **Radix Sort** ordina numeri interi (o stringhe) analizzando le singole cifre (o caratteri), dalla meno significativa (LSD, *Least Significant Digit*) alla più significativa (MSD, *Most Significant Digit*).

7.10.2.1. Idea

L'algoritmo esegue d passate sull'array, dove d è il numero di cifre del numero più lungo. Ad ogni passata, si ordina l'array rispetto a una singola cifra utilizzando un algoritmo di ordinamento **stabile** (tipicamente il Counting Sort). L'ordine di elaborazione delle cifre è cruciale: si parte dalla cifra meno significativa e si procede verso quella più significativa.

Perché dalla meno significativa? Se si procedesse dalla più significativa alla meno significativa, l'ordinamento sulle cifre successive potrebbe distruggere l'ordine già stabilito. Partendo dalla cifra meno significativa, la stabilità dell'algoritmo ausiliario garantisce che, quando si ordina per la cifra i , l'ordine relativo stabilito dalle cifre $1, \dots, i - 1$ viene preservato tra gli elementi con la stessa cifra i .

7.10.2.2. Algoritmo

Radix Sort

```
radixSort(int[] A, int d){
    // d = numero di cifre del valore massimo
    int i = 1;
    while(i <= d){
        // Ordina A con un algoritmo stabile sulla cifra i-esima
        stableSort(A, i);
        i := i + 1;
    }
}
```

7.10.2.3. Esempio passo-passo

ESEMPIO 7.10.4 (Esecuzione di Radix Sort). Input: $A = [329, 457, 657, 839, 436, 720, 355]$, con $d = 3$ cifre (in base 10).

Passo 1 – Ordinamento per UNITÀ (cifra meno significativa).

Si estraggono le cifre delle unità e si ordina stabilmente rispetto ad esse:

Numero	Cifra delle unità
329	9
457	7
657	7
839	9
436	6
720	0
355	5

Risultato dopo ordinamento stabile per unità:

[720, 355, 436, 457, 657, 329, 839] (108)

Passo 2 – Ordinamento per DECINE.

Si estraggono le cifre delle decine dall'array corrente e si ordina stabilmente:

Numero	Cifra delle decine
720	2
355	5
436	3
457	5
657	5
329	2
839	3

Risultato dopo ordinamento stabile per decine:

[720, 329, 436, 839, 355, 457, 657] (109)

Si osservi la stabilità: 720 e 329 hanno entrambi cifra delle decine = 2, ma 720 precede 329 perché era già prima nell'array dopo il Passo 1. Analogamente, 355 precede 457 e 657 (tutti con decine = 5), coerentemente con l'ordine del passo precedente.

Passo 3 – Ordinamento per CENTINAIA (cifra più significativa).

Si estraggono le cifre delle centinaia dall'array corrente e si ordina stabilmente:

Numero	Cifra delle centinaia
720	7
329	3
436	4
839	8
355	3
457	4
657	6

Risultato dopo ordinamento stabile per centinaia:

7.10.2.4. Correttezza

TEOREMA 7.10.5 (Correttezza del Radix Sort). *Se l'algoritmo ausiliario è stabile, dopo i iterazioni del Radix Sort gli elementi risultano ordinati rispetto alle ultime i cifre.*

Dimostrazione. Per **induzione** sul numero di cifre i già elaborate.

Caso base ($i = 1$): dopo la prima iterazione, gli elementi sono ordinati rispetto alla cifra meno significativa (cifra 1), per correttezza dell'algoritmo stabile.

Passo induttivo: supponiamo che dopo $i - 1$ iterazioni gli elementi siano ordinati rispetto alle cifre $1, 2, \dots, i - 1$ (ipotesi induttiva). Alla i -esima iterazione si ordina rispetto alla cifra i con un algoritmo stabile. Consideriamo due elementi x e y :

- Se la cifra i -esima di x è minore di quella di y , allora x precede y nell'output (per correttezza dell'ordinamento sulla cifra i).
- Se la cifra i -esima di x è maggiore di quella di y , allora y precede x .
- Se la cifra i -esima di x è uguale a quella di y , allora per la **stabilità** dell'algoritmo ausiliario, l'ordine relativo di x e y rimane invariato. Ma per ipotesi induttiva, questo ordine riflette correttamente l'ordinamento rispetto alle cifre $1, \dots, i - 1$.

In tutti i casi, dopo i iterazioni gli elementi sono ordinati rispetto alle cifre $1, 2, \dots, i$. Dopo d iterazioni l'array è completamente ordinato. $\square$

7.10.2.5. Complessità

Se si utilizza il Counting Sort come algoritmo stabile, e i numeri sono rappresentati in base b :

- Ogni cifra assume valori in $[0, b - 1]$, quindi il Counting Sort su ciascuna cifra richiede $\Theta(n + b)$.
- Il numero di cifre è: $d = \lceil \log_{b(M+1)} \rceil$, dove M è il valore massimo.

La complessità totale è:

$$T(n) = \Theta(d(n + b)) \quad (111)$$

LEMMA 7.10.6 (Complessità del Radix Sort (CLRS, Lemma 8.3)). *Dati n numeri di d cifre, dove ogni cifra può assumere fino a k valori possibili, la procedura Radix Sort ordina correttamente i numeri nel tempo $\Theta(d(n + k))$, se l'ordinamento stabile utilizzato impiega un tempo $\Theta(n + k)$.*

Dimostrazione. La correttezza del Radix Sort si dimostra per induzione sulla colonna da ordinare (come visto nel teorema precedente). L'analisi del tempo di esecuzione dipende dall'algoritmo stabile usato come subroutine. Se ogni cifra si trova nell'intervallo da 0 a $k - 1$ (in modo che possa assumere k valori possibili) e k non è troppo grande, il Counting Sort è la scelta naturale. Ogni passaggio su n numeri di d cifre richiede tempo $\Theta(n + k)$. Poiché ci sono d passaggi, il tempo totale del Radix Sort è $\Theta(d(n + k))$. $\square$

Quando d è costante e $k = O(n)$, il Radix Sort viene eseguito in tempo lineare.

7.10.2.6. Scelta ottimale della base

Più in generale, abbiamo una certa flessibilità nel modo in cui ripartire le singole chiavi in cifre.

LEMMA 7.10.7 (Analisi con rappresentazione in bit (CLRS, Lemma 8.4)). *Dati n numeri di b bit e un intero positivo $r \leq b$, il Radix Sort ordina correttamente questi numeri nel tempo $\Theta((b/r)(n + 2^r))$, se l'algoritmo di ordinamento stabile usato richiede tempo $\Theta(n + k)$ per input nell'intervallo da 0 a k .*

Dimostrazione. Per un valore $r \leq b$, consideriamo ogni chiave come se avesse $d = \lceil b/r \rceil$ cifre di r bit ciascuna. Ogni cifra è un numero intero compreso nell'intervallo da 0 a $2^r - 1$, quindi possiamo utilizzare il Counting Sort con $k = 2^r - 1$. Ogni passaggio di Counting Sort richiede il tempo $\Theta(n + 2^r)$; poiché ci sono d passaggi, il tempo di esecuzione totale è $\Theta(d(n + 2^r)) = \Theta((b/r)(n + 2^r))$. $\square$

TEOREMA 7.10.8 (Scelta ottimale della base per Radix Sort). *Dati n numeri interi rappresentabili con b bit (cioè con valori in $[0, 2^b - 1]$), e scegliendo $r = \lfloor \log_2 n \rfloor$ (corrispondente alla base $2^r \approx n$), il Radix Sort ordina in tempo:*

$$T(n) = \Theta\left(\frac{b}{\log_2 n} \cdot n\right) \quad (112)$$

Se $b = O(\log n)$ (cioè i valori sono polinomiali in n), la complessità diventa $\Theta(n)$: ordinamento lineare.

Dimostrazione. Dati i valori di n e b , scegliamo il valore di r (con $r \leq b$) che minimizza l'espressione $(b/r)(n + 2^r)$. Se $b < \lfloor \log_2 n \rfloor$, allora per qualsiasi valore di $r \leq b$ si ha $(n + 2^r) = \Theta(n)$. Pertanto, scegliendo $r = b$, si ottiene un tempo di esecuzione $(b/b)(n + 2^b) = \Theta(n)$, che è asintoticamente ottimale.

Se $b \geq \lfloor \log_2 n \rfloor$, allora scegliendo $r = \lfloor \log_2 n \rfloor$ si ottiene il tempo migliore a meno di un fattore costante. In questo caso si ha un tempo di esecuzione pari a $\Theta(bn/\log n)$. Aumentando r oltre $\lfloor \log_2 n \rfloor$, il termine 2^r nel numeratore cresce più rapidamente del termine r nel denominatore; quindi, aumentando r oltre $\lfloor \log_2 n \rfloor$, si ottiene un tempo di esecuzione pari a $\Omega(bn/\log n)$. Se invece diminuiamo r sotto $\lfloor \log_2 n \rfloor$, il termine b/r cresce e il termine $n + 2^r$ resta $\Theta(n)$. $\square$

Nota (Intuizione sulla scelta della base). Scegliere $b = n$ minimizza il prodotto $d \cdot (n + b)$:

- Con una base troppo piccola (es. $b = 2$), si hanno molte cifre (d grande) e ogni passata è veloce ($n + 2$), ma il costo totale cresce.
- Con una base troppo grande (es. $b = n^2$), si hanno poche cifre ma ogni passata richiede $\Theta(n + n^2)$.
- Il bilanciamento ottimale si ottiene con $b \approx n$, che dà $d = r/\log n$ passate ciascuna di costo $\Theta(n)$.

OSSERVAZIONE 7.10.9. Radix Sort è preferibile a un ordinamento basato su confronti?

Se $b = O(\log n)$, come accade spesso quando i valori sono polinomiali in n , il Radix Sort viene eseguito in tempo $\Theta(n)$, che sembra migliore di $\Theta(n \log n)$. Tuttavia, i fattori costanti nascosti nella notazione Θ sono diversi: sebbene il Radix Sort richieda meno passaggi sulle n chiavi, ogni passaggio di Radix Sort può richiedere significativamente più tempo di un confronto nel Quicksort. La scelta dell'algoritmo di ordinamento ottimale dipende dalle caratteristiche dell'implementazione, della macchina (ad esempio, Quicksort usa le cache hardware in modo più efficiente), e dei dati di input. Inoltre, il Radix Sort con Counting Sort come subroutine non ordina sul posto, mentre molti ordinamenti per confronti (come Quicksort) lo fanno. Se la memoria è limitata, un algoritmo in-place come Quicksort potrebbe essere preferibile.

7.10.3. Bucket Sort

Il **Bucket Sort** è un algoritmo di ordinamento che assume che l'input sia **distribuito uniformemente** in un intervallo noto, tipicamente $[0, 1)$.

7.10.3.1. Idea

L'algoritmo sfrutta la distribuzione uniforme dell'input per distribuire gli elementi in n **bucket** (contenitori) di uguale ampiezza. Poiché l'input è distribuito uniformemente, ci si aspetta che ogni bucket contenga pochi elementi, e quindi l'ordinamento interno a ciascun bucket sia rapido.

Le fasi dell'algoritmo sono:

1. **Distribuzione:** si dividono gli n elementi in n bucket. L'elemento $A[i]$ viene assegnato al bucket $\lfloor n \cdot A[i] \rfloor$.
2. **Ordinamento locale:** si ordina ciascun bucket con un algoritmo semplice (ad esempio Insertion Sort).
3. **Concatenazione:** si concatenano i bucket in ordine, ottenendo l'array ordinato.

7.10.3.2. Algoritmo

Bucket Sort

```
bucketSort(float[] A, int n){
    // Crea n liste (bucket) vuote
    List[] B = new List[n];

    // Distribuisci gli elementi nei bucket
    int i = 1;
    while(i <= n){
        int idx = floor(n * A[i]);
        insert(B[idx], A[i]);
        i := i + 1;
    }

    // Ordina ogni bucket con Insertion Sort
    i := 0;
    while(i <= n - 1){
        insertionSort(B[i]);
        i := i + 1;
    }

    // Concatena i bucket in ordine
    return concatenate(B[0], B[1], ..., B[n - 1]);
}
```

Nota. Si assume che gli elementi siano numeri reali in $[0, 1)$. Per input in un intervallo arbitrario $[a, b)$, si normalizza: $A[i] := \frac{A[i]-a}{b-a}$.

7.10.3.3. Esempio passo-passo

ESEMPIO 7.10.10 (Esecuzione di Bucket Sort). Input: $A = [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]$, con $n = 10$.

Fase 1 – Distribuzione. Ogni elemento $A[i]$ viene assegnato al bucket di indice $\lfloor 10 \cdot A[i] \rfloor$:

Elemento	$\lfloor 10 \cdot A[i] \rfloor$	Bucket
0.78	7	$B[7]$
0.17	1	$B[1]$
0.39	3	$B[3]$
0.26	2	$B[2]$
0.72	7	$B[7]$
0.94	9	$B[9]$
0.21	2	$B[2]$
0.12	1	$B[1]$
0.23	2	$B[2]$
0.68	6	$B[6]$

Contenuto dei bucket dopo la distribuzione:

- $B[1] = \langle 0.17, 0.12 \rangle$
- $B[2] = \langle 0.26, 0.21, 0.23 \rangle$
- $B[3] = \langle 0.39 \rangle$
- $B[6] = \langle 0.68 \rangle$
- $B[7] = \langle 0.78, 0.72 \rangle$
- $B[9] = \langle 0.94 \rangle$
- (gli altri bucket sono vuoti)

Fase 2 – Ordinamento locale. Si ordina ogni bucket con Insertion Sort:

- $B[1] = \langle 0.12, 0.17 \rangle$
- $B[2] = \langle 0.21, 0.23, 0.26 \rangle$
- $B[3] = \langle 0.39 \rangle$
- $B[6] = \langle 0.68 \rangle$
- $B[7] = \langle 0.72, 0.78 \rangle$
- $B[9] = \langle 0.94 \rangle$

Fase 3 – Concatenazione.

$$B = [0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94] \quad (113)$$

7.10.3.4. Complessità

TEOREMA 7.10.11 (Complessità del Bucket Sort).

- **Caso pessimo:** $\Theta(n^2)$. Si verifica quando tutti gli elementi finiscono nello stesso bucket, riducendo l'algoritmo a un Insertion Sort sull'intero array.
- **Caso medio** (con input distribuito uniformemente in $[0, 1)$): $\Theta(n)$.

Dimostrazione. Sia n_i il numero di elementi nel bucket i . Il tempo totale è:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \quad (114)$$

dove $\Theta(n)$ è il costo della distribuzione e della concatenazione, e $O(n_i^2)$ è il costo dell'Insertion Sort sul bucket i .

Calcoliamo il valore atteso. Se l'input è distribuito uniformemente in $[0, 1)$, ogni elemento finisce nel bucket i con probabilità $1/n$. Quindi n_i segue una distribuzione binomiale $B(n, 1/n)$, con $E[n_i] = 1$ e $\text{Var}(n_i) = 1 - 1/n$.

$$E[n_i^2] = \text{Var}(n_i) + (E[n_i])^2 = \left(1 - \frac{1}{n}\right) + 1 = 2 - \frac{1}{n} \quad (115)$$

Pertanto:

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O\left(2 - \frac{1}{n}\right) = \Theta(n) + O(n) = \Theta(n) \quad (116)$$

□

7.10.3.5. Limitazioni

- Richiede l'ipotesi di **distribuzione uniforme** dell'input per garantire complessità media lineare.
- Le prestazioni degradano significativamente con distribuzioni non uniformi (molti elementi nello stesso bucket).
- Richiede **spazio ausiliario** $\Theta(n)$ per i bucket.
- L'algoritmo **non è in-place**.

7.10.4. Riepilogo degli algoritmi di ordinamento

La tabella seguente riassume le caratteristiche di tutti gli algoritmi di ordinamento studiati.

Algoritmo	Caso pessimo	Caso medio	Caso migliore	Stabile?	In-place?
Insertion Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	Sì	Sì
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	No	Sì
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	Sì	No
QuickSort	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	No	Sì
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	No	Sì
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	Sì	No
Radix Sort	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$\Theta(d(n + b))$	Sì	No
Bucket Sort	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$	Sì	No

Tabella 20: Confronto completo degli algoritmi di ordinamento

Nota (Guida alla lettura della tabella).

- k : range dei valori nel Counting Sort ($[0, k]$).
- d : numero di cifre nel Radix Sort.
- b : base di rappresentazione nel Radix Sort (ciascuna cifra in $[0, b - 1]$).
- **Stabile**: un algoritmo è stabile se preserva l'ordine relativo degli elementi con chiave uguale.
- **In-place**: un algoritmo è in-place se utilizza solo $O(1)$ spazio ausiliario (oltre all'input).

Nota (Osservazioni finali).

- Gli algoritmi **basati su confronti** (Insertion Sort, Selection Sort, MergeSort, QuickSort, HeapSort) hanno un limite inferiore di $\Omega(n \log n)$ nel caso pessimo. MergeSort e HeapSort raggiungono questo limite e sono quindi asintoticamente ottimi.
- **Counting Sort, Radix Sort e Bucket Sort** possono raggiungere complessità lineare, ma richiedono ipotesi aggiuntive sull'input: interi in un range limitato per i primi due, distribuzione uniforme per il terzo.
- **QuickSort**, nonostante il caso pessimo $\Theta(n^2)$, è spesso preferito nella pratica per il suo eccellente caso medio e per le costanti moltiplicative basse.
- La scelta dell'algoritmo dipende dalle **caratteristiche dei dati** (tipo, distribuzione, range), dai **vincoli di spazio**, e dalla necessità di **stabilità**.

CAPITOLO 8

Strutture Dati

8.1. Strutture Dati Lineari

Una **struttura dati** è un modo sistematico di organizzare e memorizzare dati in modo da rendere efficienti le operazioni che si intendono eseguire su di essi. La scelta della struttura dati influenza direttamente la complessità temporale e spaziale degli algoritmi.

Le **strutture dati lineari** organizzano gli elementi in sequenza: ogni elemento (tranne il primo e l'ultimo) ha esattamente un predecessore e un successore. Le strutture che analizziamo in questa sezione sono **array**, **liste concatenate**, **pile** e **code**.

8.1.1. Array

DEFINIZIONE 8.1.1 (Array). Un **array** è una struttura dati che memorizza una collezione di n elementi dello stesso tipo in n locazioni di memoria **contigue**. Ogni elemento è univocamente identificato da un **indice** intero $i \in \{1, \dots, n\}$ e si accede ad esso in tempo costante tramite la formula:

$$\text{indirizzo}(A[i]) = \text{indirizzo}(A[1]) + (i - 1) \cdot \text{dimensione elemento} \quad (117)$$

L'accesso diretto tramite indice e la contiguità in memoria sono le proprietà fondamentali degli array. La contiguità garantisce un'elevata **località spaziale**, che si traduce in un utilizzo efficiente della cache del processore: quando si accede a $A[i]$, gli elementi vicini $A[i + 1]$, $A[i + 2]$, ... vengono caricati nella stessa linea di cache.

8.1.1.1. Proprietà degli array

- **Accesso diretto** in $O(1)$ tramite indice
- **Dimensione fissa**: stabilita al momento della creazione (in molti linguaggi)
- **Elementi contigui in memoria**: ottimo per accessi sequenziali
- **Tipo omogeneo**: tutti gli elementi hanno lo stesso tipo

8.1.1.2. Operazioni e complessità

Operazione	Descrizione	Complessità
Access(A, i)	Restituisce l'elemento in posizione i	$O(1)$
Search(A, x)	Cerca l'elemento x (scansione lineare)	$O(n)$
Insert(A, i, x)	Inserisce x in posizione i , traslando gli elementi successivi	$O(n)$
Delete(A, i)	Elimina l'elemento in posizione i , traslando gli elementi successivi	$O(n)$

Tabella 21: Operazioni su array e relative complessità nel caso peggiore

Nota (Costo dell'inserimento e della cancellazione). L'inserimento di un elemento in posizione i richiede lo spostamento di tutti gli elementi da $A[i]$ a $A[n]$ di una posizione a destra, per fare spazio al nuovo elemento. Analogamente, la cancellazione richiede lo spostamento degli elementi successivi a sinistra per riempire il «buco». Nel caso peggiore (inserimento o cancellazione in testa, $i = 1$) si spostano tutti gli n elementi, da cui la complessità $O(n)$.

L'inserimento **in coda** (posizione $n + 1$) costa $O(1)$ se c'è spazio disponibile, perché non richiede alcuno spostamento.

ESEMPIO 8.1.2 (Inserimento in un array). Dato l'array $A = [3, 7, 12, 5, 9]$ con $n = 5$, inseriamo il valore 8 in posizione $i = 3$:

3	7	12	5	9
1	2	3	4	5

1. Spostiamo $A[5]$ in $A[6]$:

3	7	12	5	9	9
1	2	3	4	5	6

2. Spostiamo $A[4]$ in $A[5]$:

3	7	12	5	5	9
1	2	3	4	5	6

3. Spostiamo $A[3]$ in $A[4]$:

3	7	12	12	5	9
1	2	3	4	5	6

4. Scriviamo $A[3] := 8$:

3	7	8	12	5	9
1	2	3	4	5	6

Sono stati necessari $n - i + 1 = 3$ spostamenti.

8.1.2. Liste Concatenate

DEFINIZIONE 8.1.3 (Lista concatenata). Una **lista concatenata** (linked list) è una struttura dati in cui gli elementi, detti **nodi**, sono collegati tramite **puntatori**. A differenza degli array, i nodi non occupano locazioni di memoria contigue: ogni nodo può trovarsi in una posizione arbitraria della memoria, e il collegamento tra nodi è realizzato esclusivamente attraverso i puntatori.

Il vantaggio principale delle liste rispetto agli array è che inserimenti e cancellazioni possono essere effettuati in tempo $O(1)$ senza spostare altri elementi, purché si disponga del puntatore al punto di inserimento. Lo svantaggio è la perdita dell'accesso diretto: per raggiungere l' i -esimo elemento bisogna attraversare $i - 1$ nodi.

8.1.2.1. Lista semplicemente concatenata

In una lista **semplicemente concatenata** ogni nodo contiene due campi:

- **key** : il dato memorizzato nel nodo
- **next** : un puntatore al nodo successivo nella lista (oppure **NIL** se il nodo è l'ultimo)

La lista è accessibile tramite un puntatore **L.head** al primo nodo.

Struttura di un nodo (lista semplice)

```
// Nodo di lista semplicemente concatenata
// key : dato memorizzato
// next : puntatore al nodo successivo (NIL se ultimo)
```

ESEMPIO 8.1.4 (Lista semplicemente concatenata). La lista $\langle 3, 7, 12, 5 \rangle$ è rappresentata come:

$$\underbrace{[3 \mid \cdot] \rightarrow [7 \mid \cdot] \rightarrow [12 \mid \cdot] \rightarrow [5 \mid /]}_{L.head} \quad (118)$$

Il simbolo $/$ indica il puntatore **NIL**. Ogni nodo contiene il dato e un puntatore al nodo successivo.

8.1.2.2. Lista doppiamente concatenata

In una lista **doppiamente concatenata** ogni nodo contiene tre campi:

- **key** : il dato memorizzato
- **next** : puntatore al nodo successivo
- **prev** : puntatore al nodo precedente

Struttura di un nodo (lista doppia)

```
// Nodo di lista doppiamente concatenata
// key : dato memorizzato
// next : puntatore al nodo successivo (NIL se ultimo)
// prev : puntatore al nodo precedente (NIL se primo)
```

La presenza del puntatore `prev` consente la navigazione in entrambe le direzioni e semplifica l'operazione di cancellazione: dato un puntatore a un nodo x , si può rimuoverlo in $O(1)$ senza dover cercare il suo predecessore.

8.1.2.3. Operazioni su liste doppiamente concatenate

Le tre operazioni fondamentali su liste sono la **ricerca**, l'**inserimento** e la **cancellazione**. Presentiamo lo pseudocodice per una lista doppiamente concatenata con puntatore `L.head` alla testa.

8.1.2.3.1. Ricerca

La ricerca scorre la lista dal primo nodo fino a trovare un nodo con chiave k oppure fino a raggiungere la fine della lista (`NIL`).

List-Search(L, k)

```
Node listSearch(List L, int k){
    Node x = L.head;
    while((x != NIL) && (x.key != k)){
        x := x.next;
    }
    return x;
}
```

Complessità: $O(n)$ nel caso peggiore (elemento non presente o in ultima posizione). Nel caso migliore $O(1)$ (elemento in testa).

8.1.2.3.2. Inserimento in testa

L'inserimento in testa aggiunge un nuovo nodo x come primo elemento della lista.

List-Insert(L, x)

```
listInsert(List L, Node x){
    x.next := L.head;
    if(L.head != NIL){
        L.head.prev := x;
    }
    L.head := x;
    x.prev := NIL;
}
```

Complessità: $O(1)$, poiché si aggiornano solo un numero costante di puntatori indipendentemente dalla lunghezza della lista.

8.1.2.3.3. Cancellazione

La cancellazione rimuove un nodo x dalla lista, dato il puntatore a x . Si devono aggiornare i puntatori del predecessore e del successore di x affinché si «scavalchi» il nodo rimosso.

List-Delete(L, x)

```
listDelete(List L, Node x){
    if(x.prev != NIL){
        x.prev.next := x.next;
    } else {
        L.head := x.next;
    }
    if(x.next != NIL){
        x.next.prev := x.prev;
    }
}
```

Complessità: $O(1)$ se si dispone già del puntatore al nodo x . Se occorre prima cercare x (ad esempio tramite la chiave), la complessità complessiva diventa $O(n)$ a causa della ricerca.

Nota (Gestione dei casi limite). Nell'operazione `List-Delete`, il ramo `else` gestisce il caso in cui x è il primo nodo della lista (`x.prev == NIL`): in tal caso occorre aggiornare `L.head`. Analogamente, il secondo `if` gestisce il caso in cui x è l'ultimo nodo. Questa gestione dei casi limite rende il codice più complesso; le **sentinelle** permettono di semplificarlo.

8.1.2.4. Varianti delle liste

8.1.2.4.1. Lista circolare

In una **lista circolare**, l'ultimo nodo punta al primo invece che a `NIL`. In una lista doppiamente concatenata circolare, il `prev` del primo nodo punta all'ultimo e il `next` dell'ultimo punta al primo. Questa struttura è utile quando si vuole poter ciclare sugli elementi senza un punto di inizio/fine esplicito.

8.1.2.4.2. Lista con sentinella

Una **sentinella** (o nodo fittizio) è un nodo speciale `L.nil` che non contiene dati significativi e funge da «confine» della lista. In una lista doppiamente concatenata con sentinella:

- `L.nil.next` punta al primo elemento della lista (la «testa»)
- `L.nil.prev` punta all'ultimo elemento (la «coda»)
- Il `prev` del primo elemento punta a `L.nil`
- Il `next` dell'ultimo elemento punta a `L.nil`
- Una lista vuota ha `L.nil.next == L.nil` e `L.nil.prev == L.nil`

La sentinella trasforma la lista in una struttura circolare e **elimina tutti i casi limite** (lista vuota, inserimento/cancellazione in testa o in coda), semplificando notevolmente lo pseudocodice.

Ricerca con sentinella

```
Node listSearch'(List L, int k){
    Node x = L.nil.next;
    while((x != L.nil) && (x.key != k)){
        x := x.next;
    }
    return x;
}
```

Complessità: $O(n)$ nel caso peggiore, come la versione senza sentinella. La differenza è che il ciclo testa `x != L.nil` invece di `x != NIL`.

Inserimento con sentinella

```
// Con sentinella: nessun caso speciale necessario
listInsert'(List L, Node x){
    x.next := L.nil.next;
    L.nil.next.prev := x;
    L.nil.next := x;
    x.prev := L.nil;
}
```

Complessità: $O(1)$. Non è necessario verificare se la lista è vuota, poiché la sentinella garantisce che `L.nil.next` esista sempre.

Cancellazione con sentinella

```
// Con sentinella: nessun caso speciale necessario
listDelete'(List L, Node x){
    x.prev.next := x.next;
    x.next.prev := x.prev;
}
```

Nota (Quando usare le sentinelle). Le sentinelle raramente abbassano i limiti asintotici sui tempi delle operazioni, ma possono ridurre i **fattori costanti**. Il vantaggio principale è la **chiarezza del codice**: eliminando le condizioni al contorno, lo pseudocodice risulta più semplice e meno soggetto a errori. Sono utili quando si hanno molte operazioni di inserimento e cancellazione e si vuole ridurre il numero di condizioni da verificare.

Le sentinelle non dovrebbero essere utilizzate in modo indiscriminato. Se ci sono molte liste piccole, lo spazio extra in memoria utilizzato dalle sentinelle potrebbe costituire un notevole spreco. Si utilizzano le sentinelle soltanto se semplificano davvero il codice.

8.1.2.5. Confronto Array vs Liste

Operazione	Array	Lista concatenata
Accesso per indice	$O(1)$	$O(n)$
Ricerca (non ordinato)	$O(n)$	$O(n)$
Inserimento in testa	$O(n)$	$O(1)$
Inserimento in coda	$O(1)^*$	$O(1)^{**}$
Cancellazione (dato il puntatore/indice)	$O(n)$	$O(1)$
Uso della memoria	Compatto, no overhead	Overhead per puntatori
Località di cache	Elevata	Scarsa

Tabella 22: Confronto tra array e lista concatenata. * se c'è spazio nell'array. ** se si mantiene un puntatore alla coda.

La scelta tra array e lista dipende dal profilo di utilizzo: se predominano accessi per indice, l'array è preferibile; se predominano inserimenti e cancellazioni in posizioni arbitrarie, la lista concatenata è più efficiente.

8.1.3. Pile (Stack)

DEFINIZIONE 8.1.5 (Pila). Una **pila** (stack) è una struttura dati con politica di accesso **LIFO** (Last In, First Out): l'ultimo elemento inserito è il primo ad essere rimosso. L'unico elemento accessibile è quello in **cima** (top) alla pila.

La pila è un **tipo di dato astratto**: la definizione specifica **cosa** fa (l'interfaccia), non **come** è implementata. Può essere realizzata sia con array che con liste concatenate.

8.1.3.1. Interfaccia

Le operazioni fondamentali su una pila S sono:

- $\text{Push}(S, x)$: inserisce l'elemento x in cima alla pila
- $\text{Pop}(S)$: rimuove e restituisce l'elemento in cima alla pila. Errore (**underflow**) se la pila è vuota
- $\text{Top}(S)$: restituisce l'elemento in cima senza rimuoverlo
- $\text{IsEmpty}(S)$: restituisce **true** se la pila è vuota, **false** altrimenti

Tutte le operazioni hanno complessità $O(1)$.

8.1.3.2. Implementazione con array

Si utilizza un array $S[1..n]$ e una variabile `S.top` che indica l'indice dell'elemento in cima. Una pila vuota ha `S.top == 0`.

Pila su array

```
bool stackEmpty(Stack S){
    return S.top == 0;
}

push(Stack S, int x){
    if(S.top == S.length){
        // error "overflow"
        return;
    }
    S.top := S.top + 1;
    S[S.top] := x;
}

int pop(Stack S){
    if(stackEmpty(S)){
        // error "underflow"
        return -1;
    }
    S.top := S.top - 1;
    return S[S.top + 1];
}

int top(Stack S){
    if(stackEmpty(S)){
        // error "underflow"
        return -1;
    }
    return S[S.top];
}
```

Nota (Overflow). Nell'implementazione su array, la dimensione della pila è limitata dalla dimensione dell'array. Se si tenta un `Push` quando `S.top == S.length`, si verifica un errore di **overflow**. Questo problema non si presenta nell'implementazione su lista.

8.1.3.3. Implementazione con lista concatenata

La cima della pila corrisponde alla testa della lista. Le operazioni `Push` e `Pop` corrispondono rispettivamente all'inserimento e alla cancellazione in testa.

Pila su lista concatenata

```
push(Stack S, Node x){
    x.next := S.head;
    S.head := x;
}

Node pop(Stack S){
    if(S.head == NIL){
        // error "underflow"
        return NIL;
    }
    Node x = S.head;
    S.head := S.head.next;
    return x;
}
```

Nota (Confronto delle due implementazioni).

- **Array:** più efficiente in termini di memoria (nessun overhead per puntatori) e migliore località di cache, ma ha dimensione fissa.
- **Lista:** dimensione dinamica (nessun overflow), ma ogni nodo richiede spazio aggiuntivo per il puntatore `next`.

8.1.3.4. Applicazioni delle pile

- **Call stack:** gestione delle chiamate di funzione e dei record di attivazione. Quando una funzione f chiama una funzione g , il contesto di f viene salvato sulla pila; al ritorno di g , viene ripristinato.
- **Valutazione di espressioni:** conversione e valutazione di espressioni in notazione polacca inversa (postfissa).
- **Algoritmi di backtracking:** esplorazione di spazi di ricerca (es. labirinti, puzzle) dove si «torna indietro» all'ultimo punto di scelta.
- **Undo/Redo:** negli editor di testo, ogni azione viene impilata; l'annullamento corrisponde a un `Pop`.
- **Bilanciamento delle parentesi:** verifica che ogni parentesi aperta abbia la corrispondente chiusa.

ESEMPIO 8.1.6 (Bilanciamento delle parentesi). Per verificare se una stringa di parentesi è correttamente bilanciata, si utilizza una pila: ogni parentesi aperta viene impilata, e per ogni parentesi chiusa si verifica che corrisponda alla parentesi in cima alla pila.

isBalanced(s, n)

```
bool isBalanced(char[] s, int n){
    Stack S = new Stack();
    int i = 0;
    while(i < n){
        char c = s[i];
        if((c == '(') || (c == '[') || (c == '{')){
            push(S, c);
        } else {
            if((c == ')') || (c == ']') || (c == '}')){
                if(stackEmpty(S)){
                    return false;
                }
                char open = pop(S);
                if(!match(open, c)){
                    return false;
                }
            }
        }
        i := i + 1;
    }
    return stackEmpty(S);
}
```

Ad esempio, sulla stringa "{[[]]}":

1. Leggiamo (: impiliamo → pila: ⟨⟩
2. Leggiamo { : impiliamo → pila: ⟨(, {⟩
3. Leggiamo [: impiliamo → pila: ⟨(, {, [⟩
4. Leggiamo] : Pop restituisce [, che corrisponde → pila: ⟨(, {⟩
5. Leggiamo } : Pop restituisce { , che corrisponde → pila: ⟨(⟩
6. Leggiamo) : Pop restituisce (, che corrisponde → pila: ⟨⟩ (vuota)
7. Stringa terminata e pila vuota → **bilanciata**.

ESEMPIO 8.1.7 (Esecuzione di Push e Pop). Mostriamo l'evoluzione della pila durante una sequenza di operazioni, usando l'implementazione su array $S[1..6]$ con $S.top$ inizialmente a 0:

Operazione	S.top	Contenuto array	Cima
Push(S , 4)	1	[4, —, —, —, —, —]	4
Push(S , 7)	2	[4, 7, —, —, —, —]	7
Push(S , 2)	3	[4, 7, 2, —, —, —]	2
Pop(S) $\rightarrow$ 2	2	[4, 7, —, —, —, —]	7
Push(S , 9)	3	[4, 7, 9, —, —, —]	9
Pop(S) $\rightarrow$ 9	2	[4, 7, —, —, —, —]	7
Pop(S) $\rightarrow$ 7	1	[4, —, —, —, —, —]	4

Tabella 23: Evoluzione della pila durante operazioni Push e Pop

8.1.4. Code (Queue)

DEFINIZIONE 8.1.8 (Coda). Una **coda** (queue) è una struttura dati con politica di accesso **FIFO** (First In, First Out): il primo elemento inserito è il primo ad essere rimosso. Gli inserimenti avvengono alla **fine** (tail) e le rimozioni all'**inizio** (head) della coda.

8.1.4.1. Interfaccia

Le operazioni fondamentali su una coda Q sono:

- **Enqueue**(Q , x) : inserisce l'elemento x alla fine della coda
- **Dequeue**(Q) : rimuove e restituisce l'elemento all'inizio della coda. Errore (**underflow**) se la coda è vuota
- **Front**(Q) : restituisce l'elemento all'inizio senza rimuoverlo
- **IsEmpty**(Q) : restituisce **true** se la coda è vuota, **false** altrimenti

Tutte le operazioni hanno complessità $O(1)$.

8.1.4.2. Implementazione con array circolare

Un'implementazione ingenua su array (inserimento in coda e rimozione in testa) porterebbe a uno spostamento progressivo degli elementi verso destra, spreco di spazio. La soluzione è l'**array circolare**: si utilizza un array $Q[1..n]$ in cui gli indici «ruotano», tornando alla posizione 1 dopo la posizione n .

Si mantengono due indici:

- **Q.head** : posizione del primo elemento (prossimo da rimuovere)
- **Q.tail** : posizione della prossima cella libera (dove inserire il prossimo elemento)

La coda è vuota quando **Q.head == Q.tail**. L'operazione di «avanzamento circolare» dell'indice si esprime come:

$$i := (i \bmod n) + 1 \quad (119)$$

Questo garantisce che dopo la posizione n si torni alla posizione 1.

Coda su array circolare

```
enqueue(Queue Q, int x){
    if(((Q.tail mod Q.length) + 1) == Q.head){
        // error "overflow"
        return;
    }
    Q[Q.tail] := x;
    if(Q.tail == Q.length){
        Q.tail := 1;
    } else {
        Q.tail := Q.tail + 1;
    }
}

int dequeue(Queue Q){
    if(Q.head == Q.tail){
        // error "underflow"
        return -1;
    }
    int x = Q[Q.head];
    if(Q.head == Q.length){
        Q.head := 1;
    } else {
        Q.head := Q.head + 1;
    }
    return x;
}
```

Nota (Capacità effettiva). Con questa implementazione, un array di dimensione n può contenere al massimo $n - 1$ elementi. Si «sacrifica» una posizione per poter distinguere la coda vuota ($Q.head == Q.tail$) dalla coda piena ($(Q.tail \bmod n) + 1 == Q.head$). Un'alternativa è mantenere un contatore esplicito del numero di elementi.

ESEMPIO 8.1.9 (Esecuzione su array circolare). Array $Q[1..5]$, inizialmente $Q.head = Q.tail = 1$ (coda vuota):

Operazione	head	tail	Contenuto
(iniziale)	1	1	[—, —, —, —, —]
Enqueue(Q, 3)	1	2	[3, —, —, —, —]
Enqueue(Q, 7)	1	3	[3, 7, —, —, —]
Enqueue(Q, 5)	1	4	[3, 7, 5, —, —]
Dequeue(Q) → 3	2	4	[—, 7, 5, —, —]
Enqueue(Q, 12)	2	5	[—, 7, 5, 12, —]
Enqueue(Q, 1)	2	1	[1, 7, 5, 12, —]
Dequeue(Q) → 7	3	1	[1, —, 5, 12, —]

Tabella 24: Evoluzione della coda su array circolare. Si noti come il tail «ritorna» alla posizione 1 dopo la posizione 5.

8.1.4.3. Implementazione con lista concatenata

Si mantengono due puntatori: $Q.head$ al primo elemento e $Q.tail$ all'ultimo. L'inserimento avviene in coda (tramite $Q.tail$) e la rimozione in testa (tramite $Q.head$).

Coda su lista concatenata

```
enqueue(Queue Q, Node x){
    x.next := NIL;
    if(Q.tail != NIL){
        Q.tail.next := x;
    }
    Q.tail := x;
    if(Q.head == NIL){
        Q.head := x;
    }
}

Node dequeue(Queue Q){
    if(Q.head == NIL){
        // error "underflow"
        return NIL;
    }
    Node x = Q.head;
    Q.head := Q.head.next;
    if(Q.head == NIL){
        Q.tail := NIL;
    }
    return x;
}
```

Nota (Correttezza della coda su lista). Nell'operazione `Enqueue` : se la coda è vuota (`Q.head == NIL`), il nuovo nodo diventa sia la testa che la coda. Altrimenti, si aggiunge il nodo dopo l'attuale coda.

Nell'operazione `Dequeue` : se dopo la rimozione `Q.head` diventa `NIL` , anche `Q.tail` deve essere posto a `NIL` (la coda è diventata vuota).

8.1.4.4. Applicazioni delle code

- **Scheduling di processi:** il sistema operativo gestisce i processi pronti per l'esecuzione in una coda FIFO (round-robin scheduling).
- **Buffer di I/O:** i dati in ingresso e in uscita vengono bufferizzati in code per gestire la differenza di velocità tra produttore e consumatore.
- **Gestione delle richieste:** i web server utilizzano code per servire le richieste dei client nell'ordine di arrivo.
- **BFS (Breadth-First Search):** l'algoritmo di visita in ampiezza dei grafi utilizza una coda per esplorare i nodi livello per livello.

8.1.5. Deque (Double-Ended Queue)

DEFINIZIONE 8.1.10 (Deque). Una **deque** (double-ended queue, pronunciato «deck») è una struttura dati che generalizza sia la pila che la coda, permettendo inserimenti e rimozioni da **entrambe** le estremità.

8.1.5.1. Interfaccia

- `InsertFront(D, x)` : inserisce x in testa
- `InsertBack(D, x)` : inserisce x in coda
- `RemoveFront(D)` : rimuove e restituisce l'elemento in testa
- `RemoveBack(D)` : rimuove e restituisce l'elemento in coda

Tutte le operazioni hanno complessità $O(1)$.

Nota. Una deque può simulare sia una pila (usando solo `InsertFront` e `RemoveFront`) che una coda (usando `InsertBack` e `RemoveFront`). Può essere implementata efficientemente con una lista doppiamente concatenata oppure con un array circolare con due indici.

8.2. Code con priorità

Le **code con priorità** sono trattate nella sezione dedicata agli Heap nel capitolo Algoritmi di Ordinamento. A differenza delle code FIFO, l'elemento rimosso non è il primo inserito ma quello con **priorità massima** (o minima). Le operazioni `Insert` e `Extract-Max` (o `Extract-Min`) hanno complessità $O(\log n)$ quando implementate con un heap binario.

8.3. Riepilogo e confronto

Struttura	Politica	Inserimento	Rimozione	Accesso
Array	Indice	$O(n)$	$O(n)$	$O(1)$
Lista	Posizione	$O(1)^*$	$O(1)^*$	$O(n)$
Pila	LIFO	$O(1)$	$O(1)$	$O(1)^{**}$
Coda	FIFO	$O(1)$	$O(1)$	$O(1)^{**}$
Deque	Doppia	$O(1)$	$O(1)$	$O(1)^{**}$

Tabella 25: Riepilogo delle complessità. * Con puntatore alla posizione. ** Solo all'elemento accessibile (cima o fronte).

Nota (Criteri di scelta). La scelta della struttura dati dipende dal tipo di operazioni che l'algoritmo deve eseguire con maggiore frequenza:

- **Accesso casuale frequente** → Array
- **Inserimenti e cancellazioni frequenti in posizioni arbitrarie** → Lista concatenata
- **Elaborazione LIFO** (ultimo arrivato, primo servito) → Pila
- **Elaborazione FIFO** (primo arrivato, primo servito) → Coda
- **Inserimenti/rimozioni da entrambe le estremità** → Deque

8.4. Insiemi dinamici e Dizionari

Molti algoritmi richiedono la capacità di memorizzare un insieme di dati che viene modificato nel tempo: si inseriscono nuovi elementi, se ne cancellano altri, si effettuano ricerche. Un tale insieme prende il nome di **insieme dinamico**.

DEFINIZIONE 8.4.1 (Insieme dinamico). Un **insieme dinamico** è una struttura dati che mantiene un insieme S di elementi, ciascuno identificato da una **chiave**, e supporta operazioni di modifica e di interrogazione. Le operazioni fondamentali sono:

- **Operazioni di ricerca** (query):
 - $\text{Search}(S, k)$: dato un insieme S e una chiave k , restituisce un puntatore x a un elemento in S tale che $x.\text{key} = k$, oppure NIL se tale elemento non esiste
 - $\text{Minimum}(S)$: restituisce l'elemento di S con la chiave più piccola
 - $\text{Maximum}(S)$: restituisce l'elemento di S con la chiave più grande
 - $\text{Successor}(S, x)$: restituisce l'elemento di S con la più piccola chiave maggiore di $x.\text{key}$, oppure NIL se x ha la chiave massima
 - $\text{Predecessor}(S, x)$: restituisce l'elemento di S con la più grande chiave minore di $x.\text{key}$, oppure NIL se x ha la chiave minima
- **Operazioni di modifica:**
 - $\text{Insert}(S, x)$: aggiunge l'elemento x all'insieme S
 - $\text{Delete}(S, x)$: rimuove l'elemento x dall'insieme S

DEFINIZIONE 8.4.2 (Dizionario). Un **dizionario** è un tipo di dato astratto (ADT) che supporta le operazioni `Search`, `Insert` e `Delete`. Quando un insieme dinamico supporta anche `Minimum`, `Maximum`, `Successor` e `Predecessor`, si parla spesso di **dizionario ordinato**.

OSSERVAZIONE 8.4.3. La scelta della struttura dati concreta per implementare un dizionario ha un impatto diretto sulle prestazioni: ad esempio, una lista concatenata non ordinata permette inserimento in $O(1)$ ma ricerca in $O(n)$, mentre un albero binario di ricerca bilanciato garantisce tutte le operazioni in $O(\log n)$. La scelta migliore dipende dalla frequenza relativa delle diverse operazioni.

8.5. Alberi Binari

8.5.1. Definizione e terminologia

DEFINIZIONE 8.5.1 (Albero binario). Un **albero binario** è una struttura dati definita ricorsivamente come:

- l'**albero vuoto** (indicato con `NIL`), oppure
- un **nodo** x che contiene una chiave $x.key$ e due puntatori a sottoalberi binari: il **figlio sinistro** $x.left$ e il **figlio destro** $x.right$.

DEFINIZIONE 8.5.2 (Terminologia). Dato un albero binario T :

- **Radice:** l'unico nodo senza padre, indicato con $T.root$
- **Foglia:** un nodo con entrambi i figli uguali a `NIL`
- **Nodo interno:** un nodo con almeno un figlio diverso da `NIL`
- **Profondità** di un nodo: il numero di archi dal nodo alla radice (la radice ha profondità 0)
- **Altezza** di un nodo: il numero di archi nel cammino più lungo dal nodo a una foglia
- **Altezza dell'albero:** l'altezza della radice, indicata con h
- **Livello** k : l'insieme dei nodi a profondità k
- **Padre** di un nodo x : il nodo y tale che $y.left = x$ oppure $y.right = x$

8.5.2. Rappresentazione di un nodo

Ogni nodo di un albero binario contiene:

- `key` : il dato memorizzato
- `left` : puntatore al figlio sinistro (oppure `NIL`)
- `right` : puntatore al figlio destro (oppure `NIL`)
- `parent` : puntatore al nodo padre (oppure `NIL` per la radice)

Struttura Nodo

```
// Struttura Nodo (albero binario)
// key    : dato
// left   : puntatore al figlio sinistro
// right  : puntatore al figlio destro
// parent : puntatore al padre
```

8.5.3. Rappresentazione di alberi con numero arbitrario di figli

Negli alberi binari ogni nodo ha al più due figli, ma in molte applicazioni occorre rappresentare alberi in cui ogni nodo può avere un numero arbitrario di figli (k -ari o generici). Una soluzione diretta richiederebbe di memorizzare in ogni nodo un array di puntatori ai figli, ma questo è inefficiente se il numero di figli varia molto da nodo a nodo.

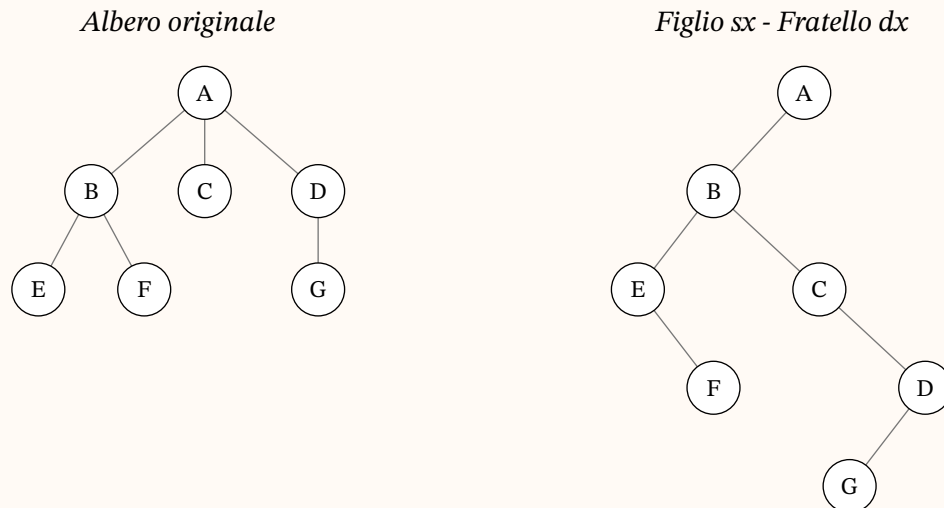
DEFINIZIONE 8.5.3 (Rappresentazione figlio sinistro - fratello destro). La rappresentazione **figlio sinistro - fratello destro** (left-child, right-sibling) consente di rappresentare un albero con numero arbitrario di figli utilizzando solo due puntatori per nodo:

- `left-child` : puntatore al **primo figlio** (il figlio più a sinistra) del nodo
- `right-sibling` : puntatore al **fratello destro** del nodo, cioè il nodo immediatamente a destra con lo stesso padre
- `parent` : puntatore al nodo padre (opzionale)

Se un nodo non ha figli, `left-child` vale NIL. Se un nodo è l'ultimo figlio del padre, `right-sibling` vale NIL.

ESEMPIO 8.5.4 (Albero generico e rappresentazione figlio-fratello). Consideriamo un albero in cui la radice A ha tre figli B, C, D ; il nodo B ha due figli E, F ; il nodo D ha un figlio G .

L'albero originale (a sinistra) e la sua rappresentazione figlio-fratello come albero binario (a destra):



Nella rappresentazione figlio sinistro - fratello destro:

- A : left-child = B , right-sibling = NIL
- B : left-child = E , right-sibling = C
- C : left-child = NIL, right-sibling = D
- D : left-child = G , right-sibling = NIL
- E : left-child = NIL, right-sibling = F
- F : left-child = NIL, right-sibling = NIL
- G : left-child = NIL, right-sibling = NIL

OSSERVAZIONE 8.5.5. Questa rappresentazione è particolarmente efficiente perché utilizza $O(n)$ spazio totale (due puntatori per nodo), indipendentemente dal grado massimo dell'albero. In pratica, trasforma un albero generico in un albero binario dove il figlio sinistro corrisponde al primo figlio e il figlio destro corrisponde al fratello successivo.

8.5.4. Proprietà degli alberi binari

TEOREMA 8.5.6 (Relazione nodi-altezza). Sia T un albero binario di altezza h . Allora:

- T ha al massimo $2^{h+1} - 1$ nodi
- T ha al massimo 2^h foglie
- Se T ha n nodi, allora $h \geq \lceil \log_2(n + 1) \rceil - 1$

Dimostrazione. Per induzione sull'altezza h .

Caso base ($h = 0$): l'albero ha un solo nodo (la radice, che è anche foglia). Si ha $2^{0+1} - 1 = 1$ nodo e $2^0 = 1$ foglia.

Passo induttivo: supponiamo la tesi vera per altezza $h - 1$. Un albero di altezza h ha una radice con due sottoalberi di altezza al più $h - 1$. Per ipotesi induttiva, ciascun sottoalbero ha al più $2^h - 1$ nodi, quindi il totale è al più $1 + 2(2^h - 1) = 2^{h+1} - 1$.

Per la terza proprietà: da $n \leq 2^{h+1} - 1$ si ottiene $h \geq \lceil \log_2(n + 1) \rceil - 1$. □

OSSERVAZIONE 8.5.7. Un albero binario **completo** (ogni nodo interno ha esattamente due figli e tutte le foglie sono allo stesso livello) di altezza h ha esattamente $2^{h+1} - 1$ nodi. Un albero binario **degenero** (ogni nodo interno ha esattamente un figlio) di altezza h ha esattamente $h + 1$ nodi ed è equivalente a una lista concatenata.

8.5.5. Visite di un albero binario

Le visite (o attraversamenti) di un albero binario permettono di esaminare tutti i nodi in un ordine specifico. Le tre visite fondamentali si distinguono per il momento in cui viene visitata la radice rispetto ai sottoalberi.

8.5.5.1. Visita in ordine (inorder)

La visita **inorder** visita prima il sottoalbero sinistro, poi la radice, poi il sottoalbero destro.

inorderTreeWalk

```
inorderTreeWalk(Node x){
    if(x != NIL){
        inorderTreeWalk(x.left);
        // visita x (es. stampa x.key)
        inorderTreeWalk(x.right);
    }
}
```

8.5.5.2. Visita anticipata (preorder)

La visita **preorder** visita prima la radice, poi il sottoalbero sinistro, poi il destro.

preorderTreeWalk

```
preorderTreeWalk(Node x){
    if(x != NIL){
        // visita x (es. stampa x.key)
        preorderTreeWalk(x.left);
        preorderTreeWalk(x.right);
    }
}
```

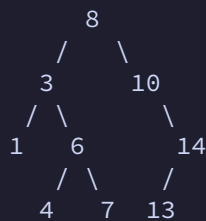
8.5.5.3. Visita posticipata (postorder)

La visita **postorder** visita prima il sottoalbero sinistro, poi il destro, poi la radice.

postorderTreeWalk

```
postorderTreeWalk(Node x){
    if(x != NIL){
        postorderTreeWalk(x.left);
        postorderTreeWalk(x.right);
        // visita x (es. stampa x.key)
    }
}
```

ESEMPIO 8.5.8 (Visite di un albero). Consideriamo il seguente albero binario:



Le tre visite producono:

- **Inorder:** 1, 3, 4, 6, 7, 8, 10, 13, 14
- **Preorder:** 8, 3, 1, 6, 4, 7, 10, 14, 13
- **Postorder:** 1, 4, 7, 6, 3, 13, 14, 10, 8

TEOREMA 8.5.9 (Complessità delle visite). Ciascuna delle tre visite di un albero binario con n nodi ha complessità $\Theta(n)$.

Dimostrazione. Sia $T(n)$ il tempo per visitare un albero con n nodi. Se l'albero è vuoto, $T(0) = c$ per una costante $c > 0$. Altrimenti, se il sottoalbero sinistro ha k nodi e il destro ne ha $n - k - 1$:

$$T(n) = T(k) + T(n - k - 1) + d \quad (120)$$

dove d è il costo costante per visitare il nodo corrente. Per sostituzione si dimostra che $T(n) = (c + d)n + c = \Theta(n)$. $\square$

8.6. Alberi Binari di Ricerca (BST)

8.6.1. Proprietà BST

DEFINIZIONE 8.6.1 (Albero binario di ricerca). Un **albero binario di ricerca** (BST, Binary Search Tree) è un albero binario in cui per ogni nodo x vale la seguente proprietà:

- per ogni nodo y nel sottoalbero sinistro di x : $y.\text{key} < x.\text{key}$
- per ogni nodo y nel sottoalbero destro di x : $y.\text{key} \geq x.\text{key}$

In altre parole, le chiavi strettamente minori si trovano nel sottoalbero sinistro, mentre le chiavi uguali o maggiori si trovano nel sottoalbero destro.

Nota. La proprietà BST è una proprietà **globale**: non basta che ogni nodo sia maggiore del figlio sinistro e minore del figlio destro; occorre che la relazione valga rispetto a **tutti** i nodi dei rispettivi sottoalberi.

TEOREMA 8.6.2 (Visita inorder e ordinamento). La visita inorder di un BST produce le chiavi in ordine non decrescente.

Dimostrazione. Per induzione sulla struttura dell'albero. Se l'albero è vuoto, la sequenza è vuota (ordinata per definizione). Altrimenti, per ipotesi induttiva, la visita inorder del sottoalbero sinistro produce le chiavi del sottoalbero sinistro in ordine. Per la proprietà BST, tutte queste chiavi sono $< x.\text{key}$. Analogamente, la visita inorder del sottoalbero destro produce chiavi $\geq x.\text{key}$ in ordine. La concatenazione (sottoalbero sinistro, radice, sottoalbero destro) è dunque ordinata. $\square$

8.6.2. Operazioni fondamentali

Tutte le operazioni fondamentali su un BST hanno complessità $O(h)$, dove h è l'altezza dell'albero.

8.6.2.1. Ricerca

DEFINIZIONE 8.6.3 (Ricerca in un BST). Data una chiave k e un BST T , l'operazione `Search( $T$ ,  $k$ )` restituisce un puntatore al nodo con chiave k , oppure NIL se la chiave non è presente.

Versione ricorsiva:

treeSearch

```
Node treeSearch(Node x, int k){
    if((x == NIL) || (k == x.key)){
        return x;
    }
    if(k < x.key){
        return treeSearch(x.left, k);
    } else {
        return treeSearch(x.right, k);
    }
}
```

Versione iterativa:

iterativeTreeSearch

```
Node iterativeTreeSearch(Node x, int k){
    while((x != NIL) && (k != x.key)){
        if(k < x.key){
            x := x.left;
        } else {
            x := x.right;
        }
    }
    return x;
}
```

Nota. La versione iterativa è generalmente preferita perché evita il costo dell'overhead delle chiamate ricorsive ed è più efficiente in termini di spazio (non usa lo stack di ricorsione).

Complessità: $O(h)$, dove h è l'altezza dell'albero. Ad ogni passo scendiamo di un livello, quindi il numero massimo di confronti è $h + 1$.

8.6.2.2. Minimo e Massimo

DEFINIZIONE 8.6.4 (Minimo e Massimo). In un BST, il **minimo** è il nodo raggiunto seguendo sempre il figlio sinistro a partire dalla radice. Il **massimo** è il nodo raggiunto seguendo sempre il figlio destro.

treeMinimum / treeMaximum

```
Node treeMinimum(Node x){
    while(x.left != NIL){
        x := x.left;
    }
    return x;
}

Node treeMaximum(Node x){
    while(x.right != NIL){
        x := x.right;
    }
    return x;
}
```

Complessità: $O(h)$ per entrambe le operazioni.

OSSERVAZIONE 8.6.5. La correttezza segue direttamente dalla proprietà BST: tutte le chiavi nel sottoalbero sinistro sono strettamente minori della radice, e tutte le chiavi nel sottoalbero destro sono maggiori o uguali.

8.6.2.3. Successore e Predecessore

DEFINIZIONE 8.6.6 (Successore). Il **successore** di un nodo x in un BST è il nodo con la più piccola chiave maggiore di $x.key$, cioè il nodo che segue x nella visita inorder.

Si distinguono due casi:

1. Se x ha un figlio destro, il successore è il **minimo del sottoalbero destro**.
2. Se x non ha figlio destro, il successore è il **primo antenato** di x il cui figlio sinistro è anch'esso antenato di x (cioè risaliamo finché non siamo un figlio sinistro).

treeSuccessor

```
Node treeSuccessor(Node x){
    if(x.right != NIL){
        return treeMinimum(x.right);
    }
    Node y = x.parent;
    while((y != NIL) && (x == y.right)){
        x := y;
        y := y.parent;
    }
    return y;
}
```

Predecessore: simmetricamente, il predecessore è il massimo del sottoalbero sinistro (se esiste), altrimenti il primo antenato di cui x è nel sottoalbero destro.

treePredecessor

```
Node treePredecessor(Node x){
    if(x.left != NIL){
        return treeMaximum(x.left);
    }
    Node y = x.parent;
    while((y != NIL) && (x == y.left)){
        x := y;
        y := y.parent;
    }
    return y;
}
```

Complessità: $O(h)$ per entrambe le operazioni, poiché si percorre al più un cammino dalla radice a una foglia.

8.6.2.4. Inserimento

DEFINIZIONE 8.6.7 (Inserimento in un BST). L'operazione `Insert( $T, z$ )` inserisce un nuovo nodo z nel BST T mantenendo la proprietà BST. Il nodo viene inserito come **foglia** nella posizione corretta.

treeInsert

```

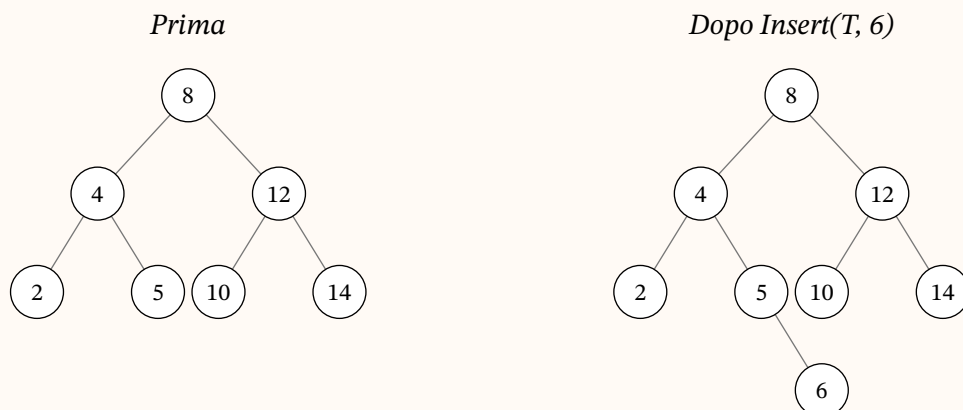
treeInsert(Tree T, Node z){
    Node y = NIL;
    Node x = T.root;
    while(x != NIL){
        y := x;
        if(z.key < x.key){
            x := x.left;
        } else {
            x := x.right;
        }
    }
    z.parent := y;
    if(y == NIL){
        T.root := z;          // l'albero era vuoto
    } else {
        if(z.key < y.key){
            y.left := z;
        } else {
            y.right := z;
        }
    }
    z.left := NIL;
    z.right := NIL;
}

```

Nota. La variabile y mantiene il puntatore al **padre** del nodo corrente x (tecnica detta «trailing pointer»). Quando x diventa NIL, y è il nodo a cui agganciare il nuovo nodo z . Si noti che le chiavi uguali vengono inserite nel sottoalbero destro, coerentemente con la proprietà BST adottata.

Complessità: $O(h)$, poiché scendiamo dalla radice fino a una posizione vuota.

ESEMPIO 8.6.8 (Inserimento in un BST). Inseriamo la chiave 6 nel seguente BST. L'algoritmo scende dalla radice confrontando 6 con ogni nodo: $6 < 8$ (sinistra), $6 > 4$ (destra), $6 > 5$ (destra). Il nodo viene inserito come figlio destro di 5.



8.6.2.5. Cancellazione

La cancellazione è l'operazione più complessa su un BST. Si distinguono tre casi in base alla struttura del nodo z da eliminare.

DEFINIZIONE 8.6.9 (Cancellazione da un BST). L'operazione `Delete( $T, z$ )` rimuove il nodo z dal BST T mantenendo la proprietà BST.

Caso 1 – z non ha figli (foglia): si rimuove z semplicemente aggiornando il puntatore del padre.

Caso 2 – z ha un solo figlio: si elimina z e si collega il figlio direttamente al padre di z .

Caso 3 – z ha due figli: si sostituisce z con il suo **successore** y (il minimo del sottoalbero destro di z). Siccome y non ha figlio sinistro (altrimenti non sarebbe il minimo), si ricade nel Caso 1 o 2 per rimuovere y dalla sua posizione originale.

Per semplificare il codice, definiamo una procedura ausiliaria che sostituisce un sottoalbero con un altro:

transplant

```
transplant(Tree T, Node u, Node v){
    if(u.parent == NIL){
        T.root := v;
    } else {
        if(u == u.parent.left){
            u.parent.left := v;
        } else {
            u.parent.right := v;
        }
    }
    if(v != NIL){
        v.parent := u.parent;
    }
}
```

Utilizzando `transplant`, la cancellazione è:

treeDelete

```

treeDelete(Tree T, Node z){
    if(z.left == NIL){
        // Caso 1 o 2: nessun figlio sinistro
        transplant(T, z, z.right);
    } else {
        if(z.right == NIL){
            // Caso 2: solo figlio sinistro
            transplant(T, z, z.left);
        } else {
            // Caso 3: due figli
            Node y = treeMinimum(z.right);
            if(y.parent != z){
                transplant(T, y, y.right);
                y.right := z.right;
                y.right.parent := y;
            }
            transplant(T, z, y);
            y.left := z.left;
            y.left.parent := y;
        }
    }
}

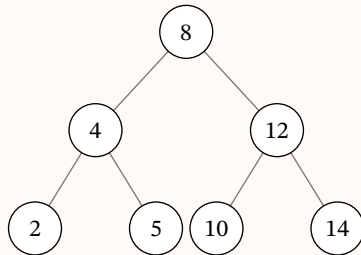
```

Complessità: $O(h)$, poiché `treeMinimum` richiede $O(h)$ nel caso peggiore e `transplant` richiede $O(1)$ (solo operazioni di puntatori).

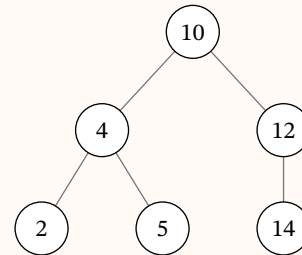
OSSERVAZIONE 8.6.10. La correttezza della procedura `treeDelete` si basa sul fatto che, nel Caso 3, il successore y di z (il minimo del sottoalbero destro) non ha figlio sinistro. Sostituendo z con y e spostando i sottoalberi in modo appropriato, la proprietà BST è preservata: tutte le chiavi nel nuovo sottoalbero sinistro di y erano nel sottoalbero sinistro di z (dunque minori di $y.key$), e tutte le chiavi nel nuovo sottoalbero destro di y erano nel sottoalbero destro di z escluso y (dunque maggiori o uguali a $y.key$).

ESEMPIO 8.6.11 (Cancellazione da un BST — Caso 3). Cancelliamo il nodo con chiave 8 (la radice) dal BST. Poiché 8 ha due figli, troviamo il suo successore: il minimo del sottoalbero destro è 10. Sostituiamo 8 con 10 e rimuoviamo 10 dalla posizione originale (Caso 1, foglia).

Prima: $Delete(T, 8)$



Dopo: 10 sostituisce 8



8.6.3. Analisi della complessità

Le prestazioni di un BST dipendono criticamente dalla sua altezza h .

TEOREMA 8.6.12 (Complessità delle operazioni su BST). Su un albero binario di ricerca di altezza h , le operazioni *Search*, *Minimum*, *Maximum*, *Successor*, *Predecessor*, *Insert* e *Delete* possono essere eseguite ciascuna in tempo $O(h)$.

Dimostrazione. Ciascuna delle operazioni di query (*Search*, *Minimum*, *Maximum*, *Successor*, *Predecessor*) percorre al più un cammino dalla radice a una foglia oppure da un nodo alla radice, visitando al più $h + 1$ nodi e spendendo tempo $O(1)$ per ciascuno.

Per *Insert*: la procedura `treeInsert` scende dalla radice fino a trovare una posizione NIL dove agganciare il nuovo nodo, percorrendo al più $h + 1$ nodi.

Per *Delete*: la procedura `treeDelete` richiede al più una chiamata a `treeMinimum` (che costa $O(h)$) e un numero costante di operazioni `transplant` (ciascuna in $O(1)$). Il costo totale è $O(h)$. $\square$

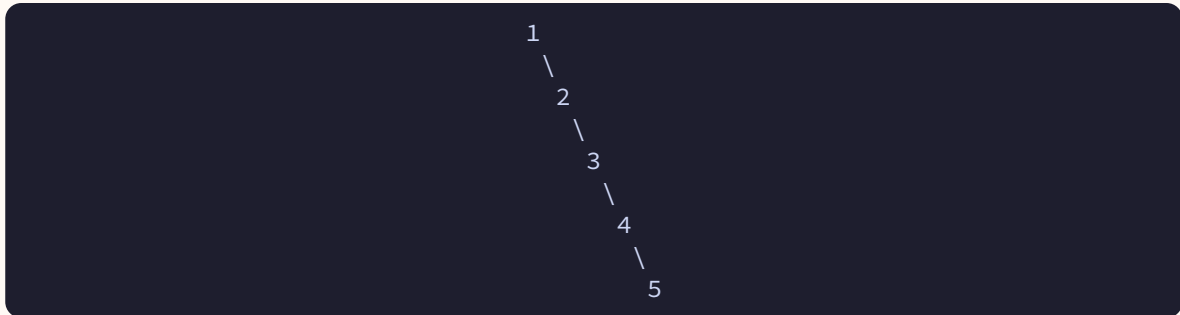
Operazione	Complessità	Descrizione
<code>Search(T, k)</code>	$O(h)$	Ricerca di una chiave
<code>Minimum(T)</code>	$O(h)$	Chiave minima
<code>Maximum(T)</code>	$O(h)$	Chiave massima
<code>Successor(x)</code>	$O(h)$	Successore di un nodo
<code>Predecessor(x)</code>	$O(h)$	Predecessore di un nodo
<code>Insert(T, z)</code>	$O(h)$	Inserimento di un nodo
<code>Delete(T, z)</code>	$O(h)$	Cancellazione di un nodo

Tabella 26: Complessità delle operazioni su BST in funzione dell'altezza h

8.6.3.1. Caso peggiore

Se le chiavi vengono inserite in ordine crescente (o decrescente), il BST degenera in una lista concatenata. In questo caso $h = n - 1$ e tutte le operazioni hanno complessità $O(n)$.

ESEMPIO 8.6.13 (BST degenero). Inserendo le chiavi 1, 2, 3, 4, 5 in quest'ordine si ottiene:



L'altezza è $h = 4 = n - 1$, e la ricerca di 5 richiede 5 confronti.

8.6.3.2. Caso migliore

Se l'albero è **bilanciato** (i sottoalberi sinistro e destro di ogni nodo differiscono in altezza di al più 1), allora $h = \Theta(\log n)$ e tutte le operazioni hanno complessità $O(\log n)$.

8.6.3.3. Caso medio

TEOREMA 8.6.14 (Altezza media di un BST). L'altezza attesa di un BST costruito inserendo n chiavi distinte in ordine casuale uniformemente distribuito è $O(\log n)$.

Nota. La dimostrazione completa di questo risultato richiede strumenti probabilistici avanzati. L'idea chiave è che un inserimento casuale produce una struttura analoga a quella del QuickSort randomizzato: la radice partiziona le chiavi e la profondità attesa è logaritmica.

Riassumendo:

Caso	Altezza h	Complessità operazioni
Peggior (degenere)	$n - 1$	$O(n)$
Migliore (bilanciato)	$\Theta(\log n)$	$O(\log n)$
Medio (casuale)	$O(\log n)$	$O(\log n)$

Tabella 27: Altezza e complessità nei diversi casi

8.6.4. Esempio completo

Mostriamo passo per passo la costruzione di un BST e le operazioni su di esso.

ESEMPIO 8.6.15 (Costruzione di un BST). Inseriamo le chiavi 8, 3, 10, 1, 6, 14, 4, 7, 13 in quest'ordine.

Passo 1 – Inserimento di 8 (radice):

```

      8
  
```

Passo 2 – Inserimento di 3 ($3 < 8$, va a sinistra):

```

      8
     /
    3
  
```

Passo 3 – Inserimento di 10 ($10 \geq 8$, va a destra):

```

      8
     / \
    3   10
  
```

Passo 4 – Inserimento di 1 ($1 < 8$, $1 < 3$, va a sinistra di 3):

```

      8
     / \
    3   10
   /
  1
  
```

Passo 5 – Inserimento di 6 ($6 < 8$, $6 \geq 3$, va a destra di 3):

```

      8
     / \
    3   10
   / \
  1   6
  
```

Passo 6 – Inserimento di 14 ($14 \geq 8$, $14 \geq 10$, va a destra di 10):

```

      8
     / \
    3   10
   / \  \
  1   6  14
  
```

Passo 7 – Inserimento di 4 ($4 < 8$, $4 \geq 3$, $4 < 6$, va a sinistra di 6):

```

      8
     / \
    3   10
   / \  \
  1   6  14
     /
    4
  
```

Passo 8 – Inserimento di 7 ($7 < 8$, $7 \geq 3$, $7 \geq 6$, va a destra di 6):

ESEMPIO 8.6.16 (Operazioni sul BST). Sull'albero costruito nell'esempio precedente eseguiamo le seguenti operazioni.

Ricerca di 6: $8 \rightarrow 3 \rightarrow 6$ (trovato dopo 3 confronti).

Ricerca di 5: $8 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow \text{NIL}$ (non trovato, 4 confronti).

Minimo: $8 \rightarrow 3 \rightarrow 1$ (il minimo è 1).

Massimo: $8 \rightarrow 10 \rightarrow 14$ (il massimo è 14).

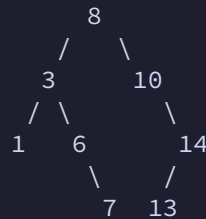
Successore di 6: il nodo 6 ha un figlio destro (7), quindi il successore è il minimo del sottoalbero destro, cioè 7.

Successore di 7: il nodo 7 non ha figlio destro, risaliamo: $7 \rightarrow 6$ (eravamo a destra), $6 \rightarrow 3$ (eravamo a destra), $3 \rightarrow 8$ (eravamo a sinistra, ci fermiamo). Il successore è 8.

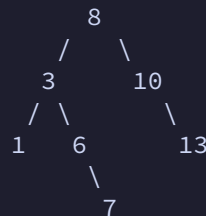
Predecessore di 10: il nodo 10 non ha figlio sinistro, risaliamo: $10 \rightarrow 8$ (eravamo a destra, ci fermiamo). Il predecessore è 8.

ESEMPIO 8.6.17 (Cancellazione dal BST). Partiamo dall'albero completo e mostriamo i tre casi di cancellazione.

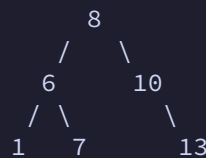
Caso 1 – Cancellazione di 4 (foglia): si rimuove il nodo e si imposta il figlio sinistro di 6 a NIL.



Caso 2 – Cancellazione di 14 (un solo figlio): si sostituisce 14 con il suo unico figlio 13.



Caso 3 – Cancellazione di 3 (due figli): il successore di 3 è 6 (minimo del sottoalbero destro). Si sostituisce 3 con 6:



La proprietà BST è mantenuta: $1 < 6 < 7 < 8 < 10 < 13$.

Nota. Per ottenere BST con altezza garantita $O(\log n)$ nel caso peggiore, si utilizzano alberi bilanciati come gli **AVL** o i **Red-Black Tree**. Queste strutture aggiungono operazioni di **rotazione** dopo inserimenti e cancellazioni per mantenere il bilanciamento, garantendo complessità $O(\log n)$ per tutte le operazioni.

8.7. Tabelle Hash

Le tabelle hash sono una realizzazione concreta del tipo di dato astratto **dizionario** che, sotto ipotesi ragionevoli, garantisce un tempo medio $O(1)$ per le operazioni fondamentali. Mentre gli ABR (Alberi Binari di Ricerca) richiedono un tempo $O(h)$ proporzionale all'altezza, le tabelle hash sfruttano un accesso diretto tramite una funzione che calcola la posizione dell'elemento a partire dalla chiave.

In questo capitolo presentiamo tre realizzazioni di dizionario con complessità crescente: le tavole a indirizzamento diretto (semplici ma con limitazioni di spazio), le tavole hash con concatenamento (pratiche ed efficienti in media), e le funzioni hash con il metodo della divisione.

8.7.1. Tavole a indirizzamento diretto

L'**indirizzamento diretto** è la tecnica più semplice: se l'universo delle chiavi è piccolo, possiamo usare un array in cui ogni posizione corrisponde a una chiave.

DEFINIZIONE 8.7.1 (Tavola a indirizzamento diretto). Sia $U = \{0, 1, \dots, m - 1\}$ l'universo delle chiavi. Una **tavola a indirizzamento diretto** è un array $T[0..m - 1]$ dove ogni posizione $T[k]$ corrisponde alla chiave k . Se l'insieme non contiene un elemento con chiave k , allora $T[k] = \text{NIL}$.

Le operazioni di dizionario sono immediate:

Operazioni su tavola a indirizzamento diretto

```
Direct-Address-Search( $T, k$ ) return  $T[k]$   
Direct-Address-Insert( $T, x$ )  $T[x.\text{key}] = x$   
Direct-Address-Delete( $T, x$ )  $T[x.\text{key}] = \text{NIL}$ 
```

TEOREMA 8.7.2 (Complessità dell'indirizzamento diretto). In una tavola a indirizzamento diretto, le operazioni *Search*, *Insert* e *Delete* richiedono ciascuna tempo $O(1)$.

Dimostrazione. Ogni operazione accede a una singola cella dell'array tramite indice diretto, senza alcuna ricerca. L'accesso a un array per indice richiede tempo costante nel modello RAM. $\square$

ESEMPIO 8.7.3 (Indirizzamento diretto). Consideriamo l'universo $U = \{0, 1, \dots, 9\}$ e l'insieme $K = \{2, 3, 5, 8\}$ di chiavi effettive. La tavola T ha 10 celle, di cui solo 4 contengono elementi:

Indice	Contenuto
0	NIL
1	NIL
2	elemento con chiave 2
3	elemento con chiave 3
4	NIL
5	elemento con chiave 5
6	NIL
7	NIL
8	elemento con chiave 8
9	NIL

Le 6 celle vuote rappresentano spazio sprecato.

OSSERVAZIONE 8.7.4. L'indirizzamento diretto è efficiente in tempo ma costoso in spazio: richiede un array di dimensione $|U|$, indipendentemente dal numero n di chiavi effettivamente memorizzate. Se l'universo U è molto grande (ad esempio, tutte le possibili stringhe di 20 caratteri), la tavola diventa impraticabile.

8.7.2. Tavole hash

Quando l'universo delle chiavi U è troppo grande per l'indirizzamento diretto, si ricorre alle **tavole hash**: strutture che usano un array di dimensione m molto più piccola di $|U|$, calcolando la posizione di ogni elemento tramite una funzione.

DEFINIZIONE 8.7.5 (Funzione hash). Una **funzione hash** è una funzione $h : U \rightarrow \{0, 1, \dots, m - 1\}$ che associa a ogni chiave k dell'universo una posizione (detta **valore hash**) nella tavola $T[0..m - 1]$. Diciamo che k viene **mappata** nella cella $h(k)$.

DEFINIZIONE 8.7.6 (Tavola hash). Una **tavola hash** è un array $T[0..m - 1]$ di dimensione m in cui un elemento con chiave k è memorizzato nella posizione $h(k)$, dove h è una funzione hash. La dimensione m è generalmente molto più piccola di $|U|$.

8.7.2.1. Collisioni

Poiché $|U| > m$, la funzione h non può essere iniettiva: esistono necessariamente chiavi distinte $k_1 \neq k_2$ tali che $h(k_1) = h(k_2)$.

DEFINIZIONE 8.7.7 (Collisione). Una **collisione** si verifica quando due chiavi distinte k_1 e k_2 hanno lo stesso valore hash: $h(k_1) = h(k_2)$.

Anche una funzione hash ben progettata non può evitare completamente le collisioni. Occorre quindi un meccanismo per gestirle. La tecnica più semplice è il **concatenamento**.

8.7.2.2. Risoluzione delle collisioni mediante concatenamento

DEFINIZIONE 8.7.8 (Concatenamento). Nel **concatenamento** (o *chaining* o *liste di trabocco*), ogni cella $T[j]$ della tavola hash contiene un puntatore alla testa di una lista concatenata di tutti gli elementi la cui chiave ha valore hash j . Se nessun elemento è mappato nella cella j , allora $T[j] = \text{NIL}$.

Le operazioni di dizionario con concatenamento sono le seguenti:

Operazioni su tavola hash con concatenamento

Chained-Hash-Insert(T, x) inserisci x in testa alla lista $T[h(x.\text{key})]$
Chained-Hash-Search(T, k) ricerca un elemento con chiave k nella lista $T[h(k)]$
Chained-Hash-Delete(T, x) cancella x dalla lista $T[h(x.\text{key})]$

OSSERVAZIONE 8.7.9. L'inserimento in testa ha costo $O(1)$ (assumendo che l'elemento da inserire non sia già presente). La ricerca richiede di scorrere la lista nella cella $T[h(k)]$: il tempo è proporzionale alla lunghezza di tale lista. La cancellazione, se le liste sono doppiamente concatenate, ha costo $O(1)$; con liste singolarmente concatenate, ha lo stesso costo della ricerca.

ESEMPIO 8.7.10 (Tavola hash con concatenamento). Sia $m = 9$ e $h(k) = k \bmod 9$. Inseriamo nell'ordine le chiavi 5, 28, 19, 15, 20, 33, 12, 17, 10.

Cella	Lista
0	$\emptyset$
1	$28 \rightarrow 19 \rightarrow 10 \rightarrow \text{NIL}$
2	$20 \rightarrow \text{NIL}$
3	$12 \rightarrow \text{NIL}$
4	$\emptyset$
5	$5 \rightarrow \text{NIL}$
6	$33 \rightarrow 15 \rightarrow \text{NIL}$
7	$\emptyset$
8	$17 \rightarrow \text{NIL}$

Ad esempio, $h(28) = 28 \bmod 9 = 1$ e $h(19) = 19 \bmod 9 = 1$: le chiavi 28 e 19 collidono nella cella 1. L'inserimento in testa fa sì che 19 preceda 28 nella lista (ma l'ordine dipende dalla sequenza degli inserimenti, non dal valore delle chiavi).

8.7.2.3. Analisi dell'hashing con concatenamento

L'analisi delle prestazioni richiede un'ipotesi sul comportamento della funzione hash.

DEFINIZIONE 8.7.11 (Fattore di carico). Data una tavola hash T con m celle in cui sono memorizzati n elementi, il **fattore di carico** è il rapporto $\alpha = \frac{n}{m}$. Il fattore α rappresenta il numero medio di elementi per cella.

Il fattore di carico α può essere minore, uguale o maggiore di 1. Se α è piccolo, la maggior parte delle liste è corta; se α è grande, le liste tendono ad allungarsi.

DEFINIZIONE 8.7.12 (Hashing uniforme semplice). L'ipotesi di **hashing uniforme semplice** richiede che qualsiasi chiave abbia la stessa probabilità di essere mappata in una qualsiasi delle m celle, indipendentemente dalla cella in cui sono mappate le altre chiavi. Indicando con n_j la lunghezza della lista $T[j]$, il numero totale di elementi è:

$$n = n_0 + n_1 + \dots + n_{m-1} \quad (121)$$

e il valore atteso della lunghezza di ciascuna lista è $E[n_j] = \alpha = \frac{n}{m}$.

Sotto questa ipotesi, possiamo analizzare il tempo medio di ricerca.

Caso pessimo. Nel caso pessimo, tutte le n chiavi sono mappate nella stessa cella, producendo una lista di lunghezza n . La ricerca richiede $\Theta(n)$, non meglio di una lista concatenata semplice. Per questo le tavole hash non si usano quando servono garanzie sul caso pessimo.

Caso medio. Il caso medio è molto più favorevole:

TEOREMA 8.7.13 (Ricerca senza successo). *In una tavola hash con concatenamento, nell'ipotesi di hashing uniforme semplice, una ricerca senza successo richiede un tempo medio $\Theta(1 + \alpha)$.*

Dimostrazione. Una chiave k non presente nella tavola viene mappata nella cella $h(k)$. Per l'ipotesi di hashing uniforme semplice, la lista in questa cella ha lunghezza attesa α . La ricerca senza successo esamina tutti gli elementi della lista (per verificare che nessuno abbia chiave k), impiegando un tempo proporzionale alla lunghezza della lista. Aggiungendo il tempo $O(1)$ per calcolare $h(k)$, si ottiene un tempo totale di $\Theta(1 + \alpha)$. $\square$

TEOREMA 8.7.14 (Ricerca con successo). *In una tavola hash con concatenamento, nell'ipotesi di hashing uniforme semplice, una ricerca con successo richiede un tempo medio $\Theta(1 + \alpha)$.*

Dimostrazione. Supponiamo che l'elemento cercato sia uno qualsiasi degli n elementi memorizzati, ciascuno con la stessa probabilità $\frac{1}{n}$. Il numero di elementi esaminati durante la ricerca di un elemento x_i (l' i -esimo inserito) è 1 più il numero di elementi inseriti dopo x_i nella stessa cella. Per l'hashing uniforme semplice, la probabilità che due chiavi collidano è $\frac{1}{m}$. Il numero atteso di elementi esaminati in una ricerca con successo è:

$$\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) = 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \quad (122)$$

Aggiungendo il tempo $O(1)$ per calcolare la funzione hash, il tempo totale è $\Theta(1 + \alpha)$. $\square$

OSSERVAZIONE 8.7.15. Il significato pratico di questa analisi è il seguente: se il numero di celle m è proporzionale al numero di elementi n , cioè $n = O(m)$, allora $\alpha = \frac{n}{m} = O(1)$ e tutte le operazioni di dizionario richiedono in media tempo $O(1)$.

ESEMPIO 8.7.16 (Scelta della dimensione della tavola). Se prevediamo di memorizzare circa $n = 1000$ elementi e vogliamo che ogni ricerca esamini in media al più 3 elementi, dobbiamo scegliere m in modo che $\alpha = \frac{n}{m} \leq 3$. Quindi $m \geq \frac{1000}{3} \approx 334$. In pratica, si sceglie m come un numero primo vicino a 334, ad esempio $m = 337$.

8.7.3. Funzioni hash

La qualità di una tavola hash dipende fortemente dalla scelta della funzione hash. Una funzione hash ideale soddisfa (approssimativamente) l'ipotesi di hashing uniforme semplice, distribuendo le chiavi il più uniformemente possibile tra le m celle.

OSSERVAZIONE 8.7.17. Purtroppo, raramente è nota la distribuzione di probabilità con cui le chiavi vengono estratte, e le chiavi potrebbero non essere generate in modo indipendente. Nella pratica si usano tecniche euristiche: un buon approccio consiste nel derivare il valore hash in modo che sia indipendente da qualsiasi regolarità presente nei dati. Ad esempio, il metodo della divisione calcola il valore hash come il resto della divisione fra la chiave e un numero primo scelto opportunamente, in modo da non essere correlato a regolarità nella distribuzione delle chiavi.

8.7.3.1. Interpretare le chiavi come numeri naturali

La maggior parte delle funzioni hash assume che l'universo delle chiavi sia un sottoinsieme di $\mathbb{N}$. Se le chiavi non sono numeri naturali, occorre prima convertirle in numeri.

ESEMPIO 8.7.18 (Conversione di stringhe in numeri). L'identificatore `pt` può essere interpretato come la coppia di interi (112, 116) (codici ASCII di `p` e `t`). Usando la base 128 (dimensione dell'alfabeto ASCII), otteniamo il numero naturale:

$$112 \cdot 128 + 116 = 14452 \quad (123)$$

In questo modo, qualsiasi stringa può essere trasformata in un numero naturale su cui applicare una funzione hash.

8.7.3.2. Il metodo della divisione

Il metodo della divisione è lo schema più semplice e diffuso per costruire funzioni hash.

DEFINIZIONE 8.7.19 (Metodo della divisione). Nel **metodo della divisione**, la funzione hash è definita come:

$$h(k) = k \bmod m \quad (124)$$

dove m è la dimensione della tavola. La chiave k viene mappata nella cella corrispondente al resto della divisione di k per m .

Il metodo è molto veloce: richiede una sola operazione di modulo.

ESEMPIO 8.7.20 (Metodo della divisione). Con $m = 12$ e $k = 100$:

$$h(100) = 100 \bmod 12 = 4 \quad (125)$$

La chiave 100 viene mappata nella cella 4.

ESEMPIO 8.7.21 (Applicazione completa). Consideriamo una tavola hash di dimensione $m = 13$ e le chiavi $K = \{5, 18, 27, 44, 31, 79, 55\}$.

Chiave k	$k \bmod 13$	Cella
5	$5 \bmod 13 = 5$	5
18	$18 \bmod 13 = 5$	5
27	$27 \bmod 13 = 1$	1
44	$44 \bmod 13 = 5$	5
31	$31 \bmod 13 = 5$	5
79	$79 \bmod 13 = 1$	1
55	$55 \bmod 13 = 3$	3

Le chiavi 5, 18, 44, 31 collidono tutte nella cella 5 (formando una lista di 4 elementi), mentre 27 e 79 collidono nella cella 1. La cella 3 contiene solo la chiave 55.

La scelta di m è cruciale per l'efficienza della funzione hash.

Nota (Scelta di m nel metodo della divisione).

- **Evitare potenze di 2:** se $m = 2^p$, allora $h(k) = k \bmod 2^p$ dipende solo dai p bit meno significativi della chiave, ignorando i bit più significativi. A meno che non sia noto che tutte le configurazioni dei bit meno significativi siano equiprobabili, questa scelta introduce un bias sistematico.
- **Evitare $m = 2^p - 1$:** se le chiavi sono stringhe di caratteri in base 2^p , la permutazione dei caratteri non cambia il valore hash, causando collisioni sistematiche.
- **Buona scelta:** un numero primo non troppo vicino a una potenza di 2. Ad esempio, per memorizzare circa 2000 stringhe e tollerare in media 3 confronti per ricerca, si sceglie $m \approx \frac{2000}{3} \approx 701$. Il numero 701 è primo e non è vicino a nessuna potenza di 2, quindi è una buona scelta.

8.7.4. Indirizzamento aperto

Nell'**indirizzamento aperto** (o *open addressing*) tutti gli elementi sono memorizzati direttamente nella tavola T , senza liste esterne. Ogni cella contiene al più un elemento: se si verifica una collisione, si cercano celle alternative all'interno della tavola stessa.

DEFINIZIONE 8.7.22 (Indirizzamento aperto). Nell'indirizzamento aperto, la tavola hash $T[0..m-1]$ contiene direttamente le chiavi (oppure NIL per le celle vuote). Il fattore di carico soddisfa sempre $\alpha = \frac{n}{m} \leq 1$. Quando si deve inserire una chiave k e la cella $h(k)$ è già occupata, si esaminano altre celle secondo una **sequenza di ispezione** determinata dalla chiave.

8.7.4.1. Sequenza di ispezione

Per gestire le collisioni, si associa a ogni chiave k una sequenza di posizioni da esaminare.

DEFINIZIONE 8.7.23 (Sequenza di ispezione). Una funzione di ispezione è una funzione $h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$ tale che per ogni chiave k , la sequenza $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ sia una permutazione di $\{0, 1, \dots, m-1\}$. In questo modo, ogni cella della tavola viene esaminata esattamente una volta.

8.7.4.2. Operazioni di dizionario

Nell'indirizzamento aperto, le operazioni di dizionario esaminano le celle nell'ordine dettato dalla sequenza di ispezione. Consideriamo prima il caso senza cancellazioni: ogni cella di T contiene una chiave oppure NIL (cella vuota).

Insert (senza cancellazioni)

```
Insert(T, k) i := 0; while (i < m) { j := h(k, i); if (T[j] == NIL)
{ T[j] := k; return j; } else i := i + 1; } error «overflow della tabella»
```

Search (senza cancellazioni)

```
Search(T, k) i := 0; while (i < m) { j := h(k, i); if (T[j] == k) return j;
if (T[j] == NIL) return -1; i := i + 1; } return -1;
```

OSSERVAZIONE 8.7.24. La Search si ferma quando trova la chiave cercata oppure quando incontra una cella vuota (NIL). L'idea chiave è: se la cella è vuota, l'algoritmo di Insert avrebbe inserito la chiave k proprio in quella cella (se fosse stata presente), quindi k non può trovarsi oltre.

8.7.4.3. Cancellazione con flag DELETED

La cancellazione nell'indirizzamento aperto non può semplicemente porre NIL nella cella, perché ciò interromperebbe le sequenze di ispezione delle chiavi inserite successivamente.

DEFINIZIONE 8.7.25 (Cancellazione logica). La cancellazione avviene in modo **logico**: la cella dell'elemento cancellato viene marcata con un valore speciale **DELETED** (diverso da NIL e da qualsiasi chiave). Una cella DELETED:

- nella **Insert** viene trattata come libera (vi si può inserire una nuova chiave);
- nella **Search** viene trattata come occupata (la ricerca prosegue oltre).

Insert (con cancellazioni)

```
Insert(T, k) i := 0; while (i < m) { j := h(k, i); if (T[j] == NIL or T[j] == DELETED) { T[j] := k; return j; } else i := i + 1; } error «overflow della tabella»
```

OSSERVAZIONE 8.7.26. L'algoritmo di Search non cambia: le celle DELETED vengono semplicemente «scavalcate» durante la ricerca, come se fossero occupate da una chiave diversa da quella cercata. Tuttavia, l'uso di DELETED degrada le prestazioni della ricerca, perché il tempo non dipende più solo dal fattore di carico α ma anche dal numero di celle marcate DELETED.

8.7.4.4. Analisi della complessità

L'analisi al caso medio richiede tre ipotesi:

1. $\alpha < 1$ (la tavola non è completamente piena);
2. non ci sono cancellazioni;
3. vale l'ipotesi di **hashing uniforme**:

DEFINIZIONE 8.7.27 (Hashing uniforme). L'ipotesi di **hashing uniforme** richiede che, per ogni chiave k , ognuna delle $m!$ permutazioni di $\{0, 1, \dots, m - 1\}$ sia equiprobabile come sequenza di ispezione di k . Questa è un'ipotesi più forte dell'hashing uniforme semplice (che riguarda solo la prima posizione).

Caso ottimo: $T(n, m) = O(1)$ — la prima ispezione trova la cella vuota o la chiave cercata.

Caso pessimo: $T(n, m) = \Theta(n)$ — tutte le chiavi collidono, si esamina l'intera tavola.

Caso medio: l'analisi si differenzia per ricerca senza successo e con successo.

TEOREMA 8.7.28 (Ricerca senza successo). Nelle ipotesi (1), (2), (3) sopra, il numero atteso di ispezioni in una ricerca senza successo è al più

$$\frac{1}{1 - \alpha} \quad (126)$$

Dimostrazione. Calcoliamo la probabilità di eseguire almeno i ispezioni:

- Probabilità di almeno 1 ispezione: 1 (si esamina sempre la prima cella).
- Probabilità di almeno 2 ispezioni (la prima cella è occupata): $\frac{n}{m} = \alpha$.
- Probabilità di almeno 3 ispezioni (le prime due occupate): $\frac{n}{m} \cdot \frac{n-1}{m-1} < \alpha^2$.
- In generale, la probabilità di almeno $i + 1$ ispezioni è al più α^i .

Il numero atteso di ispezioni è:

$$\sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha} \quad (127)$$

dove si usa la convergenza della serie geometrica per $\alpha < 1$. □

TEOREMA 8.7.29 (Ricerca con successo). *Nelle ipotesi (1), (2), (3) sopra, il numero atteso di ispezioni in una ricerca con successo è al più*

$$\frac{1}{\alpha} \ln \frac{1}{1-\alpha} \quad (128)$$

Dimostrazione. La ricerca di una chiave k presente nella tavola percorre le stesse posizioni esaminate quando k è stata inserita. Se k è stata la $(i + 1)$ -esima chiave inserita, al momento dell'inserimento il fattore di carico era $\frac{i}{m}$, e il numero atteso di ispezioni era al più $\frac{1}{1-i/m} = \frac{m}{m-i}$.

Il numero atteso di ispezioni, mediato su tutte le n chiavi presenti:

$$\frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} < \frac{1}{\alpha} \ln \frac{1}{1-\alpha} \quad (129)$$

dove l'ultimo passaggio segue dall'approssimazione della somma con un integrale. □

TEOREMA 8.7.30 (Inserimento). *Nelle ipotesi (1), (2), (3), il numero atteso di ispezioni per un inserimento è al più $\frac{1}{1-\alpha}$, lo stesso della ricerca senza successo.*

Dimostrazione. L'inserimento di una chiave k esamina le stesse celle che esaminerebbe una ricerca senza successo di k (cercando la prima cella vuota). □

ESEMPIO 8.7.31 (Prestazioni al variare di α). La tabella mostra il numero atteso di ispezioni per vari valori di α :

α	Occupazione	Ricerca senza successo ($\frac{1}{1-\alpha}$)	Ricerca con successo ($\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$)
1/10	10%	1.11	1.05
1/2	50%	2	1.38
9/10	90%	10	2.55

Con una tavola piena al 50% ($\alpha = \frac{1}{2}$), una ricerca senza successo esamina in media solo 2 celle: prestazioni eccellenti.

8.7.4.5. Ispezione lineare

Il metodo più semplice per calcolare la sequenza di ispezione è l'**ispezione lineare** (o *linear probing*).

DEFINIZIONE 8.7.32 (Ispezione lineare). Nell'**ispezione lineare**, data una funzione hash ausiliaria $h' : U \rightarrow \{0, 1, \dots, m-1\}$, la funzione di ispezione è:

$$h(k, i) = (h'(k) + i) \bmod m \quad (130)$$

La sequenza di ispezione parte dalla posizione $h'(k)$ e procede ciclicamente: $\langle h'(k), h'(k) + 1, h'(k) + 2, \dots, 0, 1, \dots, h'(k) - 1 \rangle$.

ESEMPIO 8.7.33 (Ispezione lineare). Con $m = 7$ e $h'(k) = k \bmod 7$, la sequenza di ispezione per la chiave $k = 9$ è:

- $h'(9) = 9 \bmod 7 = 2$
- Sequenza: $\langle 2, 3, 4, 5, 6, 0, 1 \rangle$

Il punto di partenza dipende dalla chiave (tramite $h'(k)$), ma il passo di ispezione è costante (+1).

OSSERVAZIONE 8.7.34. L'ispezione lineare è facile da implementare ma presenta un problema noto come **addensamento primario** (o *clustering primario*): si formano lunghe file di celle occupate (blocchi contigui), che aumentano il tempo medio di ricerca. Metodi come l'**ispezione quadratica** e il **doppio hash** riducono questo fenomeno.

8.7.4.6. Ispezione quadratica

L'**ispezione quadratica** (o *quadratic probing*) usa una funzione hash della forma:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \quad (131)$$

dove h' è una funzione hash ausiliaria, c_1 e $c_2 \neq 0$ sono costanti ausiliarie e $i = 0, 1, \dots, m - 1$. La posizione iniziale di ispezione è $T[h'(k)]$; le successive posizioni sono scostate di quantità dipendenti da un polinomio di secondo grado in i .

OSSERVAZIONE 8.7.35. Questo metodo funziona molto meglio dell'ispezione lineare, ma per poter esaminare l'intera tavola i valori di c_1 , c_2 e m devono soddisfare particolari restrizioni. Inoltre, se due chiavi hanno la stessa posizione iniziale di ispezione, esse avranno la medesima sequenza di ispezione. Questa proprietà crea una forma meno grave di clustering detta **addensamento secondario**.

8.7.4.7. Doppio hash

Il doppio hash è la tecnica più efficace per l'indirizzamento aperto, in quanto produce sequenze di ispezione che approssimano meglio l'hashing uniforme.

DEFINIZIONE 8.7.36 (Doppio hash). Nel **doppio hash**, date due funzioni hash ausiliarie h_1 e h_2 , la funzione di ispezione è:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \quad (132)$$

Sia il punto di partenza ($h_1(k)$) sia il passo ($h_2(k)$) dipendono dalla chiave. Si richiede che $h_2(k)$ sia coprimo con m per garantire che la sequenza sia una permutazione completa.

Nota (Scelta tipica per il doppio hash). Una scelta comune è m primo con $h_1(k) = k \bmod m$ e $h_2(k) = 1 + (k \bmod (m - 1))$. Il valore $h_2(k)$ è sempre compreso tra 1 e $m - 1$ ed è automaticamente coprimo con m (poiché m è primo).

ESEMPIO 8.7.37 (Doppio hash). Con $m = 13$, $h_1(k) = k \bmod 13$ e $h_2(k) = 1 + (k \bmod 12)$, inseriamo le chiavi 10, 24, 33:

- $k = 10$: $h_1(10) = 10$, cella 10 libera $\rightarrow$ inserito in posizione 10.
- $k = 24$: $h_1(24) = 11$, cella 11 libera $\rightarrow$ inserito in posizione 11.
- $k = 33$: $h_1(33) = 7$, cella 7 libera $\rightarrow$ inserito in posizione 7.
- $k = 27$: $h_1(27) = 1$, cella 1 libera $\rightarrow$ inserito in posizione 1.
- $k = 14$: $h_1(14) = 1$, collisione! $h_2(14) = 1 + 14 \bmod 12 = 3$. Provo $h(14, 1) = (1 + 3) \bmod 13 = 4$, cella 4 libera $\rightarrow$ inserito in posizione 4.

8.7.5. Confronto tra realizzazioni di dizionario

Per concludere, confrontiamo le quattro realizzazioni di dizionario studiate.

Operazione	Indir. diretto	Hash + concat. (medio)	Hash aperto (medio)	ABR (peggiore)
Search	$O(1)$	$O(1 + \alpha)$	$O(\frac{1}{1-\alpha})$	$O(h)$
Insert	$O(1)$	$O(1)$	$O(\frac{1}{1-\alpha})$	$O(h)$
Delete	$O(1)$	$O(1)$	$O(\frac{1}{1-\alpha})^*$	$O(h)$
Min/Max	$O(m)$	—	—	$O(h)$
Succ/Pred	$O(m)$	—	—	$O(h)$
Spazio	$O(U)$	$O(m + n)$	$O(m)$	$O(n)$

* Con flag DELETED; degrada con molte cancellazioni.

OSSERVAZIONE 8.7.38. Le tavole hash offrono il miglior tempo medio per le operazioni fondamentali (Search, Insert, Delete), ma non supportano efficientemente le operazioni di Minimum, Maximum, Successor e Predecessor. L'indirizzamento aperto ha il vantaggio di non usare puntatori (migliore uso della cache), ma richiede $\alpha < 1$ e la cancellazione è problematica. Il concatenamento è più flessibile (α può superare 1) e gestisce le cancellazioni in modo naturale. La tavola a indirizzamento diretto è praticabile solo quando l'universo delle chiavi è piccolo.

CAPITOLO 9

Programmazione Dinamica

9.1. Introduzione alla Programmazione Dinamica

La **programmazione dinamica** (PD) è un paradigma algoritmico per la soluzione di problemi di **ottimizzazione**. Nasce dall'osservazione che molti problemi ricorsivi ricalcolano le stesse soluzioni più volte: la PD evita questo spreco memorizzando i risultati già ottenuti.

Nota (PD, Divide et Impera e Greedy a confronto). Questi tre paradigmi risolvono problemi scomponendoli in sotto-problemi, ma differiscono in modo cruciale:

- **Divide et Impera:** i sotto-problemi sono **indipendenti** (non si sovrappongono). Ogni sotto-problema si risolve una sola volta perché nessun altro sotto-problema lo richiede. Esempio: MergeSort divide l'array in due metà che non condividono elementi.
- **Programmazione Dinamica:** i sotto-problemi si **sovrappongono** (gli stessi sotto-problemi vengono richiesti più volte da sotto-problemi diversi). La PD li risolve una sola volta e memorizza i risultati in una tabella. Esempio: Fibonacci ricorsivo ricalcola $\text{Fib}(k)$ esponenzialmente molte volte.
- **Greedy:** ad ogni passo si fa la scelta **localmente ottima**, senza mai reconsiderarla. Funziona solo quando la scelta locale è anche globalmente ottima.

Due proprietà devono essere soddisfatte perché la PD sia applicabile:

1. **Sottostruttura ottima:** la soluzione ottima del problema contiene al suo interno le soluzioni ottime dei sotto-problemi. In altre parole, se sappiamo risolvere in modo ottimo i «pezzi più piccoli», possiamo combinarli per ottenere la soluzione ottima del problema grande.
2. **Ripetizione dei sotto-problemi:** l'albero delle chiamate ricorsive risolve più volte lo stesso sotto-problema. È proprio questa ripetizione che rende la ricorsione pura inefficiente e la PD vantaggiosa.

9.1.1. Struttura di un algoritmo PD (stile bottom-up)

Nota (Top-down vs bottom-up). Esistono due modi di organizzare il calcolo di un algoritmo PD:

- **Top-down** (*dall'alto verso il basso*, per memoizzazione): si parte dalla formulazione ricorsiva del problema originale e si scende verso i sotto-problemi più piccoli, memorizzando i risultati in una tabella per non ricalcolarli.
- **Bottom-up** (*dal basso verso l'alto*, per tabulazione): si parte dai sotto-problemi più piccoli (casi base) e si riempie la tabella in ordine crescente fino al problema originale, senza usare la ricorsione.

I due approcci hanno lo stesso costo asintotico; il bottom-up è in genere preferibile perché evita l'overhead delle chiamate ricorsive.

Un algoritmo PD bottom-up si articola in quattro fasi:

1. **Definizione dei sotto-problemi e dimensionamento della tabella.** Si identificano i sotto-problemi Π_i e si alloca una struttura (array, matrice) per memorizzare i loro risultati.
2. **Soluzione diretta dei sotto-problemi elementari** (casi base) e memorizzazione del risultato nella tabella.
3. **Definizione della regola ricorsiva di riempimento della tabella**, per ottenere la soluzione di un sotto-problema a partire dalle soluzioni di sotto-problemi già risolti.
4. **Restituzione del risultato** relativo al problema originale (dimensione n).

Da ricordare. Ricetta pratica per affrontare un problema con PD:

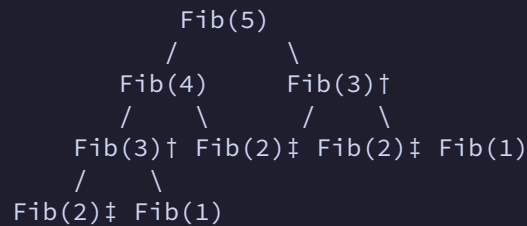
1. Verificare che il problema abbia **sottostruttura ottima** e **sotto-problemi ripetuti**.
2. Scrivere la **formula ricorsiva** che lega la soluzione di un sotto-problema a quelle più piccole, partendo dalla domanda: «Qual è l'ultima scelta? Quali sotto-problemi genera?»
3. Decidere la **struttura della tabella** (array 1D, matrice 2D) e l'**ordine di riempimento** (dai casi base al problema originale).
4. Se serve la soluzione (non solo il valore ottimo), aggiungere una struttura ausiliaria per la **ricostruzione**.

Nota. Rispetto alla ricorsione pura, l'approccio bottom-up non ha overhead di chiamate ricorsive e garantisce che ogni sotto-problema sia risolto esattamente una volta.

9.2. Fibonacci con Programmazione Dinamica

9.2.1. Motivazione: il costo della ricorsione naive

L'approccio ricorsivo diretto ha costo esponenziale. La ricorrenza $T(n) = T(n-1) + T(n-2) + O(1)$ ha soluzione $T(n) = O(\varphi^n)$ con $\varphi \approx 1.618$. Il motivo è la **ripetizione di sotto-problemi**: nell'albero delle chiamate per $n = 5$,



Fib(3) viene calcolato 2 volte (marcato con †), Fib(2) viene calcolato 3 volte (marcato con ‡).

OSSERVAZIONE 9.2.1. Nell'albero qui sopra, i sotto-problemi distinti sono solo 6 (Fib(0), ..., Fib(5)), ma le chiamate ricorsive totali sono 15. Questo rapporto tra sotto-problemi distinti e chiamate totali peggiora drasticamente all'aumentare di n : è la **ripetizione dei sotto-problemi** che la PD elimina. Con la PD, ogni sotto-problema viene risolto una sola volta, portando la complessità da $O(\varphi^n)$ a $\Theta(n)$.

9.2.2. Approccio bottom-up con tabella

DEFINIZIONE 9.2.2 (Algoritmo Fib (PD bottom-up)). L' i -esimo numero di Fibonacci è calcolato riempiendo un array F di dimensione $n + 1$:

- Π_i : calcolo dell' i -esimo numero di Fibonacci, $0 \leq i \leq n$
- Tabella PD: array F di dimensione $n + 1$

Fib(n) — versione con array

```

Fib(n) {
    F = nuovo array di dimensione n+1;
    F[0] = 0;
    F[1] = 1;
    for (i = 2; i <= n; i++) {
        F[i] = F[i-1] + F[i-2];
    }
    return F[n];
}
  
```

La complessità è $T(n) = \Theta(n)$ (numero di addizioni). Il valore esatto è $F_n \sim \frac{1}{\sqrt{5}}\varphi^n$ con $\varphi \approx 1.618$ (rapporto aureo).

9.2.3. Ottimizzazione dello spazio

Poiché ogni iterazione usa solo i due valori precedenti, è sufficiente mantenere tre variabili:

Fib(n) — versione $O(1)$ spazio

```

Fib(n) {
    if (n == 0 || n == 1) return n;
    a = 0;
    b = 1;
    for (i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

```

$T(n) = \Theta(n)$ addizioni, $S(n) = O(1)$.

9.2.4. Algoritmo pseudo-polinomiale

DEFINIZIONE 9.2.3 (Algoritmo pseudo-polinomiale). Un algoritmo è **pseudo-polinomiale** se è polinomiale (o lineare) nel **valore** dell'input, ma esponenziale nella **dimensione** della sua rappresentazione binaria.

Per Fibonacci: l'istanza è $I = n$, con $|I| = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$ bit. Quindi:

$$T(n) = \Theta(n) = \Theta(2^{\log n}) = \Theta(2^{|I|}) \quad (133)$$

L'algoritmo è lineare nel valore di n , ma esponenziale nella dimensione dell'input.

9.2.5. Memoizzazione (approccio top-down)

Un'alternativa al bottom-up è la **memoizzazione**: si parte dalla definizione ricorsiva, ma si memorizza ogni risultato in una tabella (inizializzata con un valore sentinella) prima di restituirlo.

Fib-memo(n)

```

Fib-memo(n) {
    M = nuovo array di n+1 elementi, tutti -1;
    return Fib-aux(n, M);
}

Fib-aux(n, M) {
    if (M[n] != -1) return M[n];
    if (n == 0 || n == 1) { M[n] = n; return n; }
    M[n] = Fib-aux(n-1, M) + Fib-aux(n-2, M);
    return M[n];
}

```

$T(n) = \Theta(n)$: ogni sotto-problema è risolto una sola volta, le chiamate successive trovano il valore in $O(1)$.

Nota. Bottom-up vs memoizzazione.

- **Bottom-up:** riempie la tabella in ordine topologico, nessun overhead di ricorsione, accesso sequenziale alla memoria (cache-friendly). Preferibile quando tutti i sotto-problemi devono essere risolti.
- **Memoizzazione:** segue l'ordine naturale della ricorsione, risolve solo i sotto-problemi effettivamente raggiungibili (utile con tabelle sparse), ma ha overhead di chiamate ricorsive e rischio di stack overflow per n grande.

9.3. Il problema del Taglio delle Corde

9.3.1. Definizione del problema

DEFINIZIONE 9.3.1 (Taglio delle Corde (Rod Cutting)). Data una corda di acciaio di n cm e un listino prezzi p_i (prezzo di un pezzo di i cm, $1 \leq i \leq n$), trovare il **ricavo massimo** ottenibile tagliando la corda e vendendo i singoli pezzi.

ESEMPIO 9.3.2 ($n = 4$). Listino: $p_1 = 1, p_2 = 5, p_3 = 8, p_4 = 9$.

Una corda di 4 cm ha $2^{n-1} = 8$ tagli possibili. Il ricavo massimo è $r_4 = 10$, ottenuto con due pezzi da 2 cm ($p_2 + p_2 = 5 + 5$).

La sottostruttura ottima è: fissato il primo taglio a i cm, il rimanente pezzo di $(n - i)$ cm va tagliato in modo ottimale. Quindi:

$$r_n = \max_{1 \leq i \leq n} \{p_i + r_{n-i}\} \quad (134)$$

con $r_0 = 0$.

Nota. Leggere la formula: stiamo provando ogni possibile primo taglio ($i = 1, 2, \dots, n$ cm) e per ciascuno calcoliamo il ricavo come «prezzo del pezzo tagliato (p_i) + ricavo ottimo del pezzo rimanente (r_{n-i})». Il massimo tra tutte queste opzioni è il ricavo ottimo r_n . Il caso $i = n$ corrisponde a non tagliare affatto (vendere la corda intera).

9.3.2. Soluzione ricorsiva

CUT-ROD(p, n)

```

CUT-ROD(p, n) {
    // p: array dei prezzi (dim n); n: lunghezza del pezzo da tagliare
    if (n == 0) return 0;
    q = -inf;
    for (i = 1; i <= n; i++) {
        q = max{q, p(i) + CUT-ROD(p, n-i)};
    }
    return q;
}

```

9.3.2.1. Analisi della complessità

Sia $T(n)$ il numero di chiamate ricorsive:

$$T(n) = \begin{cases} 1 & n = 0 \\ 1 + \sum_{i=1}^n T(n-i) & n > 0 \end{cases} \quad (135)$$

Per sostituzione: $T(n) = 1 + \sum_{j=0}^{n-1} T(j)$. Per induzione su n si dimostra $T(n) = 2^n$.

Dimostrazione (per induzione). Base: $n = 0: T(0) = 1 = 2^0$. ✓

Ipotesi induttiva: $\forall j, 0 \leq j < n : T(j) = 2^j$.

Passo: $T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 1 + \sum_{j=0}^{n-1} 2^j = 1 + \frac{2^n - 1}{2 - 1} = 2^n$. ✓

□

9.3.3. Soluzione con Programmazione Dinamica

Le quattro fasi:

1. Π_i : taglio ottimale di una corda di i cm, $0 \leq i \leq n$. Tabella PD: array r di dimensione $n + 1$.
2. $\Pi_0: r_0 = 0 \rightarrow r[0] = 0$.
3. $r[j] = \max_{1 \leq i \leq j} \{p(i) + r[j-i]\}$, calcolato per $j = 1, \dots, n$.
4. Restituire $r[n]$.

CUT-ROD-PD(p, n)

```

CUT-ROD-PD(p, n) {
    r = nuovo array di dimensione n+1; // 0(1)
    r[0] = 0;
    for (j = 1; j <= n; j++) {           // pezzo di corda di j cm
        q = -inf;
        for (i = 1; i <= j; i++) {       // 0(j)
            if (q < p(i) + r[j-i]) {
                q = p(i) + r[j-i];
            }
        }
        r[j] = q;
    }
    return r[n];
}

```

$$T(n) = \Theta(1) + \sum_{j=1}^n \Theta(j) = \Theta\left(\sum_{j=1}^n j\right) = \Theta(n^2), S(n) = \Theta(n).$$

ESEMPIO 9.3.3 (Simulazione con $n = 4$). $p = [0, 1, 5, 8, 9]$, $r = [0, -, -, -, -]$.

- $j = 1$: un solo taglio possibile, $i = 1$: $p(1) + r[0] = 1 + 0 = 1$. $\rightarrow r[1] = 1$
- $j = 2$: proviamo $i = 1$: $p(1) + r[1] = 1 + 1 = 2$; $i = 2$: $p(2) + r[0] = 5 + 0 = 5$. Il massimo è 5 (meglio vendere un pezzo da 2 che due da 1). $\rightarrow r[2] = 5$
- $j = 3$: proviamo $i = 1$: $p(1) + r[2] = 1 + 5 = 6$; $i = 2$: $p(2) + r[1] = 5 + 1 = 6$; $i = 3$: $p(3) + r[0] = 8 + 0 = 8$. Il massimo è 8 (vendere la corda intera). $\rightarrow r[3] = 8$
- $j = 4$: proviamo $i = 1$: $p(1) + r[3] = 1 + 8 = 9$; $i = 2$: $p(2) + r[2] = 5 + 5 = 10$; $i = 3$: $p(3) + r[1] = 8 + 1 = 9$; $i = 4$: $p(4) + r[0] = 9 + 0 = 9$. Il massimo è 10 (due pezzi da 2). $\rightarrow r[4] = 10$

Risultato finale: $r = [0, 1, 5, 8, 10]$.

9.3.4. Ricostruzione della soluzione

Si aggiunge un array s per memorizzare il primo taglio ottimale per ogni lunghezza:

CUT-ROD-PD con ricostruzione

```

CUT-ROD-PD(p, n) {
    r = nuovo array di dimensione n+1;
    s = nuovo array di dimensione n    // [1..n]
    r[0] = 0;
    for (j = 1; j <= n; j++) {
        q = -inf;
        for (i = 1; i <= j; i++) {
            if (q < p(i) + r[j-i]) {
                q = p(i) + r[j-i];
                s[j] = i;
            }
        }
        r[j] = q;
    }
    return s, r;
}

```

PRINT-CUT-ROD(p, n)

```

PRINT-CUT-ROD(p, n) {
    (r, s) = CUT-ROD-PD(p, n);
    ricavo = r[n];
    while (n > 0) {
        print s[n];
        n = n - s[n];
    }
    print "ricavo massimo: " ricavo;
}

```

ESEMPIO 9.3.4 (Ricostruzione per $n = 4$). $s = [0, 1, 2, 3, 2]$. Esecuzione di PRINT-CUT-ROD:

- stampa $s[4] = 2$, $n := 4 - 2 = 2$
- stampa $s[2] = 2$, $n := 2 - 2 = 0$
- fine

Il taglio ottimale è: due pezzi da 2 cm, ricavo = 10.

9.4. Longest Common Subsequence (LCS)

La **Longest Common Subsequence** (sottosequenza comune massima) è un problema classico con importanti applicazioni pratiche: gli strumenti `diff` per confrontare file, l'allineamento di sequenze DNA in bioinformatica e il rilevamento di plagio lo usano come componente fondamentale.

9.4.1. Definizioni

DEFINIZIONE 9.4.1 (Sottosequenza). Z è una **sottosequenza** di $X = x_1x_2\dots x_m$ se Z si ottiene da X cancellando zero o più caratteri (non necessariamente consecutivi, ma mantenendo l'ordine).

X di lunghezza m ha 2^m sottosequenze.

ESEMPIO 9.4.2. $X = \text{SPIEGARE}$. Sono sottosequenze di X : SPIA, SPIE, SPIARE, PERE, SPG, ...

I caratteri non devono essere consecutivi, ma devono mantenere l'ordine relativo.

Attenzione: Non confondere **sottosequenza** con **sottostringa**:

- **Sottostringa:** i caratteri devono essere **consecutivi**. Ad esempio, PIEG è sottostringa di SPIEGARE.
- **Sottosequenza:** i caratteri mantengono l'ordine ma possono **non essere consecutivi**. Ad esempio, SPIARE è sottosequenza di SPIEGARE (si salta la G), ma non è una sottostringa.

Un anagramma (stessi caratteri in ordine diverso) **non** è una sottosequenza.

DEFINIZIONE 9.4.3 (Sottosequenza Comune Massima (LCS)). Date due sequenze X di m caratteri e Y di n caratteri, una **sottosequenza comune** (CS) è una sequenza Z che è contemporaneamente sottosequenza di X e di Y .

Una **sottosequenza comune massima** (LCS) è una CS di lunghezza massima.

ESEMPIO 9.4.4. $X = \text{SPIEGARE}$, $Y = \text{OSPITARE}$.

$Z = \text{SPIARE}$ è una LCS di X e Y (lunghezza 6).

Il problema con forza bruta richiede di enumerare tutte le 2^m sottosequenze di X e verificare ciascuna come sottosequenza di Y : $T(n, m) = O(2^m \cdot n)$.

9.4.2. Sottostruttura ottima

Denotiamo con $X_i = x_1x_2\dots x_i$ l' i -esimo **prefisso** di X ($0 \leq i \leq m$, con $X_0 = \varepsilon$, la stringa vuota).

I sotto-problemi sono: $\Pi_{i,j} = |\text{LCS}(X_i, Y_j)|$, con $0 \leq i \leq m$, $0 \leq j \leq n$. Si usa una matrice c di dimensione $(m+1) \times (n+1)$, dove $c[i, j] = |\text{LCS}(X_i, Y_j)|$.

Per ricavare la formula ricorsiva, ragioniamo sugli ultimi caratteri dei prefissi X_i e Y_j .

Nota (Intuizione: cosa succede con gli ultimi caratteri?). Supponiamo di conoscere una LCS Z di X_i e Y_j . Confrontiamo x_i con y_j :

- **Se $x_i = y_j$ (match):** questo carattere comune *deve* far parte della LCS. Lo «consumiamo» e cerchiamo la LCS dei prefissi accorciati X_{i-1} e Y_{j-1} . La lunghezza è $c[i-1, j-1] + 1$.
- **Se $x_i \neq y_j$ (mismatch):** almeno uno dei due ultimi caratteri *non* è nella LCS. Proviamo due strade: (a) scartare x_i e cercare LCS di X_{i-1} e Y_j ; oppure (b) scartare y_j e cercare LCS di X_i e Y_{j-1} . Prendiamo la più lunga: $\max(c[i-1, j], c[i, j-1])$.
- **Se $i = 0$ o $j = 0$ (caso base):** una delle due sequenze è vuota, quindi non esistono sotto-sequenze comuni: $c[i, j] = 0$.

Questo ragionamento ci permette di ridurre il problema a sotto-problemi più piccoli — esattamente la struttura che serve alla PD.

Il seguente teorema formalizza questa intuizione.

TEOREMA 9.4.5 (Sottostruttura ottima di LCS). Siano $X = x_1 \dots x_m$ e $Y = y_1 \dots y_n$, e sia $Z = z_1 \dots z_k$ una LCS di X e Y :

1. Se $x_m = y_n$, allora $z_k = x_m = y_n$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1} .
2. Se $x_m \neq y_n$ e $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y .
3. Se $x_m \neq y_n$ e $z_k \neq y_n$, allora Z è una LCS di X e Y_{n-1} .

In sintesi: una LCS di due sequenze contiene come prefisso una LCS dei loro prefissi.

Dimostrazione (per assurdo). **Caso 1** ($x_m = y_n$): supponiamo per assurdo $z_k \neq x_m$. Allora $W = Zx_m$ è CS di X e Y con $|W| = k + 1 > k$, contraddizione con $Z \in \text{LCS}$.

Per dimostrare che Z_{k-1} è LCS di X_{m-1} e Y_{n-1} : per assurdo, esiste W CS di X_{m-1} e Y_{n-1} con $|W| > k - 1$, cioè $|W| \geq k$. Allora Wx_m è CS di X e Y con $|Wx_m| \geq k + 1$, contraddizione.

Caso 2 ($x_m \neq y_n$, $z_k \neq x_m$): Z non usa x_m , quindi è CS di X_{m-1} e Y . Per assurdo, se non fosse LCS esisterebbe W , CS di X_{m-1} e Y , con $|W| > k$; ma W è anche CS di X e Y , contraddizione.

Caso 3: analogo al Caso 2. □

9.4.3. Formula ricorsiva e algoritmo PD

Dal teorema si ricava direttamente la formula ricorsiva per il riempimento della tabella:

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i-1, j-1] + 1 & \text{se } x_i = y_j, \ i, j > 0 \\ \max(c[i-1, j], c[i, j-1]) & \text{se } x_i \neq y_j, \ i, j > 0 \end{cases} \quad (136)$$

Nota (Collegamento formula — teorema). Ogni riga della formula corrisponde a un caso del ragionamento:

- $c[i, j] = 0$ (caso base): una delle due sequenze è vuota — non esistono sottosequenze comuni.
- $c[i - 1, j - 1] + 1$ (Caso 1 — match): gli ultimi caratteri coincidono e fanno parte della LCS. Li consumiamo e aggiungiamo 1 alla LCS dei prefissi accorciati.
- $\max(c[i - 1, j], c[i, j - 1])$ (Casi 2 e 3 — mismatch): uno dei due ultimi caratteri non è nella LCS. Proviamo a scartarli uno alla volta e prendiamo il risultato migliore.

La soluzione ricorsiva diretta è inefficiente perché i sotto-problemi non sono indipendenti: $\text{LCS}(X_{m-1}, Y)$ e $\text{LCS}(X, Y_{n-1})$ condividono il sotto-problema $\text{LCS}(X_{m-1}, Y_{n-1})$. Con la PD bottom-up, ogni sotto-problema è risolto esattamente una volta.

LCS-length(X, Y, m, n)

```
LCS-length(X, Y, m, n) {
    c = nuova matrice (m+1)×(n+1);
    for (i = 0; i <= m; i++) { c[i,0] = 0; }
    for (j = 1; j <= n; j++) { c[0,j] = 0; }
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) {
            if (X[i] == Y[j]) {
                c[i,j] = c[i-1, j-1] + 1;
            } else {
                c[i,j] = c[i-1, j];
                if (c[i,j-1] > c[i,j]) {
                    c[i,j] = c[i,j-1];
                }
            }
        }
    }
    print c[m,n];
    return c;
}
```

$$T(n, m) = \Theta(m \cdot n), S(n, m) = \Theta(m \cdot n).$$

9.4.4. Riempimento della matrice passo per passo

Per capire concretamente come funziona l'algoritmo, riempiamo la matrice per un esempio piccolo, spiegando la decisione presa in ogni cella.

ESEMPIO 9.4.6 (X = CASA, Y = COSA — riempimento passo per passo). $m = 4, n = 4$. La matrice c ha dimensione 5×5 .

Fase 2 — casi base: la riga 0 e la colonna 0 sono tutte zero (una delle due sequenze è vuota).

Fase 3 — riempimento riga per riga:

Riga $i = 1$ ($x_1 = C$): per ogni colonna, confrontiamo C con il carattere corrispondente di Y:

- $c[1, 1]: C = C \rightarrow \text{match} \rightarrow c[0, 0] + 1 = 0 + 1 = 1$
- $c[1, 2]: C \neq O \rightarrow \text{mismatch} \rightarrow \max(c[0, 2], c[1, 1]) = \max(0, 1) = 1$
- $c[1, 3]: C \neq S \rightarrow \text{mismatch} \rightarrow \max(c[0, 3], c[1, 2]) = \max(0, 1) = 1$
- $c[1, 4]: C \neq A \rightarrow \text{mismatch} \rightarrow \max(c[0, 4], c[1, 3]) = \max(0, 1) = 1$

Riga $i = 2$ ($x_2 = A$):

- $c[2, 1]: A \neq C \rightarrow \max(c[1, 1], c[2, 0]) = \max(1, 0) = 1$
- $c[2, 2]: A \neq O \rightarrow \max(c[1, 2], c[2, 1]) = \max(1, 1) = 1$
- $c[2, 3]: A \neq S \rightarrow \max(c[1, 3], c[2, 2]) = \max(1, 1) = 1$
- $c[2, 4]: A = A \rightarrow \text{match} \rightarrow c[1, 3] + 1 = 1 + 1 = 2$

Riga $i = 3$ ($x_3 = S$):

- $c[3, 1]: S \neq C \rightarrow \max(c[2, 1], c[3, 0]) = \max(1, 0) = 1$
- $c[3, 2]: S \neq O \rightarrow \max(c[2, 2], c[3, 1]) = \max(1, 1) = 1$
- $c[3, 3]: S = S \rightarrow \text{match} \rightarrow c[2, 2] + 1 = 1 + 1 = 2$
- $c[3, 4]: S \neq A \rightarrow \max(c[2, 4], c[3, 3]) = \max(2, 2) = 2$

Riga $i = 4$ ($x_4 = A$):

- $c[4, 1]: A \neq C \rightarrow \max(c[3, 1], c[4, 0]) = \max(1, 0) = 1$
- $c[4, 2]: A \neq O \rightarrow \max(c[3, 2], c[4, 1]) = \max(1, 1) = 1$
- $c[4, 3]: A \neq S \rightarrow \max(c[3, 3], c[4, 2]) = \max(2, 1) = 2$
- $c[4, 4]: A = A \rightarrow \text{match} \rightarrow c[3, 3] + 1 = 2 + 1 = 3$

Matrice completa:

	ε	C	O	S	A
ε	0	0	0	0	0
C	0	1	1	1	1
A	0	1	1	1	2
S	0	1	1	2	2
A	0	1	1	2	3

Risultato: $c[4, 4] = 3$, quindi $|\text{LCS}(\text{CASA}, \text{COSA})| = 3$.

9.4.5. Ricostruzione della LCS

Per ottenere la LCS stessa (non solo la sua lunghezza), si «risale» la matrice dalla cella $c[m, n]$ fino a raggiungere una riga o colonna zero. Ad ogni cella, la decisione è:

- Se $x_i = y_j$ (**match**): il carattere fa parte della LCS. Ci si sposta in **diagonale** verso $c[i - 1, j - 1]$.
- Se $x_i \neq y_j$ (**mismatch**): ci si sposta verso la cella con il valore **maggiore** tra $c[i - 1, j]$ (sopra) e $c[i, j - 1]$ (sinistra). In caso di parità, si va sopra.

Nota (Le tre direzioni nella matrice). Ogni cella della matrice è stata calcolata guardando al massimo tre celle vicine. La ricostruzione percorre il cammino inverso:

- **Diagonale** ($\nearrow$) — match: il carattere fa parte della LCS, entrambi gli indici decrescono.
- **Su** ($\uparrow$) — mismatch: il valore viene dalla riga sopra, x_i non è nella LCS, solo i decresce.
- **Sinistra** ($\leftarrow$) — mismatch: il valore viene dalla colonna a sinistra, y_j non è nella LCS, solo j decresce.

PRINT-LCS(c, X, Y, i, j)

```
PRINT-LCS(c, X, Y, i, j) {
    // Prima chiamata: PRINT-LCS(c, X, Y, m, n)
    if (i == 0 || j == 0) return;
    if (X[i] == Y[j]) { // match → diagonale
        PRINT-LCS(c, X, Y, i-1, j-1);
        print X[i];
    } else { // mismatch → su o sinistra
        if (c[i-1, j] >= c[i, j-1]) {
            PRINT-LCS(c, X, Y, i-1, j);
        } else {
            PRINT-LCS(c, X, Y, i, j-1);
        }
    }
}
```

Ad ogni chiamata i oppure j (o entrambi) decrescono: $T(n, m) = O(n + m)$.

ESEMPIO 9.4.7 (Ricostruzione per X = CASA, Y = COSA). Partiamo dalla cella $c[4, 4] = 3$ e risaliamo:

1. $c[4, 4]$: $x_4 = A, y_4 = A \rightarrow$ **match** ($\nearrow$). Il carattere A è nella LCS. Vai a $c[3, 3]$.
2. $c[3, 3]$: $x_3 = S, y_3 = S \rightarrow$ **match** ($\nearrow$). Il carattere S è nella LCS. Vai a $c[2, 2]$.
3. $c[2, 2]$: $x_2 = A, y_2 = O \rightarrow$ **mismatch**. $c[1, 2] = 1 \geq c[2, 1] = 1 \rightarrow$ vai **su** ($\uparrow$) a $c[1, 2]$.
4. $c[1, 2]$: $x_1 = C, y_2 = O \rightarrow$ **mismatch**. $c[0, 2] = 0 < c[1, 1] = 1 \rightarrow$ vai **sinistra** ($\leftarrow$) a $c[1, 1]$.
5. $c[1, 1]$: $x_1 = C, y_1 = C \rightarrow$ **match** ($\nearrow$). Il carattere C è nella LCS. Vai a $c[0, 0]$.
6. $c[0, 0]$: $i = 0$, stop.

Caratteri raccolti (nell'ordine della ricorsione): C, S, A. **LCS = CSA**, lunghezza 3. ✓

9.4.6. Ulteriori esempi

ESEMPIO 9.4.8 (X = PICCOLA, Y = PIANTA). Tabella c (8×7). Il valore finale è $c[7, 6] = 3$, quindi $|LCS| = 3$. Una LCS è PIA.

	ε	P	I	A	N	T	A
ε	0	0	0	0	0	0	0
P	0	1	1	1	1	1	1
I	0	1	2	2	2	2	2
C	0	1	2	2	2	2	2
C	0	1	2	2	2	2	2
O	0	1	2	2	2	2	2
L	0	1	2	2	2	2	2
A	0	1	2	3	3	3	3

ESEMPIO 9.4.9 (X = RISTOTTO, Y = RISTORO). RISTOTTO ha $m = 8$ caratteri, RISTORO ha $n = 7$. Tabella c di dimensione 9×8 . Il valore finale $c[8, 7] = 6$: una LCS è RISTOO (R, I, S, T, O, O).

	ε	R	I	S	T	O	R	O
ε	0	0	0	0	0	0	0	0
R	0	1	1	1	1	1	1	1
I	0	1	2	2	2	2	2	2
S	0	1	2	3	3	3	3	3
T	0	1	2	3	4	4	4	4
O	0	1	2	3	4	5	5	5
T	0	1	2	3	4	5	5	5
T	0	1	2	3	4	5	5	5
O	0	1	2	3	4	5	5	6

9.5. Partizione di un Insieme di Interi

9.5.1. Definizione del problema

DEFINIZIONE 9.5.1 (Problema della Partizione). Dato un insieme $A = \{a_1, a_2, \dots, a_n\}$ di interi positivi con somma $\text{somma}(A) = \sum_{i=1}^n a_i = 2s$, stabilire se esiste un sottoinsieme $A' \subseteq A$ tale che $\text{somma}(A') = s$.

ESEMPIO 9.5.2. $A = \{1, 3, 7, 5, 4\}$, $\text{somma}(A) = 20$, $s = 10$. $A' = \{3, 7\}$ ha somma 10. ✓
 $A = \{2, 2, 7, 7, 2\}$, $\text{somma}(A) = 20$, $s = 10$. Nessun sottoinsieme ha somma 10. ×

La forza bruta genera tutti i 2^n sottoinsiemi: $T(n) = O(n \cdot 2^n)$.

9.5.2. Soluzione con Programmazione Dinamica

I sotto-problemi sono: $\Pi_{i,j}$: esiste $A' \subseteq \{a_1, \dots, a_i\}$ tale che $\text{somma}(A') = j$? Con $0 \leq i \leq n$, $0 \leq j \leq s$.

Tabella PD: matrice booleana p di dimensione $(n+1) \times (s+1)$: $p[i, j] = \text{true}$ se esiste $A' \subseteq \{a_1, \dots, a_i\}$ con $\text{somma}(A') = j$.

Nota (Leggere la tabella). $p[i, j]$ risponde alla domanda: «Posso ottenere la somma j usando solo i primi i elementi di A ?»

Ad esempio, $p[3, 7] = \text{true}$ significa: «Sì, esiste un sottoinsieme di $\{a_1, a_2, a_3\}$ la cui somma è 7.» La risposta al problema originale è nella cella $p[n, s]$.

Fase 2 — sotto-problemi elementari ($i = 0$, nessun elemento):

- $p[0, 0] = \text{true}$ (sottoinsieme vuoto ha somma 0)
- $p[0, j] = \text{false}$ per $1 \leq j \leq s$ (senza elementi, non posso ottenere somma positiva)

Fase 3 — regola ricorsiva per $\Pi_{i,j}$ con $i \geq 1$:

Nota (Intuizione: prendere o non prendere). Per ogni elemento a_i abbiamo esattamente **due** scelte:

- **Non prendo a_i :** la somma j deve essere raggiungibile con i soli primi $i - 1$ elementi, cioè $p[i - 1, j] = \text{true}$.
- **Prendo a_i :** la somma rimanente $j - a_i$ deve essere raggiungibile con i primi $i - 1$ elementi, cioè $p[i - 1, j - a_i] = \text{true}$. Questa opzione è disponibile solo se $a_i \leq j$.

Basta che **una** delle due scelte funzioni per avere $p[i, j] = \text{true}$.

Formalmente:

- Se $a_i > j$: non posso prendere a_i (è troppo grande), quindi $p[i, j] = p[i - 1, j]$.
- Se $a_i \leq j$: $p[i, j] = \text{true}$ se e solo se $p[i - 1, j] = \text{true}$ (non prendo a_i) oppure $p[i - 1, j - a_i] = \text{true}$ (prendo a_i).

Partizione(a)

```

Partizione(a) {
    // a: array di n elementi interi positivi
    somma = 0;
    for (i = 0; i < n; i++) { somma = somma + a[i]; }
    if (somma % 2 == 1) return FALSE;
    s = somma/2;
    p = nuova matrice (n+1)×(s+1);
    for (i = 0; i <= n; i++) { p[i,0] = true; }
    // tutto il resto della matrice si inizializza a false
    for (i = 0; i <= n; i++) {
        for (j = 1; j <= s; j++) { p[i,j] = false; }
    }
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= s; j++) {
            if (p[i-1, j] == true) { p[i,j] = true; }
            else if (a[i-1] <= j && p[i-1, j-a[i-1]] == true) {
                p[i,j] = true;
            }
        }
    }
    print p[n,s];
    return p;
}

```

$T(n, s) = \Theta(n \cdot s)$: polinomiale in n e nel **valore** di s , ma esponenziale nella **dimensione** di s (algoritmo **pseudo-polinomiale**).

9.5.3. Ricostruzione del sottoinsieme**Sottoinsieme(a, p, n, s)**

```

Sottoinsieme(a, p, n, s) {
    if (p[n,s] == true) {
        i = n;
        j = s;
        while (i > 0) {
            if (p[i-1, j] == false) { // prendo a[i]
                print a[i-1];
                j = j - a[i-1];
            }
            i--;
        }
    } else {
        print "non esiste sottoinsieme di somma s";
    }
}

```

$T(n) = O(n)$.

ESEMPIO 9.5.3 ($A = \{2, 4, 1, 3\}$, $s = 5$ — **riempimento e ricostruzione**). $n = 4$, $\text{somma}(A) = 10 = 2s$.

Riempimento passo per passo (prime righe):

Riga $i = 0$ (nessun elemento): $p[0, 0] = T$, tutto il resto F . Senza elementi, posso fare solo somma 0.

Riga $i = 1$ ($a_1 = 2$): per ogni j , posso non prendere 2 (guardo sopra: $p[0, j]$) o prendere 2 (guardo $p[0, j - 2]$):

- $j = 1$: $p[0, 1] = F$ e $2 > 1$ (non posso prendere 2) $\rightarrow F$
- $j = 2$: $p[0, 2] = F$ ma $p[0, 0] = T$ (prendo il 2) $\rightarrow T$
- $j = 3, 4, 5$: nessuna opzione funziona $\rightarrow F$

Riga $i = 2$ ($a_2 = 4$): con gli elementi $\{2, 4\}$:

- $j = 2$: $p[1, 2] = T$ (non prendo 4) $\rightarrow T$
- $j = 4$: $p[1, 4] = F$ ma $p[1, 0] = T$ (prendo 4) $\rightarrow T$
- il resto: F

Righe $i = 3$ ($a_3 = 1$) e $i = 4$ ($a_4 = 3$): proseguendo con la stessa logica.

La tabella p completa, di dimensione 5×6 :

	0	1	2	3	4	5
ε	T	F	F	F	F	F
2	T	F	T	F	F	F
4	T	F	T	F	T	F
1	T	T	T	T	T	T
3	T	T	T	T	T	T

$p[4, 5] = \text{true}$: esiste il sottoinsieme.

Ricostruzione (traccia di Sottoinsieme):

- $i = 4, j = 5$: $p[3, 5] = T \rightarrow$ non prendo $a_4 = 3$. $i = 3$.
- $i = 3, j = 5$: $p[2, 5] = F \rightarrow$ prendo $a_3 = 1$. $j = 4$. $i = 2$.
- $i = 2, j = 4$: $p[1, 4] = F \rightarrow$ prendo $a_2 = 4$. $j = 0$. $i = 1$.
- $i = 1, j = 0$: $p[0, 0] = T \rightarrow$ non prendo. Fine.

$A' = \{1, 4\}$, somma = 5. ✓

9.6. Riepilogo

Da ricordare. Confronto tra i problemi PD visti in questo capitolo:

Problema	Forza bruta	PD	Tabella
Fibonacci	$O(\varphi^n)$	$\Theta(n)$	Array 1D, $n + 1$
Taglio Corde	$O(2^n)$	$\Theta(n^2)$	Array 1D, $n + 1$
LCS	$O(n \cdot 2^m)$	$\Theta(m \cdot n)$	Matrice $(m + 1) \times (n + 1)$
Partizione	$O(n \cdot 2^n)$	$\Theta(n \cdot s)^*$	Matrice $(n + 1) \times (s + 1)$

* Pseudo-polinomiale: polinomiale nel valore di s , esponenziale nella sua rappresentazione binaria.

Come affrontare un nuovo problema PD:

1. Chiedersi: «Qual è l'ultima scelta/decisione?» Questo rivela i sotto-problemi.
2. Verificare **sottostruttura ottima**: la soluzione ottima si compone di soluzioni ottime dei sotto-problemi.
3. Verificare **sotto-problemi ripetuti**: la ricorsione ricalcola gli stessi sotto-problemi.
4. Scrivere la **formula ricorsiva** e identificare i **casi base**.
5. Scegliere la **tabella** e l'**ordine di riempimento** (dai casi base al problema originale).
6. Se serve, aggiungere una struttura ausiliaria per la **ricostruzione** della soluzione.

CAPITOLO 10

Grafi

10.1. Definizione di Grafo

DEFINIZIONE 10.1.1 (Grafo). Un **grafo** $G = (V, E)$ è una coppia di insiemi finiti:

- V : insieme dei **vertici** (o **nodi**), che rappresentano oggetti;
- $E \subseteq V \times V$: insieme degli **archi** (coppie di nodi), che rappresentano relazioni tra oggetti.

I grafi si dividono in:

- **non orientati**: E è un insieme di coppie *non ordinate* $\{u, v\}$. L'arco può essere percorso in entrambe le direzioni: (u, v) e (v, u) rappresentano lo stesso arco.
- **orientati** (o **diretti**): E è un insieme di coppie *ordinate* (u, v) . L'arco ha una direzione fissa da u verso v : (u, v) e (v, u) sono archi distinti.

La **dimensione** del grafo è $|V| + |E| = n + m$, con $n = |V|$ (**ordine**) e $m = |E|$.

Nota (Coppia ordinata vs non ordinata). Una **coppia non ordinata** $\{u, v\}$ è un insieme di due elementi: $\{u, v\} = \{v, u\}$ (l'ordine non conta). Una **coppia ordinata** (u, v) distingue il primo dal secondo elemento: $(u, v) \neq (v, u)$. Intuitivamente, una strada a doppio senso è un arco non orientato; una strada a senso unico è un arco orientato.

DEFINIZIONE 10.1.2 (Adiacenza e incidenza). Dato un arco $(u, v) \in E$:

- i vertici u e v sono **adiacenti** (si «vedono» direttamente tramite un arco);
- l'arco (u, v) è **incidente** sui vertici u e v (li «tocca»);
- u e v sono gli **estremi** dell'arco.

Nei grafi orientati si specifica inoltre che l'arco (u, v) **esce** da u ed **entra** in v .

Nota. Per grafi non orientati vale $0 \leq m \leq \binom{n}{2} = \frac{n(n-1)}{2}$ (ogni coppia di vertici può avere al più un arco); per grafi orientati $0 \leq m \leq n(n-1)$ (ogni coppia ordinata può avere al più un arco, quindi il doppio).

La **densità** di un grafo misura quanti archi contiene rispetto al massimo teorico. Poiché il numero massimo di archi cresce come $\Theta(n^2)$, si distinguono due categorie in base alla **velocità di crescita** di m al crescere di n :

- **Grafo sparso:** $m = O(n)$. Il numero di archi cresce *linearmente* con il numero di vertici. Significa che, in media, ogni vertice è collegato a un numero *costante* di altri vertici (non dipendente da n).

Esempi:

- Una rete stradale: ogni incrocio ha tipicamente 3-4 strade, indipendentemente da quanti incroci totali ci siano in città.
 - Un social network con le sole «amicizie strette»: ogni persona ne ha circa 5-10.
 - Un albero: ha sempre $m = n - 1$ archi, qualunque sia n .
- **Grafo denso:** $m = \Theta(n^2)$. Il numero di archi cresce *quadraticamente*: ogni vertice è collegato a una *frazione* di tutti gli altri (ad es. alla metà).

Esempio estremo: il **grafo completo**, indicato con K_n , in cui ogni coppia di vertici distinti è collegata. Ha $m = \binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$ archi: è il grafo più denso possibile.

OSSERVAZIONE 10.1.3. La parola *asintotica* significa: guardiamo come si comporta m quando n diventa grande, non il valore preciso di m per un n fissato. Per questo non esiste una soglia esatta «sotto k archi è sparso, sopra è denso»: dipende da come m e n si relazionano al crescere di n .

Per dare un'idea concreta: un grafo con $n = 10$ vertici e $m = 40$ archi è *denso* (molto vicino al massimo di $\binom{10}{2} = 45$), mentre $n = 10^6$ vertici con $m = 10^6$ archi è *sparso* (in media ogni vertice ha un solo arco).

La distinzione è importante perché determina **due scelte pratiche**:

1. Scelta della rappresentazione in memoria:

- Per grafi **densi** conviene la **matrice di adiacenza** (una tabella $n \times n$): occupa $\Theta(n^2)$ celle, che è lo stesso ordine di m , quindi nessuno spreco; in compenso permette di verificare in $O(1)$ se due vertici sono adiacenti.
- Per grafi **sparsi** conviene le **liste di adiacenza** (un array di n liste): occupa $\Theta(n + m)$ spazio, che per grafi sparsi equivale a $\Theta(n)$ — molto meno di $\Theta(n^2)$.

Entrambe le rappresentazioni sono descritte in dettaglio più avanti.

- ### 2. Scelta dell'algoritmo:
- alcuni algoritmi hanno due versioni con costi diversi a seconda di come sono implementate le strutture interne. Ad esempio, l'algoritmo di Dijkstra costa $O(n^2)$ se PQ è un array (adatto ai grafi densi) e $O((n + m) \log n)$ se PQ è un MIN-heap binario (adatto ai grafi sparsi).

10.2. Grafi non orientati

10.2.1. Grado, cammino, ciclo

DEFINIZIONE 10.2.1 (Grado di un vertice). Il **grado** di un vertice v è $\delta(v)$ = numero di archi incidenti su v . Un vertice è **isolato** se $\delta(v) = 0$.

Vale la seguente proprietà:

$$\sum_{v \in V} \delta(v) = 2|E| = 2m \quad (137)$$

poiché ogni arco contribuisce per un fattore 2 (incrementa il grado dei due estremi).

DEFINIZIONE 10.2.2 (Cammino). Un **cammino** da u a v in G è una sequenza di vertici $x_0, x_1, \dots, x_k$ tale che:

- $x_0 = u, x_k = v$
- $\forall i, 1 \leq i \leq k : (x_{i-1}, x_i) \in E$

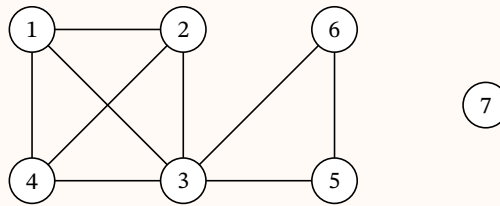
La **lunghezza** del cammino è k (numero di archi). Un cammino è **semplice** se tutti i vertici sono distinti. Un **ciclo** è un cammino che torna al vertice di partenza ($x_k = u$).

DEFINIZIONE 10.2.3 (Distanza e cammino minimo). La **distanza** $\delta(u, v)$ tra due vertici è il numero minimo di archi da percorrere per spostarsi da u a v :

$$\delta(u, v) = \begin{cases} \text{lunghezza minima di un cammino } u \rightsquigarrow v & \text{se esiste un cammino } u \rightsquigarrow v \\ +\infty & \text{altrimenti} \end{cases} \quad (138)$$

Un **cammino minimo** da u a v è un cammino semplice di lunghezza $\delta(u, v)$.

ESEMPIO 10.2.4. Grafo con $V = \{1, 2, 3, 4, 5, 6, 7\}$, $E = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4), (3, 5), (3, 6), (5, 6)\}$.



- $\delta(1, 5) = 2$: cammino $\langle 1, 3, 5 \rangle$
- $\langle 1, 4, 2, 3, 5 \rangle$: cammino di lunghezza 4, non minimo
- $\langle 3, 6, 5, 3 \rangle$: ciclo
- $\langle 1, 2, 4, 1, 3 \rangle$: non è un cammino semplice (il vertice 1 appare due volte)
- $\delta(1, 7) = +\infty$: il vertice 7 è isolato, non raggiungibile da nessun altro vertice
- $\sum_{v \in V} \delta(v) = 18 = 2 \cdot 9$, coerente con $\sum_{v \in V} \delta(v) = 2|E|$

10.2.2. Grafo completo, connesso, sottografo

DEFINIZIONE 10.2.5 (Grafo completo (Clique)). Un grafo è **completo** (o **clique**) se ogni coppia di vertici distinti è adiacente: $\forall u, v \in V, u \neq v : (u, v) \in E$. In un grafo completo $m = \binom{n}{2}$.

DEFINIZIONE 10.2.6 (Grafo connesso). Un grafo è **connesso** se ogni coppia di vertici è connessa, cioè per ogni $u, v \in V$ esiste un cammino da u a v ($\delta(u, v) < +\infty$).

In un grafo connesso, da ogni vertice è possibile raggiungere tutti gli altri.

DEFINIZIONE 10.2.7 (Sottografo). $G' = (V', E')$ è un **sottografo** di $G = (V, E)$ se $V' \subseteq V$ e $E' \subseteq V' \times V', E' \subseteq E$.

DEFINIZIONE 10.2.8 (Componente connessa). Una **componente connessa** di G è un sottografo G' di G che è connesso e **massimale** (non può essere esteso: non esistono altri nodi in G connessi ai nodi di G').

Le componenti connesse sono le classi di equivalenza della relazione «è raggiungibile da». Un grafo connesso è composto da una sola componente connessa.

10.3. Grafi orientati

10.3.1. Grado, cammino orientato, connessione forte

DEFINIZIONE 10.3.1 (Grafici orientati). In un **grafo orientato** $G = (V, E)$, l'arco $(u, v) \in E$ è **diretto** da u a v : può essere percorso solo in quella direzione. Si noti che $(u, v) \neq (v, u)$.

Per ogni vertice $v \in V$:

- **grado uscente** $\delta_{u(v)}$ = numero di archi che escono da v
- **grado entrante** $\delta_{e(v)}$ = numero di archi che entrano in v
- $\delta(v) = \delta_{u(v)} + \delta_{e(v)}$

Vale: $\sum_{v \in V} \delta_{e(v)} = \sum_{v \in V} \delta_{u(v)} = m$.

Per grafi orientati: $0 \leq m \leq n(n-1)$.

DEFINIZIONE 10.3.2 (Cammino orientato e ciclo orientato). Un **cammino orientato** $u \rightsquigarrow v$ è un cammino i cui archi sono orientati nel verso corretto: $\forall i, 1 \leq i \leq k : (x_{i-1}, x_i) \in E$.

Un **ciclo orientato** è un cammino orientato che torna al vertice di partenza.

DEFINIZIONE 10.3.3 (Connessione forte). Due vertici $u, v \in V$ sono **fortemente connessi** se esistono sia un cammino orientato $u \rightsquigarrow v$ sia uno $v \rightsquigarrow u$ (sono **mutuamente raggiungibili**).

Un grafo orientato è **fortemente connesso** se ogni coppia ordinata di nodi è fortemente connessa.

DEFINIZIONE 10.3.4 (Componenti fortemente connesse). Le **componenti fortemente connesse** di un grafo orientato sono i sottografi fortemente connessi e massimali. Sono le classi di equivalenza della relazione «sono mutuamente raggiungibili».

ESEMPIO 10.3.5. Grafo orientato con $V = \{1, 2, 3, 4\}$, archi $(1, 2), (2, 4), (4, 3), (3, 2), (1, 3)$:

- $\langle 2, 4, 3, 2 \rangle$ è un ciclo orientato
- $\langle 1, 2, 4, 3 \rangle$ è un cammino orientato
- I vertici 2, 3, 4 sono fortemente connessi (componente $\{2, 3, 4\}$)
- Il vertice 1 non è fortemente connesso agli altri (non esiste cammino verso 1)

10.3.2. Grafi aciclici, alberi, foreste

DEFINIZIONE 10.3.6 (Grafo aciclico). Un grafo (orientato o non orientato) è **aciclico** se non contiene cicli.

DEFINIZIONE 10.3.7 (Albero (libero)). Un **albero (libero)** è un grafo non orientato, connesso e aciclico.

TEOREMA 10.3.8 (Proprietà degli alberi liberi [CLRS: Teo. B.2]). Sia $G = (V, E)$ un grafo non orientato. Le seguenti affermazioni sono equivalenti:

1. G è un albero.
2. Due vertici qualsiasi di G sono collegati da un unico cammino semplice.
3. G è connesso, ma la rimozione di un arco qualsiasi lo disconnette.
4. G è connesso e $|E| = |V| - 1$.
5. G è aciclico e $|E| = |V| - 1$.
6. G è aciclico, ma l'aggiunta di un arco qualsiasi crea un ciclo.

DEFINIZIONE 10.3.9 (Foresta). Una **foresta** è un grafo non orientato e aciclico le cui componenti connesse sono alberi.

10.4. Rappresentazione dei grafi in memoria

10.4.1. Matrice di adiacenza

DEFINIZIONE 10.4.1 (Matrice di adiacenza). Dato $G = (V, E)$ con $|V| = n$, la **matrice di adiacenza** è una matrice A di dimensione $n \times n$ con valori in $\{0, 1\}$:

$$A[i, j] = \begin{cases} 1 & (i, j) \in E \\ 0 & (i, j) \notin E \end{cases} \quad (139)$$

Per grafi non orientati, A è simmetrica ($A[i, j] = A[j, i]$).

Costo in spazio: $S(n, m) = \Theta(n^2)$. Va bene per grafi densi; inefficiente per grafi sparsi.

Operazione	Restituisce	Costo	Come
<code>adiacenti(u, v)</code>	booleano: <code>true</code> se $(u, v) \in E$, <code>false</code> altrimenti	$O(1)$	return $A[u, v]$
<code>grado(u)</code>	intero: numero di archi incidenti su u	$\Theta(n)$	scorrere la riga u di A e contare gli 1
<code>aggiungiArco(u, v)</code>	nulla (effetto: aggiunge l'arco al grafo)	$O(1)$	$A[u, v] = 1$ (e $A[v, u] = 1$ se non orientato)
<code>rimuoviArco(u, v)</code>	nulla (effetto: rimuove l'arco dal grafo)	$O(1)$	$A[u, v] = 0$ (e $A[v, u] = 0$ se non orientato)

10.4.2. Liste di adiacenza

DEFINIZIONE 10.4.2 (Liste di adiacenza). La rappresentazione con **liste di adiacenza** associa ad ogni vertice u la lista $\text{Adj}[u]$ dei vertici adiacenti:

$$\text{Adj}[u] = \begin{cases} \text{NIL} & \delta(u) = 0 \\ \text{lista dei vertici } v \text{ t.c. } (u, v) \in E & \text{altrimenti} \end{cases} \quad (140)$$

Adj è un array di $n = |V|$ liste.

Costo in spazio: $S(n, m) = \Theta(n + m)$ (lineare nella dimensione del grafo).

Per grafi non orientati: $\sum_{v \in V} |\text{Adj}[v]| = 2|E|$ (ogni arco è rappresentato due volte).

Per grafi orientati: $\sum_{v \in V} |\text{Adj}[v]| = |E|$ (ogni arco è rappresentato una sola volta).

Operazione	Restituisce	Costo	Come
<code>adiacenti(u, v)</code>	booleano: <code>true</code> se $(u, v) \in E$, <code>false</code> altrimenti	$O(\delta(u))$	cercare v scorrendo la lista $\text{Adj}[u]$
<code>grado(u)</code>	intero: numero di archi incidenti su u	$\Theta(\delta(u))$	calcolare la lunghezza di $\text{Adj}[u]$
<code>aggiungiArco(u, v)</code>	nulla (effetto: aggiunge l'arco al grafo)	$O(1)$	inserire v in $\text{Adj}[u]$ (e u in $\text{Adj}[v]$ se non orientato), liste non ordinate
<code>rimuoviArco(u, v)</code>	nulla (effetto: rimuove l'arco dal grafo)	$O(\delta(u) + \delta(v))$	cercare ed eliminare v da $\text{Adj}[u]$ e u da $\text{Adj}[v]$

OSSERVAZIONE 10.4.3. Per grafi sparsi ($m = O(n)$), le liste di adiacenza usano $\Theta(n)$ spazio contro $\Theta(n^2)$ della matrice. Lo svantaggio è che determinare se $(u, v) \in E$ richiede $O(\delta(u))$ invece di $O(1)$.

Da ricordare. La scelta della rappresentazione dipende dalla densità del grafo e dalle operazioni richieste:

- **Matrice di adiacenza:** adiacenza in $O(1)$, ma spazio $\Theta(n^2)$. Preferibile per grafi densi.
- **Liste di adiacenza:** spazio $\Theta(n + m)$, adiacenza in $O(\delta(u))$. Preferibile per grafi sparsi.

10.5. Visita in Ampiezza (BFS)

Una **visita** (o **attraversamento**) di un grafo è l'esame sistematico di tutti i suoi vertici e archi. Può essere **completa** (visita tutto il grafo, se connesso) o **parziale** (visita solo la componente (fortemente) connessa raggiungibile dalla sorgente). Le due strategie principali sono la **BFS** (*Breadth First Search*, visita in ampiezza), che scopre i vertici in ordine di distanza crescente dalla sorgente, e la **DFS** (*Depth First Search*, visita in profondità, trattata nella sezione successiva), che esplora il più lontano possibile prima di tornare indietro.

10.5.1. Struttura e proprietà

DEFINIZIONE 10.5.1 (BFS(G, s)). La **visita in ampiezza** prende in input:

- $G = (V, E)$: grafo rappresentato con liste di adiacenza
- $s \in V$: vertice **sorgente**

L'algoritmo scopre tutti i vertici raggiungibili da s in ordine di distanza crescente, usando una **coda** con disciplina **FIFO** (*First-In First-Out*: il primo elemento inserito è anche il primo a uscire). Al termine, $v.d = \delta(s, v)$ per ogni vertice raggiungibile.

Per ogni vertice $v \in V$ si mantengono tre attributi:

- $v.d$: distanza da s (inizializzata a $+\infty$; al termine vale $\delta(s, v)$)
- $v.color$: colore che indica lo stato di elaborazione
- $v.\Pi$: predecessore di v nell'albero BFS

I colori hanno il seguente significato:

Colore	Significato
B (Bianco)	v non è stato ancora scoperto
G (Grigio)	v è stato scoperto (è nella coda)
N (Nero)	tutti gli archi uscenti da v sono stati esaminati (v è uscito dalla coda)

BFS(G, s)

```

BFS(G, s) {
  for (v in V \ {s}) {           // inizializzazione
    v.color = B;
    v.d = +inf;
    v.π = NIL;
  }
  s.color = G;
  s.d = 0;
  s.π = NIL;
  Q = coda vuota;
  enqueue(Q, s);
  while (Q non è vuota) {
    u = dequeue(Q);
    for (v in Adj[u]) {
      if (v.color == B) {        // v non ancora scoperto
        v.color = G;
        v.d = u.d + 1;
        v.π = u;
        enqueue(Q, v);
      }
    }
    u.color = N;
  }
}

```

10.5.2. Correttezza e complessità

TEOREMA 10.5.2 (Correttezza di BFS). *Al termine di $\text{BFS}(G, s)$, per ogni vertice $v \in V$ raggiungibile da s , vale $v.d = \delta(s, v)$.*

Complessità: ogni vertice viene accodato e rimosso dalla coda al più una volta: costo $O(n)$. Per ogni vertice rimosso si esaminano tutti gli archi della sua lista di adiacenza. Poiché la somma delle lunghezze di tutte le liste è $\Theta(m)$ (grafi non orientati) o $\Theta(m)$ (orientati), il costo totale è:

$$T(n, m) = O(n + m) \quad (141)$$

10.5.3. Albero BFS

DEFINIZIONE 10.5.3 (Albero BFS). Al termine di BFS, il sottografo dei predecessori $G_\Pi = (V_\Pi, E_\Pi)$ con $V_\Pi = \{v : v.\Pi \neq \text{NIL}\} \cup \{s\}$ e $E_\Pi = \{(v.\Pi, v) : v \in V_\Pi \setminus \{s\}\}$ è un **albero BFS** radicato in s .

L'albero BFS contiene tutti i cammini minimi da s ai vertici raggiungibili.

ESEMPIO 10.5.4 (BFS($G, 1$) su grafo con due componenti connesse). Grafo con 11 vertici e due componenti connesse. Archi nella componente di 1: $(1, 2), (1, 3), (1, 4), (1, 6), (2, 3), (3, 4), (4, 7), (6, 5), (6, 8)$; archi nella componente $\{9, 10, 11\}$: $(9, 10), (10, 11)$, non raggiungibile da 1.

Evoluzione passo-passo (si assume Adj ordinata in modo crescente):

Passo	u estratto	Vicini scoperti ($B \rightarrow G$)	Coda Q dopo
0	— (init)	$s = 1$ grigio, $1.d = 0$	[1]
1	1	2, 3, 4, 6 con $d = 1$	[2, 3, 4, 6]
2	2	— (tutti già grigi)	[3, 4, 6]
3	3	— (tutti già grigi)	[4, 6]
4	4	7 con $d = 2$	[6, 7]
5	6	5, 8 con $d = 2$	[7, 5, 8]
6	7	—	[5, 8]
7	5	—	[8]
8	8	—	[]

Distanze finali: $1.d = 0$; $2.d = 3.d = 4.d = 6.d = 1$; $5.d = 7.d = 8.d = 2$.

Albero BFS radicato in 1:

- livello 0: $\{1\}$
- livello 1: $\{2, 3, 4, 6\}$
- livello 2: $\{5, 7, 8\}$ con $5.\pi = 6, 7.\pi = 4, 8.\pi = 6$

I vertici 9, 10, 11 restano **bianchi** con $d = +\infty$: BFS ha esplorato solo la componente connessa di 1.

OSSERVAZIONE 10.5.5. BFS calcola la distanza da s a **tutti** i vertici raggiungibili in una sola visita. Se il grafo non è connesso, BFS esplora solo la componente connessa di s . Per visitare l'intero grafo non connesso, è necessario avviare BFS da ogni vertice non ancora scoperto.

10.5.4. Correttezza formale di BFS

La correttezza di BFS si dimostra con una serie di lemmi [CLRS: 22.2].

LEMMA 10.5.6 (Triangolare [CLRS: Lemma 22.1]). Sia $G = (V, E)$ un grafo orientato o non orientato e $s \in V$. Per ogni arco $(u, v) \in E$:

$$\delta(s, v) \leq \delta(s, u) + 1 \quad (142)$$

Dimostrazione. Se u è raggiungibile da s , lo è anche v : il cammino minimo da s a v non può essere più lungo del cammino minimo da s a u seguito dall'arco (u, v) , quindi $\delta(s, v) \leq \delta(s, u) + 1$. Se u non è raggiungibile, $\delta(s, u) = +\infty$ e la disuguaglianza è banalmente soddisfatta. $\square$

OSSERVAZIONE 10.5.7. Il lemma vale anche quando v non è raggiungibile da s . In quel caso $\delta(s, v) = +\infty$, ma allora anche $\delta(s, u) = +\infty$ (perché se u fosse raggiungibile, attraversando l'arco (u, v) lo sarebbe anche v): la disuguaglianza $+\infty \leq +\infty + 1$ è soddisfatta.

LEMMA 10.5.8 (Limite superiore). Al termine di $\text{BFS}(G, s)$, per ogni vertice $v \in V$: $v.d \geq \delta(s, v)$.

Dimostrazione (per induzione). **Ipotesi induttiva:** $v.d \geq \delta(s, v)$ per ogni $v \in V$, dopo ogni operazione ENQUEUE .

Base: dopo la prima ENQUEUE , $s.d = 0 = \delta(s, s)$ e $v.d = +\infty \geq \delta(s, v)$ per ogni $v \neq s$. ✓

Passo: sia v un vertice bianco scoperto da u . L'algoritmo pone $v.d = u.d + 1$. Per ipotesi induttiva $u.d \geq \delta(s, u)$, dunque:

$$v.d = u.d + 1 \geq \delta(s, u) + 1 \geq \delta(s, v) \quad (143)$$

dove l'ultima disuguaglianza segue dal Lemma Triangolare. Il valore $v.d$ non cambierà più. ✓ $\square$

LEMMA 10.5.9 (Proprietà della coda [CLRS: Lemma 22.3]). Durante l'esecuzione di BFS , se $Q = [v_1, v_2, \dots, v_r]$ con v_1 in testa e v_r in fondo, allora:

- $v_r.d \leq v_1.d + 1$
- $v_i.d \leq v_{i+1}.d$ per $i = 1, 2, \dots, r - 1$

Da ricordare. Intuizione fondamentale: in ogni istante la coda Q contiene al più **due valori distinti** di distanza dalla sorgente — precisamente k e $k + 1$ per qualche k . È questa proprietà che garantisce a BFS di scoprire i vertici in ordine di distanza crescente.

COROLLARIO 10.5.10 (Monotonia delle distanze in coda [CLRS: Cor. 22.4]). Se il vertice v_i è inserito nella coda prima di v_j , allora $v_i.d \leq v_j.d$ al momento dell'inserimento di v_j .

TEOREMA 10.5.11 (Correttezza di BFS [CLRS: Teo. 22.5]). Sia $G = (V, E)$ e $s \in V$. BFS scopre tutti i vertici raggiungibili da s e, al termine, $v.d = \delta(s, v)$ per ogni $v \in V$. Inoltre, per ogni $v \neq s$ raggiungibile, uno dei cammini minimi da s a v è un cammino minimo da s a v . Il seguito dall'arco $(v.\pi, v)$.

Dimostrazione (per assurdo). Procediamo in tre fasi.

(1) Scelta di un controesempio minimo. Supponiamo per assurdo che esista un vertice v con $v.d \neq \delta(s, v)$. Sia v il vertice a distanza minima da s che riceve un valore d errato. Per il Lemma del Limite Superiore si ha $v.d \geq \delta(s, v)$: dunque $v.d > \delta(s, v)$, e v è raggiungibile (altrimenti $\delta(s, v) = +\infty$ contraddirebbe la disuguaglianza).

(2) Derivazione della disuguaglianza chiave. Sia u il vertice che precede v su un cammino minimo da s , quindi $\delta(s, v) = \delta(s, u) + 1$ (sottostruttura ottima). Poiché $\delta(s, u) < \delta(s, v)$, per la scelta di v si ha $u.d = \delta(s, u)$. Dunque:

$$v.d > \delta(s, v) = \delta(s, u) + 1 = u.d + 1 \quad (*) \quad (144)$$

(3) Analisi al momento in cui u viene estratto da Q . Al momento dell'estrazione di u , il vertice v può essere in uno dei tre stati:

- **Bianco** (B): v non è ancora stato scoperto. La riga 15 di BFS pone $v.d = u.d + 1$, che contraddice (*).
- **Nero** (N): v è stato estratto da Q prima di u . Per il Corollario di monotonia, $v.d \leq u.d \leq u.d + 1$, contro (*).
- **Grigio** (G): v è stato scoperto da un vertice w estratto da Q prima di u , quindi $v.d = w.d + 1$; per il Corollario di monotonia $w.d \leq u.d$, da cui $v.d \leq u.d + 1$, contro (*).

Tutti i casi contraddicono (*): quindi $v.d = \delta(s, v)$ per ogni $v \in V$. Infine, se $v.\pi = u$, l'algoritmo impone $v.d = u.d + 1$, cioè un cammino minimo $s \rightsquigarrow v$ si ottiene estendendo con l'arco (u, v) un cammino minimo $s \rightsquigarrow u$. $\square$

10.5.5. PRINT-PATH

Dopo aver eseguito $\text{BFS}(G, s)$, è possibile stampare un cammino minimo da s a qualsiasi vertice v raggiungibile tramite la procedura ricorsiva PRINT-PATH [CLRS: p. 503].

PRINT-PATH(G, s, v)

```
PRINT-PATH( $G, s, v$ ) {
    // da chiamare dopo BFS( $G, s$ )
    if ( $v == s$ )
        print  $s$ ;
    else if ( $v.\pi == \text{NIL}$ )
        print " $v$  non è raggiungibile da  $s$ ";
    else {
        PRINT-PATH( $G, s, v.\pi$ );
        print  $v$ ;
    }
}
```

$$T(|V|, |E|) = \begin{cases} \Theta(1) & \text{se } s \rightsquigarrow v \text{ non esiste} \\ \Theta(\delta(s, v)) & \text{se } s \rightsquigarrow v \text{ esiste} \end{cases}$$

poiché ogni chiamata ricorsiva riguarda un cammino con un vertice in meno: la profondità della ricorsione è $\delta(s, v)$.

10.5.6. Applicazioni della BFS

OSSERVAZIONE 10.5.12. BFS si può applicare anche a grafi il cui insieme di vertici non è noto a priori (ad esempio, grafi dinamici). In tal caso si sostituiscono i colori con un **dizionario** D dei vertici già visitati. Il costo dipende dalle operazioni sul dizionario: con tabelle hash il caso medio è $\Theta(n_s + m_s)$, dove $n_s = |V_s|$ e $m_s = |E_s|$ sono i vertici e gli archi raggiungibili da s .

ESEMPIO 10.5.13 (Connessività e componenti connesse). Per verificare se un grafo non orientato G è connesso:

```
Connesso(G) {
    scegli un vertice s in V;
    BFS(G, s);
    for (v in V)
        if (v.color == B) return FALSE;
    return TRUE;
}
```

$$T(|V|, |E|) = \Theta(|V| + |E|).$$

Per contare le componenti connesse si esegue BFS sull'intero grafo riavviandola da ogni vertice non ancora scoperto:

```
BFS(G) {
    for (v in V) { v.color = B; }
    cc = 0;
    for (v in V) {
        if (v.color == B) {
            cc++;
            BFSmod(G, v); // BFS senza π e d; colora solo
        }
    }
    return cc;
}
```

Al termine, cc è il numero di componenti connesse. $T(|V|, |E|) = \Theta(|V| + |E|)$.

10.6. Visita in Profondità (DFS)

La **visita in profondità** (Depth-First Search, DFS) esplora il grafo seguendo i percorsi più profondi possibili prima di fare **backtrack** — cioè tornare indietro al vertice precedente quando non ci sono più archi da esplorare in avanti. A differenza della BFS (che usa una coda e una sorgente fissa), la DFS:

- esplora **tutto il grafo** (non solo la componente di una sorgente);
- produce una **foresta DF** (più alberi, non uno solo);
- assegna a ogni vertice due **timestamp** (marcatori temporali, numeri interi che indicano in che istante si verifica un evento): $v.d$ (istante di scoperta) e $v.f$ (istante di fine ispezione).

10.6.1. Algoritmo

Per ogni vertice $v \in V$ si mantengono:

- $v.color \in \{B, G, N\}$: stato di visita (bianco, grigio, nero)
- $v.\pi$: predecessore nella foresta DF
- $v.d$: tempo di scoperta
- $v.f$: tempo di completamento

I timestamp sono interi in $[1, 2|V|]$ con $v.d < v.f$.

DFS(G)

```
DFS(G) {
    for (u in V) {
        u.color = B;
        u.π = NIL;
    }
    time = 0;
    for (u in V) {
        if (u.color == B)
            DFS-VISIT(G, u);
    }
}
```

DFS-VISIT(G, u)

```
DFS-VISIT(G, u) {
    time = time + 1;
    u.d = time;           // u scoperto
    u.color = G;
    for (v in Adj[u]) {
        if (v.color == B) {
            v.π = u;
            DFS-VISIT(G, v);
        }
    }
    u.color = N;
    time = time + 1;
    u.f = time;           // u completato
}
```

Ogni volta che si chiama `DFS-VISIT(u)` nella procedura principale, u diventa la radice di un nuovo albero della foresta DF.

10.6.2. Complessità

Ogni vertice viene colorato di grigio esattamente una volta (la prima istruzione di `DFS-VISIT` colora grigio): `DFS-VISIT` è chiamata esattamente una volta per vertice.

- Cicli di inizializzazione e for principale: $\Theta(|V|)$
- Corpo di `DFS-VISIT`: per ogni vertice u , si esaminano $|\text{Adj}[u]|$ vicini; la somma è $\Theta(|E|)$

$$T(|V|, |E|) = \Theta(|V| + |E|) \quad (145)$$

10.6.3. Foresta DF e struttura di parentesi

DEFINIZIONE 10.6.1 (Foresta DF). Il sottografo dei predecessori $G_\Pi = (V_\Pi, E_\Pi)$ con $V_\Pi = V$ e $E_\Pi = \{(v.\Pi, v) : v \in V, v.\Pi \neq \text{NIL}\}$ è la **foresta DF** (depth-first forest). Ogni albero della foresta è un **albero DF** (depth-first tree).

OSSERVAZIONE 10.6.2. $u = v.\Pi$ se e solo se `DFS-VISIT(v)` è stata chiamata durante l'ispezione della lista di adiacenza di u . Il vertice v è discendente di u nella foresta DF se e solo se v è stato scoperto mentre u era grigio.

I timestamp hanno una struttura di **parentesi**: se rappresentiamo la scoperta di u con $(u$ e il suo completamento con $u)$, la storia della visita forma un'espressione con parentesi correttamente annidate.

TEOREMA 10.6.3 (Teorema delle Parentesi [CLRS: Teo. 22.7]). In qualsiasi DFS di G (orientato o non orientato), per ogni coppia $u, v \in V$ vale esattamente una delle tre condizioni:

1. $[u.d, u.f]$ e $[v.d, v.f]$ sono **disgiunti**: u e v non sono discendenti l'uno dell'altro.
2. $[v.d, v.f] \subset [u.d, u.f]$: v è **discendente** di u in un albero DF.
3. $[u.d, u.f] \subset [v.d, v.f]$: u è **discendente** di v in un albero DF.

Dimostrazione. Supponiamo $u.d < v.d$ (l'altro caso è simmetrico).

Caso 1: $u.f < v.d$. Gli intervalli sono disgiunti ($u.d < u.f < v.d < v.f$). Poiché nessuno dei due è stato scoperto mentre l'altro era grigio, non c'è relazione di discendenza.

Caso 2: $u.f > v.d$. Il vertice v è stato scoperto mentre u era grigio, quindi v è discendente di u . Poiché v è stato scoperto dopo u , tutti i suoi archi vengono ispezionati e la sua visita completata prima di u : $v.d < v.f < u.f$, e quindi $[v.d, v.f] \subset [u.d, u.f]$. $\square$

COROLLARIO 10.6.4 (Annidamento degli intervalli [CLRS: Cor. 22.8]). *Il vertice v è un discendente proprio di u nella foresta DF se e solo se:*

$$u.d < v.d < v.f < u.f \quad (146)$$

10.6.4. Teorema del Cammino Bianco

TEOREMA 10.6.5 (Teorema del Cammino Bianco [CLRS: Teo. 22.9]). *Nella foresta DF di G , il vertice v è un discendente di u se e solo se, al tempo $u.d$ in cui u viene scoperto, il vertice v è raggiungibile da u lungo un cammino composto esclusivamente da vertici **bianchi**.*

Dimostrazione. $\Rightarrow$) Se $v = u$: il cammino contiene solo u , bianco al tempo $u.d$. Se v è discendente proprio di u : per il Corollario, $u.d < v.d$, quindi v è bianco al tempo $u.d$. Tutti i vertici sul cammino unico da u a v nella foresta DF sono bianchi al tempo $u.d$ (scoperti dopo u).

$\Leftarrow$) Per assurdo: esiste cammino bianco da u a v al tempo $u.d$, ma v non diventa discendente di u . Sia v il vertice più vicino a u lungo il cammino che non diventa discendente. Sia w il suo predecessore sul cammino (w è discendente di u , quindi $w.f \leq u.f$). Poiché $v \in \text{Adj}[w]$ e v è bianco al tempo $u.d < w.d$, la visita di w scopre v : v diventa discendente di w e quindi di u . Contraddizione. $\square$

10.6.5. Classificazione degli archi

La DFS classifica ogni arco (u, v) in base al colore di v quando l'arco viene ispezionato per la prima volta [CLRS: p. 509]:

Tipo	Condizione (colore di v)	Descrizione
Arco d'albero	v è Bianco	$(u, v) \in E_{\Pi}$; v scoperto esaminando (u, v)
Arco all'indietro	v è Grigio	v è antenato di u nella foresta DF
Arco in avanti	v è Nero , $u.d < v.d$	v è discendente non diretto di u
Arco trasversale	v è Nero , $u.d > v.d$	u e v non sono in relazione di discendenza

OSSERVAZIONE 10.6.6. Caratterizzazione tramite timestamp:

- Arco d'albero o in avanti: $u.d < v.d < v.f < u.f$
- Arco all'indietro: $v.d \leq u.d < u.f \leq v.f$
- Arco trasversale: $v.d < v.f < u.d < u.f$

TEOREMA 10.6.7 (Archi nei grafi non orientati [CLRS: Teo. 22.10]). *In una DFS di un grafo **non orientato** G , ogni arco è un arco d'albero o un arco all'indietro. Non esistono archi in avanti né archi trasversali.*

Dimostrazione. Sia $(u, v) \in E$ con $u.d < v.d$ (senza perdita di generalità). Il vertice v è nella lista di adiacenza di u , quindi viene scoperto e completato mentre u è grigio: $[v.d, v.f] \subset [u.d, u.f]$.

Caso 1: (u, v) viene ispezionato per la prima volta da u verso v . Allora v è bianco (non ancora scoperto, poiché avremmo già visto l'arco da v verso u): arco d'albero.

Caso 2: (u, v) viene ispezionato per la prima volta da v verso u . Allora u è grigio (ancora in visita): arco all'indietro. $\square$

10.6.6. Grafi ciclici e archi all'indietro

LEMMA 10.6.8 (Aciclicità e archi all'indietro [CLRS: Lemma 22.11]). *Un grafo orientato G è **aciclico** se e solo se una DFS di G non genera archi all'indietro.*

Dimostrazione. $\Rightarrow$) Se (u, v) è un arco all'indietro, v è antenato di u : esiste cammino $v \rightsquigarrow u$, e l'arco (u, v) forma un ciclo.

$\Leftarrow$) Se G contiene un ciclo c , sia v il primo vertice scoperto in c e (u, v) l'arco che lo precede. Al tempo $v.d$, tutti i vertici di c sono bianchi, quindi esiste cammino bianco da v a u . Per il Teorema del Cammino Bianco, u diventa discendente di v , e (u, v) è un arco all'indietro. $\square$

10.7. Cammini minimi da sorgente unica

Il problema dei **cammini minimi da sorgente unica** si pone su grafi **pesati**: dato $G = (V, E)$ e una sorgente $s \in V$, si vuole determinare un cammino di peso minimo da s a ciascun vertice $v \in V$.

Applicazioni tipiche sono il **routing su rete** (instradamento di pacchetti su internet, navigatori satellitari): si opera sulla base di algoritmi per la ricerca di cammini minimi. Per grafi **non pesati** il problema si risolve con $\text{BFS}(G, s)$.

10.7.1. Grafi pesati

DEFINIZIONE 10.7.1 (Grafo pesato e peso di un cammino). Dato $G = (V, E)$, una **funzione peso** è una funzione $\omega : E \rightarrow \mathbb{R}$ che associa a ciascun arco un peso (costi, perdite, distanze: quantità che occorre minimizzare).

Il **peso di un cammino** $p = \langle v_0, v_1, \dots, v_k \rangle$ è:

$$\omega(p) = \sum_{i=1}^k \omega(v_{i-1}, v_i) \quad (147)$$

Il **peso di un cammino minimo** da u a v è:

$$\delta(u, v) = \begin{cases} \min \{ \omega(p) : u \overset{p}{\rightsquigarrow} v \} & \text{se esiste un cammino } u \overset{p}{\rightsquigarrow} v \\ +\infty & \text{altrimenti} \end{cases} \quad (148)$$

Un **cammino minimo** da u a v è un cammino p di peso $\omega(p) = \delta(u, v)$.

10.7.2. Varianti del problema

- **Cammini minimi con sorgente unica** $s \in V$: cammino di peso minimo dal vertice s a ciascun vertice $v \in V$.
- **Cammini minimi con destinazione unica** $t \in V$: cammino di peso minimo da ciascun vertice $v \in V$ al vertice t . Invertendo la direzione degli archi, si riconduce al caso precedente.
- **Cammino minimo per una coppia di vertici**: cammino minimo da u a v fissati. Si risolve con il problema della sorgente unica scegliendo $s = u$ (asintoticamente, al caso pessimo, stesso costo di algoritmi specifici).
- **Cammini minimi tra tutte le coppie di vertici**: si può risolvere iterando l'algoritmo con sorgente unica scegliendo ogni vertice come sorgente (esistono metodi più efficienti).

10.7.3. Proprietà dei cammini minimi

LEMMA 10.7.2 (Sottostruttura ottima dei cammini minimi). Dato un grafo orientato pesato $G = (V, E)$ con funzione peso $\omega : E \rightarrow \mathbb{R}$, sia $p = \langle v_1, v_2, \dots, v_k \rangle$ un cammino minimo $v_1 \rightsquigarrow v_k$ e, per $1 \leq i \leq j \leq k$, sia $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ il sottocammino di p da v_i a v_j . Allora p_{ij} è un cammino minimo da v_i a v_j .

I sottocammini di cammini minimi sono cammini minimi.

Dimostrazione. Scomponiamo p come $v_1 \xrightarrow{p_{1i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, da cui:

$$\omega(p) = \omega(p_{1i}) + \omega(p_{ij}) + \omega(p_{jk}) \quad (149)$$

Per assurdo, esista un cammino p'_{ij} da v_i a v_j con $\omega(p'_{ij}) < \omega(p_{ij})$. Allora il cammino p' ottenuto sostituendo p_{ij} con p'_{ij} avrebbe peso:

$$\omega(p') = \omega(p_{1i}) + \omega(p'_{ij}) + \omega(p_{jk}) < \omega(p) \quad (150)$$

che contraddice la minimalità di p . $\square$

10.7.4. Archi di peso negativo e cicli di peso negativo

In alcuni problemi di cammini minimi possono presentarsi archi con peso negativo. Se il grafo G **non** contiene cicli di peso negativo raggiungibili da s , allora $\delta(s, v)$ resta ben definito per ogni $v \in V$ (anche se può avere valore negativo).

Se invece esiste un ciclo di peso negativo raggiungibile da s e, da un vertice del ciclo, si raggiunge v , allora $\delta(s, v)$ **non** è ben definito: percorrendo il ciclo un numero arbitrario di volte si ottengono cammini di peso arbitrariamente piccolo. In questo caso si pone per convenzione $\delta(s, v) = -\infty$.

OSSERVAZIONE 10.7.3. I cammini minimi sono sempre cammini semplici, privi di cicli:

- se il ciclo ha peso > 0 , eliminandolo dal cammino se ne ottiene uno di peso minore;
- se il ciclo ha peso 0 , possiamo eliminarlo e ottenere un cammino semplice con lo stesso peso;
- un ciclo di peso < 0 non può comparire in un cammino minimo (altrimenti δ non sarebbe definito).

Un cammino semplice contiene al più $|V|$ vertici, e dunque $|V| - 1$ archi.

I cammini minimi si rappresentano con l'attributo π (predecessore): la catena dei predecessori a partire da v descrive all'indietro il cammino minimo da s a v , e può essere stampato con PRINT-PATH.

L'**algoritmo di Dijkstra** assume che tutti i pesi siano non negativi ($\omega(u, v) \geq 0$). L'**algoritmo di Bellman-Ford** (non trattato qui) gestisce il caso generale con archi di peso negativo, rilevando inoltre l'eventuale presenza di cicli di peso negativo raggiungibili dalla sorgente.

10.7.5. Disuguaglianza triangolare pesata

LEMMA 10.7.4 (Disuguaglianza triangolare [CLRS: Lemma 24.10]). Sia $G = (V, E)$ un grafo orientato pesato con funzione peso ω e sorgente $s \in V$. Per ogni arco $(u, v) \in E$:

$$\delta(s, v) \leq \delta(s, u) + \omega(u, v) \quad (151)$$

Dimostrazione. Se u è raggiungibile da s , estendendo un cammino minimo $s \rightsquigarrow u$ con l'arco (u, v) si ottiene un cammino da s a v di peso $\delta(s, u) + \omega(u, v)$: non può essere più corto del cammino minimo $s \rightsquigarrow v$. Se u non è raggiungibile, $\delta(s, u) = +\infty$ e la disuguaglianza è banalmente vera. $\square$

Nota. È la versione pesata del Lemma 22.1 visto per BFS: qui al posto di «+1» (lunghezza dell'arco non pesato) compare il peso $\omega(u, v)$.

10.7.6. Inizializzazione e rilassamento

Gli algoritmi per cammini minimi operano su due attributi per ogni vertice v : la **stima** $v.d$ (limite superiore di $\delta(s, v)$) e il **predecessore** $v.\pi$. Entrambi vengono inizializzati da una procedura INITIALIZE-SINGLE-SOURCE e modificati esclusivamente dal **rilassamento** di un arco.

INITIALIZE-SINGLE-SOURCE(G, s)

```
INITIALIZE-SINGLE-SOURCE( $G, s$ ) {
  for ( $v$  in  $V$ ) {
     $v.d = +\infty$ ;
     $v.\pi = \text{NIL}$ ;
  }
   $s.d = 0$ ;
}
```

Relax(u, v, ω)

```
RELAX( $u, v, \omega$ ) {
  if ( $v.d > u.d + \omega(u, v)$ ) {
     $v.d = u.d + \omega(u, v)$ ;
     $v.\pi = u$ ;
  }
}
```

Il rilassamento di (u, v) **verifica** se, passando da u , si può migliorare la stima corrente di v ; in caso affermativo aggiorna $v.d$ e $v.\pi$. Costo: $O(1)$.

Nota. Il termine «rilassamento» è tradizionale: l'operazione riduce un limite superiore, quindi «rilassa» il vincolo $v.d \leq u.d + \omega(u, v)$ (che deve valere all'ottimo per la disuguaglianza triangolare).

10.8. Algoritmo di Dijkstra

L'**algoritmo di Dijkstra** risolve il problema dei cammini minimi da sorgente unica in un grafo orientato pesato $G = (V, E)$ con pesi **non negativi**: $\forall (u, v) \in E, \omega(u, v) \geq 0$.

Per ogni vertice $v \in V$ mantiene due attributi:

- $v.\pi$: predecessore di v nell'albero dei cammini minimi;
- $v.d$: **stima del cammino minimo**, ovvero un **limite superiore** per $\delta(s, v)$, inizializzato a $+\infty$ (vale sempre $v.d \geq \delta(s, v)$).

Tutti i vertici sono inseriti in una **coda di MIN-priorità** PQ con priorità $v.d$.

Nota (Coda di MIN-priorità). Una *coda di MIN-priorità* è una struttura dati che mantiene un insieme di elementi, ciascuno con una **chiave** (o priorità), e supporta tre operazioni fondamentali:

- `INSERT(PQ, x)` : inserisce l'elemento x con la sua chiave corrente;
- `EXTRACT-MIN(PQ)` : rimuove e restituisce l'elemento con chiave **minima**;
- `DECREASE-KEY(PQ, x, k)` : diminuisce la chiave dell'elemento x a k (solo se k è minore della chiave attuale).

In Dijkstra la chiave di v è la stima $v.d$: `EXTRACT-MIN` restituisce il vertice con la minor stima corrente di cammino minimo. L'implementazione tipica è un **MIN-heap binario** (tipicamente trattato nel capitolo sugli heap).

L'algoritmo seleziona ripetutamente il vertice u con la minor stima del cammino minimo (estraendolo da PQ) ed esamina gli archi uscenti da u : per ogni $v \in \text{Adj}[u]$, controlla se passando da u si trova un cammino meno costoso da s a v ; in tal caso aggiorna $v.d$ e $v.\pi$ (**rilassamento** dell'arco (u, v)).

Dijkstra(G, ω, s)

```

DIJKSTRA( $G, \omega, s$ ) {    //  $\omega$ : funzione peso,  $s$ : sorgente
    INITIALIZE-SINGLE-SOURCE( $G, s$ );
    PQ = nuova coda di MIN-priorità;
    for ( $v$  in  $V$ )
        INSERT(PQ,  $v$ );                // priorità:  $v.d$ 
    while (PQ  $\neq \emptyset$ ) {
         $u$  = EXTRACT-MIN(PQ);
        for ( $v$  in  $\text{Adj}[u]$ ) {           // rilassamento degli archi
            if ( $v.d > u.d + \omega(u, v)$ ) { // RELAX( $u, v, \omega$ )
                 $v.d = u.d + \omega(u, v)$ ;
                 $v.\pi = u$ ;
                DECREASE-KEY(PQ,  $v, v.d$ );
            }
        }
    }
}

```

OSSERVAZIONE 10.8.1. Rispetto al RELAX astratto, qui il test e l'aggiornamento sono esplicitati perché quando si abbassa $v.d$ occorre invocare DECREASE-KEY sulla coda di priorità — altrimenti PQ non manterrebbe la proprietà di heap.

10.8.1. Simulazione

ESEMPIO 10.8.2 (Esecuzione di Dijkstra su grafo pesato). Grafo orientato G con $V = \{s, t, x, y, z\}$ e archi pesati: $\omega(s, t) = 10, \omega(s, y) = 5, \omega(t, x) = 1, \omega(t, y) = 2, \omega(y, t) = 3, \omega(y, x) = 9, \omega(y, z) = 2, \omega(x, z) = 4, \omega(z, s) = 7, \omega(z, x) = 6$.

Dopo l'inizializzazione, $s.d = 0$ e tutti gli altri vertici hanno $d = +\infty$. Ad ogni iterazione si estrae da PQ il vertice con stima minima e si rilassano gli archi uscenti. La tabella mostra le stime **dopo** il rilassamento e sottolinea il prossimo vertice che verrà estratto da PQ.

Iter.	u estratto	$s.d$	$t.d$	$x.d$	$y.d$	$z.d$
0 (init)	—	<u>0</u>	∞	∞	∞	∞
1	s	0	10	∞	<u>5</u>	∞
2	y	0	8	14	5	<u>7</u>
3	z	0	<u>8</u>	13	5	7
4	t	0	8	<u>9</u>	5	7
5	x	0	8	9	5	7

Distanze finali: $s.d = 0, t.d = 8, x.d = 9, y.d = 5, z.d = 7$.

Predecessori: $t.\pi = y$ (cammino $s \rightarrow y \rightarrow t$), $x.\pi = t$ ($s \rightarrow y \rightarrow t \rightarrow x$), $y.\pi = s$, $z.\pi = y$ ($s \rightarrow y \rightarrow z$).

Si noti che all'iterazione 2, rilassando gli archi uscenti da y , la stima di t scende da 10 a $5 + 3 = 8$: il cammino $s \rightarrow y \rightarrow t$ è migliore dell'arco diretto $s \rightarrow t$.

10.8.2. Correttezza

TEOREMA 10.8.3 (Correttezza dell'algoritmo di Dijkstra). L'algoritmo di Dijkstra, eseguito su un grafo orientato pesato $G = (V, E)$ con funzione peso non negativa ω e sorgente s , termina con $v.d = \delta(s, v)$ per ogni $v \in V$.

Dimostrazione (sketch). (1) Per ogni vertice $v \in V$ vale $v.d \geq \delta(s, v)$.

- All'inizio $v.d = +\infty \geq \delta(s, v)$.
- $v.d$ viene aggiornato solo dal rilassamento, che pone $v.d$ uguale al peso di un qualche cammino $s \rightsquigarrow v$, che è certamente $\geq \delta(s, v)$.
- Sulla sorgente si pone $s.d = 0 = \delta(s, s)$, e non viene più aggiornato.

(2) Per induzione sul numero di vertici estratti da PQ. Sia S l'insieme dei vertici già estratti da PQ.

Ipotesi induttiva: per ogni $v \in S$, $v.d = \delta(s, v)$.

Base: $S = \{s\}$ e $s.d = 0 = \delta(s, s)$. ✓

Passo: sia v il prossimo vertice estratto da PQ e inserito in S . Sia P un cammino $s \rightsquigarrow v$ di peso minimo, e sia $(x, y) \in E$ il primo arco di P che esce da S : $x \in S$ e y è ancora in PQ (s può coincidere con x , e y può coincidere con v).

Mostriamo che $y.d = \delta(s, y)$: quando x è stato estratto da PQ, l'arco (x, y) è stato rilassato, dunque per ipotesi induttiva su x e per la sottostruttura ottima:

$$y.d \leq x.d + \omega(x, y) = \delta(s, x) + \omega(x, y) = \delta(s, y) \quad (152)$$

Combinando con il punto (1), $y.d = \delta(s, y)$.

Ora, essendo y sul cammino minimo P prima di v e i pesi non negativi:

$$v.d \geq \delta(s, v) \geq \delta(s, y) = y.d \geq v.d \quad (153)$$

dove l'ultima disuguaglianza vale perché v è stato estratto prima di y da PQ (scelta del minimo). Dunque $v.d = \delta(s, v)$.

Al termine dell'algoritmo $PQ = \emptyset$ e $S = V$: per ogni $v \in V$, $v.d = \delta(s, v)$. □

10.8.3. Analisi di complessità

Il tempo di esecuzione dipende dall'implementazione della coda di MIN-priorità. In generale:

$$T(|V|, |E|) = O(|V| \cdot \text{costo}(\text{EXTRACT-MIN}) + |E| \cdot \text{costo}(\text{DECREASE-KEY})) \quad (154)$$

- Si eseguono $|V|$ operazioni EXTRACT-MIN.
- La lista di adiacenza di ciascun vertice viene esaminata esattamente una volta: $|E|$ iterazioni del ciclo `for`, con al più $|E|$ chiamate di DECREASE-KEY.
- La costruzione di PQ richiede $\Theta(|V|)$ (ogni inserzione costa $O(1)$: solo due valori, 0 e $+\infty$).

Implementazione di PQ	EXTRACT-MIN	DECREASE-KEY	Totale
Array indicizzato sui vertici	$O(V)$	$O(1)$	$O(V ^2)$
MIN-heap binario	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$

OSSERVAZIONE 10.8.4.

- **Array:** si ispeziona l'intero array per trovare il vertice con stima minima. Costo complessivo $\Theta(|V|) + O(|V|^2 + |E|) = O(|V|^2)$. Preferibile per grafi **densi**.
- **MIN-heap binario:** la costruzione di PQ richiede $\Theta(|V|)$. Costo complessivo $\Theta(|V|) + O(|V| \log |V| + |E| \log |V|) = O((|V| + |E|) \log |V|)$. Preferibile per grafi **sparsi**.

Per implementare DECREASE-KEY in $O(\log |V|)$ con lo heap binario, occorre trovare v in PQ in tempo costante: i vertici devono mantenere un riferimento alla posizione del corrispondente elemento nell'heap (e viceversa).

10.8.4. Esempio di implementazione di DECREASE-KEY

Per ogni $v \in V$, $v.pos$ è la posizione del vertice v in PQ. PQ è un array di coppie (stima, vertice), dove **stima** è il valore della stima di cammino minimo e **vertice** è il riferimento al vertice.

DECREASE-KEY(PQ, v, dnew)

```
DECREASE-KEY(PQ, v, dnew) {
    if (dnew ≥ v.d)
        error "la nuova stima non è minore della precedente";
    i = v.pos;                // posizione di v in PQ
    PQ[i].stima = dnew;       // PQ è uno heap tranne in posizione i
    while (i > 1 && PQ[i].stima < PQ[Parent(i)].stima) {
        Scambia(PQ, i, Parent(i));
        i = Parent(i);
    }
}
```

Scambia(PQ, i, j)

```
SCAMBIA(PQ, i, j) {
    tmp = PQ[i];
    PQ[i] = PQ[j];
    PQ[j] = tmp;
    (PQ[i].vertice).pos = i;    // aggiorna i riferimenti
    (PQ[j].vertice).pos = j;
}

PARENT(i) { return ⌊i/2⌋; }
```

La DECREASE-KEY costa $O(\log |V|)$ (risalita nello heap); **Scambia** e **Parent** costano $O(1)$.

CAPITOLO 11

Calcolabilità e Complessità Computazionale

11.1. Problemi computazionali, calcolabilità e complessità

In tutto il resto della dispensa abbiamo progettato algoritmi **efficienti**: $O(n \log n)$ per l'ordinamento, $O(n)$ per Fibonacci con la programmazione dinamica, $O(n + m)$ per la visita di un grafo. In questo capitolo conclusivo cambiamo prospettiva e ci chiediamo quali siano i **limiti intrinseci** del calcolo: esistono problemi che nessun calcolatore potrà mai risolvere? Esistono problemi risolvibili in linea di principio ma per i quali ogni algoritmo richiede tempo proibitivo nella pratica?

Le risposte affermative a queste domande — e la loro dimostrazione — costituiscono i fondamenti di due teorie distinte ma intrecciate: la **teoria della calcolabilità** e la **teoria della complessità**.

DEFINIZIONE 11.1.1 (Problema computazionale). Un **problema computazionale** è una funzione matematica che associa ad ogni **istanza** di ingresso un **risultato**. Formalmente, fissati interi $k, j \geq 1$, un problema è una funzione

$$f : \mathbb{N}^k \rightarrow \mathbb{N}^j \quad (155)$$

e **risolvere** il problema significa esibire un algoritmo che, su ogni istanza valida, produca in tempo finito il valore corrispondente.

I problemi computazionali si classificano gerarchicamente:

Da ricordare.

- **Problemi indecidibili (non calcolabili):** nessun algoritmo può risolverli, nemmeno con tempo e memoria illimitati.
- **Problemi decidibili (calcolabili):** esiste almeno un algoritmo che li risolve in tempo finito su ogni istanza.
 - **Trattabili:** esiste un algoritmo di costo **polinomiale** nella dimensione dell'istanza ($O(n^k)$).
 - **Intrattabili:** tutti gli algoritmi noti hanno costo **esponenziale** (e in alcuni casi si può dimostrare che è inevitabile).

Nota (Calcolabilità vs. Complessità). La **calcolabilità** studia il confine fra problemi risolvibili e non risolvibili (qualitativo: *esiste* un algoritmo?). La **complessità** studia il confine fra problemi facili e difficili all'interno di quelli risolvibili (quantitativo: *quanto costa* il miglior algoritmo?).

11.2. Insiemi numerabili e diagonalizzazione

Per dimostrare l'esistenza di problemi indecidibili, mostreremo che gli **algoritmi** sono enumerabili come $\mathbb{N}$, mentre i **problemi** no. Da qui segue per cardinalità che esistono problemi privi di un corrispondente algoritmo.

11.2.1. Insiemi numerabili

DEFINIZIONE 11.2.1 (Insiemi equipotenti e numerabili). Due insiemi A e B si dicono **equipotenti** (hanno lo stesso numero di elementi) se esiste una corrispondenza biunivoca $f : A \rightarrow B$.

Un insieme A si dice **numerabile** se esiste una corrispondenza biunivoca con i numeri naturali $\mathbb{N}$, ovvero se i suoi elementi possono essere disposti in una sequenza

$$a_1, a_2, \dots, a_n, \dots \quad (156)$$

che li raggiunge tutti.

ESEMPIO 11.2.2 (Insiemi numerabili classici).

- $\mathbb{N}$ stesso, banalmente.
- $\mathbb{Z}$: si enumera $0, -1, 1, -2, 2, -3, 3, \dots$ associando all'intero k il naturale $2k$ se $k \geq 0$ e $2|k| - 1$ se $k < 0$ (così $0 \rightarrow 0, -1 \rightarrow 1, 1 \rightarrow 2, -2 \rightarrow 3, \dots$).
- I numeri naturali pari: $0, 2, 4, 6, \dots$ con la biiezione $n \leftrightarrow 2n$.
- Le coppie $\mathbb{N} \times \mathbb{N}$, i numeri razionali $\mathbb{Q}$, e in generale ogni unione numerabile di insiemi numerabili.

11.2.2. Enumerazione delle sequenze finite

Un punto chiave: l'insieme delle sequenze *finite* di simboli su un alfabeto *finito* è numerabile, anche se infinito.

DEFINIZIONE 11.2.3 (Ordinamento canonico). Fissato un alfabeto finito Γ con un ordine totale fra i suoi caratteri (per es. ordine alfabetico), si elencano le sequenze finite su Γ in **ordine canonico**:

1. per **lunghezza crescente**;
2. a parità di lunghezza, in **ordine alfabetico**.

ESEMPIO 11.2.4 (Ordinamento canonico su $\Gamma = \{a, b, c\}$).

$$a, b, c, \quad aa, ab, ac, ba, bb, bc, ca, cb, cc, \quad aaa, aab, \dots \quad (157)$$

Ogni sequenza σ di lunghezza $|\sigma| = \ell$ appare a una distanza finita dall'inizio, perché prima di lei compaiono al più $|\Gamma| + |\Gamma|^2 + \dots + |\Gamma|^\ell$ sequenze, che è un numero finito.

OSSERVAZIONE 11.2.5. La numerazione funziona perché le sequenze hanno lunghezza *finita*. Per sequenze di lunghezza infinita la numerazione fallisce: questo distinguerà gli oggetti «rappresentabili» (algoritmi) da quelli «infiniti» (alcune funzioni).

11.2.3. Insiemi non numerabili: la diagonalizzazione di Cantor

Ci sono insiemi i cui elementi non possono essere disposti in alcuna sequenza che li raggiunga tutti. Lo strumento standard per dimostrarlo è il **metodo della diagonale**.

TEOREMA 11.2.6 (Cantor). *L'insieme $\mathcal{F} = \{f \mid f : \mathbb{N} \rightarrow \{0, 1\}\}$ delle funzioni booleane sui naturali non è numerabile.*

Dimostrazione (per assurdo e diagonalizzazione). Supponiamo per assurdo che $\mathcal{F}$ sia numerabile. Allora possiamo disporre le sue funzioni in una sequenza $f_0, f_1, f_2, \dots$ e costruire la tabella infinita

x	0	1	2	3	4	5	6	7	8	...
$f_0(x)$	1	0	1	0	1	0	0	0	1	...
$f_1(x)$	0	0	1	1	0	0	1	1	0	...
$f_2(x)$	1	1	0	1	0	1	0	0	1	...
$f_3(x)$	0	1	1	0	1	0	1	1	1	...
$f_4(x)$	1	1	0	0	1	0	0	0	1	...
...										

Definiamo la funzione $g : \mathbb{N} \rightarrow \{0, 1\}$ «anti-diagonale»:

$$g(x) = \begin{cases} 1 & \text{se } f_{x(x)} = 0 \\ 0 & \text{se } f_{x(x)} = 1 \end{cases} \quad (158)$$

Per costruzione g differisce da f_0 in 0, da f_1 in 1, da f_2 in 2, e in generale da f_x nel valore x . Quindi $g \neq f_j$ per ogni $j \in \mathbb{N}$, ma $g \in \mathcal{F}$. Contraddizione: la nostra enumerazione non era totale, quindi $\mathcal{F}$ non è numerabile. $\square$

OSSERVAZIONE 11.2.7. L'argomento è robustissimo: per *qualunque* enumerazione si scelga, la funzione «anti-diagonale» che si costruisce da quella enumerazione è sempre esclusa. Variando la «linea diagonale» (qualsiasi linea che attraversi tutte le righe e tutte le colonne esattamente una volta) si trovano *infinite* funzioni escluse da ogni enumerazione: $\mathcal{F}$ è strettamente più grande di $\mathbb{N}$.

COROLLARIO 11.2.8. Non sono numerabili gli insiemi delle funzioni $f : \mathbb{N} \rightarrow \mathbb{N}$, $f : \mathbb{N} \rightarrow \mathbb{R}$, $f : \mathbb{R} \rightarrow \mathbb{R}$ né, in generale, $f : \mathbb{N}^k \rightarrow \mathbb{N}^j$.

Dimostrazione (inclusione). Una funzione $f : \mathbb{N} \rightarrow \{0, 1\}$ è anche una particolare funzione $f : \mathbb{N} \rightarrow \mathbb{N}$: se l'insieme più ristretto è non numerabile, a maggior ragione lo sono i suoi sovrainsiemi. $\square$

11.3. Esistenza di problemi indecidibili

Mettiamo insieme due osservazioni di natura diversa.

PROPOSIZIONE 11.3.1 (Numerabilità degli algoritmi). L'insieme degli algoritmi (in qualunque modello di calcolo ragionevole) è numerabile.

Dimostrazione (rappresentazione). Un algoritmo, una volta fissato un modello di calcolo (Macchina di Turing, RAM, linguaggio C, ...), è una **sequenza finita di simboli** di un alfabeto finito (il suo «codice sorgente»). Per la sezione precedente, l'insieme di tali sequenze è numerabile mediante l'ordinamento canonico. $\square$

PROPOSIZIONE 11.3.2 (Non numerabilità dei problemi). L'insieme dei problemi computazionali $f : \mathbb{N}^k \rightarrow \mathbb{N}^j$ non è numerabile.

Dimostrazione (riduzione a Cantor). Restringendo il codominio a $\{0, 1\}$ otteniamo le funzioni booleane $f : \mathbb{N}^k \rightarrow \{0, 1\}$, che a loro volta contengono come sotto-insieme proprio le $f : \mathbb{N} \rightarrow \{0, 1\}$ (basta usare una sola variabile). Per il Teorema di Cantor questo sotto-insieme non è numerabile. Un sovrainsieme di un insieme non numerabile non è numerabile, da cui la tesi. $\square$

TEOREMA 11.3.3 (Esistenza di problemi indecidibili). Esistono problemi computazionali per i quali non esiste alcun algoritmo di risoluzione.

Dimostrazione (cardinalità). Se ad ogni problema corrispondesse un algoritmo, l'insieme dei problemi risolvibili sarebbe in iniezione con l'insieme degli algoritmi, quindi numerabile. Ma l'insieme dei problemi non lo è. La cardinalità degli algoritmi è strettamente minore di quella dei problemi:

$$|\{\text{Algoritmi}\}| \ll |\{\text{Problemi}\}| \quad (159)$$

Esistono dunque problemi privi di algoritmo. $\square$

Attenzione: Questa dimostrazione è puramente **esistenziale**: ci dice che ci sono problemi indecidibili, ma non ne esibisce nessuno. Per averne uno concreto serve un argomento costruttivo, fornito da Turing (1936) col problema dell'arresto.

11.4. Il problema dell'arresto

Storicamente, il primo esempio esplicito di problema indecidibile è il **problema dell'arresto** (*Halting Problem*) introdotto da Alan Turing nel 1936.

DEFINIZIONE 11.4.1 (Problema dell'arresto). Dati un algoritmo A e un'istanza di input D , **decidere in tempo finito** se la computazione $A(D)$ termina (si arresta) oppure non termina (entra in un ciclo infinito).

OSSERVAZIONE 11.4.2. Si tratta di un **problema decisionale**: la risposta è 1 (« A su D termina») o 0 (« A su D non termina»). Per problemi decisionali la calcolabilità prende il nome di **decidibilità**. Inoltre è un problema **meta**: indaga proprietà di un algoritmo, trattandolo come dato di input. Questo è legittimo perché tanto l'algoritmo A quanto il dato D sono sequenze finite di simboli sullo stesso alfabeto.

11.4.1. Un caso reale: l'algoritmo Goldbach

Per capire perché il problema è non banale, consideriamo:

Goldbach() — cerca un controesempio alla congettura di Goldbach

```
Goldbach() {
  n = 2;
  do {
    n = n + 2;
    controesempio = true;
    for (p = 2; p <= n - 2; p = p + 1) {
      q = n - p;
      if (Primo(p) AND Primo(q))
        controesempio = false;
    }
  } while (NOT controesempio);
  return n;
}
```

L'algoritmo cerca il più piccolo intero pari $n \geq 4$ che *non* sia somma di due primi e si arresta non appena lo trova.

Nota (Congettura di Goldbach (1742)). Ogni intero pari $n \geq 4$ è esprimibile come somma di due numeri primi.

La congettura è aperta dal XVIII secolo. Allora:

- Se la congettura è **falsa**, esiste un controesempio e Goldbach() **si arresta**.
- Se la congettura è **vera**, non c'è controesempio e Goldbach() **non si arresta**.

Quindi un ipotetico algoritmo che decide l'arresto risolverebbe immediatamente la congettura — e in generale qualsiasi congettura aritmetica esprimibile come «esiste un controesempio». Questo è un primo segnale che un tale algoritmo non può esistere.

11.4.2. Il teorema di Turing

TEOREMA 11.4.3 (Turing, 1936 — Indecidibilità dell'arresto). Non esiste alcun algoritmo che, dato in input un arbitrario algoritmo A e un arbitrario dato D , decida in tempo finito se $A(D)$ termina.

Dimostrazione (per assurdo, paradosso di auto-applicazione). Supponiamo per assurdo che esista un algoritmo Arresto tale che, su ogni input (A, D) ,

$$\text{Arresto}(A, D) = \begin{cases} 1 & \text{se } A(D) \text{ termina} \\ 0 & \text{se } A(D) \text{ non termina} \end{cases} \quad (160)$$

e che Arresto termini sempre in tempo finito.

Osservazione preliminare. L'algoritmo Arresto non può limitarsi a simulare A su D : se $A(D)$ non termina, la simulazione gira all'infinito e Arresto non può rispondere 0 in tempo finito. Deve allora ispezionare il codice di A .

Costruzione dell'algoritmo paradosso. Definiamo l'algoritmo

Paradosso(A) — usa Arresto come oracolo

```
Paradosso(A) {
    while (Arresto(A, A) == 1) {
        ; // ciclo vuoto
    }
}
```

Notiamo che è legittimo passare A sia come algoritmo sia come dato: entrambi sono sequenze di simboli sullo stesso alfabeto e una stessa stringa può essere interpretata, a seconda del contesto, come codice o come input. L'ispezione di Paradosso mostra:

$$\text{Paradosso}(A) \text{ termina} \leftrightarrow \text{Arresto}(A, A) = 0 \leftrightarrow A(A) \text{ non termina.} \quad (161)$$

L'auto-applicazione. Cosa succede se applichiamo Paradosso a se stesso? Sostituendo $A = \text{Paradosso}$:

$$\begin{aligned} \text{Paradosso}(\text{Paradosso}) \text{ termina} &\leftrightarrow \text{Arresto}(\text{Paradosso}, \text{Paradosso}) = 0 \\ &\leftrightarrow \text{Paradosso}(\text{Paradosso}) \text{ non termina.} \end{aligned} \quad (162)$$

Cioè Paradosso(Paradosso) termina se e solo se non termina: **contraddizione**.

L'unico modo per uscire dalla contraddizione è negare l'esistenza di Paradosso, ma poiché Paradosso è ottenuto da Arresto con una semplice costruzione, deve essere Arresto a non esistere. Il problema dell'arresto è dunque indecidibile. $\square$

OSSERVAZIONE 11.4.4. La struttura della dimostrazione è ancora una **diagonalizzazione**: l'algoritmo Paradosso «fa il contrario» della tabella di tutti gli algoritmi quando applicato a se stesso, esattamente come la g di Cantor faceva il contrario della diagonale di tutte le funzioni.

Attenzione: Il teorema dice che è indecidibile la coppia (A, D) **arbitraria**, non che sia impossibile decidere la terminazione di un singolo programma fissato. Per esempio l'algoritmo `Primo(p)` (cerca un divisore di p partendo da 2) termina certamente per ogni $p > 1$, e questo lo possiamo dimostrare.

11.4.3. Altri problemi indecidibili

Una volta noto un problema indecidibile, se ne ottengono molti altri:

- **Equivalenza di programmi:** dati due algoritmi A_1, A_2 , decidere se per ogni input producono lo stesso output. Indecidibile.
- **Decimo problema di Hilbert (Matijasevich, 1970):** data un'arbitraria equazione diofantea $p(x_1, \dots, x_n) = 0$ (polinomio a coefficienti interi), decidere se ammette soluzioni intere. Indecidibile.

Nota (Lezione di Turing). Non esistono algoritmi che decidono il comportamento di altri algoritmi **esaminandoli dall'esterno**, cioè senza ricorrere alla loro simulazione completa. Ogni proprietà non banale del comportamento di un algoritmo arbitrario è indecidibile (Teorema di Rice, esula da questo corso).

11.4.4. La tesi di Church-Turing

Sembrerebbe che la decidibilità di un problema dipenda dal modello di calcolo (RAM? Macchina di Turing? Linguaggio C? Quantum computer?). Non è così.

Da ricordare. Tesi di Church-Turing. Tutti i ragionevoli modelli di calcolo risolvono esattamente la stessa classe di problemi. Modelli più sofisticati (più istruzioni, più memoria, parallelismo) possono ridurre il **tempo** di esecuzione e rendere la programmazione più ergonomica, ma *non* ampliano la classe dei problemi decidibili.

In particolare, la decidibilità è una proprietà del **problema**, non dell'apparato che cerca di risolverlo.

11.5. Problemi intrattabili

Esiste un'ulteriore frontiera, *interna* ai problemi decidibili: i **problemi intrattabili** sono quelli decidibili ma che richiedono tempo esponenziale nella dimensione n dell'istanza — e dunque sono praticamente irrisolvibili per n già moderato. Vediamo tre classici esempi prima di formalizzare le classi di complessità.

11.5.1. Le Torri di Hanoi

DEFINIZIONE 11.5.1 (Problema delle Torri di Hanoi). Tre pioli p, t, s e n dischi di dimensioni decrescenti, inizialmente tutti impilati in p in ordine decrescente (il più grande in basso). Spostare tutti i dischi su t rispettando le regole:

1. si sposta un solo disco alla volta;
2. un disco non può essere posto su uno più piccolo.

TorriHanoi(n, p, t, s) — sposta n dischi da p a t usando s come appoggio

```
TorriHanoi(n, p, t, s) {
  if (n == 1) print "p -> t";
  else {
    TorriHanoi(n - 1, p, s, t);
    print "p -> t";
    TorriHanoi(n - 1, s, t, p);
  }
}
```

Sia $M(n)$ il numero di mosse generate dall'algoritmo. La ricorrenza è

$$M(n) = \begin{cases} 1 & n = 1 \\ 2M(n-1) + 1 & n > 1 \end{cases} \quad (163)$$

TEOREMA 11.5.2. $M(n) = 2^n - 1$.

Dimostrazione (induzione su n). Base. $M(1) = 1 = 2^1 - 1$.

Ipotesi induttiva. $M(n-1) = 2^{n-1} - 1$.

Passo. $M(n) = 2M(n-1) + 1 = 2(2^{n-1} - 1) + 1 = 2^n - 2 + 1 = 2^n - 1$. □

ESEMPIO 11.5.3 (L'esponenziale è davvero proibitivo). Per $n = 64$ dischi, anche a una mossa al secondo, $2^{64} - 1 \approx 1.8 \cdot 10^{19}$ secondi $\approx 585 \cdot 10^9$ anni: oltre 40 volte l'età dell'universo. Un computer veloce non aiuta granché: se fa 10^9 mosse al secondo, servono ancora ≈ 585 anni.

Possiamo fare meglio? La risposta è no.

TEOREMA 11.5.4 (Limite inferiore per le Torri di Hanoi). Sia $L(n)$ il numero di mosse necessarie a qualsiasi algoritmo corretto per risolvere il problema. Allora $L(n) \geq 2^n - 1$.

Dimostrazione (argomento di necessità). Per spostare il disco più grande dal piolo p al piolo t , occorre che (a) i restanti $n - 1$ dischi siano fuori da p e t , quindi tutti su s , (b) il piolo t sia libero. Quindi prima del singolo spostamento del disco grande dobbiamo aver eseguito $L(n - 1)$ mosse per impilare gli $n - 1$ dischi piccoli su s . Dopo lo spostamento dobbiamo eseguire altre $L(n - 1)$ mosse per riportarli sopra il disco grande in t . Vale dunque

$$L(n) \geq 2L(n - 1) + 1 \quad \text{con } L(1) = 1, \quad (164)$$

da cui $L(n) \geq 2^n - 1$. Poiché $M(n) = 2^n - 1$, l'algoritmo è ottimo: l'esponenziale è inerente al problema, non all'algoritmo. $\square$

OSSERVAZIONE 11.5.5. Hanoi è un problema **intrinsecamente intrattabile**: nessun algoritmo, presente o futuro, potrà mai risolverlo in tempo subesponenziale. Per altri problemi intrattabili, come quelli che vedremo nella prossima sezione, *non sappiamo* se sia così: forse esistono algoritmi polinomiali che nessuno ha ancora trovato.

11.5.2. Generazione di tutte le sequenze binarie

Dato un insieme $A = \{a_0, \dots, a_{n-1}\}$, ogni sottoinsieme $S \subseteq A$ è codificato da un array binario B_S di n bit: $B_S[i] = 1$ se $a_i \in S$, $B_S[i] = 0$ altrimenti. I sottoinsiemi sono dunque 2^n , e per generarli tutti basta generare le 2^n sequenze binarie di n bit.

GeneraBinarie(B[], i, n) — riempie ricorsivamente B in posizione i

```
GeneraBinarie(B, i, n) {
    if (i == n) Elabora(B);
    else {
        B[i] = 0;
        GeneraBinarie(B, i + 1, n);
        B[i] = 1;
        GeneraBinarie(B, i + 1, n);
    }
}
// Chiamata iniziale: GeneraBinarie(B, 0, n)
```

L'albero della ricorsione ha 2^n foglie (una per sequenza). Costo: $T(n) = \Theta(2^n \cdot c_{\text{Elabora}})$ dove c_{Elabora} è il costo di trattare ciascuna sequenza prodotta. **L'esponenziale è inevitabile** perché l'output stesso ha dimensione esponenziale.

11.5.3. Generazione di tutte le permutazioni

Le permutazioni di un array P di n elementi sono $n!$. L'idea: ogni permutazione si fissa scegliendo l'elemento in posizione 0 (e ce ne sono n scelte), poi si generano ricorsivamente le permutazioni dei restanti $n - 1$ elementi.

GeneraPermutazioni($P[], i, n$)

```
GeneraPermutazioni(P, i, n) {
    if (i == n - 1) Elabora(P);
    else {
        for (j = i; j < n; j = j + 1) {
            scambia P[i] e P[j];
            GeneraPermutazioni(P, i + 1, n);
            scambia P[i] e P[j]; // ripristina l'ordine
        }
    }
}
// Chiamata iniziale: GeneraPermutazioni(P, 0, n)
```

Costo: $T(n) = \Theta(n! \cdot c_{\text{Elabora}})$. Anche qui, la dimensione dell'output (numero di permutazioni stampate) è $n!$, quindi l'esponenziale è inerente al problema di enumerazione.

Nota. Questi due algoritmi sono usatissimi come «motori di forza bruta» sopra problemi più ricchi: per esempio, per cercare la migliore CLIQUE in un grafo si possono enumerare i 2^n sottoinsiemi di vertici e per ciascuno verificare se è una clique. Da qui in poi la domanda interessante è: si può *evitare* questa enumerazione esaustiva?

11.6. Le classi di complessità

Per studiare la difficoltà dei problemi **decidibili** in termini di tempo, è utile separarli in tipologie e poi raggruppare i problemi che richiedono risorse confrontabili.

11.6.1. Tipologie di problemi computazionali

DEFINIZIONE 11.6.1 (Decisionali, di ricerca, di ottimizzazione). Un problema Π ha un insieme $\mathcal{I}$ di istanze e un insieme $\mathcal{S}$ di soluzioni.

- **Decisionale:** $\mathcal{S} = \{0, 1\}$. Esempi: « G è connesso?», « p è primo?». Le istanze su cui $\Pi(x) = 1$ si dicono **positive** (o **accettabili**); quelle su cui $\Pi(x) = 0$, **negative**.
- **Di ricerca:** data x , restituire una qualsiasi soluzione $s \in \mathcal{S}$ (per es. trovare un cammino fra due vertici).
- **Di ottimizzazione:** fra tutte le soluzioni ammissibili, restituire quella di costo minimo o di valore massimo (per es. clique di cardinalità massima, cammino minimo).

Nota (Perché studiare i decisionali). La teoria della complessità si concentra su problemi decisionali per due motivi: (a) la risposta è un singolo bit, quindi tutto il tempo dell'algoritmo è speso nel calcolo e non nella scrittura dell'output; (b) la versione decisionale è un **limite inferiore** alla versione di ottimizzazione: se sappiamo trovare la clique massima in G , sappiamo anche rispondere a « G ha una clique di almeno k vertici?». Quindi dimostrare che la versione decisionale è difficile rende difficile anche quella di ottimizzazione.

11.6.2. Le classi P, PSPACE, EXP

DEFINIZIONE 11.6.2 (Classe P). **P** è la classe dei problemi decisionali risolvibili in **tempo polinomiale** nella dimensione n dell'istanza, ossia per i quali esiste un algoritmo di costo $O(n^k)$ con k costante.

DEFINIZIONE 11.6.3 (Classe PSPACE). **PSPACE** è la classe dei problemi decisionali risolvibili in **spazio polinomiale** nella dimensione n dell'istanza.

DEFINIZIONE 11.6.4 (Classe EXP (o EXPTIME)). **EXP** è la classe dei problemi decisionali risolvibili in **tempo esponenziale**, ossia $O(2^{n^k})$ per qualche costante k .

PROPOSIZIONE 11.6.5 (Inclusioni fra classi). Vale la catena di inclusioni $P \subseteq \text{PSPACE} \subseteq \text{EXP}$.

Dimostrazione (intuitiva). $P \subseteq \text{PSPACE}$: in tempo polinomiale si possono visitare al più un numero polinomiale di celle di memoria distinte (in ordine di grandezza). $\text{PSPACE} \subseteq \text{EXP}$: con $s(n)$ bit di memoria si hanno al più $2^{s(n)}$ configurazioni distinte; un algoritmo che usa spazio polinomiale ha al più $2^{\text{poly}(n)}$ configurazioni e termina entro questo numero di passi (altrimenti entrerebbe in ciclo). $\square$

ESEMPIO 11.6.6 (Problemi in P). Ordinamento ($O(n \log n)$), ricerca in array/lista/albero, ordinamento topologico di un DAG, connettività di un grafo, ricerca di un ciclo euleriano, test di primalità.

ESEMPIO 11.6.7 (Problemi presumibilmente intrattabili). Per i problemi seguenti conosciamo solo algoritmi esponenziali, ma *nessuno ha mai dimostrato* che non possa esistere un algoritmo polinomiale.

CLIQUE. Dato un grafo $G = (V, E)$ e un intero $k > 0$, G contiene una clique (sottografo completo) di almeno k nodi? L'algoritmo banale prova tutti i 2^n sottoinsiemi di vertici.

Cammino Hamiltoniano. Dato $G = (V, E)$, esiste un cammino semplice che attraversa *tutti* i vertici esattamente una volta? L'algoritmo banale prova tutte le $n!$ permutazioni dei vertici.

SAT (soddisfacibilità booleana). Data una formula F in **forma normale congiuntiva** (FNC), cioè $F = c_1 \wedge c_2 \wedge \dots \wedge c_m$ dove ogni clausola c_i è una disgiunzione (OR) di letterali (variabili o loro negazioni), esiste un'assegnazione di verità alle variabili che rende F vera? Per n variabili, l'algoritmo banale prova tutti i 2^n assegnamenti.

ESEMPIO 11.6.8 (Una formula SAT). $V = \{x, y, z, w\}$, $F = (x \vee y \vee z) \wedge (\bar{x} \vee w) \wedge y$. L'assegnazione $x = 1, y = 1, z = 0, w = 1$ rende ogni clausola vera, dunque F è soddisfacibile.

11.7. Verifica e classe NP

L'intuizione chiave per definire NP è la differenza fra *trovare* una soluzione e *verificare* che una soluzione candidata sia corretta. Spesso le due cose hanno costi radicalmente diversi: trovare i fattori di un intero di 200 cifre è proibitivo, ma se qualcuno ti dà i fattori, moltiplicarli e controllare richiede un microsecondo.

DEFINIZIONE 11.7.1 (Certificato). Per un problema decisionale Π e un'istanza accettabile x (con $\Pi(x) = 1$), un **certificato** è un'informazione che permette di verificare in tempo polinomiale che effettivamente $\Pi(x) = 1$.

ESEMPIO 11.7.2 (Certificati naturali).

- **CLIQUE:** il certificato è il sottoinsieme di k vertici. La verifica controlla in tempo $O(k^2)$ che ogni coppia sia collegata da un arco.
- **SAT:** il certificato è un'assegnazione α alle variabili. La verifica valuta F su α in tempo $O(|F|)$.
- **Cammino Hamiltoniano:** il certificato è una sequenza di vertici. La verifica controlla in $O(n)$ che ogni vertice compaia una volta e che ogni transizione sia un arco.
- **Fattorizzazione (versione decisionale: n ha un fattore $\leq k$?):** il certificato è il fattore $d \leq k$. La verifica esegue una sola divisione.

DEFINIZIONE 11.7.3 (Classe NP). NP è la classe dei problemi decisionali per i quali esiste un **algoritmo di verifica** polinomiale, ossia un algoritmo $V(x, c)$ tale che:

1. V termina in tempo $\text{poly}(|x|)$;
2. $\Pi(x) = 1$ se e solo se esiste un certificato c di lunghezza $\text{poly}(|x|)$ tale che $V(x, c) = 1$.

Nota (Perché si chiama NP?). NP sta per **Nondeterministic Polynomial time**. La definizione equivalente «storica» è: $\Pi \in \text{NP}$ se esiste una macchina di Turing *non deterministica* (capace di «indovinare» la scelta giusta a ogni passo) che lo risolve in tempo polinomiale. Le due definizioni coincidono perché la macchina non deterministica può essere simulata dalla coppia «indovina il certificato + verifica».

Attenzione: NP *non* significa «non polinomiale».

PROPOSIZIONE 11.7.4. $P \subseteq \text{NP}$.

Dimostrazione (diretta). Se $\Pi \in P$, esiste un algoritmo A polinomiale che risolve Π . Definiamo l'algoritmo di verifica $V(x, c) = A(x)$, che ignora il certificato. Questo soddisfa la definizione di NP. $\square$

OSSERVAZIONE 11.7.5. Rapporto P-NP. P è dunque contenuto in NP. La grande domanda aperta della complessità computazionale è: $P = \text{NP}$ oppure $P \subsetneq \text{NP}$? In altre parole, **verificare velocemente è equivalente a risolvere velocemente?**

La maggior parte degli informatici teorici crede che $P \neq \text{NP}$, ma nessuno è mai riuscito a dimostrarlo. La domanda è uno dei sette **Problemi del Millennio** del Clay Mathematics Institute, con un premio di un milione di dollari per chi la risolverà.

11.8. Riduzioni polinomiali

L'idea: per confrontare la difficoltà di due problemi, si «trasforma» un problema nell'altro. Se sappiamo risolvere il secondo, sappiamo risolvere anche il primo.

DEFINIZIONE 11.8.1 (Riduzione polinomiale). Siano Π_1 e Π_2 problemi decisionali con insiemi di istanze $\mathcal{I}_1$ e $\mathcal{I}_2$. Si dice che Π_1 **si riduce in tempo polinomiale** a Π_2 , e si scrive

$$\Pi_1 \leq_p \Pi_2 \quad (165)$$

se esiste una funzione $f : \mathcal{I}_1 \rightarrow \mathcal{I}_2$ calcolabile in tempo polinomiale tale che, per ogni istanza $x \in \mathcal{I}_1$,

$$\Pi_1(x) = 1 \iff \Pi_2(f(x)) = 1. \quad (166)$$

OSSERVAZIONE 11.8.2. Cosa significa. Una riduzione $\Pi_1 \leq_p \Pi_2$ è un **traduttore**: trasforma istanze di Π_1 in istanze di Π_2 preservandone la risposta. Il fatto che f sia polinomiale è cruciale: garantisce che la traduzione non «distrugge» l'efficienza.

PROPOSIZIONE 11.8.3 (La riduzione trasporta la trattabilità). Se $\Pi_1 \leq_p \Pi_2$ e $\Pi_2 \in P$, allora $\Pi_1 \in P$.

Dimostrazione (composizione di algoritmi). Sia f la riduzione, calcolabile in tempo $O(n^k)$, e sia A_2 un algoritmo polinomiale per Π_2 di costo $O(m^h)$. Dato $x \in \mathcal{I}_1$ di taglia n :

1. calcoliamo $y = f(x)$ in tempo $O(n^k)$, ottenendo un'istanza di Π_2 la cui taglia m è al più $O(n^k)$ (non si può scrivere più di quanto si possa calcolare);
2. restituiamo $A_2(y)$, in tempo $O(m^h) = O(n^{kh})$.

Il costo totale è polinomiale e la risposta è corretta per definizione di riduzione. $\square$

Nota. La proposizione si legge anche al contrario (per contrapposizione): se $\Pi_1 \notin P$ e $\Pi_1 \leq_p \Pi_2$, allora $\Pi_2 \notin P$. **Una riduzione di A a B certifica che B è almeno difficile quanto A .**

11.9. NP-hard e NP-completi

Le riduzioni polinomiali permettono di individuare i problemi **più difficili** dentro NP.

DEFINIZIONE 11.9.1 (NP-hard, NP-completo). Un problema decisionale Π si dice:

- **NP-hard** se ogni problema $\Pi' \in \text{NP}$ si riduce polinomialmente a Π :

$$\forall \Pi' \in \text{NP}, \quad \Pi' \leq_p \Pi. \quad (167)$$

- **NP-completo** se è NP-hard e inoltre $\Pi \in \text{NP}$.

Da ricordare. Intuitivamente, i problemi **NP-completi** sono i più difficili dentro NP. Un problema **NP-hard** può anche stare fuori da NP (è «almeno» così difficile, ma potrebbe esserlo di più).

TEOREMA 11.9.2 (Conseguenze della NP-completezza). Sia Π NP-completo.

1. Se $\Pi \in P$, allora $P = \text{NP}$.
2. Se $P \neq \text{NP}$, allora $\Pi \notin P$.

Dimostrazione (uso della riduzione). **Punto 1.** Sia Π' un qualsiasi problema in NP. Per NP-completezza $\Pi' \leq_p \Pi$. Se $\Pi \in P$, per la proposizione precedente $\Pi' \in P$. Vale per ogni $\Pi' \in \text{NP}$, quindi $\text{NP} \subseteq P$. Combinato con $P \subseteq \text{NP}$, si ottiene $P = \text{NP}$.

Punto 2. Contropositiva del Punto 1. □

Nota (Conseguenza pratica). Tutti i problemi NP-completi sono **equivalenti** dal punto di vista della complessità: o sono *tutti* trattabili (e $P = \text{NP}$), o sono *tutti* intrattabili. Trovare un algoritmo polinomiale per uno *qualunque* di essi risolverebbe automaticamente l'intera classe NP.

11.9.1. Il teorema di Cook–Levin

Il primo problema del quale si è dimostrata la NP-completezza è SAT. Questo fornisce un «punto di ancoraggio»: dimostrare che un nuovo problema Π è NP-completo si riduce a:

1. verificare che $\Pi \in \text{NP}$ (di solito facile: si esibisce un certificato);
2. esibire una riduzione $\text{SAT} \leq_p \Pi$ (oppure da un altro problema NP-completo già noto).

TEOREMA 11.9.3 (Cook (1971), Levin (1973)). SAT è NP-completo.

(La dimostrazione, tecnica, non viene riportata in questo corso.)

PROPOSIZIONE 11.9.4 (Caratterizzazione operativa della NP-completezza). Un problema $\Pi \in \text{NP}$ è NP-completo se e solo se $\text{SAT} \leq_p \Pi$ (o, equivalentemente, $\Pi^* \leq_p \Pi$ per un qualsiasi problema Π^* NP-completo).

Dimostrazione (transitività delle riduzioni). Le riduzioni polinomiali sono transitive: se $A \leq_p B$ tramite f e $B \leq_p C$ tramite g , allora $A \leq_p C$ tramite $g \circ f$ (composizione di funzioni polinomiali è polinomiale). Per ogni $\Pi' \in \text{NP}$, $\Pi' \leq_p \text{SAT}$ (Cook) e per ipotesi $\text{SAT} \leq_p \Pi$, dunque $\Pi' \leq_p \Pi$. $\square$

11.9.2. Una riduzione esemplare: $\text{SAT} \leq_p \text{CLIQUE}$

Mostriamo come la riduzione di Cook–Levin si propaga ad altri problemi.

TEOREMA 11.9.5. CLIQUE è NP-completo.

Dimostrazione (riduzione esplicita). CLIQUE è in NP. Certificato: il sottoinsieme di k vertici. Verifica in $O(k^2)$.

Riduzione $\text{SAT} \leq_p \text{CLIQUE}$. Data una formula in FNC $F = c_1 \wedge c_2 \wedge \dots \wedge c_k$, costruiamo in tempo polinomiale un grafo $G = (V, E)$ tale che G contenga una k -clique se e solo se F è soddisfacibile.

Vertici. Per ogni occorrenza di un letterale ℓ nella clausola c_i inseriamo il vertice ℓ_i . Se F ha k clausole e ognuna ha al più r letterali, $|V| \leq kr$.

Archi. Mettiamo l'arco (x_i, y_j) se e solo se:

1. $i \neq j$ (i due letterali appartengono a **clausole diverse**);
2. $x \neq \bar{y}$ (i due letterali sono **consistenti**: possono essere veri contemporaneamente).

Costruzione in tempo polinomiale: $|E| \leq |V|^2 \leq (kr)^2$, e ogni arco si decide in tempo costante.

Equivalenza.

($\Leftarrow$) Se F è soddisfacibile, esiste un'assegnazione α che rende vera ogni clausola. Per ogni clausola c_i scegliamo un letterale ℓ_i^* vero in α e prendiamo i vertici corrispondenti $\ell_1^*, \dots, \ell_k^*$. Sono vertici di clausole diverse, e sono mutualmente consistenti (perché tutti veri sotto α). Quindi formano una k -clique.

($\Rightarrow$) Se G contiene una k -clique, i suoi k vertici devono appartenere a k clausole *distinte* (per la condizione (1)). Inoltre i corrispondenti letterali sono tutti consistenti (per la condizione (2)), dunque possiamo assegnare valore *vero* a tutti questi letterali simultaneamente, soddisfacendo tutte e k le clausole. Estendendo arbitrariamente l'assegnazione alle variabili rimaste libere, otteniamo un'assegnazione che soddisfa F . $\square$

ESEMPIO 11.9.6 ($F = (a \text{ or } b) \text{ and } (\overline{a} \text{ or } \overline{b} \text{ or } c) \text{ and } \overline{c}$). La formula ha $k = 3$ clausole; il grafo costruito ha 6 vertici: $a_1, b_1, \overline{a}_2, \overline{b}_2, c_2, \overline{c}_3$.

Gli archi collegano coppie di vertici (i) di clausole diverse, (ii) consistenti. Ad esempio: a_1 è collegato a $\overline{b}_2$ e a c_2 ma *non* a $\overline{a}_2$ (sono inconsistenti); a_1 non è collegato a b_1 (stessa clausola).

Una 3-clique nel grafo è $\{b_1, \overline{a}_2, \overline{c}_3\}$, che corrisponde all'assegnazione parziale $b = 1, a = 0, c = 0$: si verifica che soddisfa F .

Nota (Problemi NP-equivalenti). Per la transitività delle riduzioni, da $\text{SAT} \leq_p \text{CLIQUE}$ e dal fatto che SAT è NP-completo segue anche $\text{CLIQUE} \leq_p \text{SAT}$: SAT e CLIQUE sono **NP-equivalenti**. Tutti i problemi NP-completi sono mutuamente equivalenti: o sono *tutti* facili (e $P = \text{NP}$), o sono *tutti* difficili.

11.9.3. Altri NP-completi notevoli

Una volta dimostrato che CLIQUE è NP-completo, si possono dimostrare altri risultati di NP-completeness riducendosi a CLIQUE (o a SAT, o a uno qualsiasi dei nuovi NP-completi). Si è creata così una rete di migliaia di problemi NP-completi: se uno solo di essi fosse in P , lo sarebbero tutti.

ESEMPIO 11.9.7 (Altri famosi NP-completi).

- **Cammino e Ciclo Hamiltoniano:** dato $G = (V, E)$, esiste un cammino (rispettivamente, un ciclo) semplice che attraversa *tutti* i vertici una ed una sola volta?
- **Commesso Viaggiatore (TSP):** dato un grafo completo G con pesi w sugli archi ed un intero k , esiste un ciclo di peso al più k che attraversa ogni vertice una ed una sola volta?
- **Zaino (Knapsack):** dato uno zaino di capacità c , n oggetti di dimensioni $s_1, \dots, s_n$ con profitti $p_1, \dots, p_n$ ed un intero k , esiste un sottoinsieme degli oggetti di dimensione totale al più c che garantisca profitto totale almeno k ?
- **Map Coloring:** è possibile colorare i vertici di un grafo con un numero limitato di colori in modo che a due vertici adiacenti sia assegnato un colore diverso? Sui grafi non planari è un problema difficile.

11.10. La gerarchia completa e la questione di P vs NP

Riassumiamo le inclusioni note (e le poche che siamo certi essere strette):

$$P \subseteq \text{NP} \subseteq \text{PSPACE} \subseteq \text{EXP} \subseteq \text{Decidibili} \subsetneq \text{Tutti i problemi.} \quad (168)$$

OSSERVAZIONE 11.10.1. Sebbene si sappia che la classe dei problemi decidibili contiene strettamente la classe dei problemi indecidibili (per esempio, il problema dell'arresto sta fuori dalla prima), le inclusioni intermedie $P \subseteq NP \subseteq PSPACE \subseteq EXP$ sono in larga parte non ancora note essere strette. La domanda più celebre è la prima della catena.

Nota (Cosa fare quando si incontra un NP-completo). Se la soluzione ottima è troppo difficile da ottenere, una soluzione quasi ottima ottenibile facilmente potrebbe essere già abbastanza buona. In pratica si ricorre a una di queste strategie:

- **Algoritmi approssimati:** restituiscono una soluzione la cui qualità non è troppo lontana da quella ottima.
- **Algoritmi probabilistici:** la soluzione ottima si ottiene con alta probabilità.
- **Euristiche:** simulazioni statistiche mostrano la bontà del metodo in molti casi, anche senza garanzie formali.

Da ricordare. Conseguenze sulla crittografia moderna. Buona parte della **crittografia a chiave pubblica** (RSA, DSA, ...) si fonda sull'ipotesi che la fattorizzazione di interi e il logaritmo discreto *non* siano in P . Curiosamente questi due problemi sono in NP ma *non* si sa se siano NP -completi. Una dimostrazione di $P = NP$ avrebbe conseguenze drammatiche: si «romperebbero» molti protocolli crittografici. Una dimostrazione di $P \neq NP$ confermerebbe che la sicurezza è fondata.

11.11. Esercizi

ESERCIZIO 11.11.1 — Discussione. Spiegare con parole proprie perché l'argomento di Turing per l'indcidibilità dell'arresto è una **diagonalizzazione**. Quale «tabella» stiamo costruendo, e qual è il «valore opposto» che mette in contraddizione l'enumerazione?

ESERCIZIO 11.11.2 — Vero/Falso. Stabilire se ciascuna delle seguenti affermazioni è vera o falsa, motivando.

1. Se $P = NP$, ogni problema in NP è risolvibile in tempo polinomiale.
2. Se $\Pi_1 \leq \Pi_2$ e Π_1 è NP -completo, allora Π_2 è NP -completo.
3. Se $\Pi_1 \leq_p \Pi_2$ e Π_2 è NP -completo, allora Π_1 è NP -completo.
4. L'esistenza di un algoritmo polinomiale per CLIQUE implicherebbe l'esistenza di un algoritmo polinomiale per SAT.
5. Il problema dell'arresto è in EXP .

ESERCIZIO 11.11.3 — Calcolo. Costruire esplicitamente il grafo G ottenuto dalla riduzione SAT-CLIQUE applicata alla formula

$$F = (x \vee y) \wedge (\bar{x} \vee y) \wedge (\bar{y} \vee z). \quad (169)$$

Quanti vertici ha? Quanti archi? Esibire una 3-clique e l'assegnazione corrispondente.

ESERCIZIO 11.11.4 — Dimostrazione. Dimostrare per induzione che $L(n) = 2^n - 1$, dove $L(n)$ è la ricorrenza $L(1) = 1$, $L(n) = 2L(n-1) + 1$. (Oltre alla base e al passo, mostrare anche che il «+1» è essenziale: cosa succederebbe se la ricorrenza fosse $L(n) = 2L(n-1)$?)

ESERCIZIO 11.11.5 — Controesempio. Mostrare che la condizione « f calcolabile in tempo polinomiale» nella definizione di riduzione $\leq_p$ è essenziale: esibire (anche informalmente) due problemi $\Pi_1 \notin P$ e $\Pi_2 \in P$ tali che esiste una funzione f (non polinomiale) che riduce Π_1 a Π_2 . Spiegare perché ciò non contraddice la proposizione « $\Pi_1 \leq_p \Pi_2 \wedge \Pi_2 \in P \Rightarrow \Pi_1 \in P$ ».

ESERCIZIO 11.11.6 — Discussione. L'algoritmo `Goldbach()` termina se e solo se la congettura di Goldbach è falsa. Supponiamo che un giorno si dimostri che Goldbach è vera. Cosa cambierebbe rispetto al teorema di Turing? E se invece si trovasse un controesempio?

ESERCIZIO 11.11.7 — Riduzione. Riprodurre, senza guardare il testo, la riduzione SAT $\leq_p$ CLIQUE: definizione dei vertici, regola degli archi, dimostrazione delle due implicazioni e analisi del costo.

ESERCIZIO 11.11.8 — Discussione. Spiegare a parole proprie perché, se un singolo problema NP-completo fosse risolvibile in tempo polinomiale, allora **tutti** i problemi in NP sarebbero risolvibili in tempo polinomiale.